

noVa64

SYNTHESIS DOCUMENT – INDEX AND CONTENT OUTLINE

W65C816S · 3× iCE40 TQ144 · RP2354B

Helium · Neon · Argon

all 3.3V · hand-solderable

100% open toolchain

MILESTONE 1 · APPLE II – autonomous boot to prompt

MILESTONE 2 · AMIGA – multitasking + windowed GUI

SHEET INDEX				
1		INTRODUCTION no sheets yet		
2		HARDWARE no sheets yet		
3		SOFTWARE no sheets yet		
4		PROJECT no sheets yet		
5		SPECIFICATIONS no sheets yet		
6		APPENDICES no sheets yet		
7		AI Assembled by AI and not yet reviewed. Every sheet starts here and moves to its part once it has been read.		
INTRODUCTION				
01	A	Project vision and philosophy · how to read this document.....		5 –
ARCHITECTURE				
02	B	Global architecture.....		9 Fig. 1
03	H	Peripherals and their location.....		12 table
04	X	The expansion bus · parallel slot and the slow I2C tier.....		16 tables
POWER				
05	C	Power supply.....		26 Fig. 2
06	S	Power control, shutdown and telemetry · \$FF power block.....		33 Fig. 9
EMBEDDED CONTROL AND BOOT				
07	D	Embedded controller · boot, USB topology, on-board hub and the recovery paths.....		41 Fig. 3
08	R	Debug agent and console · SPI register map.....		48 Fig. 8
CPU AND SYSTEM				
09	E	CPU – W65C816S and the softcore option.....		60 table
10	M	Kernel–hardware interface · Helium control registers.....		66 tables
MEMORY AND STORAGE				
11	F	Physical memory structure.....		71 Fig. 4
12	K	Virtual memory – concepts and glossary.....		75 Fig. 5

13	L	Virtual memory management.....	77	Fig. 6
14	G	Persistent storage – microSD.....	82	–
15	Y1	Filesystems · the VFS layer, vnode model and mount table.....	85	tables
16	Y2	Filesystems · NVFS, the native volume and its journal.....	91	tables
17	Y3	Filesystems · ext2, the transfer medium.....	101	tables
VIDEO + AUDIO				
18	T	Neon – the GPU · text, graphics, blitter and command lists...	108	Fig. 10
19	U	Neon – blitter, arbitration and the window compositor.....	130	Fig. 11
20	W	Audio · PCM5102A, four channels, hardware mixer and /dev/audio	142	tables
SOFTWARE				
21	I	BIOS – what it provides.....	154	–
22	J	Operating system.....	155	–
23	N	Process management.....	158	–
24	O	Programming and toolchain.....	158	–
25	V	The windowing OS · window server, client protocol and toolkit	160	Fig. 12
PROJECT AND BUILD				
26	P	Step-by-step build · prototype P0–P5 · target E0–E8 · laptop L1– L8 · host filesystem track F0–F9.....	171	Fig. 7
27	Q	Decision log · open questions.....	195	–
APPENDIX				
28	Z1	Appendix · glossary of terms and acronyms.....	225	tables
29	Z2	Appendix · index of figures.....	243	table

INTRODUCTION

A

Vision and philosophy

WHAT IT IS · WHAT FOR · UNDER WHAT RULES · HOW TO READ IT

SHEET 01

Project framework: what is being built, what finishing it means, what principles govern every decision, and how the rest of these sheets are meant to be used.

- A.1** What it is: an original portable personal computer built around the W65C816S, designed from scratch as a learning exercise in electronics, PCB design, gateway, computer architecture, and OS design. **From scratch is meant literally:** the schematic, the board, the FPGA gateway, the BIOS, the kernel and the windowing system are all written here. What is bought is bought as a part and not as a solution – a CPU, three FPGAs, a microcontroller, memories, a panel and a DAC – and everything that turns them into a machine is ours. **The learning is the deliverable, and it settles the build-or-buy calls.** A subsystem that could be finished faster by dropping in someone else's block, but would teach nothing on the way, is designed here rather than imported – and where the trade was made the other way it is recorded as a decision in sheet Q.
-
- A.2** The counterfactual it answers: **the 16-bit machine Commodore never built.** Had it not bought Amiga and gone looking elsewhere for a 16-bit future, the obvious move was to follow its own 8-bit line onto the 6502's natural successor. noVa64 is that machine – the Commodore that would have come next. **It is a compass, not a costume.** The lineage is inherited in character – a shared bus, custom chips beside the CPU, video and audio doing the work the processor cannot – and not in the bill of materials: those custom chips are FPGAs. The question asked of a decision is what that engineering culture would have done with parts a hobbyist can buy and solder today, not what 1987 happened to stock.
-
- A.3** Which settles why the CPU is a 65816 and not something better behaved. Lineage first: it is where the 6502 family goes. Then the practical case – 6502 simplicity kept, 24-bit addressing, 16-bit registers, and, decisively, the **ABORT** pin. ABORT is what separates a machine that can protect memory from one that merely divides it (→ sheet E).
The CPU is the premise, not a choice still under review; what is open is whether a softcore ever stands in for the physical part – the second half of sheet E, resting on a footprint that ships unpopulated and on the pin list of E.13.

A.4 Inspiration and goals: two milestones define success. **Apple II Milestone** – the machine boots on its own to a prompt. **Amiga Milestone** – preemptive multitasking with a windowed GUI.

The second is named for the capability, not the lineage: it is the bar the Amiga set, cleared by the machine Commodore would have built instead of buying it.

Both have an exact position in the build rather than a mood: the first closes with stage **E6** – SD, a minimal kernel and a serial shell – and the second with **E8**, the panel, the battery and the compositor. Everything in sheet P is either on the path to one of them or explicitly marked as being off it.

On its own means from the card, with no host attached – which is why P5 reaching a prompt on the carrier does not close it and E6 does.

A.5 **Compatible with nothing.** This machine runs its own software; it is not an emulator and no existing binary is expected to work on it. That is a freedom, not a gap – it is what removes any need for cycle-exact behaviour, and therefore what licenses a cache with variable latency, a stalled clock, and a soft core that need not match the silicon cycle for cycle. Several later decisions rest on this one and would collapse without it.

The freedom is paid for once, in full. Nothing above the transistors exists yet, so the BIOS, the filesystem, the scheduler, the drivers, the toolchain and the GUI are all inside the scope of this project – which is what the whole Software area is for. It also removes the last court of appeal: when a sheet asks whether some behaviour is *correct*, the answer can only come from this document and from the tests it sets, because there is no reference machine to defer to and no existing program whose expectations have to be met.

A.6 Non-negotiable principles: every component hand-solderable (TQFP / PLCC / TSOP / 0.5 mm max.), everything at 3.3V, open toolchain (KiCad · Yosys · nextpnr · IceStorm · pico-sdk), incremental progress with an explicit verification criterion per stage, and unpopulated footprints to preserve future expansion.

Each of those has already cost something and been paid. Hand-solderable is why the FPGAs are three iCE40 in TQ144 rather than one BGA, and the small block RAM that follows is why the MMU walks its tables in hardware (D01). An open toolchain rules out any part whose use depends on an NDA or a vendor blob, which is what removed a panel-bridge candidate outright (D05). One rail keeps every level shifter off the board, and an unpopulated footprint costs pennies at fabrication and is unbuyable afterwards.

A.7 **Scope of this document: it is the specification, not an index to one.** It began as an outline tying together detail that lived in separate design documents, and for a while the two had to be read side by side. That is over – every one of those documents has been folded in, and where they disagreed with this board the reconciliation is recorded here rather than left for the reader to perform. **Nothing in this project is documented anywhere else**, and a claim that cannot be found in these sheets should be treated as not having been made. The converse is the working rule: a decision taken but not written down is not a decision, and when the board and a sheet disagree one of the two is a bug – which one gets settled in sheet Q before anything is cut.

How to read this document – seven parts, ten areas, twenty-seven sheets, and a letter that never moves.

The document is three levels deep. A **part** is the coarsest division and there are seven of them – Introduction, Hardware, Software, Project, Specifications, Appendices, and AI. Inside a part the sheets are grouped into **areas**: Introduction, Architecture, Power, Embedded control and boot, CPU and system, Memory and storage, Video + audio, Software, Project and build, and the Appendix. Inside an area is the

sheet, and inside the sheet is the numbered item. The sheet *number* follows that order end to end. **The sheet letter does not, and that is deliberate.**

A.7a Every sheet is currently in the AI part, and that is a statement about review rather than about subject matter. The seven parts are declared and six of them are empty: a sheet sits in **AI** until it has been read by a human, and moves to the part it belongs to when it has. So the index is also a progress bar – **what is still under 7 · AI is what has not yet been checked**, and an empty part is not a gap in the design but a queue that has not been worked yet.

The mechanism is one field. A sheet names its part in `manifest.json` and nothing else records it – not the file name, not the letter, not the prose – so migrating one is a single edit and the numbering, the sidebar, the index and the printed page breaks all follow. Areas are unaffected: they keep their own grouping *inside* whichever part their sheets end up in.

A.8 The letter is the sheet's address, not its position. Letters were handed out in the order the sheets were written, and every cross-reference in the prose points at one – sheet T for Neon, T.53 for a single item inside it. Renumbering them to match the areas would rewrite several hundred references at once and break every link anyone has kept, so it is not done: a new sheet still takes the next free letter and is placed in whichever area it belongs to.

A suffix means one subject that outgrew a single sheet, and there are two such runs. The appendix is Z1 and Z2, glossary and figures; the filesystem stack is Y1, Y2 and Y3 – the VFS layer, NVFS and ext2 – because it is one subject three ways and not three subjects. Splitting cost a suffix rather than three consecutive letters, and the letters run A, B, C... underneath either way.

A.9 Every item is numbered and individually addressable. **F.4** is the fourth item of sheet F, and it can be linked to directly; the reference (`→ [Q41]` (`sec_q#q41`)) means an open question is tracked in sheet Q and this item depends on how it closes. Items are the unit the document argues in – a sheet is a container, an item is a claim.

The item is also the unit a *reference* points at, never a page. This document is produced from markdown at whatever paper size the printer is handed, so a page number belongs to one printing while **T.53** survives every reflow. **The sheet index of a printed copy is the one exception and it proves the rule:** it carries the page each sheet opens on, generated for that PDF at that paper size, and it is there to be leafed through rather than cited.

A.10 A gold TEST ▶ line is a verification criterion, not commentary: it says what has to be observed before the item is considered true. A dimmed line below an item is a note – context, a caveat, or the reason a decision moved.

A.11 Chips mark the state of a question. A mint chip `open` is something undecided but not in the way; a gold chip `blocking` is a contradiction or a dependency that stops work until it closes. Both are resolved in sheet Q, which holds the decision log and the open questions with their numbers.

A.12 The checkboxes belong to sheet P and only to it: that sheet is the build, and its boxes carry the four states below. The gold banner marks a milestone from A.4.

Pending, done, in progress, optional. **Those boxes are the only claim this document makes about what physically exists;** every other sheet describes the design as specified, whether or not anyone has yet soldered it.

- A.13 The twelve figures are numbered across the whole document**, not per sheet, and each one lives in the sheet it illustrates. Z2 lists all twelve with their sheet, and Z1 is the glossary for every acronym used anywhere – deliberately redundant, since each acronym is also expanded where it is first used.
- A.14 Conventions in the prose.** An address is written `$FF:8000` when the bank matters and flat, `$FE0000`, when it is being used as an operand; register and signal names are in `code` font. **Polarity is stated, never inferred from a name** – the idle and asserted states of the inter-device nets are given explicitly in B.6's table, because the naming across the board is not consistent enough to be read as a convention.
- A.15 Three ways in.** Front to back is one reading, and the areas are ordered for it. A hardware reading can stop after Memory and storage; a software one can start at the BIOS and take everything below the kernel–hardware interface of sheet M as given. Anyone who only wants to know what is being worked on right now should read sheet P and sheet Q and nothing else – between them they hold the build state and every question still open.
- A.16 One machine, two build tracks – and every sheet specifies the target board.** A third series, F0–F9, builds the filesystem stack on a development PC and touches no hardware at all. Sheet P runs a **P-series** prototype on a commercial Colorlight i9 module, with Helium and Neon merged into a single ECP5 and the CPU, SRAM, EC and audio outside it, beside the **E-series** that builds the three-iCE40 board this document describes. The prototype is temporary packaging of the same design, not a variant of it (D40): where the two differ – no battery, no charger, no panel, so sheets C and S go unexercised – the difference is a gap recorded in sheet P, never a second specification competing with the sheet it came from. The revision line under the masthead carries **the document's own version first, then a revision per subsystem**. `v0.1` is this document as issued – the point at which every separate design document had been folded in (A.7); the letters after it belong to the subsystems, which revise on their own schedule and independently of the version, and **REV A means first issue, not absent** – an area is listed at REV A because nothing in it has yet been superseded. A superseded revision is not silently overwritten either – Neon REV A's single 102 MHz clock domain does not exist on this board, and the correction to 103.125 MHz lives where the reader meets the problem, in B.7 and T.4, with the reasoning in sheet Q.

MARK	MEANING
A.7 in mint	Item number – the anchor a cross-reference points at (<code>sec_a#a7</code>)
TEST ▶ in gold	The verification criterion for the item above it
Dimmed line	A note: context, caveat, or the history of a decision
Mint chip <code>open</code>	Open, tracked in sheet Q, not blocking
Gold chip <code>blocking</code>	Blocking – a contradiction or an unresolved dependency
Checkbox	Build state in sheet P: pending · done · in progress · optional
Gold banner	A milestone: Apple II, or Amiga

B**Global architecture**

PHYSICAL CPU + OPTIONAL SOFTCORE · HELIUM = NORTHBRIDGE · NEON = VIDEO/AUDIO

SHEET 02

An Amiga-style shared bus with four inhabitants: the physical CPU, two always-present FPGAs – Helium (FPGA-A) and Neon (FPGA-B) – and an optional third, Argon (FPGA-C). The RP2354B stays off the bus, acting as the board controller.

B.1 CPU: W65C816S in a PLCC-44 socket, 24-bit native mode. PHI2 generated by FPGA-A (Helium), target 8 MHz at 3.3V. The BE signal lets the CPU release the bus – used both by Argon and, far more often, by Helium itself to take the shared nets during a fill (→ F.4). Detail in sheet E.

B.2 FPGA-A (Helium) – the "northbridge": MMU + TLB + walker, cache, bus arbiter, prioritized interrupt controller, system timer, UART, SPI-SD, I2C, the debug agent (→ sheet R), and CPU glue logic. **A pure state machine**, never a soft CPU: anything in the critical memory path pays for itself on every access. Intelligence – filesystems, protocol stacks – lives in the RP2354B or in 65816 software.

iCE40HX8K TQ144 · budget: 90/107 pins for CPU bus + SDRAM + SRAM, 17 free. [open](#) the free pins do not yet cover SPI-SD, UART and the debug port (→ Q8).

B.3 FPGA-B (Neon) – video and audio: taps the bus directly; the CPU reaches its **data** through a 64 KB VRAM window in bank `$FE` and its **registers** through a 256-byte window at `$FF:8000`, both validated by the MMU's NEON_BUS_SEL signal. Its own 64 MB SDRAM (chip RAM – same part and same controller as Helium's), RGB-TTL output to the panel, bring-up VGA, and PCM5102A over I2S. **Fully specified in sheet T**: three video modes, a blitter, and a command processor that renders whole frames from a list in SDRAM with the CPU doing nothing. **Audio is sheet W** – four channels mixed in hardware, because the 65816 has 181 cycles per sample frame and cannot mix anything (W.1).

Local memory rather than streaming pixels from Helium: video across a link would need ~26 MB/s sustained with hard deadlines and would make Helium the bottleneck for the whole machine.

Neon is the one subsystem on this board that is useful before anything else works. Its text mode lives entirely in the FPGA's own block RAM, initialised from the bitstream, and depends on no SDRAM, no arbiter and no bus: a correct screen exists the moment `CDONE` rises, with Helium unconfigured and the 65816 still in reset (→ T.1, T.53). That ordering is deliberate and should be preserved as the design evolves.

B.4 FPGA-C (Argon) – optional, unpopulated at first: a 65816 softcore (P65816 reference) that takes the same bus with BE=0. Allows switching from the real CPU to a softcore without touching the rest.

Whether that is worth doing is a live question – on this platform the win is CPI and features, not clock (→ sheet E, Q13).

B.5 RP2354B – embedded controller off the bus: configures, boots, and watches over the system (→ sheet D). **Its 2 MB of flash is in the package**, and it is the board's only mandatory non-volatile storage (D.8).

B.6 Interconnect: a single shared A/D/control bus. The dedicated *data* link between FPGAs was removed, but **three** control lines survive it, and the naming says what they are for: **we select Neon's bus, and while Neon's bus is busy Helium holds the CPU.**

NET	DIRECTION	ASSERTED MEANS	IDLE / PULLED STATE
NEON_BUS_SEL	Helium → Neon	"Helium has validated this cycle for you" – for \$FE , after translation and the permission check; for \$FF:8000–\$FF:80FF , after the bank decode and the privilege check	Deasserted. Active low with a pull-up, so an unconfigured Helium cannot select Neon by accident (B.8)
NEON_BUS_BSY	Neon → Helium	"I cannot serve this cycle yet." Its meaningful edge is the deassertion , which says the data is in Neon's output register (T.18)	Asserted. Active high with a pull-up – the one place where the pulled state is the <i>asserted</i> one, and deliberately so
NEON_IRQ	Neon → Helium's PIC	Vblank, raster compare, or a display list's SIGNAL ; aggregated inside Neon and demultiplexed by the kernel reading IRQ_STATUS (T.45)	Deasserted

NEON_BUS_SEL qualifies two windows, not one, and the second costs Neon no pin: it already taps A0–A15 and captures the bank byte off D0–D7 during PHI2 low, so **\$FF** is the same comparator as **\$FE** with a different constant (→ T.21, D36). **NEON_BUS_BSY** stalls PHI2, not RDY. D16 rejected RDY for cache stalls because some W65C816S revisions ignore it during writes, and the same argument applies here; gating the clock sidesteps the question entirely and Helium already owns that gate. It is worth more than it looks – it is the reason Neon's graphics aperture can be *read* at all, which deleted a whole diagnostic register path REV A carried (→ T.7, D32, D37). **Busy-by-default is a safety property.** Were the net idle-low, Helium could sample "not busy" before Neon had noticed the cycle and resume onto a bus nobody was driving. It also makes an **absent, unconfigured or reset Neon read as busy**, so the machine stalls and the watchdog reports it rather than returning whatever a floating bus settles at – and E1.4's staged population guarantees stages where Neon is not fitted (→ F.11). **These three nets are the whole of the inter-device contract, and what the prototype board must emulate.** Sheet P's carrier merges Helium and Neon into one ECP5, where the temptation is to let them talk through anything handy – an earlier carrier proposal invented a 56-signal command and peek link, unaware that D04 had deleted it. On one die **neon.v** still takes A0–A15, D0–D7, PHI2, RWB and **NEON_BUS_SEL**, still drives **NEON_BUS_BSY** and **NEON_IRQ**, and nothing else crosses (P.13). **A merged die that respects this table is a re-partitioning; one that does not is a rewrite.** Those three nets now have a second tenant, and the contract was written to allow it. Sheet X gives an expansion slot **/EXP_SEL_n**, **/EXP_BSY** and **/EXP_IRQ** – the same shape, the same stall-first protocol, the same watchdog – with one polarity deliberately reversed: a slot's busy line idles *not busy*, because an empty slot must not stall the machine the way an unfitted Neon does (X.11). **The**

inter-FPGA contract is untouched; what generalises is its shape – which is what Q56 chose the names for.

B.7 Clocking: **one 25 MHz oscillator** in a star to the three FPGAs, **33 Ω in series in each branch**, entering each through a GBIN pin; every device's PLL then synthesises what it needs – Helium's core and PHI2, Neon's pixel clock, Argon's core. One part, one net, no cross-chip skew – and the series resistors are not decoration: an unterminated star into three CMOS inputs rings.

Neon's specification got this wrong at REV A: it derived a single 102.000 MHz domain from a 12 MHz crystal, which this board neither has nor can reach from 25 MHz. Nearest legal PLL: **103.125 MHz core, 51.5625 MHz pixel**, vertical total 635 → 640 to land back at ~60 Hz (→ T.4, Q43). Its second PLL is then unspent (→ T.9).

The star gains a fourth destination the moment a slot exists. X.15 puts the 25 MHz reference on the connector with its own series resistor – and makes a fan-out buffer the answer to a second slot, rather than a fifth branch hung on the same net.

B.8 Safe state with no bitstream loaded: **RESB pulled down** so the CPU stays in reset, control lines pulled up so nothing reads as asserted, and the shared data bus held by weak pull-ups so it never floats.

The board can be powered with all three FPGAs blank and nothing fights for the bus – which is exactly what makes the staged bring-up of E1.5–E1.7 possible.

One exception is electrical, not logical. **RDY** is not an input: it is **bidirectional and open-drain**, pulled low by the CPU on **WAI**. So it needs ~3.3 k Ω where the rest take 10 k Ω , or the rising edge is slow and **WAI** is slow to release. Every other control net is push-pull from one device, its pull-up only for the blank case.

NEON_BUS_BSY is the one exception to "nothing reads as asserted": there the safe state *is* the asserted one, and its pull-up means busy (B.6).

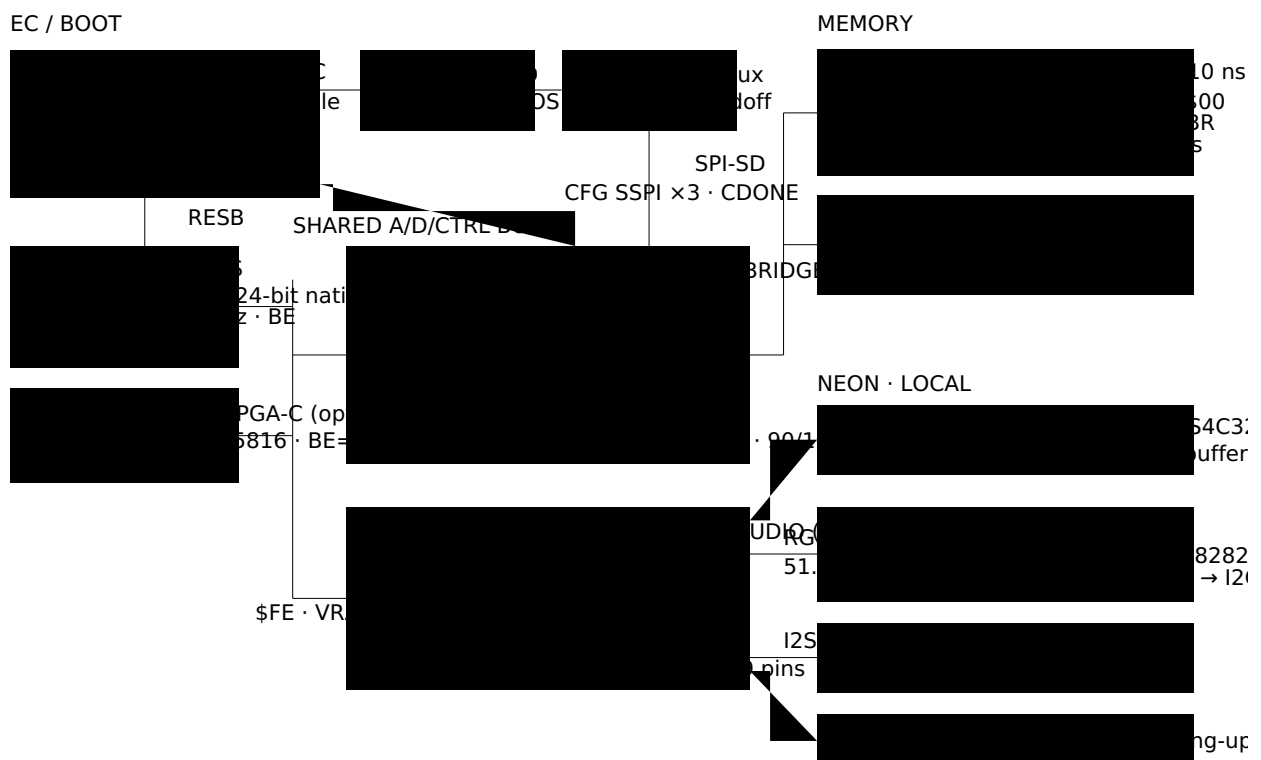


Fig. 1 – Global architecture. Four inhabitants of the shared bus; the RP2354B governs configuration and reset from outside.

Trace legend: **mint** = data · **gold** = power / EC control · dashed = optional or future.

Every peripheral has a clear physical owner and a uniform interface toward software: a `/dev/*` node served by a kernel driver.

PERIPHERAL	CHIP / INTERFACE	HANGS OFF	ACCESS FROM THE OS
10.1" 1024×600 panel	ER-TFT101-1 · HX8282 · RGB-TTL FPC-50 · 18 bpp · 51.5625 MHz → 59.95 Hz (T.5). The 60 Hz is physics, not preference: TFT pixels are capacitors that need refreshing, and polarity inversion becomes visible flicker as the rate drops – 30 Hz is out of spec and ~50 Hz buys only ~17% for a worse image	Neon (FPGA-B)	<code>/dev/fb</code> – VRAM mapping via <code>ioctl</code> , for asset upload, not per-pixel drawing (J.5)
Screen console	Mode 0 · 128 × 32 cells of 8 × 16 px, text buffer and font in Neon's block RAM, both initialised from the bitstream – it displays before any software exists (T.53)	Neon (FPGA-B)	<code>/dev/con</code> – the kernel's own console, distinct from the serial <code>/dev/tty</code> below (→ Q47)
Video modes	0 text · 1 hires mono 1024×600 at 1 bpp, the GUI mode · 2a 320×200 and 2b 512×300 at 8 bpp with a 256-entry palette, both replicated to the panel. Px-doubling is Mode 2b, not a property of the machine (T.33)	Neon (FPGA-B)	<code>ioctl</code> on <code>/dev/fb</code> ; drawing goes through the command channel, never through the CPU
Drawing engine	Blitter – fill, copy, keyed copy, tiled copy, 1-bpp expand, lines – plus a command processor executing display lists out of Neon's SDRAM with no CPU involvement (sheet T)	Neon (FPGA-B)	commands written to <code>CMD_PORT</code> , or a list the kernel builds once and patches
VGA (bring-up)	R-2R ladders hung off the top bits of the panel's own RGB bus through isolating jumpers – so it costs only 2 dedicated sync pins · 640×480 / 800×600. Its pixel clock does not fall out of the video domain and wants Neon's free second PLL (→ Q54)	Neon (FPGA-B)	same framebuffer, same timing generator with different parameters

PERIPHERAL	CHIP / INTERFACE	HANGS OFF	ACCESS FROM THE OS
Capacitive touch	GT911 · I2C + INT (module's capacitive variant)	I2C-SW , the switched-domain bus – its pull-ups would back-feed a dead rail on the always-on one (C.13)	<code>/dev/touch</code>
Audio	Fully specified in sheet W – four channels, a hardware mixer, and a sample rate of 44,146 Hz because no PLL setting on this board reaches 44,100 (W.11). PCM5102A · I2S · DMA buffers in Neon's SDRAM. Strapped <code>FLT / DEMP / FMT</code> low, <code>SCK</code> to GND for the part's internal PLL, and <code>XSMT</code> to 3V3 through an RC – that delay is the anti-pop, and without it every power-up is a click in the headphones	Neon (FPGA-B)	<code>/dev/audio</code>
Keyboard / mouse	USB HID · native host controller + on-board hub IC · switched VBUS per port (D.10)	RP2354B → EC channel	<code>/dev/kbd</code> · <code>/dev/mouse</code>
Serial console	UART ↔ the EC's <code>J-DBG</code> UART (D.4)	Helium ↔ RP2354B	<code>/dev/tty</code>
Storage	microSD · SPI · '3257 mux · a register block in bank <code>\$FF</code> with a FIFO path and a DMA path, the latter depositing blocks straight into SDRAM (G.6, D45)	Helium (FPGA-A)	<code>/dev/sd</code> → FS
Real-time clock	PCF8563 · I2C	Unassigned since the bus split ; it wants to keep time while the machine is off. <code>I2C-EXP</code> is the third answer, with a backup cell doing what no bus can (H.5, X.22, Q35)	<code>/dev/rtc</code>
Battery and power	MAX17048 + BQ25896 · I2C-A0N, EC-owned	RP2354B → telemetry snapshot in Helium → bank <code>\$FF</code> (sheet S)	<code>/dev/power</code> – latch, then read; <code>ioctl</code> for poweroff and reboot
Power button	Momentary, wired to both <code>Q0N</code> and an A0N GPIO	RP2354B (duration) and BQ25896 (hardware floor)	Arrives as an interrupt and a <code>/dev/power</code> state, not as a key event

PERIPHERAL	CHIP / INTERFACE	HANGS OFF	ACCESS FROM THE OS
Timer + interrupts	100 Hz timer · prioritized PIC · IRQ/NMI/ABORT	Helium (FPGA-A)	scheduler tick
VSync, raster and SIGNAL	One IRQ line, three sources, demultiplexed by the kernel reading <code>IRQ_STATUS</code> (T.47)	Neon → PIC	GUI sync, raster effects, and the display list telling the CPU a frame is safe to patch
Backlight	PT4110 boost · <code>EN</code> and PWM. Owned by the EC, not Neon – the only party awake before Neon is configured and while the switched domain comes down (T.52)	RP2354B GPIO (D.6)	brightness through <code>/dev/power</code> , not <code>/dev/fb</code>
Expansion slot	Shared bus: a 256-byte register window in <code>\$FF</code> , a 1 MB aperture the MMU can map, and the three nets of X.2. Unpopulated in build 1 – three Helium pins and a connector (sheet X)	Helium (FPGA-A)	whatever the card is; the slot itself has no node
Slow bus – I2C-EXP	Third I2C bus, Helium its only master, devices mirrored into I/O windows by a sequencer – a byte is ~90 μs and nothing may stall PHI2 for that (X.23). Two pins already budgeted	Helium (FPGA-A)	per device – the PCF8563 above is its first candidate tenant

The rules the table encodes – one owner, one node, and a bus chosen by domain rather than by convenience.

H.1 One owner per peripheral, and the third column says which. Every row names exactly one device that drives the part – Neon, Helium or the RP2354B – and nothing on this board answers to two masters at once. Where two parties need the same device it is **handed over, not shared**: the microSD passes from the RP2354B to Helium through the '3257 mux, one owner at a time.

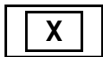
H.2 One face toward software. Everything software touches is a `/dev/*` node behind J.4's five-function driver interface, and the table's two exceptions are deliberate. The power button arrives as an interrupt and a `/dev/power` state, not a key event, because it must work with no process listening; and the slot has no node of its own, because what it is depends on the card (sheet X). NOTE: `/dev/con` is the panel, `/dev/tty` the serial line (→ Q47).

H.3 Which I2C bus a device sits on is a power-domain decision, not a convenience one (C.13). `I2C-AON` carries the charger and the gauge and nothing else, because anything on it must be safe to leave powered whenever SYS exists. `I2C-SW` carries the touch controller, whose pull-ups would back-feed a dead rail from the always-on side. `I2C-EXP` is Helium's own, and its devices are mirrored into windows rather than reached transactionally (X.22).

H.4 The three owners divide by deadline. Neon takes what has a hard per-frame deadline – pixels, the display list, the audio DMA – the only device with local memory and nothing in front of it. Helium takes what lives in the memory path – storage, the timer, the PIC – a request answered inside a bus cycle cannot cross a link first. The EC takes what must be awake when the machine is not: power, the backlight, USB HID long before a kernel exists. Nothing in the table hangs off Argon: an unpopulated device owns no peripheral.

H.5 Two rows are still open, and both are placement questions, not design ones. The **RTC** has no bus: it wants to keep time while the machine is off, which argues the always-on side, against a bus reserved to charger and gauge – and X.22 now offers a third answer (→ Q35). The **VGA output** has no clock: its pixel rate does not fall out of the video domain, and it is now the only claimant on Neon's free second PLL, W.11 having removed audio's (→ Q54). Neither is on the path to a milestone, which is why both have stayed open.

H.6 The table is the whole of the board's I/O. A device that is not in it is not fitted, and adding one is not a soldering job: it needs an owner, a bus that suits its power domain, a node and a driver – or, if it is slow and trivial, the tier of X.21. The two deliberate blanks are in the table: the slot, unpopulated in the first build, and an **I2C-EXP** with no tenant assigned yet.



The expansion bus

A SLOT THAT COSTS THREE PINS · A SLOW BUS THAT COSTS NONE · WHAT A CARD MAY NEVER ASSUME

SHEET 04

The machine grows in two directions and they are not the same problem. A **parallel slot** gives a card the machine's own bus – addresses, data, the clock and a validated window – and it is specified here as a second instance of a contract that already exists rather than as a new bus. A **slow bus** carries everything that never needed the bus at all, as a set of I2C devices mirrored into I/O windows. Neither is populated in the first build; both constrain the schematic, and one of them constrains a pin budget that is already short.

X.1 The currency is pins on Helium, not pins on a connector, and that single fact shapes every decision below. Q8 is still blocking: seventeen free pins that already fail to cover SPI-SD, the UART console and the debug port. A proposal that spends ten is not a proposal. So the target this sheet sets itself is **three pins for the first slot, one for each slot after it, and zero for the slow bus** – the last because its two pins have been sitting in the budget since before the I2C bus split gave them nothing to do.

D.6 closes with a prediction – *"both Helium and the EC ran out of pins, and both were saved by moving slow signals onto a cheaper transport. Expect it a third time."* This sheet is the third time, and it is the reason the two tiers are specified together rather than in separate sheets: the slow bus exists so that the parallel slot never has to justify itself for a temperature sensor.

The three pins are spent at **layout**, not at population. An unpopulated connector still needs its nets routed to Helium, which is why X.30 puts this decision before the board rather than before the first card.

The slot – one more instance of the contract

X.2 The slot is not a new bus. It is B.6's three-net contract with a second tenant, and the naming of those nets already anticipated it: Q56 chose `NEON_BUS_SEL` over `VRAM_SEL` precisely because the net means *"Helium has validated this cycle for you"* and nothing narrower. A card gets the same three nets, the same stall protocol, the same watchdog and the same "Helium decodes the window, the tenant decodes inside it" split. **A slot that respects this table is a second Neon; one that does not is a second bus, and the board cannot afford a second bus.**

It also settles what a second slot is – another tenant of the same contract, never a second master: only the card whose `/EXP_SEL_n` is asserted may drive anything.

NET	DIRECTION	ASSERTED MEANS	IDLE / PULLED STATE
<code>/EXP_SEL_n</code>	Helium → slot <i>n</i>	"This cycle is yours" – after the bank decode and the privilege check for a register cycle, after translation and the permission check for an aperture cycle (X.7, X.8)	Deasserted. Push-pull, one driver, pull-up for the unconfigured case (B.8)

NET	DIRECTION	ASSERTED MEANS	IDLE / PULLED STATE
/EXP_BSY	slot → Helium, common , open- drain	"I cannot serve this cycle yet." Driven only by the card whose / EXP_SEL_n is asserted; released before any other slot can be selected	Not busy. Pull-up to 3V3_MAIN – the opposite of NEON_BUS_BSY , and X. 11 is why

NET	DIRECTION	ASSERTED MEANS	IDLE / PULLED STATE
<code>/EXP_IRQ</code>	slot → Helium's PIC, common , open- drain	"Somebody in a slot wants attention", demultiplexed by reading each slot's <code>SLOT_STATUS</code> (X.13)	Deasserted

- X.3 What it costs, counted honestly.** `/EXP_SEL_n` is one pin per slot; `/EXP_BSY` and `/EXP_IRQ` are one pin each however many slots there are. Everything else on the connector – A0–A23, D0–D7, `RWB`, `PHI2`, the 25 MHz reference, `RESB`, 3V3 and ground – is a net that already exists on the board and reaches the connector through discrete logic, at **no cost in FPGA pins at all**. One slot is three pins, which is exactly what Neon costs; two slots are four. **How many slots the board carries is therefore the only variable in this sheet that fights Q8**, and it is left open deliberately (→ Q81).
- A second slot is the cheapest thing on this list, and the 25 MHz reference is the only part of it that is not free: B.7's star already says a fan-out buffer is the answer to a second destination rather than a fifth branch hung on the same net (X.15).
- X.4 The board demultiplexes the bank byte, and no card ever learns that the 65816 multiplexes it.** F.3 latches the bank inside Helium and T.51 does the same inside Neon, both of them saving eight pins by it – but making a third-party card capture D0–D7 during the low phase of `PHI2` is making the author of a serial port understand the CPU's half-cycle pipeline. **One '573 on the motherboard, latching D0–D7 on the rising edge of PHI2**, and the connector presents A0–A23 static for the whole cycle. One package, zero FPGA pins.
- It also pays for itself a second time, in X.7 and X.8: a card with the latched bank in front of it can tell a register cycle from an aperture cycle by itself – bank `$FF` or not – so the two address spaces need no signalling pin to separate them.
- The '573 is also the one part of this sheet that has to exist before any card does, and it is not free of consequence: it latches from the same D0–D7 the CPU and both FPGAs already load (F.4), so it is one more load on the most layout-critical net on the board. That is the argument X.5 then makes for buffering everything else.
- X.5 The slot is buffered, and this is not optional.** F.4 calls D0–D7 and A0–A10 the most layout-critical nets on the board, with three loads already on them and an SRAM that must deliver inside a ~62 ns high phase. Hanging a connector, its stub and an unknown card off those nets is the cheapest way to lose that margin. So: a '245 on the data lines, direction from `RWB`, **enabled only while some `/EXP_SEL_n` is asserted**; a '244 on the address lines the '573 does not already buffer. The slot is electrically absent for the all traffic that is not addressed to it.
- One rule inherited from T.18 governs the transceiver and it is the one that will be got wrong: **nothing may drive the data bus toward the CPU while `PHI2` is low**, because the bank byte is travelling on it. The '245's output enable is qualified by `PHI2` high, not by `/EXP_SEL_n` alone.
- X.6 A card is reachable in two spaces, and the split is the same one T.21 draws for Neon:** control in bank `$FF`, where the MMU can never take a process, and bulk through a window the MMU *can* map with permissions in front of it. Neither is a novelty; both are a comparator widened.

X.7 Space one: a 256-byte register window per slot in bank \$FF. This is the block Helium deliberately does not decode: it recognises the window, asserts `/EXP_SEL_n`, releases D0–D7 on a read and performs the privilege check – and **the card decodes A0–A7 inside it**, exactly as Neon owns everything above `$FF:8000`. It inherits the keystone of L.10 for free: bank `$FF` is unmappable to user space, so **no card in any slot is reachable from user code by construction**, whatever the card is and whoever wrote it. The proposal, on Q22's terms of one 256-byte wall per block, is `$FF:7E00` for slot 0 and `$FF:7F00` for slot 1 – the top of Helium's half, against Neon at the bottom of the other one (→ Q82).

In a register cycle Helium performs no translation (F.3), so **A11–A19 carry nothing** and a card must not decode them. The registers are byte-wide and byte-addressable, unlike M.4's block: a card is an eight-bit device on an eight-bit bus and there is no reason to make a serial port pretend otherwise.

X.8 Space two: a 1 MB aperture per slot, mapped by the MMU, at zero pin cost – and this is the part that is not obvious. F.10 sets the ceiling at 128 MB and populates half of it, so **the upper half of the frame-number space decodes to nothing**. Give a range of it to the slots: a PTE naming a frame there is a legal mapping that reaches a card instead of SDRAM, with the TLB, the permission bits and `ABORTB` still in front of it. And the address gets there on wires that are already driven: during a slot cycle the SRAM's `/CE` is idle, so **A11–A19 – which Helium drives anyway (F.2) – carry the offset within the aperture**, and A0–A10 come from the CPU as they always do. Twenty bits, 1 MB per slot, not one new pin.

The proposal is frames `$F000–$F1FF` for slot 0 and `$F200–$F3FF` for slot 1 – 512 frames of 2 KB each. The range is arbitrary and the size is not: it is what A0–A19 reaches, and widening it means new pins (→ Q83).

What this buys is the thing a register window cannot: **a card's memory can be mapped into a process with protection**. A frame buffer, a packet ring, a sample buffer – reachable at bus speed by user code that still cannot escape its own address space, and still cannot reach the card's control registers, because those are in `$FF`. It is J.5's framebuffer mapping generalised to devices nobody has designed yet.

X.9 Aperture frames are non-cacheable, always, and the gateway must enforce it rather than trust the flag. The `NC` bit exists (M.9) and the kernel must set it – but a cached aperture is not a performance question, it is a correctness one: a cache line is 2 KB of a device's registers held in SRAM and written back whenever the cache feels like it. Helium must treat a frame number in the expansion range as `NC` regardless of what the PTE says, and a fill must never be started against one.

X.10 The read protocol is T.18's, unchanged: Helium stalls first and asks afterwards. Helium knows a slot cycle is in progress at the same instant the card does, since it decoded it. So it asserts `/EXP_SEL_n`, gates `PHI2` low, and only *then* samples `/EXP_BSY`. A card that answers inside the high phase – a register file, a FIFO – never asserts it and costs nothing; a card that needs longer holds it and the CPU is frozen, not misinformed. As with Neon, **the clock is gated and RDY is not touched**, for D16's reason.

Sampling starts after a mandatory minimum window of two core clocks rather than immediately, and that window is the whole of what makes X.11 safe.

- X.11** `/EXP_BSY` idles *not busy*, which inverts Neon's polarity, and the inversion is the point. B.6 makes `NEON_BUS_BSY` busy-by-default so that an unfitted Neon stalls the machine and the watchdog reports it – correct for a soldered device that is either there or is a bring-up stage. **An empty slot is neither:** busy-by-default in a slot means a machine with nothing plugged in does not boot. The race that argument was defending against – Helium sampling "not busy" before the card has noticed the cycle – is killed by the mandatory stall window of X.10 instead of by the pull-up, which is a better place for it: the guarantee becomes a property of the protocol rather than of a resistor. The safety property is not lost, only moved. **An empty slot answers with the pull-ups and an absent card is indistinguishable from a broken one – until enumeration**, where `SLOT_ID` reading `$00` says "nothing there" in one register access (X.14).
- X.12** **The fill watchdog gains a third client.** F.11's counter already resumes PHI2 and raises NMI or `ABORT` for an overrunning cache fill, and T.18 gave it a second trigger in a Neon that never answers. A card that asserts `/EXP_BSY` and dies is exactly the same failure with a third source: **resume the clock, return `$00`, raise the fault**, and give it its own `FAULT_CAUSE` code so the kernel can say *which slot* rather than *something timed out*. The timeout value is Q57's and is still unassigned; this makes it one number serving three clients, which is an argument for settling it rather than against. Three clients, one escalation path – an overrunning fill, a silent Neon, a card holding `/EXP_BSY`. A hung machine reports the same way whatever hung it.
- X.13** **One interrupt line for every slot, demultiplexed by a mandatory register – because that is what Neon already does.** A line per slot into the PIC gives exact vectoring and costs a pin per slot, which X.3 will not pay. T.47 aggregates three sources inside Neon and has the kernel read `IRQ_STATUS`; the slot does the same across devices instead of within one. What makes it cheap rather than ISA-shaped is that the demultiplexing offset is **normative, not negotiated**: bit 7 of `SLOT_STATUS` at a fixed offset in every card's window, so an ISR costs one read per populated slot and the answer is deterministic. `/EXP_IRQ` is **level-sensitive and open-drain**, so two cards asserting simultaneously is an ordinary wired-OR rather than a lost edge, and the handler must loop until no slot claims it.
- X.14** **Enumeration costs no pins either.** M.4 already rules that an address nothing answers reads `$00`, so an empty slot identifies itself: `SLOT_ID` reading `$00` means the slot is empty, and no card-present pin is needed. It is T.54's `NEON_ID == $4E` generalised – the BIOS reads it once, expects a card or does not, and continues either way. A card learns **which slot it is in** from two connector pins strapped differently in each position, so the same board works in either slot with no jumper; those straps are board traces to 3V3 and ground and cost nothing anywhere.
- X.15** **PHI2 goes to the connector, and the first thing a card author must be told is that it is not a clock.** Helium gates it – for cache fills (F.4), for Neon reads (T.18), and now for the slot itself. A card that derives a baud rate, a sample rate or a timeout from PHI2 will drift by an amount that depends on what the rest of the machine is doing, which is the least debuggable class of fault this board can produce. **PHI2 is a bus qualifier; the 25 MHz reference is the time base.** That reference is a branch of B.7's single-oscillator star with its own 33 Ω in series, and a second slot means a fan-out buffer rather than a fifth branch on the same net.

- X.16 A card is never a bus master in this revision, and the reason is coherence rather than arbitration.** M.12 states the rule that governs any master writing memory behind the cache: invalidate before a read into memory, flush before a write out of it, and each mistake is a silent corruption that only appears under memory pressure. Helium's own SD engine can honour it because Helium owns the cache; **a third-party card cannot honour it and cannot be made to.** DMA therefore stays a privilege of the northbridge. `/EXP_BR` and `/EXP_BG` are routed to the connector and left unconnected at the FPGA, in the same spirit as F.10's second SDRAM `/CS`: the board does not foreclose the decision, and this revision does not take it.
- X.17 Reset, insertion and the blank-bitstream case.** `RESB` reaches the connector buffered, so a card comes up with the CPU. **There is no hot-plug:** cards are inserted with the machine off, which deletes sequencing, inrush and connector-order questions that would otherwise each need an answer. And a card must be safe with a blank Helium – every slot control net is pulled to its idle state by B.8's rule, so an unconfigured board selects nothing and a card that drives the bus only when selected drives nothing.
- X.18 A card is testable before a kernel exists, and this falls out of M.10 at no cost.** The debug agent's internal accesses land on the same address comparators a CPU store does, so the RP2354B can read a card's `SLOT_ID`, poke its registers and watch its interrupt with the 65816 held in reset and no software anywhere. For a subsystem whose whole purpose is to host hardware this document did not design, that is worth more than it is for any of the blocks that already have it.
So a card is brought up with the 65816 socket still empty, exactly as Neon is.
- X.19 Connector: a 2 × 26 header on 2.54 mm pitch,** which is the project's hand-solderable rule applied to a part that also has to survive being a mechanical interface. Signal count below. Ground is alternated along one row rather than gathered at the ends: at 8 MHz the risk is not the frequency, it is the return path under a 25 MHz clock edge and a bus turnaround.

GROUP	PINS	NOTES
A0–A23	24	A16–A23 from the board's '573 (X.4); A11–A19 meaningful only in an aperture cycle
D0–D7	8	Through the '245, never driven toward the CPU while PHI2 is low
RWB · RESB	2	Buffered
PHI2 · CLK25	2	A qualifier and a time base – not interchangeable (X.15)
/EXP_SEL_n · /EXP_BSY · /EXP_IRQ	3	The contract of X.2
SDA · SCL	2	The slow bus, so that a card can be a slow device without touching any of the above (X.27)
SLOTID0 · SLOTID1	2	Geographic strapping, board traces (X.14)
/EXP_BR · /EXP_BG	2	Routed, not connected at the FPGA (X.16)

GROUP	PINS	NOTES
3V3 · GND	3 · 4	

X.20 The card contract, in one place, because a slot is a promise made to people who will not read the rest of this document. A card may assume: static A0–A23, RWB and a full PHI2 high phase to answer in; that it is selected only when the cycle is genuinely its own; that it will never be selected by an unconfigured board. A card must: drive D0–D7 only while selected, on a read, during PHI2 high; release /EXP_BSY before deasserting; implement SLOT_ID, SLOT_CLASS and SLOT_STATUS at the offsets below; hold /EXP_IRQ no longer than its own status bit. A card must never: master the bus; use PHI2 as a time base; decode A11–A19 in a register cycle; decode bank \$FF in an aperture cycle; hold /EXP_BSY longer than Q57's window.

Most of that list is enforced by the board rather than trusted to the card: the select line, the '245's direction and the watchdog do it. Only the decode rules depend on the card, and getting them wrong misbehaves inside its own window.

Register map – the slot window. Offsets from \$FF:7E00 (slot 0) and \$FF:7F00 (slot 1), still to be ratified (→ Q82). Byte-wide.

OFFSET	NAME	ACCESS	MEANING
+\$00 – +\$EF	–	–	The card's own. Helium decodes nothing here and reads \$00 when no card answers
+\$F0	SLOT_STATUS	read	Bit 7 IRQ_PENDING – normative, this is what the ISR demultiplexes on (X.13) · 6:0 the card's
+\$F2	SLOT_CTRL	write	Bit 0 IRQ enable · 1 clear IRQ_PENDING · 7:2 the card's
+\$FC	SLOT_CLASS	read	What kind of device this is. Class numbering unallocated (→ Q84)
+\$FE	SLOT_ID	read	Non-zero identifies the card; \$00 means the slot is empty (X.14)

The slow bus – I2C devices as I/O windows

- X.21 The second tier exists so the first one is never asked to carry a temperature sensor.** An RTC, an expander, a sensor, an EEPROM: none of them needs an address bus, a stall protocol or a megabyte of aperture, and putting them behind a parallel slot means a connector and a buffer per trivial device. D.6 already solved this problem twice on this board – the keyboard matrix onto USB HID, the rail enables onto an I2C expander – and both times the pattern was *move the slow signals onto a cheaper transport*.
The tier has a floor: a device with a deadline does not belong on it. A byte is $\sim 90\ \mu\text{s}$ (X.23), so anything that must answer inside a bus cycle or a frame belongs on the slot or inside Helium – this tier is for state that changes at human rates.
- X.22 It is a third I2C bus, Helium's own, and it must not be either of the existing two.** C.13's rule R3 split I2C in two for a reason that still holds: **I2C-AON** belongs to the charger and the gauge, and every device added to it costs standby current for the life of a charge; **I2C-SW** is the EC's, and E8.1 has the EC reading the GT911 on it. Making Helium a master on either means genuine multi-master arbitration in gateway, against B.2's rule that Helium is a pure state machine – and it would collide head-on with Q4's proposal that the HID path run with the EC as master and Helium as *slave*. So: **I2C-EXP, two pins, Helium the only master, pulls to 3V3_MAIN under rule R2, in the switched domain because Helium is.** The pins are not new – Q8 has been carrying "I2C 2" in Helium's allocation since before the split left them without a device. **This gives Q35 an answer it did not have.** The PCF8563 was left unassigned because it wants to keep time while the machine is off, which argues the always-on bus, against a bus reserved to the charger and the gauge. On **I2C-EXP** with its own backup cell it keeps time with no bus at all – the crystal does not care who is powered – and the kernel then reads the clock directly instead of asking the EC for it. It becomes window 0 and the first tenant of this tier.
- X.23 A window is a mirror, not a proxy, and the arithmetic is what forces that.** The tempting design has a CPU read of a window perform the I2C transaction – but a byte over I2C is nine bit-times plus addressing: $\sim 90\ \mu\text{s}$ at 100 kHz, $\sim 23\ \mu\text{s}$ at 400 kHz, against a 125 ns bus cycle. Serving it by stalling PHI2 violates M.6 by three orders of magnitude, and there is no version of "the kernel polls instead" that survives being spelled `lda $FF7C04`. So the window **never touches the wire**: it is a copy in Helium's block RAM, refreshed by a sequencer, and a CPU read of it is a register read at full bus speed like any other.
- X.24 The refresh mechanism already exists: it is S.16's, with a device table in front of it.** Sheet S solved exactly this problem for one device – a slow instrument, an asynchronous updater, an eight-bit CPU that must not see a multi-byte value tear – and solved it with **staging → live → shadow**: the slow side writes staging in any order, a commit copies it to live in one cycle, the CPU latches live into a shadow bank and reads the shadow as slowly as it likes. Here the slow side is the I2C sequencer rather than the EC, the device is whatever the descriptor names, and everything else is the same machinery. **The atomicity and staleness properties come along unchanged**, which is the whole argument for this shape over a transaction register: one mechanism, already specified, already justified, now serving n devices instead of one. Per-window refresh period rather than one global rate – an RTC at 1 Hz and a sensor at 10 Hz have nothing to say to each other, and S.20 already makes the point that cadence costs power on the slow side.

- X.25 Writes are queued, never immediate, and the completion story is the one the rest of the document uses.** A CPU write into a window lands in the shadow and sets a dirty bit; the sequencer carries it to the device on its next pass. A `WIN_FLUSH` trigger forces the pass for the case where ordering matters. Completion is a status bit and, optionally, the same aggregated interrupt the slots use – because a driver that has to wait 90 μ s for an acknowledgement should be blocked on an interrupt, exactly as G.10 argues for the SD engine. Ordering *between* windows is not defined: the sequencer visits descriptors on its own schedule, so writes to two devices land in whatever order the pass reaches them. Where order matters the driver serialises with `WIN_FLUSH`, never with timing.
- X.26 A device that stops answering must read as a fault, not as a plausible zero –** S.19's rule, and it is the one thing about this tier that is not negotiable. Every window carries a `VALID` bit, an age, and a NAK/bus-error flag; a window whose device has gone silent stops updating and says so, rather than mirroring the last good value forever. A driver that ignores the flag reads a battery frozen at whatever it was when the wire fell off. The rule has a kernel-side half: a driver reads `VALID` and the age *before* the data, and a window in fault is reported upward, not smoothed over. S.19 states it for the battery, where a frozen but plausible value is the most dangerous reading there is. A `VALID` bit costs a flip-flop; not having one costs a wrong battery.
- X.27 SDA and SCL are also on the slot connector, and that makes the cheapest possible card.** A board that only wants an ID, a couple of GPIOs or a sensor implements no bus protocol at all: it is an I2C device, it gets a window, and it never asserts `/EXP_BSY` because it never sees a cycle. It is also the natural home for a per-card identity EEPROM, if X.14's `SLOT_ID` ever proves too thin. NOTE: Such a card needs no buffer, no '245, no select line – two wires.
- X.28 What this tier deliberately does not specify yet,** and the honest list is short because the mechanism is borrowed rather than invented: how many windows, how large each is, the descriptor encoding, the sequencer's scheduling policy when several windows come due at once, and whether the raw-transaction escape hatch of X.29 is enough to initialise every device the tier is likely to host. The register map below is a proposal at that resolution and nothing more ($\rightarrow$ Q85). None of those five gaps blocks the only decision this sheet must settle before layout – the pins (X.30). They block the first driver, and the first driver does not exist until a device does.
- X.29 Device initialisation stays in software, behind a raw-transaction register,** for G.11's reason applied one tier down: Helium's gateway does the shifting and the sequencing, and nothing above that. A device needing a setup sequence – a mode register, a resolution, an alert threshold – gets it through `I2C_RAW_*` from the kernel driver, and only then is its window enabled and mirrored. This is B.2's rule about where intelligence lives, applied to the tier most tempting to violate it for. The escape hatch doubles as the audit: anything a driver can do *only* through `I2C_RAW_*` is a candidate for a descriptor field that does not exist yet, and the list of those cases is the evidence X.28 is waiting for.

Register map – the slow-bus block. **Proposal, not ratified** (`open` $\rightarrow$ Q85). One 256-byte wall, base unassigned (Q22). Byte-wide.

OFFSET	NAME	ACCESS	MEANING
+\$00	I2C_CTRL	write	Bit 0 sequencer enable · 1 bus speed (100/400 kHz) · 2 bus reset · 3 IRQ enable
+\$01	I2C_STATUS	read	Bit 0 sequencer running · 1 raw transaction busy · 2 any window in fault · 3 IRQ pending
+\$02	I2C_SEQ	read	Bumped on every completed refresh pass – the staleness reference of S.16
+\$03	I2C_LATCH	write	Copy live → shadow in one cycle. Read the shadow afterwards, tear-free
+\$08 – +\$0F	I2C_RAW_*	read/ write	Device address, register, length, data and trigger for an unmirrored transaction (X.29)
+\$10 +	WINn_DEV · _BASE · _LEN · _PERIOD · _STAT	read/ write	One descriptor per window: 7-bit device address and flags, first device register mirrored, byte count, refresh period, and VALID /age/NAK
+\$80 +	Window shadow data	read/ write	The mirrors themselves. A read is a register read; a write sets the dirty bit of X.25

X.30 None of this is on the critical path, and saying so is part of the specification. The slot has no tenant, the slow bus has one candidate, and neither is required by any milestone: E6 boots without them and E8 is a laptop without them. What they require *now* is schematic space and three or four pins reserved before layout, because both are cheap to route and impossible to add afterwards – and that is exactly the class of decision E1.4's staged population was designed to protect. Where the gateway lands in the E-series is open, and it should land late (→ Q86).

The dates say the same thing. The slot's cost falls at E1.1, when the schematic sheets are captured; its gateway is not owed until something is plugged in. Reserving routing is cheap, and a respin to add a connector is not.

REV C, and the revision that changes the shape of the sheet rather than its parts. A 1S pack behind a power-path charger still means the machine runs on USB with the battery flat or absent – but the 3.3 V rail is now **split in two**, and that split is what turns the RP2354B from a sequencer into an actual power controller. The registers, the shutdown paths and the battery telemetry that ride on it are sheet S.

C.1 The tree in one line: **USB-C sink → CH224K negotiates 9 V → BQ25896 charger with power-path → SYS → five converters**. Everything hangs off SYS and nothing off the battery terminals, so no rail cares whether the pack is charged, flat or missing.

It also makes the board bring-uppable with no pack at all: C.5's SYS floor runs the machine from the adapter alone, which is what E1.4 needs.

C.2 **The rail split is the whole revision.** REV B had one 3V3, which the EC ran from – and a controller cannot switch the rail that powers it. So 3V3 becomes **3V3_AON**, permanently on and carrying only what must be reachable while the machine is off, and **3V3_MAIN**, switched, carrying everything else. Every switched rail gets an **EN** line from the EC and returns **PG** to it.

The AON rail's tenants are exactly: the RP2354B with its in-package flash, the CH224K, the charger and gauge I2C bus with its pull-ups, and the power button GPIO. Nothing else belongs there, and the selection criterion for its converter is **Iq**, not current – it is energised for the entire life of a charge.

C.3 **Input and PD.** CH224K in standalone resistor mode, **6.8 kΩ from CFG1 to GND = 9 V**; CC1/CC2 terminate at it and the REV A 5.1 kΩ pull-downs come off, since a PD sink and a passive sink are different terminations and fitting both conflicts. 9 V rather than 12 because the BQ25896 trips input OVP at 14 V and 12 V leaves no headroom for hot-plug transients; $9\text{ V} \times 2\text{ A} = 18\text{ W}$ is comfortably over requirement.

Do not connect the CH224K to D+/D–. It can negotiate legacy fast-charge protocols over the data lines, but those lines are the RP2354B's USB device port – console and programming. Forfeiting Quick Charge is the correct trade. Its VDD comes from 3V3_AON, so negotiation is possible before any switched rail exists.

Both USB connectors need ESD protection, and it is the only protection this sheet asks for that is not a rail provision. On the USB-C input, **D+**, **D–** and **VBUS** go through a low-capacitance TVS array – a USBLC6-2 or equivalent – because they are nets a user physically touches and the RP2354B's USB device port sits directly behind them. **The USB-A host port needs its own array for the same reason**, and its polyfuse is not a substitute: the fuse answers a peripheral that draws too much, the array answers a discharge, and the two fail in different directions.

C.4 Cold start is the one sequence nobody controls: the adapter supplies 5 V by default → the power-path raises SYS → the AON converter comes up → the EC boots → the CH224K negotiates → the adapter steps VBUS to 9 V **with the EC already running**. The BQ25896 is built for that input transition; confirming it in the datasheet is Q32.

The order matters in one direction only: it is the *step* to 9 V that has to find the EC awake. An adapter that never negotiates leaves the machine on 5 V – inside the charger's 3.9–14 V window – so it still runs and still charges, only more slowly.

C.5 Charger: BQ25896, replacing the MCP73871. The pack grew to ~10 Ah and linear charging at that current is not viable – it is a switching charger at 90–92 %, 3 A capable and programmed to ~2 A, input window 3.9–14 V, input current limit in 50 mA steps over I2C, and a programmable **SYS floor of 3.5 V** so the system runs with the battery flat or absent. Its integrated ADC (VBUS, BAT, SYS, TS, charge current) gives the EC rail telemetry at no extra part cost.

The **OTG boost is present and deliberately unused**. An earlier proposal had it replace the TPS61023 for the host port; it cannot. The boost drives VBUS, physically the same node as the USB-C input, so a 9 V contract would put 9 V on an attached peripheral – and OTG only enables when VBUS is below the battery, i.e. only when unplugged, so the host port would die whenever the machine was charging.

C.6 Pack: 2× 5000 mAh 1S LiPo in parallel – ~10 Ah, ~37 Wh, under the 100 Wh air-travel line. Two cells rather than one thick one is a mechanical choice: the pack splits across the enclosure. Parallel 1S cells need no balancing, but **must be the same model and production lot, and must be at the same state of charge when first joined** – charge them individually to matching voltage before the first parallel connection. NTC in the same JST-PH harness, into the charger's TS pin, JEITA profile in firmware.

The NTC is not decoration – the charger's TS pin is what applies the JEITA profile, and an absent or open thermistor must read as a fault, not as room temperature.

C.7 Fuel gauge: MAX17048, kept despite the charger's ADC – the ADC reports a voltage, the gauge models a state of charge, and those are different claims. I2C on the AON bus, ALRT to an AON GPIO, **CONFIG.ATHD** set for the low-battery threshold plus the 1 % SOC-change alert, so crossings arrive without polling for them.

The gauge is where S.19's rule earns its keep. Its reading reaches the CPU as a snapshot in Helium (S.16) carrying a **VALID** bit and an age, because a battery frozen at a plausible value is worse than one admitting it does not know.

The iCE40 sequencing constraint – the reason the tree looks the way it does.

C.8 The datasheet order is **VCC and VCCPLL first · then VCC_SPI, any time after · then VPP_2V5, after all three**, each supply reaching 0.5 V before the next is raised, every ramp monotonic – and the same order again whenever partially powered-down rails are brought back up, which is precisely what the EC does on every resume. REV B violated it twice: VCCIO rose before VCC because the 1.2 V LDO hung off 3V3, and VPP_2V5 rose before VCC because the BAT54 did too.

REV B proposed holding **CRESET_B** low as the mitigation. It is not one – it prevents configuration, not parasitic conduction through the I/O clamp structures into an unpowered core rail. The order is fixed properly instead, and the fix is one wire: **the 1.2 V converter sources from SYS, not from 3V3**, which deletes the dependency that made the wrong order unavoidable.

C.9 The order the EC therefore produces is **1V2 (VCC + VCCPLL) → 3V3_MAIN (VCC_SPI + every VCCIO bank) → VPP_2V5**, and the last of those is automatic: VPP_2V5 is a BAT54 from 3V3_MAIN, so it can never lead. The constraint is satisfied by construction rather than by firmware discipline.

The rails – five converters off SYS, one of them unswitchable by design.

RAIL	PART	TOPOLOGY	CONTROL	LOAD
3V3_AON	TPS63900	buck-boost	none – lives whenever SYS lives	RP2354B, flash included · CH224K VDD · charger/gauge I2C · power button
1V2	TLV62568 / TPS62825	buck from SYS	EN ← EC · PG → EC	VCC of both iCE40s · VCCPLL through an RC filter per chip
3V3_MAIN	TPS63020	buck-boost	EN ← EC · PG → EC	VCC_SPI + all VCCIO · SRAM · 2× SDRAM · W65C816S · panel logic · PCM5102A · GT911 · microSD · the VPP_2V5 diodes
backlight	PT4110	boost	EN /PWM ← EC	LED string, ~1.5 W reserved
5 V host	TPS61023	boost	EN ← EC	USB-A VBUS, 500 mA, behind a polyfuse

C.10 Two details in that table carry weight. **EN is pulled down on every switched rail**, to 3V3_MAIN where the net crosses domains, so the machine is off by default at EC reset rather than relying on firmware to keep it off. And the TPS63020 runs **forced PWM** (PS/SYNC tied): ripple matters to SDRAM and to video timing, light-load efficiency does not matter on a rail that is switched off when idle.

C.11 Killing the LD0 was worth more than the sequencing fix alone. Dropping 3.3 V to 1.2 V at 250 mA dissipated **~0.5 W** in a SOT-23-5 – roughly a 100 °C rise – and burned it as heat on a battery-powered machine. The buck recovers ~0.6 W and the thermal problem disappears with it.

C.12 The two boosts stay on **SYS, not VBAT** (D09), so backlight and host port behave identically on adapter and on battery. The backlight comes up **late** in the power-up order and goes down **early** in shutdown, so the user never sees an unconfigured panel and gets immediate feedback that a shutdown command was accepted.

The LED string's voltage and current are still pending the ER-TFT101-1 module datasheet, and with them the PT4110's inductor, Schottky and sense resistor (→ Q1).

Domain crossings – three mandatory rules, and the sheet's most error-prone area.

C.13 Splitting the rail creates a hazard REV B could not have: while the switched domain is down, the EC is alive at 3.3 V with GPIO wired into unpowered silicon. A driven-high output injects current through the target's ESD clamps and back-feeds the dead rail through a path nobody designed. **R1 – tri-state before power-down:** before de-asserting any rail `EN`, the EC places every pin crossing into the switched domain in high-impedance – configuration SPI, `CRESET_B`, the debug agent's SPI including `DBG_CSN`, and the two power handshake lines. It is firmware, but it is a hardware-correctness requirement, not an optimisation. **R2 – pull resistors belong to the destination domain:** every pull on a crossing net ties to `3V3_MAIN`, never to `3V3_AON`, because an AON pull-up back-feeds continuously and defeats R1 whatever firmware does. **R3 – two separate I2C buses**, and sheet X has since made it three.

The third bus belongs to Helium and changes nothing about these two. `I2C-EXP` (X.22) is the slow expansion tier, it lives wholly in the switched domain, its pulls tie to `3V3_MAIN` under R2 like any other, and it crosses no domain boundary at all – so it adds no row to the crossings audit. What R3 forbids is a bus with two masters, not a third bus: the EC keeps `I2C-AON` and `I2C-SW`, Helium gets its own, and no arbitration logic is written anywhere.

BUS	DOMAIN	DEVICES	WHY THERE
I2C-AON	<code>3V3_AON</code>	BQ25896 · MAX17048	Must be reachable while the machine is off
I2C-SW	<code>3V3_MAIN</code>	GT911 touch	Its pull-ups would back-feed it whenever the machine is off
I2C-EXP	<code>3V3_MAIN</code>	Helium's slow expansion tier – RTC candidate, slot connector (X.22)	A different master : Helium, not the EC. Two pins already carried in Q8's allocation

C.14 Sharing one bus would need a bus isolator; a second bus costs two pins on an RP2354B that has peripherals to spare, and is strictly simpler. **The audit that closes this sheet is a net-by-net list of every AON↔switched crossing checked against R1–R3** – not folded into general ERC, a pass of its own, because it is the likeliest source of a board-level bug that is hard to diagnose (→ Q31).

blocking

Sequencing – the EC governs both directions; the CPU-facing half is sheet S.

#	POWER-UP	#	POWER-DOWN
1	Adapter or battery raises SYS through the power path	1	65816 held in reset
2	TPS63900 brings up <code>3V3_AON</code> – no gating, it follows SYS	2	USB-A 5 V boost disabled
3	EC boots; every switched <code>EN</code> is low by its pull-down	3	Backlight disabled – early , so the user sees the command land
4	<code>I2C-AON</code> up; charger configured, gauge read. Below the operating threshold with no adapter, the EC stops here and signals the fault rather than browning out mid-configuration	4	Both FPGAs held in reset via <code>CRESET_B</code>

#	POWER-UP	#	POWER-DOWN
5	1V2 EN → wait for PG	5	R1 tri-state pass over every crossing pin
6	3V3_MAIN EN → wait for PG ; VPP_2V5 follows through the BAT54	6	3V3_MAIN EN de-asserted; VPP_2V5 falls with it
7	CRESET_B released, bitstreams loaded over SPI	7	1V2 EN de-asserted
8	CDONE verified on both devices	8	EC enters dormant mode, or commands ship mode over I2C for a true off
8b	EC reads gauge and charger and commits an initial telemetry snapshot into Helium, so the kernel's first read is not zeros (→ S.16)		
9	I2C-SW up, GT911 initialised		
10	Backlight, then the USB-A 5 V boost		
11	BIOS preloaded into SRAM, 65816 released from reset		

C.15 Power-down step 5 sits between 4 and 6 deliberately: **after** the FPGAs are quiescent, **before** their rails vanish. The datasheet specifies no power-down order, so reverse is taken as the conservative choice, and it costs nothing.

C.16 The button is one switch wired to two destinations: the charger's QON pin, which gives wake-from-ship-mode and a firmware-independent hardware reset intrinsic to the silicon, and an AON GPIO, which gives the EC press-duration discrimination. The charger is the floor of last resort; the EC is the graceful path. Three levels of shutdown fall out of that, in S.6.

Budget and runtime – datasheet typicals, to be replaced by measurement at E1.

3V3_MAIN LOAD	TYP	PEAK	DRAWN FROM SYS	TYP	WORST
W65C816S @ ~14 MHz	15 mA	25 mA	3V3_MAIN (~90 %)	3.0 W	4.1 W
System SRAM	70 mA	90 mA	3V3_AON	0.23 W	0.31 W
2× AS4C32M16SB SDRAM	250 mA	360 mA	1V2 (~88 %)	0.20 W	0.34 W
2× iCE40 HX8K VCCIO	160 mA	220 mA	Backlight	1.5 W	1.8 W
Panel logic (HX8282)	200 mA	250 mA	USB-A host, loaded	0 W	2.8 W
PCM5102A + GT911 + microSD	95 mA	130 mA	Total	~4.9 W	~9.4 W
Misc, LEDs, pulls	30 mA	45 mA	3V3_AON, EC active	60 mA	–

3V3_MAIN LOAD	TYP	PEAK	DRAWN FROM SYS	TYP	WORST
Total 3V3_MAIN	~0.82 A	~1.12 A	3V3_AON, EC dormant	5 mA	–

C.17 Against ~0.82 A typical the TPS63020 supplies about 2 A in this region – roughly 45 % margin. **The SDRAM and VCCIO rows are the two most likely to be wrong**, both being dominated by bus activity, and they are the two that would spend that margin; measuring them at E1 and revising this table is Q33. 1V2 is ~150 mA typical and ~250 mA peak for two HX8K cores, well inside either candidate part.

Two rows are deliberately pessimistic against the rest of this document. The SRAM row was costed on the x16 IS61WV102416, which D15 replaced with a 1 MB x8 part; and the SDRAM row counts **two** devices where D14 leaves the second footprint unpopulated. Both err toward more current than the board will draw, which is the right direction for a converter sizing, but neither figure should be carried into a thermal or runtime claim without being re-derived.

C.18 Runtime: 37 Wh at ~92 % usable ≈ 34 Wh → ~7 h typical, ~3.6 h with the backlight at full and a powered host port. **Standby** with the rails off and the EC dormant is ~17 mW, so the pack self-discharges before it flattens; **ship mode** leaves charger leakage only, and shelf life becomes a property of the chemistry rather than of the board. **Charging** at the programmed 2 A: ~5 h empty-to-full while in use, ~2.5 h with the machine off.

These are bench figures derived from the table above, C.17's two pessimistic rows included. The number that counts is measured inside the closed enclosure under sustained load, which is what L6 exists for.

Ship mode exists because 17 mW is still a draw.

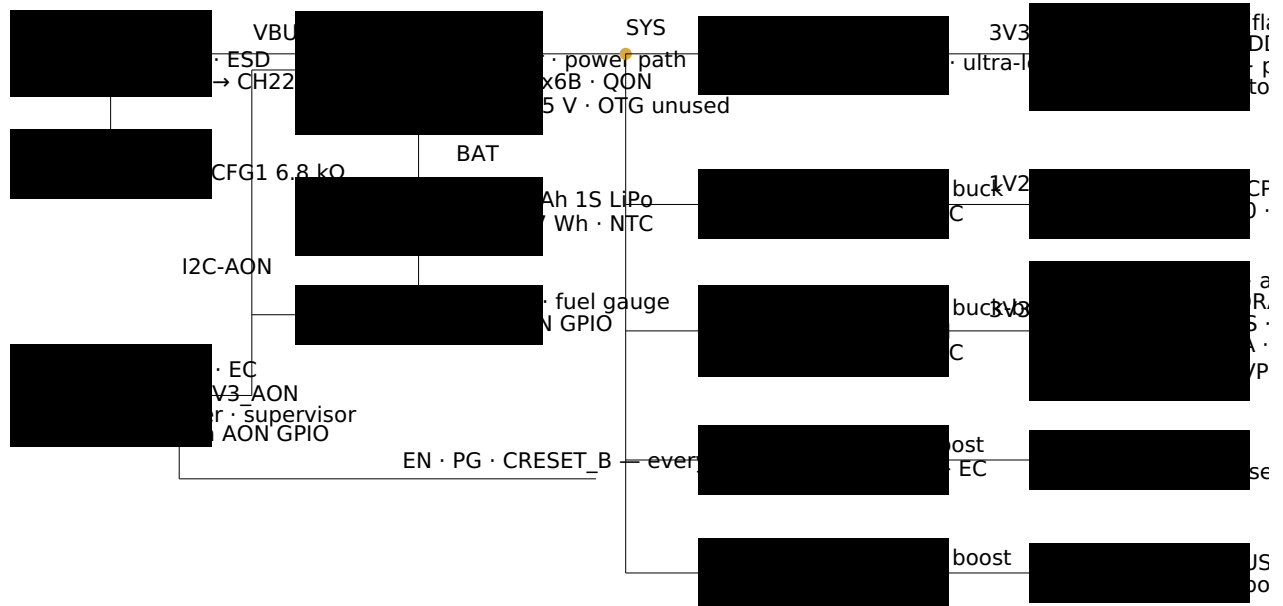
C.19 Bring-up provisions, every rail without exception: a 0 Ω series jumper so the converter can be tested into a dummy load before any downstream silicon sees voltage, a labelled test point on its output side, and enough separation between test points to land two probes without shorting. E0 order is charger and SYS alone → each rail into a resistive load → jumpers fitted one at a time, checking draw against the table above at every step (→ E1.5).

Bring the EN lines out to a header too. Driving a rail by hand, independently of EC firmware, is worth a great deal on a board where the EC is itself one of the things under test.

The jumpers stay fitted after bring-up. They are the isolation E1.4 relies on to attribute a fault to one block, and pulling one is how a suspect rail leaves the machine without anything being unsoldered.

C.20 Net cost of REV C against REV B: one extra IC and one extra inductor – the AON converter – plus the 1.2 V LDO becoming a buck. Bought with it: correct iCE40 sequencing, full EC power control, ~0.6 W recovered and a 0.5 W hot spot deleted. [open](#)

Still open on this sheet, beyond the crossings audit: TPS63900 availability and its actual Iq (→ Q34), CH224K versus its pin-compatible successor CH224A as the K is phased out, and where the PCF8563 RTC now lives – sheet H records it as unassigned and X.22 offers **I2C-EXP** as the third answer (→ Q35).



Order up: 1V2 (VCC + VCCPLL) → 3V3_MAIN (VCC_SPI + all VCCIO) → VPP_2V5 through the BAT54. Down is the reverse. Every switched rail: EN pulled down to 3V3_MAIN, PG back to an AON GPIO, a 0 Ω jumper and a labelled test point. The AON rail has no enable — a controller cannot switch the rail that powers it. Only ship mode removes it, by removing the AON rail.

Fig. 2 – Power tree REV C: USB-C + CH224K → BQ25896 power-path → SYS, and the five converters hanging off it. The AON rail has no enable; every other rail is switched by the EC and reports PG back.

Trace legend: gold = power · mint = EC control and telemetry · dashed = I2C / status.

The hardware half of power management is sheet C; this is the half software can see. Three parties have to agree on it and none of them can currently reach the others: the 65816 talks only to Helium, the EC talks to Helium only over the debug agent's SPI link, and only the EC can actually cut a rail. This sheet is that bridge – the \$FF registers the kernel drives, the SPI registers the EC reads, and the battery telemetry that travels the same path in the opposite direction.

S.1 The requirement, stated plainly because everything else is derived from it: **shutdown must be initiatable both by software on the 65816 and by a physical button, and must be graceful in both cases** – the kernel flushes the card before any rail drops. Neither the CPU nor the kernel can cut power; only the EC can. So a path from CPU to EC has to exist, and it did not. Graceful has a bound, and it is S.7's: the EC waits ten seconds for an acknowledgement and then stops waiting. So the requirement is ordered shutdown *by default*, not unconditionally – a kernel that never answers loses the argument.

S.2 **The pins carry attention, the SPI link carries detail.** Two dedicated wires, both **level-held until the transaction completes, never pulses** – a level survives a missed edge and a pulse does not. `EC_PWR_REQ` (RP2354B → Helium) says the EC wants the system down; `SYS_PWR_REQ` (Helium → RP2354B) says the system has a request outstanding. *Which* request – poweroff, reboot, acknowledge, refusal – is read over SPI. Encoding several meanings in pulse widths on one wire is a bug class nobody should sign up for on a board where both endpoints are also being brought up.

`EC_PWR_REQ` is asynchronous to Helium's core clock and takes a **two-flop synchroniser**; the acknowledgement path takes the equivalent inside RP2354B firmware.

None of this is the floor. A button held long enough reaches the charger's `QON` and resets the BATFET with no firmware anywhere in the path (C.16), which is what makes a hung EC recoverable and why S.6's levels are ordered as they are.

A level also survives a restart at either end: Helium can be *asked* what is outstanding, rather than having had to witness the transition.

S.3 Both pins cross the AON↔switched boundary and are therefore subject to C.13 in full: the EC tri-states `EC_PWR_REQ` before dropping `3V3_MAIN`, and any pull on either net ties to `3V3_MAIN`. **Pin cost is 2 of Helium's spare I/O**, to be allocated together with `DBG_CSN` – which lands on a budget that Q8 already records as not closing (→ Q27). blocking

Those two pins are the only ones this sheet spends. Everything else it needs – the command, the source, the acknowledgement, the telemetry – rides the SPI link that already exists for sheet R.

POWER_CTRL – write-only, one byte, address selects nothing: the value *is* the command.

VALUE	COMMAND	EFFECT
\$01	POWEROFF	Kernel-initiated shutdown. Helium asserts <code>SYS_PWR_REQ</code> , reason = poweroff
\$02	REBOOT	Kernel-initiated restart. Same, reason = reboot

VALUE	COMMAND	EFFECT
\$03	ACK_POWEROFF	Acknowledges a pending EC request – the kernel has flushed and is ready
\$04	ACK_REBOOT	As above, for a reboot
\$05	NAK	The kernel refuses, or cannot service the request now
\$10	CLEAR_PENDING	Clears the pending bit and any latched request; for the kernel to reset state after a NAK
\$20	TELEM_LATCH	Snapshot the telemetry bank for reading. Not privileged (→ S.16)
\$21	CLEAR_TELEM_EVENT	Clear <code>TELEM_EVENT</code> in <code>POWER_STATUS</code>

S.4 This is the one register in the block that departs from sheet M's "address as opcode" model, and deliberately: the power commands are a small closed enumeration with no arguments, so a value decoder is cheaper here than eight more addresses. Every command is **synchronous and completes within the write cycle** – none sets `BUSY`. Writes are **privileged**, faulting through ABORTB exactly as any other bank `$FF` write, with `TELEM_LATCH` as the single documented exception: reading the battery level is a legitimate unprivileged operation and the shadow bank is read-only, so the latch grants nothing. **The gateware must implement that exception on purpose rather than arrive at it by accident.**

The enumeration is closed on purpose: a value the gateware does not recognise is a no-op, never an undefined state.

TEST ▶ a user-mode power command aborts; a user-mode `TELEM_LATCH` completes.

POWER_STATUS – read-only.

BIT	NAME	MEANING
0	REQ_PENDING	An EC-initiated request is outstanding, awaiting acknowledgement
1	REQ_REBOOT	The pending request is a reboot; clear means poweroff
2	SRC_BUTTON	It came from the physical button
3	SRC_BATTERY	It came from a critical battery condition
4	SRC_THERMAL	Reserved for a future thermal trip
5	ACK_SENT	An acknowledgement has been issued; the EC has not yet acted
6	TELEM_EVENT	A discrete power or battery event since last cleared (→ S.20)
7	–	Reserved, reads zero

S.5 Bits 2–4 are mutually exclusive and meaningful only while `REQ_PENDING` is set. They exist so the kernel can behave differently **by cause**: a button press deserves a dialogue and a chance to cancel, a critical-battery request deserves an immediate flush and no prompt. `REQ_PENDING` raises a **level-based** interrupt through Helium's existing aggregation, held until acknowledged, so a kernel that masks interrupts briefly cannot lose the request.

Mutually exclusive means the kernel switches on the cause and never ranks two of them; a source that is none of the three is a gateware bug, not a fourth case.

S.6 Three levels of off, and each exists because the level above it can fail.

The levels are ordered by what each can still assume: level 1 needs a working kernel, level 2 only a working EC, level 3 nothing but the charger.

LEVEL	TRIGGER	PATH	COVERS
1 · Ordered	<code>poweroff</code> from the CPU, or a short press (< 1 s)	Kernel flushes and confirms → EC drops rails in reverse order → ship mode	Normal operation
2 · Forced	Long press, ~4 s	EC drops the rails without asking	A hung kernel
3 · Hardware	Very long press on <code>QON</code>	BQ25896 BATFET reset, no firmware involved	A hung EC

The three durations must be checked against the BQ25896's own `QON` timing so the EC's 4 s does not collide with the charger's hardware reset threshold. **If they overlap, the EC's threshold moves** – the charger's is fixed in silicon (→ Q36).

S.7 The watchdog is mandatory, not defensive polish.

When the EC asserts `EC_PWR_REQ` it starts a **10 s timer**; with no acknowledgement it escalates to level 2 and drops the rails anyway. Without it, a kernel hung during flush leaves a machine that can only be turned off by the 4-second press – a poor experience, and worse, one that hides the fault. The same timer covers reboot. Forced does not mean careless: the R1 tri-state pass still runs on the way down. The timer is the EC's, not Helium's – the party that can act on expiry owns it.

The four sequences – everything the two pins and the register block do.

CASE	FLOW
Button, ordered	EC detects short press → asserts <code>EC_PWR_REQ</code> , starts watchdog → Helium synchronises, latches, sets <code>REQ_PENDING</code> + <code>SRC_BUTTON</code> , raises IRQ → kernel reads <code>POWER_STATUS</code> , notifies processes, flushes and unmounts → writes <code>ACK_POWEROFF</code> → Helium sets <code>ACK_SENT</code> , asserts <code>SYS_PWR_REQ</code> with <code>IS_ACK</code> → EC confirms over SPI, cancels the watchdog, runs the power-down sequence of sheet C
Software, ordered	Kernel flushes and unmounts → writes <code>POWEROFF</code> → Helium asserts <code>SYS_PWR_REQ</code> , <code>IS_ACK</code> clear → EC reads a fresh system request → power-down. No watchdog : the work was done before the signal
Kernel declines	EC asserts and starts the watchdog → kernel writes <code>NAK</code> → Helium asserts <code>SYS_PWR_REQ</code> with <code>IS_NAK</code> → EC de-asserts, cancels the watchdog, takes no further action. A second short press restarts it
Watchdog expiry	EC asserts → <code>REQ_PENDING</code> , IRQ → kernel never writes → 10 s elapse → EC escalates to forced, R1 pass, rails down

S.8 Reboot is poweroff plus one bit.

From Helium's and the kernel's point of view the only difference is `REQ_REBOOT`; after the rails are down the EC re-runs the power-up sequence instead of entering ship or dormant mode. `NAK` is provided deliberately – a kernel midway through something uninterruptible needs a way to decline, and the watchdog bounds how long a refusal can stall things. Whether a *repeated* refusal should escalate is EC firmware policy and is left open (→ Q37).

S.9 On the EC's side a single read-only SPI register, `EC_PWR_STATE : SYS_REQ` mirroring the pin · `REQ_REBOOT` · `IS_ACK` (this is an acknowledgement of the EC's own request, not a fresh one) · `IS_NAK` · `CPU_ACK_SEEN`. **It must be readable regardless of the `DEBUG_ENABLE` strap.** The debug agent's privileged registers are gated by that strap by design (→ R.18), but power management is a production function, not a debug one, and putting it behind the strap would break shutdown on every board built with debug disabled. `EC_PWR_STATE` and the telemetry staging registers therefore live in a small **always-accessible region** of the SPI map alongside `DBG_ID` – read-only, carrying no ability to initiate bus cycles, so nothing the strap exists to withhold is exposed (→ Q38).

S.10 Three gateway requirements, each of them a failure mode rather than a preference. **The request latch is reset-safe:** a Helium reset with a request pending must clear `REQ_PENDING`, not leave a phantom that re-interrupts the kernel the instant it boots. **`SYS_PWR_REQ` survives a CPU reset:** if the kernel acknowledges and the CPU then resets before the EC services the pin, the shutdown must still complete. And **none of this may depend on the cache controller, the arbiter or SDRAM** – it is pure register logic, so that shutdown still works when the memory subsystem is wedged, which is exactly when a user reaches for the button.

That last one is the constraint that outranks the rest of the sheet. **The power path has to be the most robust thing in Helium, because it is the escape hatch for everything else** – and it is why the telemetry below is register logic too, rather than the much cheaper trick of letting the EC drop a structure into SRAM.

The kernel side – `/dev/power`, and no new syscalls.

S.11 The project rule is that drivers expose no syscalls of their own and userspace reaches hardware through `/dev/` (→ J.4). Power management honours it and **adds nothing to the dispatch table**. A dedicated `sys_poweroff()` was considered and rejected: it would be a single-purpose entry in a table otherwise built from general primitives, and it would need its own privilege check duplicating one the filesystem already performs. Routing through `/dev/power` gets that check for free – the node is writable only by privileged processes, for the same reason any other privileged device node is.

IOCTL	ARGUMENT	EFFECT
<code>PWR_POWEROFF</code>	–	Begin ordered shutdown
<code>PWR_REBOOT</code>	–	Begin ordered restart
<code>PWR_GET_STATE</code>	pointer	Fill the state structure – the only unprivileged command
<code>PWR_CANCEL</code>	–	Cancel a pending request, only before the point of no return

PWR_ACK	–	The session manager signalling that userspace cleanup is done
----------------	---	---

S.12 The driver implements the standard five functions: `init` clears any stale request and registers the handler, `read` returns the state structure, `write` is unsupported (control goes through `ioctl` so arguments are typed), `ioctl` takes the table above, `irq_handler` services the request. **The interrupt handler must not flush the filesystem** – flushing is unbounded work and cannot run in interrupt context. It captures request type and source, posts a message to the session manager, and returns; the level-based interrupt means nothing is lost between those steps.

S.13 The shutdown procedure itself, in process context: **notify** every registered process and start the grace period → **wait**, bounded, `PWR_CANCEL` still honoured → **point of no return**, cancellation now fails → **terminate** stragglers in reverse creation order → **stop the window server**, releasing the framebuffer aperture → **flush and unmount** → **quiesce the MMU** → **halt** with interrupts off, having written `POWER_CTRL` first.

The MMU quiesce – range-flush the cache, then poll `MMU_STATUS.BUSY` until clear (→ M. 6) – is the step most likely to be forgotten and the most damaging to omit. **The cache is write-back: a shutdown that flushes the filesystem but not the cache can still lose the very writes the filesystem just made**, which is precisely the data loss the ordered path exists to prevent.

S.14 `PWR_CANCEL` exists so a desktop session can put a confirmation dialogue in front of a button press. It is honoured up to the point of no return and fails after it, because from there onward continuing to run is worse than finishing.

PHASE	BUDGET
Interrupt → session manager notified	< 10 ms
Userspace grace period	3 s
Process termination	500 ms
Filesystem flush and unmount	2 s
Cache flush and MMU quiesce	100 ms
Worst case	~5.6 s, against a 10 s watchdog

S.15 That leaves ~4.4 s of margin, and the grace period is the tunable. **A period longer than the watchdog turns every shutdown into a forced one, and the failure is silent** – a forced shutdown looks identical to a successful one from outside, until someone loses data. The two numbers are a matched pair and must live in one place and change together (→ Q39). On a **critical battery** the EC raises the request with `SRC_BATTERY`; the kernel treats it as an ordinary ordered shutdown with two differences – no dialogue, no cancellation, and a grace period shortened to 1 s. Whether the kernel may `NAK` it at all is open policy. The argument against is short: the battery does not negotiate.

Battery telemetry – the same path, travelling the other way.

S.16 The instruments are on the A0N I2C bus and belong to the EC; the 65816 cannot reach them, and any path that fixes that owes three properties. **The CPU must never block** on an I2C transaction it did not initiate and cannot bound. **Reads must not tear** – a multi-byte value updated asynchronously and read a byte at a time otherwise yields impossible combinations: a low byte from one sample, a high byte from the next. **Stale data must be detectable**, or a hung EC reports a battery frozen at whatever it last said. The answer is **two atomic copies, one in each direction**: the EC dribbles bytes into a **staging** bank over SPI in any order and at any speed, then sets **TELEM_COMMIT** – Helium copies staging → **live** in one cycle and bumps **TELEM_SEQ**. The CPU writes **TELEM_LATCH**, Helium copies live → **shadow** in one cycle, and the kernel reads it as slowly as it likes without tearing. No retry loop, no generation counter: a latch, then a read. Two alternatives were rejected, both for reasons that recur across this document. A **seqlock** – one bank plus a counter the CPU reads before and after, retrying on mismatch – saves the shadow bank's ~128 flip-flops but moves complexity into kernel code on the constrained processor, and a retry loop against a fast committer has no hard bound; the flip-flops are cheaper than the reasoning. And letting the **EC write telemetry into SRAM** through the debug agent's bus path would drag battery reporting through the cache controller and the arbiter, making it depend on the health of the memory subsystem – which is exactly what S.10 forbids. Total cost of the chosen path is ~16 bytes of register file doubled for the snapshot, about 256 flip-flops.

The telemetry registers are in the switched domain and lose their contents whenever 3V3_MAIN drops. So after every power-up and every resume, the EC must repopulate staging and commit **before releasing the 65816 from reset** – step 8b of sheet C's power-up table – or the kernel's first read is zeros, indistinguishable from a flat battery. **TELEM_SEQ** resets to zero on Helium reset, so sequence zero with a fresh age means "first commit after boot" rather than a wrap.

The CPU's only write anywhere in this path is **TELEM_LATCH**, and it carries no data – which is what makes the block safe to expose to user space at all.

S.17 **The EC publishes human units; the CPU never sees a raw sensor code.** The 65816 has no multiplier and no divider, so making the kernel scale a VCELL code at 78.125 $\mu\text{V}/\text{LSB}$ or a CRATE code at 0.208 $\%/ \text{hour}/\text{LSB}$ would burn cycles on arithmetic the RP2354B does in nanoseconds. All scaling, sign extension and clamping happen before the push: VCELL → millivolts, SOC → integer percent plus a separate 1/256 fraction byte, CRATE → signed tenths of a percent per hour, the charger's ADC → millivolts and milliamps, **CHRG_STAT** → an enumerated byte.

Human units also fix the ABI. A future gauge that reports differently is the EC's problem.

S.18 **TELEM_AGE counts eighths of a second since the last commit, saturating at 255** (~32 s). Helium supplies the raw counter and takes no view; the kernel derives **PWR_F_TELEM_STALE** from it, treating anything above the threshold as *unknown* rather than as zero. With a 1 Hz cadence, a threshold of 8 is a sensible default. One byte and one small counter, and it is the whole difference between "the battery is at 47 %" and "the OS does not currently know."

Saturating rather than wrapping is the point: 255 means *at least* 32 s and can never be read as fresh, which is exactly what a wrap would allow.

Eighths are enough resolution to tell a half-second-old reading from a four-second-old one, and coarse enough that no timestamp has to travel with the data.

Telemetry block – sixteen consecutive read-only addresses, all served from the shadow bank.

OFF	NAME	DESCRIPTION	OFF	NAME	DESCRIPTION
+\$00	TELEM_SEQ	Increments per commit; wraps	+\$08	PWR_FLAGS	See below
+\$01	TELEM_AGE	Eighths of a second, saturating	+\$09	CHG_STATE	0 none · 1 pre · 2 fast · 3 terminated
+\$02	BATT_SOC	State of charge, 0–100 %	+	VBUS_MV	Adapter voltage, mV
			\$0A · +		
			\$0B		
+\$03	BATT_SOC_FRAC	Fraction, 1/256 %	+	SYS_MV	SYS node voltage, mV
			\$0C · +		
			\$0D		
+	BATT_MV	Cell voltage, mV	+	ICHG_MA	Charge current, mA
\$04 · +			\$0E · +		
\$05			\$0F		
+	BATT_RATE	Signed tenths of %/hour; negative discharges			
\$06 · +					
\$07					

PWR_FLAGS BIT	0	1	2	3	4	5	6
	ON_ADAPTER	CHARGING	BATT_LOW	BATT_CRIT	PD_ACTIVE	GAUGE_FAULT	CHARGER_FAULT

S.19 `GAUGE_FAULT` is the flag that earns its bit: **an I2C failure must be visible as a fault, not as a plausible-looking zero.** The same reasoning as `TELEM_AGE`, applied to the instrument rather than to the link. Where this block sits inside bank `$FF` is still unassigned – it wants its own 256-byte wall on Q22's terms, and the base address blocks both the gateway and the kernel headers (→ Q40).

A fault flag the driver ignores is a fault the user meets as a wrong number.

S.20 **The CPU is not interrupted on every commit.** At 1 Hz that is a context switch to observe a number that moved by a fraction of a percent. The EC sets `TELEM_EVENT` only on discrete transitions – adapter in or out, charging started, finished or faulted, a low or critical threshold crossed, a gauge or charger fault appearing or clearing, the battery removed or inserted – and everything else is picked up by polling. Cadence is 1 s awake either side of the adapter, an immediate push after any event, and 60 s gauge-only with the EC dormant; the MAX17048 refreshes VCELL every 250 ms, so nothing is gained by going faster and the I2C traffic costs EC wake time.

The 1 Hz commit is the EC's cadence, not a promise to the kernel. A driver needing a fresher figure latches and reads rather than waiting for the next commit, and `TELEM_SEQ` is what tells it whether anything moved in between.

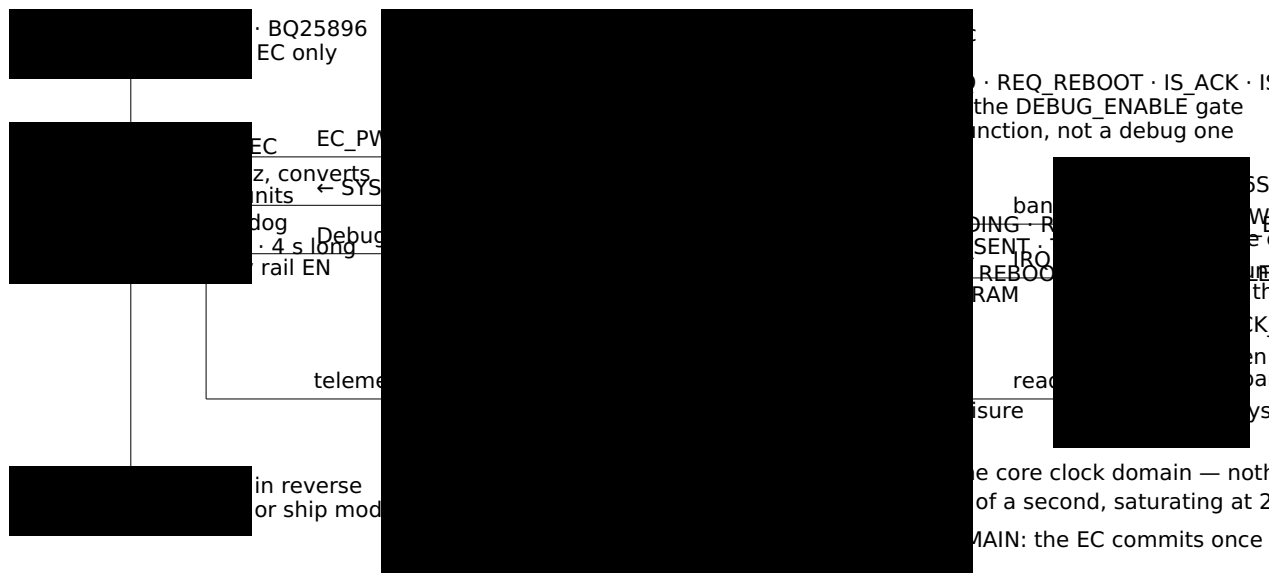
S.21 Time to empty is $\text{minutes_left} = (\text{battery_pct} * 600) / |\text{battery_rate}|$ – 16-bit integer arithmetic, one division, which is as much as the CPU should be asked to do. Two cautions, both of which produce nonsense if ignored. **CRATE is noisy:** a load transient swings the instantaneous rate wildly, so the driver keeps a ring of the last 8 samples and uses their mean. **A near-zero rate is not a long runtime:** below about 0.1 %/hour report `0xFFFF`, unknown, rather than a figure in the thousands of minutes. The same formula with a positive rate gives time-to-full, and the driver must not report a time-to-empty while the flags say charging.

That division is the only one in the path, and it is where a zero has to be caught: `BATT_RATE` is signed and sits at zero on a machine resting at full charge, so the driver reports "not discharging" rather than dividing by it.

One division, 16-bit, and no floating point anywhere in the path.

S.22 On `TELEM_EVENT` the driver refreshes, clears the event and posts to the session manager so the indicator updates at once; ordinary drift is left to the GUI polling `/dev/power` every 10–30 s, which is ample. **The read path is ordered deliberately:** latch, read `TELEM_AGE` and stop there if it is stale, and only then spend cycles on the rest. There is no point converting data already known to be worthless. `open`

What is still open here beyond the pin allocation: the block's base address (→ Q40), the low and critical thresholds in percent and millivolts, whether `TELEM_AGE` should be visible to the EC too so it can detect that its own pushes are not landing, the BQ25896 ADC's conversion cadence, and how SOC is recovered after a long shelf period – the MAX17048's ModelGauge state is lost with power and its recovery leans on an open-circuit reading taken while the cell is at rest, so a machine woken from ship mode straight into a heavy load may report poorly for its first minutes.



The pins carry attention, the SPI link carries detail: a level held until the transaction completes, never a pulse encoding. If the acknowledgement never comes, the watchdog expires at 10 s and the EC drops the rails — after the tri-state p

Fig. 9 – The power control path. Two level pins carry attention between the EC and Helium, the debug agent's SPI link carries the detail, and telemetry travels the same link into a staging bank that is committed and latched into two atomic snapshots before the CPU ever reads it.

Trace legend: mint = attention pins, SPI and CPU register access · gold = telemetry and rail control · dashed = I2C and the interrupt to the CPU.

D RP2354B – embedded controller

BOOT · FPGA PROGRAMMING · USB AND HID · THE WAYS BACK IN

SHEET 07

It's the laptop's "management microcontroller": it handles everything that happens before a live CPU exists, and stays on watch afterward. **The part is now a single RP2354B in QFN-80**, and that change is the subject of D.8 onward, along with the USB topology, the on-board hub and the three recovery paths that replace the one this design used to rely on.

D.1 Triple role: **programmer** (bitstreams, BIOS, its own firmware), **bootstrapper** (full sequence in Fig. 3), and permanent **embedded controller**. **Its 2 MB of flash is in the package**, and it is the board's only mandatory non-volatile storage.

The flash moving inside the package removes a part from the always-on domain and removes a net from the board, and it costs the recovery argument that used to sit here. The old one was that a bad write to the external W25Q16 took the whole boot with it, but **BOOTSEL** over USB answered whatever the flash held and took a UF2, so a bricked EC needed no programmer and no header. **That path is gone** – D.9 commits the USB port to host mode and it cannot be borrowed back – and it is replaced by three paths that do not involve USB at all (D.13).

D.2 9-step boot (Fig. 3): button → rails → SSPI configuration of Helium/Neon(/ Argon) with CRESET_B/CDONE → SDRAM initialized by gateway → BIOS from SD to SRAM → SD handoff → RESB↑ → \$FFFC vector.

Configure Neon first, and the rest of that sequence becomes visible while it happens. Neon's **CDONE** is the point at which the panel starts showing a correct blank screen with no register written; from there the EC switches its SPI into service-port mode (T.46) and writes each subsequent step to the text buffer – Helium configuring, the card mounting, the BIOS loading, the CPU released. **A failure in any of those steps is then on the panel instead of being inferred from a dark one**, which is the whole argument for the service port and the reason the configuration order is a requirement rather than a preference (→ T.53, T.64).

D.3 Debugging: a debug port into FPGA-A (Helium) – live read/write of system memory, a free "ICE" for the kernel. The EC does not master the bus to do it: it drives a **debug agent inside Helium** over a dedicated SPI chip select, and that block issues the accesses as a peer of the CPU port. Specified in sheet R.

That was one half of the debug story and the other half used to be missing. The EC can halt and inspect the whole machine; nothing could halt or inspect the EC, and by D.7 its firmware holds the 10-second watchdog and the shutdown state machine – the parts whose misbehaviour looks like a hung kernel or a corrupt filesystem rather than like a firmware bug. **It is closed: J-DBG carries SWCLK, SWDIO, GND and the console UART on one 5-pin header** (D.12), one Raspberry Pi Debug Probe services all of it over a single cable, and SWD is now the *daily* programming path rather than a concession (D.13).

- D.4** Console: **a UART on J-DBG**, and it is the EC's only console. The USB-CDC console the earlier arrangement carried is gone with the device-mode port (D.9); the same Debug Probe that programs the EC bridges this UART to the host, so the cable count did not change. It's the terminal for the monitor (E2) and for development work throughout.
- It is **no longer the only console, and no longer the first one**. Neon's text mode reaches the panel at E1.7, before Helium's UART exists and before the CPU runs at all, and the EC drives it over the service port with no bus in the picture (T.64). The two are complementary rather than redundant: the serial line survives a dead panel and carries a host debugger, the screen survives a dead CPU and needs no PC attached.
- D.5** HID host: **the native USB controller in host mode, behind an on-board hub** – USB keyboard and mouse (PS/2 and matrix removed). It powers the downstream ports on demand and translates HID → events to the kernel over the EC channel.
- The hub is part of the machine, not an accessory the user supplies**, and it is now an IC on the board rather than something plugged into one port (D.10). A portable computer needs more than one socket without a dongle, and a known-good on-board hub also removes a variability source during bring-up.
- The channel those events ride on is the half of this link still unwritten: sheet H names its endpoints, sheet S settled the power traffic, and HID has no map yet.
- D.6** Its own pin budget nearly failed too, and was rescued the same way the FPGA's was. Straight onto GPIO the EC needed roughly 38 pins against 30 available; an **MCP23017 I2C expander** absorbed everything with no timing requirement – the three CDONE lines, the charger's STAT and PG, the fuel gauge's alert, and the enables for the 1.2V rail, the panel and the host boost – bringing it to 28 of 30.
- The pattern had appeared twice: both Helium and the EC ran out of pins, and both were saved by moving slow signals onto a cheaper transport – the keyboard matrix onto USB HID, the enables and status lines onto I2C. **Expect it a third time**, and sheet X is where it happened.
- The expander is retired, and this is the clearest thing the part change bought**. QFN-80 gives **48 GPIO** against the 30 that forced the expander, which absorbs REV C's per-rail **EN** and **PG** (sheet C), the two Helium handshake pins, and the hub's reset line with room left – and it removes the question of which power domain the expander itself lives in, which was open and awkward because an expander must sit on AON to drive an enable at all while its outputs cross into the switched domain (Q41 closes on this). One constraint disappears with it: PIO-USB's requirement that **D+** and **D-** sit on two consecutive GPIOs was a **pico-pio-usb** restriction, and there is no PIO-USB any more.
- D.7** **EC in operation, and this is no longer a sketch**. The button (short press ordered, ~4 s forced, **QON** for the hardware floor), the gauge and charger readout at 1 Hz, the telemetry push into Helium, and a shutdown that stays graceful with the OS hung – all specified in sheet S, which also closes what used to be the EC↔kernel protocol question.
- What the EC now owns that it did not before: **it is the only thing that can cut a rail**, and it holds the 10-second watchdog that decides when a kernel has stopped answering. That makes its firmware state machine part of the machine's data-integrity story, not just its housekeeping.

The part – what changed, and what it deletes rather than adds.

D.8 One RP2354B, QFN-80, A4 stepping, and the three things it removes are worth more than the pin count on its own. **48 GPIO** removes the pressure that motivated both the expander and, in an earlier arrangement, a second microcontroller. **2 MB of flash in the package** removes a component from the always-on domain and the net that went with it. And a **hardware USB host controller** removes the bit-banged PIO stack from the critical path – the silicon handles hub enumeration, the **PRE** packets Low Speed devices need, and bulk transfers, none of which anybody wants to debug in software. **Confirm the stepping from the package marking on the parts as received, not from the distributor's part number.** RP2354 never shipped as A2, but the marking is the only thing that settles it (→ Q104). Every GPIO assignment must also be validated against the function-mux table before capture, which is not the same exercise as counting pins (→ Q105).

D.9 The native controller is assigned to host mode, and that assignment is one-way. The USB-A receptacles source VBUS as a host, so attaching a PC would need an A-to-A cable with two 5 V supplies opposing each other: **BOOTSEL over USB is not structurally available, not merely unsupported.** Recovery is therefore restructured into three tiers that touch no USB port at all (D.13), and the debug console is a UART (D.4) rather than a second USB device.

The USB-C connector carries power only – VBUS and the two CC lines into the CH224K, with **D+ / D-** unconnected. Which also settles C.11's warning by construction rather than by discipline: the CH224K cannot be tempted onto the data lines because there are none on that connector.

The alternative arrangements were examined and rejected. **Native as device with PIO as host** preserves drag-and-drop programming but puts the hub, the Low Speed **PRE** handling and the bulk transfers on the most fragile protocol path in the most fragile implementation. **Native as host with PIO as device** for a CDC console is defensible and is held as a fallback if a USB console ever proves necessary – it would *not* restore **BOOTSEL**, which needs the native controller in device mode. **A USB role mux** would restore it, at the cost of a part in the differential path, a connector and VBUS interlock logic, to recover a capability three independent tiers already cover.

USB topology – one Full Speed bus, shared.

D.10 The hub's upstream port is Full Speed, and that is specification-mandated rather than a compromise. It gets no answer to its Hi-Speed chirp from a Full Speed host, so it reverts to Full Speed and behaves as a pure repeater with the transaction translator bypassed – exactly what happens when a USB 2.0 hub is plugged into a USB 1.1 port. Everything downstream shares 12 Mbps; a Full Speed hub is a repeater and not a switch, so downstream traffic is retransmitted to every enabled port and only the device matching the token's address answers.

Low Speed devices behind a hub need the host to emit a PRE packet before each transaction. It is supported in silicon here, and it is historically the most fragile path in this family's host stack. **Verify it early, with a real keyboard** (→ Q102).

The on-board hub consumes the single supported hub level, and in a portable computer a user will plug another one in. The tier limit has to be raised and multi-level handling verified rather than assumed, and if cascading proves unreliable the fallback is stated now: **detect a downstream hub during enumeration and refuse it with a message on the console**, rather than failing silently (→ Q103).

D.11 JP-BYPASS puts a series jumper on D+ / D- ahead of the hub with an alternate route straight to the first receptacle, and its value is entirely diagnostic. At 12 Mbps a jumper costs nothing in signal integrity. What it buys is the difference between "does the host stack work at all" and "does the hub work" – without it, a board that enumerates nothing tells you nothing about which of the two is at fault. Three-pin header with a shunt, or two 0 Ω pads with one populated (→ Q106).

D.12 Silkscreen the pin order on both debug headers. Transposing SWCLK and SWDIO during bring-up costs hours, and so does guessing at J-QSPI.

Connector inventory – the EC's, not the machine's.

REF	TYPE	FUNCTION	SIGNALS
J-PWR	USB-C receptacle	PD sink input	VBUS, CC1, CC2, GND. D+ / D- not connected
J-HOST1 · J-HOST2	USB-A receptacles	Downstream ports 1 and 2	Switched VBUS, D+, D-, GND
J-EXP	4-pin internal header	Downstream port 3, expansion	5 V, D+, D-, GND
J-DBG	5-pin header, 2.54 mm	SWD + console UART	SWCLK, SWDIO, GND, UART_TX, UART_RX
J-QSPI	8-pin header	Bootrom UART boot and direct flash	QSPI_SS, SCLK, SD0 – SD3, GND, 3V3
JP-BYPASS	3-pin jumper	Hub bypass (D.11)	D+ / D- routing select
SW-RUN · SW-BOOT · SW-UBOOT	Buttons or pads	Reset · BOOTSEL entry · UART boot select	RUN → GND · QSPI_SS → GND via 1 kΩ · QSPI_SD1 → 3V3 via 1 kΩ

The 1 kΩ in series with QSPI_SS is **required**, so the button cannot fight the EC when it drives chip select during normal flash access. Place it close to the pin. Put the three buttons adjacent and document the entry sequence – hold SW-BOOT and SW-UBOOT, pulse SW-RUN, release – on the silkscreen beside them.

Programming and recovery – three tiers, none of which touches a USB port.

TIER	PATH	INTERFACE	WRITES	SURVIVES
Daily	SWD	J-DBG	Flash + SRAM	–
1	SWD rescue debug port	J-DBG	Chip reset	Hung firmware, code that reconfigures the debug pins, corrupt flash
2	Bootrom UART boot	J-QSPI	SRAM only	A missing or broken SWD probe

TIER	PATH	INTERFACE	WRITES	SURVIVES
3	Direct QSPI to the flash die	J-QSPI + RUN low	Flash die	Everything short of physical damage

D.13 Tier 0 is SWD and it is two signals plus ground. Flash programming works by loading a flash algorithm into SRAM which then calls the bootrom's flash routines; OpenOCD handles that transparently, and **picotool is not available over SWD** – it speaks its protocol over USB only, so OpenOCD is the tool for this project.

Tier 1 is the reason SWD is the primary path rather than merely the convenient one. The debug architecture exposes a **rescue debug port** alongside the two cores, and it can reset the chip even when firmware is actively hostile to debugging – a tight loop, a routine that reconfigures the debug pins, a crashed sequencer. For an EC whose firmware sequences the rails that feed the FPGAs, that matters more than usual: **a bug in the sequencer must never leave the board unreachable.**

D.14 Tier 2 is the bootrom's UART boot, and its limitation is the whole reason D.15 exists. The bootrom checks two straps at reset – `QSPI_SS` low enters BOOTSEL, `QSPI_SD1` high selects UART rather than USB – and then speaks 1 Mbaud on `QSPI_SD2` and `QSPI_SD3`. Those are flash-interface pins, not GPIO, and are not remappable; it is a different UART from the console on `J-DBG`. **It writes only to SRAM. It cannot program flash.**

The strap is safe because the bootrom leaves chip select output-disabled and pulled high, so the in-package flash stays deselected, is never clocked, and its data pins stay high-impedance rather than opposing the `SD1` strap.

The RAM flash-writer stub is what makes tier 2 useful, and it is a real deliverable that must exist before it is needed. Built to link entirely into SRAM, it receives the payload over that same bootrom UART and writes flash by calling the bootrom's own flash functions, with a **CRC-32 readback** to verify – and the assembly CRC-32 from Y2.4 is reusable for it unchanged. About 200 lines. **Archive the built binary alongside every EC firmware release: a recovery tool that has to be rebuilt from source during an emergency is not a recovery tool** (→ Q107).

D.15 Tier 3 addresses the flash die directly, and it works because holding RUN low puts the processor's QSPI pins in high impedance. The in-package die is a standard part that speaks single-bit SPI, so four signals – chip select, clock, and the two data lines – are enough, and all four are on `J-QSPI`. Any SPI flash programmer will do.

The hazard is that those nets are simultaneously exposed on a header and connected to the in-package flash die, so external noise on them can destabilise the running system. Keep the header stubs as short as physically possible; **route them away from the SDRAM, the backlight boost and the boost converter's switching node;** ground-guard them if there is room; and consider 33 Ω series resistors on the header side to damp reflections from the unterminated stub (→ Q108).

The hub IC and its power.

D.16 The candidate is a four-port hub that runs standalone from pin strapping, with per-port power control and overcurrent sense, chosen over two cheaper parts on documentation grounds: it is the only candidate whose datasheet meets the project's no-black-boxes rule, and per-port control lets the EC kill a misbehaving port without cycling the whole bus. **The cost delta is irrelevant against the schedule cost of debugging an undocumented part.** A cheaper SSOP part with gang-only power control remains defensible if hand-assembly ease dominates on the prototype board, since it is the easiest of the three to rework (→ Q109).

Configuration rules that are easy to get wrong. **Strap it self-powered**, so it can declare 500 mA per downstream port – a bus-powered hub cannot. Follow the crystal's load capacitance exactly; **crystal error is a common cause of intermittent enumeration.** Strap unused downstream ports off rather than leaving them floating. **Do not populate the configuration EEPROM** – pin strapping is sufficient and removes a failure mode. And **do not release the hub from reset with an RC**: it must be powered and out of reset before the EC begins enumerating, so the ordering belongs in EC firmware where it is explicit.

D.17 Per-port current-limit switches driven by the hub's port-power outputs, with the fault line returned to the hub, and 120 µF of bulk capacitance downstream of each switch to absorb hot-plug transients. **Do not omit the per-port limiting: a short on a user-accessible connector must not take down the system.**

The current budget does not close on a portable, and that is a policy question rather than a component one. Typical draw is the hub's 30 mA plus a keyboard and a mouse at 50 mA each – 130 mA, 0.65 W. Worst case is three ports at 500 mA plus the hub: **1.55 A, 7.75 W**, which a 1S battery is not going to deliver. Two answers: a single aggregate current limit ahead of the hub, letting the per-port switches handle faults only; or **the EC reading each device's declared maximum during enumeration, keeping a running total, and refusing to configure a device that would exceed the budget, saying so on the console.** The second is preferable – it is the honest behaviour, it is visible to the user, and it is exactly the kind of policy an EC should own (→ Q110).

VBUS_EN for the whole USB subsystem becomes another switched rail in the REV C sequence: **the hub and every port stay dead until the system is up** (C.19). Verify the boost converter's output capability at the chosen aggregate limit with the battery low, at 3.4 V in, before the schematic is frozen (→ Q111).

D.18 Signal integrity, four pairs of it: 27 Ω series resistors on the EC's D+ / D– close to the pin, 90 Ω differential routing matched in length on the upstream pair and all three downstream ones, and **ESD protection on every downstream pair and its VBUS** – these are user-accessible connectors. The host-mode pull-downs are internal to the part and need no external components.

Do not do – the list of irreversible or expensive mistakes, kept together deliberately.

D.19 **Do not program the OTP.** Secure boot and the bootrom-UART disable are irreversible, and programming the security bits **disables SWD permanently and removes tiers 1 and 2 at the same stroke.** The EC has no threat model that requires them.

D.20 **Do not route the J-QSPI stubs near the SDRAM, the backlight boost or the boost converter's switching node** (D.15) · **do not release the hub from reset with an RC** (D.16) · **do not omit per-port current limiting** (D.17) · **do not populate the hub's configuration EEPROM.**

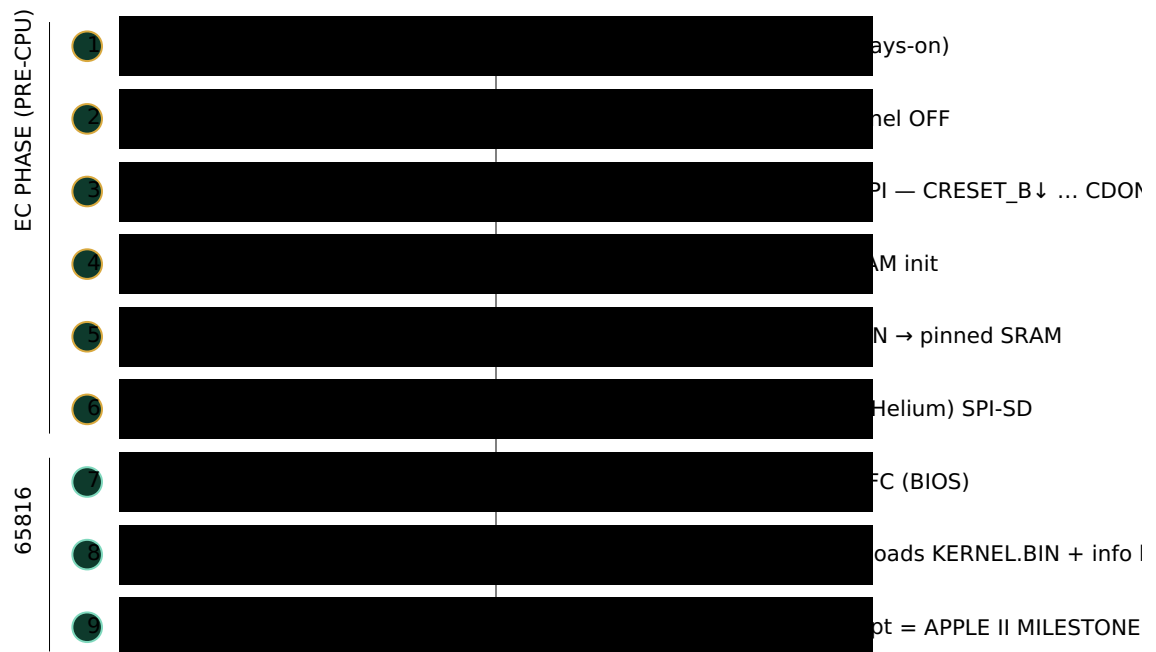


Fig. 3 – Boot sequence. Steps 1–6 are executed by the EC (gold); from step 7 onward the 65816 (mint) takes over.

Gate plan · U0–U8 – the USB and programming path, brought up separately from everything else.

- U0

Hub IC selected, power policy decided, J-QSPI layout guarded. Q109, Q110 and Q106 closed.

- U1

Schematic captured, with every USB and debug net reviewed against this sheet.

- U2

Board powered and the EC programmed over SWD. OpenOCD programs and verifies; console output on J-DBG .

- U3

The RAM flash-writer stub validated – firmware written to flash over the J-QSPI UART and verified by CRC-32 readback.

- U4

The host stack enumerates with JP-BYPASS in bypass. A Full Speed device enumerates directly and reports on the console.

- U5

The host stack enumerates through the hub. Hub descriptor read, port power applied, a device on port 1 enumerated.

- U6

A Low Speed device behind the hub. A keyboard reports HID data. Closes Q102.

- U7

Cascade behaviour characterised. An external hub either works or is refused gracefully. Closes Q103.

- U8

The power policy enforced. A device exceeding the budget is refused at configuration and the refusal appears on the console.

Instrumentation, not a product feature. A hardware block inside Helium performs memory and bus accesses on command, and a text monitor on the RP2354B drives it – so the board is observable from the first stage at which anything can be powered, long before a BIOS or a kernel exists. It is expected to stay in the design permanently, gated off rather than removed.

R.1 Five capabilities define it, and everything else on this sheet is their cost: read and write **any physical location** (SRAM and SDRAM), read and write **through the MMU** once translation exists, generate **real bus cycles** other devices can see, **halt · single-step · trace** the W65C816S, and do all of it from a terminal over the EC's console UART (R.19).

The five arrive in that order rather than together. Physical access exists at E2, translation at E4, real bus cycles at E5; the staging table at the foot of this sheet is the honest form of the list above, and reading it as a set available on day one is the one way to be disappointed by this block.

Reading *through the MMU* is the capability that arrives last and matters most: it alone shows what the kernel believes, not what the memory holds.

Nothing on that list needs a running CPU, which is what makes E2 possible.

R.2 **The RP2354B is not a bus master.** It sends commands; the debug agent performs the access. Three reasons, none of them stylistic. **Pins:** mastering directly needs ~24 address plus 8 data lines plus control, well beyond what the EC has left after its committed functions – and its budget already had to be rescued once (→ D.6). **Reuse:** Helium already owns the SRAM controller, the SDRAM controller with refresh, the cache and the arbiter; a second path to memory would duplicate all of it and become a second source of truth about memory state. **Coherence:** an external master bypassing the cache reads stale data whenever a dirty sub-block is resident in SRAM.

The consequence is the whole design in one line – the debug agent is **a peer of the CPU port in Helium's arbiter**, not a special case bolted to the side of it.

The cost of that decision is that the agent inherits Helium's health. Every capability is reached through a block which is itself part of the machine under test, so a Helium that did not configure, or whose clock is wrong, takes the whole debug path down with it – `DBG_ID` reading `$0000` is not a link fault (R.8) but the instrument gone. It is the deliberate price of refusing a second path to memory, and it is why Neon's service port (R.4) counts for more than it first appears: two blocks that fail apart.

Being a peer also means competing for bandwidth like one: a march test running while the CPU runs slows both, which is honest behaviour rather than a surprise.

R.3 **Helium drives BE**, not an RP2354B GPIO. Taking the bus requires two things in a defined order: the CPU releases its drivers, then Helium begins driving. Across the SPI link that ordering is a race across a clock-domain crossing with millisecond latency; inside Helium it is a synchronous state transition. Schematic consequence: **BE** routes Helium → CPU socket, and Helium's CPU-side address and data pins must be **bidirectional**, which lands on the I/O bank plan (→ Q27).

R.4 The register file is SPI-only and is not in bank \$FF. The agent is unconditionally privileged – it writes \$FF, bypasses permission checks and halts the CPU – so exposing it to software would make the entire protection model depend on kernel correctness. Keeping the control path physically separate means no software failure can reach it, and the console stays usable precisely when software has failed, which is when it is needed (→ M.1, Q22). There is a second debug path on this board now, and the two are worth seeing as a pair. Neon reuses its four configuration SPI pins as a service port once CDONE rises, reaching its register file, its text buffer and font in block RAM, and its SDRAM (T. 46). This agent gives the EC memory and bus visibility from inside Helium; the service port gives it a screen and Neon's state from inside Neon. They share no logic and no chip select, so a fault in one does not take the other – and between them a machine with a dead CPU still has a way to say what is wrong, with nothing attached to it.

R.5 Two orthogonal axes characterise every transaction: **which address space** and **how far the cycle reaches**. They compose freely, and the console shows both in the prompt because getting them wrong is the commonest source of confusing results. Four combinations, and the one that surprises is **virtual + external cycle**: translation and a cycle other devices can see. It is the last to arrive, at E5, because it needs both halves – which is why the prompt shows the axes rather than assuming them.

The two axes – address space and transaction scope, chosen independently.

AXIS	MODE	WHAT IT MEANS	AVAILABLE FROM
Address space	Physical	27-bit raw address, straight to cache and memory controllers. What is needed while the MMU is itself under test	E2
Address space	Virtual	24-bit address + ASID, through translation and, on a miss, the walker. Shows what the kernel believes it is looking at	E4
Scope	Internal	SRAM, SDRAM and Helium's own registers. Nothing appears on the CPU bus	E2
Scope	External cycle	Everything the CPU can reach – the \$FE VRAM aperture and the rest of \$FF included. Indistinguishable, from outside, from a cycle issued by the 65816	E5

R.6 Link: SPI mode 0, RP2354B controller, Helium peripheral, 16 MHz initially – the limit is the synchroniser depth on Helium's side, not the wire. It shares SCK / MOSI / MISO with the configuration SPI but takes a dedicated DBG_CSN: sharing data lines is acceptable, sharing the chip select is not, because the configuration path has to keep working when the agent is in an unknown state. 16 MHz is a starting value, to be re-measured if the synchroniser depth changes – the wire is nowhere near its limit.

R.7 Frame: one header byte – bit 7 is 0 write / 1 read, bits 6:0 the register address – then data, **least-significant byte first**. DBG_CSN deasserts between frames; a frame commits on its rising edge, and a truncated one is discarded and sets ERR_FRAME. Two registers, DBG_FIFO and TRC_FIFO, stream for as long as DBG_CSN stays low, so dumping a kilobyte of memory does not cost a frame per byte.

R.8 `DBG_ID` reads `$6516`, and the console reads it first: a correct value proves the SPI link, Helium's configuration and the clock all at once, before anything else is attempted. `$0000` or `$FFFF` means the bitstream did not load. A second read in the same frame returns the gateway build revision, which is bumped on every synthesis.

`$6516` is not arbitrary – it reads as the part number, which makes a wrong value obvious on a console with no other output yet.

R.8a **First contact, in the order the console performs it – each step discriminates a different failure.** `DBG_ID` first (R.8) – link, configuration and clock in one read. Then the build revision from the second read of the same frame, which says whether the part runs the bitstream just synthesised or a stale image from flash. Then `DBG_STATUS`, which answers even with `DEBUG_ENABLE` deasserted (R.18), so "jumper off" and "agent broken" stop looking alike. Then clear `DBG_ERR`: R.9's latch may be set from a previous session, and every command after it refused. Only then an **internal** read of a known SRAM location, proving the arbiter path and the cache controller with no cycle on the CPU bus – and only after that anything virtual (E4) or external (E5).

`$0000` or `$FFFF` from `DBG_ID` means no bitstream; a correct ID with `DBG_STATUS` refusing is `DEBUG_ENABLE` off; a correct ID with the first internal read timing out is R.17's watchdog – the agent is alive, the memory path is not.

R.9 Two disciplines the console must observe. **Never assume a command completes inside the frame that issued it** – an SDRAM access with a page miss and an intervening refresh takes far longer than an SRAM hit, so poll `BUSY`. And **the agent latches the first error and refuses further commands until `DBG_ERR` is cleared**, which is what stops a scripted burst from quietly producing a page of garbage after its first failure.

Both disciplines are the console's: gateway cannot enforce either.

R.10 Access width, corrected against the memory the board actually has. The system SRAM is **1 MB ×8 with D0–D7 shared with the CPU nets** (→ D15), so it has no upper/lower byte enables and a 16-bit access is simply two consecutive byte cycles. Byte granularity does bite on the SDRAM side, which is ×16 and masks with `DQM`. `DBG_WIDTH` therefore selects **8-bit low · 8-bit high · 16-bit**, and the E2 memory test must still exercise byte writes explicitly: a byte write that corrupts its neighbour is a very specific and very confusing failure, and it is cheaper to find it with `mt` than through a kernel bug.

This register was specified against an IS61WV102416 ×16 SRAM with `UB# / LB#`. That part predates D15 and is not on this board; the register keeps its encoding, its justification changes.

On the SRAM side an odd byte count is no special case – only a last access.

On an ×8 memory low-versus-high is an address, not a byte enable: `DBG_WIDTH` picks which of the two cycles happens, and the SRAM never learns which half it served.

Register map – SPI address space, 7 bits, entirely separate from the `$FF` block of sheet M.

ADDR	NAME	ACCESS	WIDTH	MEANING
<code>0x00</code>	<code>DBG_ID</code>	R0	16	Magic <code>\$6516</code> ; second read returns the build revision

ADDR	NAME	ACCESS	WIDTH	MEANING
0x01	DBG_CTRL	RW	8	ENABLE · VIRT · EXTCYC · AUTOINC – advances DBG_ADDR by the access width after each access, and not after an error, so a failed burst leaves the address on the element that failed · NOCACHE · STALL – 0 steals cycles where it can, 1 stalls PHI2 for every access
0x02	DBG_STATUS	RO	8	BUSY · DONE · ERROR · FIFO_EMPTY · FIFO_FULL · CPU_HALTED · TRC_TRIGD · DBG_READY
0x03	DBG_ERR	RO/W1C	8	TIMEOUT · UNMAPPED · PERM · ARB · CMD · STATE · FRAME · ALIGN
0x04	DBG_ADDR	RW	32	27-bit physical, or 24-bit virtual; unused high bits must be zero
0x08	DBG_DATA	RW	16	Data port for single accesses
0x0A	DBG_WIDTH	RW	8	1 low byte · 2 high byte · 3 16-bit (→ R. 10)
0x0B	DBG_ASID	RW	16	ASID for virtual accesses. Narrows when Q24 lands
0x0C	DBG_CMD	WO	8	Command trigger; rejected with ERR_STATE while BUSY
0x0D	DBG_COUNT	RW	16	Burst length in elements
0x10	CPU_CTRL	RW	8	RESET · HALT · STEP · STEP_INSN · IRQ_MASK · NMI_MASK
0x11	CPU_STATUS	RO	8	CPU state
0x12	CPU_CYCLES	RO	32	Free-running bus-cycle counter
0x18	TRC_CTRL	RW	8	EN · ARM · ONESHOT · POS · HALT_ON_TRIG · FILTER_OPFETCH
0x19	TRC_STATUS	RO	8	Trace state and fill level
0x1A	TRC_TRIG_ADDR	RW	32	Trigger address, with RWB / VPA / VDA qualifiers in the upper bits
0x1E	TRC_TRIG_MASK	RW	32	Trigger don't-care mask
0x1F	TRC_FIFO	RO	burst	Trace record readout
0x20	DBG_FIFO	RW	burst	512-byte bulk data port, sized to stay ahead of the link across an SDRAM page miss. The agent never stretches the SPI clock , so the console is the side that honours FIFO_FULL on writes and FIFO_EMPTY on reads
0x30	TLB_INDEX	RW	16	TLB entry selector
0x31	TLB_ENTRY	RO	64	TLB entry readout
0x38	MMU_MIRROR	RO	8	Read-only view of MMU_STATUS (→ sheet M)

ADDR	NAME	ACCESS	WIDTH	MEANING
0x40	EC_PWR_STATE	RO	8	Always accessible , <code>DEBUG_ENABLE</code> notwithstanding: <code>SYS_REQ</code> · <code>REQ_REBOOT</code> · <code>IS_ACK</code> · <code>IS_NAK</code> · <code>CPU_ACK_SEEN</code> (→ S.9)
0x41	TELEM_CTRL	WO	8	Always accessible : <code>TELEM_COMMIT</code> · <code>TELEM_EVENT_SET</code>
0x42+	TELEM_STAGE	WO	16 B	Always accessible : the staging bank, written byte by byte in any order (→ S.16)

Commands – written to `DBG_CMD`, one at a time.

CODE	NAME	ACTION
0x00	<code>CMD_NOP</code>	No operation; clears <code>DONE</code>
0x01 · 0x02	<code>CMD_READ</code> · <code>CMD_WRITE</code>	One element through <code>DBG_DATA</code>
0x03 · 0x04	<code>CMD_READ_BURST</code> · <code>CMD_WRITE_BURST</code>	<code>DBG_COUNT</code> elements through the FIFO
0x05	<code>CMD_FILL</code>	Write <code>DBG_DATA</code> to <code>DBG_COUNT</code> consecutive elements
0x10 · 0x11	<code>CMD_CACHE_FLUSH</code> · <code>CMD_CACHE_INVALID</code>	Flush the whole cache to SDRAM · invalidate it without writeback. INVAL discards dirty data and exists only to recover a wedged cache during gateway bring-up
0x12	<code>CMD_CACHE_FLUSH_LINE</code>	Flush the line holding <code>DBG_ADDR</code> – the same operation the kernel needs before paging a frame out (→ F.7)
0x20 · 0x21	<code>CMD_TLB_PROBE</code> · <code>CMD_TLB_FLUSH</code>	Translate <code>DBG_ADDR</code> into <code>TLB_ENTRY</code> · invalidate the whole TLB
0x22	<code>CMD_PTWALK</code>	Full page-table walk, bypassing the TLB
0x30	<code>CMD_ABORT</code>	Cancel the in-flight command

R.11 `CMD_PTWALK` earns its place by differing from `CMD_TLB_PROBE` in exactly one way: it always reads the table from memory. **Comparing the two is the direct test for a stale TLB entry** – which is the precise failure the `CTX_SET_PTBASE` before `CTX_SET_ASID` ordering invariant exists to prevent, and which otherwise has no symptom until something is corrupted (→ M.8).

The comparison runs the other way too, and that is its use at E4. With no kernel in existence the console writes a page table by hand, walks it with `CMD_PTWALK` and probes it with `CMD_TLB_PROBE`: agreement validates the walker against a table the engineer wrote, disagreement localises the fault to one of the two. A walker is otherwise hard to test before anything exists that can fault (→ M.2).

The comparison only means anything with the same ASID loaded for both.

- R.12** The access path: `IDLE` → decode → translate if `VIRT` → arbitrate → then either an internal access through the cache, or `BUS_TAKE` · `BUS_CYCLE` · `BUS_RELEASE` for an external one → complete. Only the external branch touches the CPU, and its sequence is fixed: request the bus, stall `PHI2` at a clean cycle boundary, assert `BE` low, wait the CPU's tristate turnaround, drive the cycle, release the drivers, wait turnaround, deassert `BE`, release `PHI2` and the arbiter. The restart must meet the minimum `PHI2` pulse width. The existing glitch-free gating logic is reused unchanged – the agent is one more client of it, not a second implementation (→ D16).
- R.13** Coherence, four rules. **One: the agent goes through the cache controller**, exactly as the CPU port does, so a physical read of an SDRAM address whose sub-block is dirty in SRAM returns the cached value. This is not negotiable – a debug tool that reports memory contents differing from what the CPU sees is worse than no tool at all, because it produces confident wrong answers during exactly the sessions where the engineer can least detect them. **Two: `NOCACHE` is for gateway bring-up only** – its one legitimate use is validating the cache itself (write through the cache, flush, read with `NOCACHE`, compare), and the console prints a warning banner whenever it is set. **Three: external cycles are coherent by construction**, being decoded by the same logic as a CPU cycle. **Four: writing a PTE does not invalidate the TLB** – the console's `pt write` issues `CMD_TLB_FLUSH` afterwards; anyone driving the registers directly must do the same (→ M.7).
- R.14** CPU control is an **indefinite `PHI2` stall**, legal because the core is fully static (→ E.4, D16). `STEP` advances one bus cycle, `STEP_INSN` runs to the next opcode fetch – which `VPA` & `VDA` identifies, the same pins the MMU already needs for the `X` permission (→ E.3). `IRQ_MASK` and `NMI_MASK` matter more than they look: a 100 Hz timer that keeps running while the CPU is stopped presents a backlog of pending interrupts on resume, which looks exactly like a kernel bug and is not. `STEP_INSN` finds the next opcode fetch, and with interrupts unmasked that fetch may be the handler's first: stepping over an instruction and into an interrupt look identical from outside. That is what the masks are for.
- R.15** **The agent cannot read the CPU's registers.** The W65C816S has no scan chain and no debug port; `A`, `X`, `Y`, `S`, `DP`, `DBR`, `PBR` and `P` are not observable from outside, and no amount of gateway changes that. Two workarounds, in order of preference: **inferred from trace** – a halt at an instruction boundary plus the preceding cycles lets the console reconstruct PC and much of the state, passively and always; and a **debug stub** in bank `$00` that pushes every register to a known location, entered either by forcing an instruction onto the data bus during a fetch or from a `BRK` breakpoint trap (→ J.3). The console exposes the first as `reg` and the second as `reg -f`, and **the distinction stays visible to the user**: silently perturbing state during inspection is unacceptable in a debugger.

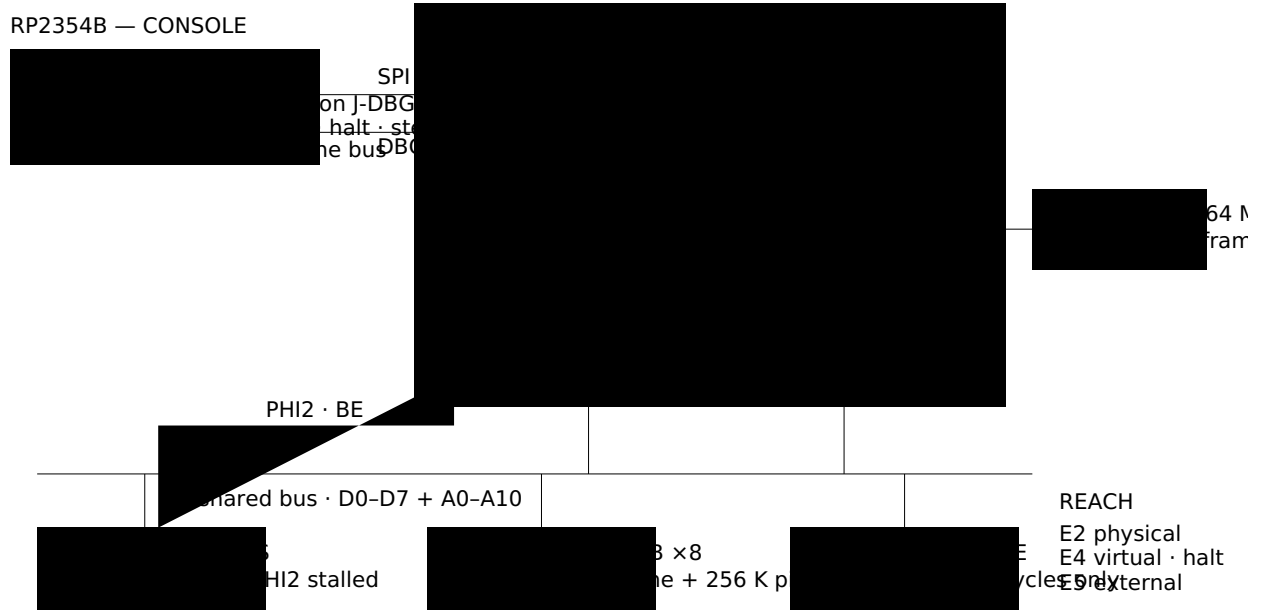


Fig. 8 – The debug agent as a requester inside Helium. The RP2354B commands over SPI and never drives the bus; internal accesses go through the cache, external cycles are taken with PHI2 stalled and BE low.

Trace legend: mint = command and data path · gold = CPU clock and bus control · dashed = optional.

VPA / VDA are all the visibility the part gives, and the trace records both.

R.16 Trace: a ring buffer in EBR capturing, per bus cycle, address (24), data (8), RWB, VPA, VDA, VPB, MLB, E and a cycle-counter delta – about 48 bits per record. The trigger compares (address & ~TRC_TRIG_MASK) against TRC_TRIG_ADDR, optionally qualified by RWB and VPA / VDA, and can position the capture at the start, centre or end of the window, halt the CPU when it fires, or filter down to opcode fetches only. **This is a logic analyser for one EBR block and a comparator, and it must be built with the CPU stage rather than retrofitted after the first hard kernel bug.** The mask is a don't-care mask rather than a match mask – TRC_TRIG_MASK bits set are the bits ignored – so an exact address is mask zero and a 2 KB page is a mask of eleven ones, which is what makes "trigger anywhere in this page" a single comparator rather than a range check. The buffer is read back through TRC_FIFO, which streams for as long as DBG_CSN stays low (R.7), so dumping the whole ring costs one frame rather than one per record. [open](#)

1024 records is the starting proposal, but the depth is a synthesis parameter against an EBR budget that the TLB and the cache tags are already spending (→ Q30). That figure was costed on an HX4K; Helium is an HX8K (→ B.2).

The depth is not free and the arithmetic is the reason it is a parameter: 48 bits is six bytes, so 1,024 records are 6 KB of a device that has 16 KB of EBR in total – and the TLB and the cache tags are spending it first (B.2).

The cycle-counter delta is counted in Helium's core clock, not in PHI2 edges, which is the only reason it survives L.15: a stalled cycle appears as a long delta rather than as a gap the trace cannot account for.

Filtering to opcode fetches turns the same buffer into an instruction trace.

R.17 Every transaction is covered by a watchdog – 100 μ s at the Helium core clock as a starting value, generous enough for a worst-case SDRAM access with an intervening refresh and short enough that a human reads it as a delay. On expiry the agent abandons the transaction, releases the bus and PHI2, sets `ERR_TIMEOUT` and returns to idle. **The agent must never be able to hang the console**, and that constraint outranks completing any transaction: if it hangs, the engineer loses the only view into a board that is, by hypothesis, already misbehaving. Abandoning is not simply stopping: the external branch of R.12 is unwound in reverse – drivers released, turnaround waited, `BE` deasserted, PHI2 restarted at a legal pulse width – since a timeout that left the CPU tristated or its clock stopped would turn a slow transaction into a dead machine. `ERR_TIMEOUT` then latches like any other error and blocks further commands until it is cleared (R.9), so a script cannot walk past the first timeout and report a page of zeros.

The reverse interaction has to be handled too – the system watchdog that turns a hung fill into NMI or ABORT must be **suppressed while the CPU is halted by the agent**, or every debug session ends in a spurious abort.

100 μ s sits between two bounds: longer than the slowest legal transaction – an SDRAM access whose row is closed and whose refresh lands mid-way – and short enough to read as a delay rather than a hang.

This board now has three watchdogs and they do not overlap. F.11's bounds a cache fill in microseconds, this one a debug transaction on the SPI side, and S.7's a kernel's shutdown acknowledgement in seconds. Each covers a case the others cannot see, so none can be dropped for another.

A watchdog that fires is itself data – it says where answering stopped.

R.18 `DEBUG_ENABLE`, a Helium input on a jumper, gates the whole thing: deasserted, it forces `DBG_CTRL.ENABLE` to zero, ignores writes to it, and rejects every command with `ERR_STATE` – while still answering `DBG_ID` and `DBG_STATUS`, so the console can report *why* it is refusing rather than appearing broken. The point is not security against physical access, which is unachievable and not a goal, but making the privileged path an **explicit, visible, deliberate state of the board**.

The gate now has a **documented hole, and it has to be one**: `EC_PWR_STATE` and the telemetry staging registers sit outside it. Power management is a production function, not a debug function, and putting it behind the strap would break shutdown and battery reporting on every board built with debug disabled. Neither register can initiate a bus cycle or reach memory, so the region grants nothing the strap exists to withhold – but it must be carved out of the map deliberately, and confirming it can be without disturbing what is already there is Q38.

The jumper is a production decision as much as a security one. R.4's argument – the agent bypasses every permission check and halts the CPU – applies to a stranger with four wires as much as to a kernel bug: a shipped board with the agent enabled hands over the whole address space.

R.19 The console is a **physical UART**, 115200 8N1, on the **J-DBG** header – and it is the only transport, because the EC's USB is committed to host mode and there is no device port to put a CDC console on (D.9). The mirrored USB-CDC of an earlier arrangement is gone with it; nothing above this line depended on having two.

What the loss costs is a diagnostic, and it is worth naming rather than glossing. Two transports used to let you tell a firmware fault from a USB stack fault at the first stage, by seeing which one still answered. With one transport that discrimination is gone – and it is largely paid back by there no longer being a USB stack under the console to be at fault, and by the rescue debug port of D.13, which reaches a chip whose firmware has stopped answering anything.

The header that carries it is the same one that carries SWD, which closes the case this NOTE used to make for putting two headers beside each other: **J-DBG** is five pins – **SWCLK**, **SWDIO**, **GND**, **TX**, **RX** – and one probe services both over one cable (D.12). They survive onto the target board for E1.4's reason: a header cannot be added to a board already spun.

Console grammar – hexadecimal by default, **#** prefixes decimal, **p:** and **v:** qualify an address.

GROUP	COMMANDS	NOTES
Memory	md · mw · mf · mc · ms · mt	dump · write · fill · compare · search · test (march-c, addr-in-addr, walking-1)
Mode	mode phys / virt · int / ext · asid <n> · width 8 / 16 · nocache on / off	bare mode prints the current flags; nocache prints a warning
CPU	halt · run · step [n] · stepi [n] · reset [hold] · reg · reg -f · dis	reg infers from trace, reg -f forces the stub and perturbs state (→ R.15)
Trace	trace arm [addr] [mask] · on / off · pos · dump [n] · filter opfetch	dump disassembles the records
MMU	tlb dump / probe / flush · pt walk / dump / write · cache flush / inval / stats	pt walk shows each level; pt write flushes the TLB after it
Storage	ld <file> <addr> · sv <addr> <len> <file> · boot <file>	straight from the microSD the EC already owns (→ G.2)

GROUP	COMMANDS	NOTES
System	<code>id · status · err · help [cmd]</code>	<code>status</code> decodes every status bit; <code>err</code> decodes and clears

R.20 Five behavioural requirements, each of them a lesson rather than a preference. **Every error is reported with its decoded cause** – never a bare failure. **Long operations show progress**: a march test over 64 MB of SDRAM takes minutes, and a silent console for that long is indistinguishable from a hang. **Ctrl-C aborts** through `CMD_ABORT`. **md output is annotated** with the region it fell in – cached SRAM, pinned SRAM, SDRAM, VRAM aperture, `$FF` – derived from the physical map of L.6. **The prompt carries the mode**: `p16>` for physical 16-bit, `v8:0042>` for virtual 8-bit under ASID `$42`.

The console is the reference implementation of R.9, not a convenience laid over it. Each of the five is a discipline that any other driver of these registers – a script, a GDB stub, a test harness – has to reproduce for itself, because the agent enforces none of them: it latches an error and refuses, and turning that into a legible failure is entirely the business of whatever holds the SPI.

R.21 What the agent settles that was open before it: **Q25 – how the RP2354B reaches Helium's control registers with the CPU held in reset.** It does not master the bus; it issues an internal access, and an internal write to a `$FF:00xx` address meets the same address comparator a CPU store would. So the whole "address as opcode" command model of sheet M – TLB flushes, context loads, cache flushes and their `BUSY` discipline – becomes testable at E4 with no CPU, no BIOS and no kernel in the picture (→ M.2, D18).

What it does not settle is worth listing beside it: it cannot show the CPU's registers (R.15); it inherits Helium's health, so a Helium that did not configure takes the whole path with it; and it says nothing before that configuration, which is the half Neon's service port covers (R.4).

Staging – the agent's capability mapped onto this document's stages (sheet P).

STAGE	DEBUG AGENT CAPABILITY	WHAT IT VALIDATES
E1.6 · RP2354B alone	Console up on the <code>J-DBG</code> UART; microSD; bitstream load	EC firmware, SD stack, the console transport
E1.7 · the three FPGAs	<code>DBG_ID</code> reads <code>\$6516</code> ; and on Neon's side, a correct text screen plus glyphs written over the service port – Neon stage N0 (T.64)	SPI link, Helium clock, bitstream actually loaded – and, on Neon, the PLL, the timing generator, block-RAM initialisation, the panel and the FPC
E2 · SRAM + monitor	Physical peek/poke; <code>mt</code> march test; byte-write check	SRAM controller, shared-net timing, <code>md / mw</code> end to end
E3 · SDRAM	Physical SDRAM access; <code>CMD_CACHE_FLUSH / INVAL</code> ; <code>cache stats</code>	SDRAM controller, refresh under load, cache coherence
E4 · MMU + cache + protection	Halt · step · trace; virtual mode; <code>TLB_PROBE</code> vs <code>PTWALK</code> ; internal writes to <code>\$FF</code>	PHI2 gating, <code>BE</code> handoff, arbiter, walker, ASID, sheet M's command model

STAGE	DEBUG AGENT CAPABILITY	WHAT IT VALIDATES
E5 · video + audio	External cycles into the \$FE aperture, reads included – with BE low there is no PHI2 to stall, so the agent simply waits for NEON_BUS_BSY to clear before sampling D0–D7 (T.18)	Inter-FPGA bus visibility, the VRAM window, Neon's decode – and framebuffer dumps, which is how blitter output gets compared byte-for-byte against a software reference with no kernel in existence

R.22 E2 is the stage that pays for the whole design. Being able to march-test SRAM before any CPU exists converts the hardest class of bring-up bug – intermittent memory faults surfacing as random software misbehaviour – into a direct, reproducible measurement, and it does so at the exact moment the board is least able to explain itself. The alternative is not a slower measurement but a circular one: without the agent, the first read of that SRAM comes from a CPU fetching its own instructions out of it, so the instrument and the subject are the same part – and a memory fault presents as a processor that misbehaves.

The free-run stage before it (E1.8) gains too, if the trace buffer is ready: a captured NOP run reads PHI2, reset and the address bus with no analyser.

It is also what makes the sheet worth its pins: everything after E2 extends an existing path, so translation, bus cycles and trace cost gateware, not board.

The economics are worth stating once. A marginal SRAM fault found at E2 costs a march test and a reflow; the same fault at E6 costs days of suspecting the kernel, the toolchain and the cache in turn, since each is a better-known suspect than the memory. The stage does not make the bug rarer, it makes it cheap to name.

R.23 Honest cost. In pins: `DBG_CSN` on both ends, `DEBUG_ENABLE`, `BE` from Helium, optionally `DBG_IRQN` – which asserts on a trace trigger or a CPU halt, and buys only the difference between an interrupt and a poll – the 3-pin UART header, and bidirectional CPU-side pins on Helium. That lands on a budget which **does not close as written** – Q8 already had the free 17 pins failing to cover SPI-SD, the console UART and this port. Two of those are not chargeable to the agent, and the accounting should say so: `BE` is driven by Helium for cache fills whether or not a debug port exists (F.4), and the UART header hangs off the RP2354B, not Helium. The agent's own marginal cost is `DBG_CSN`, `DEBUG_ENABLE` and an optional `DBG_IRQN` – two pins, or three with the interrupt. blocking

In gateware the open items are the SPI-to-core clock-domain crossing (→ Q28), the `BE` turnaround count at 3.3 V (→ Q27), where the agent sits in the arbiter's priority order (→ Q29), and the trace depth against remaining EBR (→ Q30).

If the budget gives, it gives in a known order: `DBG_IRQN` first, since polling costs only latency; then nothing – `DBG_CSN` and `DEBUG_ENABLE` cannot be shared, the second because a gate software can reach is not a gate (R.18).

Nor are these pins allocated alone: S.3 assigns `EC_PWR_REQ` and `SYS_PWR_REQ` together with `DBG_CSN`, so port and handshake close or fail as one item (→ Q27).

R.24 Deferred, not ruled out: **breakpoint comparators** – address match forcing a halt, a natural extension of the trace trigger, held back to keep the CPU stage small · **watchpoints on data value** as well as address · and a **GDB remote serial protocol stub** on the RP2354B, so a host debugger drives the agent directly. The last is attractive and cheap to get wrong early; it is worth doing only once the command set above has stopped moving. The costs are known even if the dates are not: a comparator is a register pair compared against the address the trace already carries, watchpoints add the data byte to that compare, and the stub is firmware. None of the three needs a board change, which is what makes deferring them safe rather than merely convenient. [open](#)

The order is not arbitrary. Comparators and watchpoints are gateway, competing for the EBR Q30 is already short of; the GDB stub is firmware and costs no silicon at all, which makes it the tempting one to begin with and the wrong one – a remote protocol frozen against a command set still moving becomes a second thing to keep in step, and the audit it then needs is the one this sheet has so far been able to avoid.

The stub carries a dependency the other two do not. A host debugger asks for registers first, and R.15 says the part cannot give them – so a GDB stub is only as good as the inference from trace, or it drags in the intrusive stub and perturbs the state it was asked to report. That decision belongs with the stub, not after it.

E**CPU – W65C816S and the softcore option**

THE PART · THE PINS THE DESIGN LEANS ON · WHAT ACTUALLY CAPS SPEED

SHEET 09

The 65816 is the premise of the project, not a component choice open for review – the lineage argument is in A.3. This sheet gathers what the rest of the design assumes about it, and settles what an FPGA softcore in Argon's slot would and would not buy – a question worth closing before anyone populates that footprint.

- E.1** The part: W65C816S in a PLCC-44 socket, in **24-bit native mode** (**CLC/XCE** at reset). WDC grades the silicon 1–14 MHz; this machine targets **8 MHz at 3.3V**, PHI2 from Helium. The 14 MHz is a property of the chip, not a ceiling anyone is pushing against – the limit that binds is memory (→ sheet F).
- E.2** The programmer's model the whole system rests on: 16-bit accumulator and index registers with widths switched at runtime by the **M and X flags**, the **E flag** for 6502 emulation mode, 24-bit addressing split into 64 KB banks via **DBR** and **PBR**, a relocatable direct page, and a stack that always lives in bank **\$00** – that bank is pinned (→ L.10).
- E.3** Four pins carry more architectural weight here than is usual. **ABORTB** cancels an instruction with no side effects, turning page permissions into real protection. **VPA/VDA** distinguish an instruction fetch from a data access, which is what makes the **X** permission checkable. **BE** tristates the CPU's buses – used by Argon, and far more often by Helium to take the shared nets during a fill. **RDY** freezes the CPU, and is bidirectional: the CPU pulls it low on **WAI**. Three consequences of those four that are easy to miss and expensive to discover. **RDY is an I/O in Helium, not an input** – the arbiter and the debug halt logic have to treat it as one, or **WAI** never releases (E.13). **Every chip select must be qualified with VDA v VPA**, because with both low the cycle is internal and the bank bits are don't-care (F.3). And **M/X are invalid during the cycle after REP, SEP or PLP** – anything sampling them to size an access has to skip that cycle. **ABORTB** is the pin with the most conditions attached, and they are the subject of E.14 onward.
- E.4** The core is **fully static**: WDC's datasheet sanctions holding PHI2 in either state indefinitely as a standby mode, with no minimum clock frequency. That property is not a curiosity – it is what licenses the cache-miss stall (→ D16), and it makes a stopped clock the machine's low-power mode for free. The same property makes the debug agent's indefinite halt legal (R.14).
- E.5** Argon (FPGA-C) holds an unpopulated softcore slot that takes the same bus with BE=0. The point of the slot is optionality – it lets the physical chip be replaced without redesigning anything around it. Its 3V3 and 1V2 rails sit behind their own 0Ω jumpers, so an empty footprint cannot load the bus. Giving the softcore its own device rather than a corner of Helium was deliberate twice over: it hands Helium back the block RAM and routing the memory subsystem needs, and it lets the core's clock be tuned without dragging MMU timing along with it. Whether populating it is worth doing is the rest of this sheet.

- E.6** What a softcore would have to preserve is narrower than it looks: only **programmer-visible compatibility**. The 92 opcodes and every addressing mode, the E and M/X flags, the full status register, and the interrupt vectors and entry semantics. Microarchitecture, pipelining, cycle counts and the external bus protocol are all free to change – and nearly all the available speed lives in what can be thrown away, starting with the faithful reproduction of the original multiplexed bus dance and its dummy reads.
- E.7** Two bottlenecks that must not be conflated. **Core Fmax**: the critical path runs microcode ROM → ALU → address calculation; registering the ROM output is free in EBR, and the decimal-mode adder is a habitual offender. **CPI**: the original spends 2–7 cycles, many wasted on the physical bus. CPI is the larger prize and the cheaper – throughput off the critical path.
- E.8** Reality check on this platform. The often-quoted 80–100 MHz and "20–30× an original 3 MHz machine" are figures for mid-range parts (Artix-7, Cyclone V) and **do not transfer to iCE40**, where complex logic tops out near 50 MHz. A full microcoded 65816 is complex logic; expect the low tens of MHz. [open](#)
No number here should be believed until a candidate core has been placed and routed on the target part (→ Q15).
- E.9** The "run from block RAM" path that makes softcores fast elsewhere is **closed on iCE40**: 128 Kbit of EBR total, ~16 KB, already shared with the TLB and the cache tags. There is no room to execute a meaningful program on-chip. That budget is not merely small, it is committed: the TLB, the cache tags and the trace ring (R.16) spend the same 128 Kbit, and Argon would be asking after them.
- E.10** Which leaves the real conclusion: **the ceiling is memory, not the core**. Argon would ride the same shared bus, the same Helium cache and the same PHI2 window as the physical chip, so its effective speed is gated by the identical hierarchy. A softcore can win on CPI, on added features, or on deleting a PLCC part from the BOM – **not on raw clock**, and only if given a wider or more direct path into Helium's cache than the multiplexed external bus allows. No such path exists in the design today (→ Q14).
- E.11** Honest caveat: the 65816 is an accumulator architecture – memory-traffic-heavy and fairly serial, with little instruction-level parallelism to exploit. Even pipelined, data dependencies cap the gains. "The fastest possible 65816" is a well-defined goal and a bounded one; it will not scale the way a modern core does.
- E.12** **Three of the five native vectors are load-bearing here**. Entry pushes PBR, PC and P, clears D, sets I, forces PBR = \$00, and vectors through bank \$00: \$FFE4 COP, \$FFE6 BRK, \$FFE8 ABORT, \$FFEA NMI, \$FFEE IRQ. COP is the syscall entry (M.1), ABORT the page fault (L.4), BRK the debug breakpoint (M.3). Those pushes are stack writes in bank \$00, and an ABORT taken while entering the ABORT handler has nowhere to go – which is what pinning that bank (L.11) forecloses. Five vectors in twelve bytes at the top of bank \$00, \$FFEC reserved. The rest of that bank belongs to the machine, not the CPU (L.11).

E.13 What a softcore owes the board is a pin list, wider than E.6's programmer-visible set. `VPA / VDA` valid every cycle, or `X` cannot be checked; `ABORTB` honoured with side effects suppressed, the instruction restartable; `BE` releasing the buses; `RDY` bidirectional, or `WAI` never releases. And a fifth E.3 omits: `VPB`, the privilege transition Helium reads on the vector fetch (L.12).

A cycle-exact core carries these because it reproduces everything; an accelerator such as FT816 is exactly where the audit below has to start.

A core that keeps those pins and changes everything else is still a drop-in.

The ABORT pin – what it guarantees, and the four documented cases where it does not.

E.3 names `ABORTB` as one of four pins carrying architectural weight, and sheet L builds real memory protection on it. This section is the audit that claim needs, because **no fielded 65816 virtual-memory machine exists to copy from**, and because WDC's own datasheet enumerates the cases in which an abort modifies state rather than suppressing it.

E.14 `ABORTB` has no timing parameter of its own. It is governed by the generic *processor control* setup and hold times it shares with `IRQB`, `NMIB`, `RESB` and `RDY` – 10 ns each at the 5 V / 14 MHz grade, setup relaxing to 15 ns at the 3.3 V / 8 MHz grade this machine runs at. **Anyone citing a dedicated abort timing symbol is mistaken**; there is none.

The budget is comfortable here and would not be at the top of the range. A 14 MHz cycle is ~71 ns, so with setup and hold the MMU has roughly 35–50 ns after `VDA / VPA` and the address go valid to complete a TLB lookup, a permission check, and drive the pin – tight enough to argue for a registered TLB whose hit or miss is a single combinational term. At 8 MHz a cycle is 125 ns (E.1), and the same decision has nearly twice the room. It is one more reason the clock target is where it is, and it should be re-derived rather than assumed if anyone ever raises it.

E.15 The assertion rules are three, and all three are conditions on *when*, not on *how long*. Assert only once `VDA` or `VPA` says the cycle is a real bus access – the two read `00` during internal dead cycles, and aborting one of those is meaningless. **Hold until `PHI2` falls, then release immediately**, and never hold longer than one cycle. And **never assert while `VPB` is low**: the pin is level-or-pulse sensitive, its internal latch clears on the second cycle of the abort sequence, and an abort asserted during the abort's own vector fetch aborts the abort (E.18).

E.16 In the good case the sequence is register-preserving, and the one detail that makes demand paging possible is which address is stacked. The instruction runs to completion with **no register or memory changed**; then `PB`, then the **aborting instruction's own address** – not the following one – then `SR`; `I` is set, `D` cleared, `PB` forced to `$00`, and `PC` loaded from the abort vector at `$00:FFE8` in native mode (E.12). **RTI therefore returns to the faulting instruction and re-executes it**, which is the whole mechanism: resolve the fault, return, and the access simply happens.

The CPU saves neither `A`, `B`, `X`, `Y`, `DB` nor `DP`, and changes no status bit but `D` and `I`. **The handler preserves them itself**, and that save set is the same audit N.1 needs for the context switch.

Priority is `RESB` > `ABORTB` > `NMIB` > `IRQB`, and abort is **non-maskable**. An `NMIB` asserted at the same moment is taken after the abort handler returns.

E.17 A write that has already driven the bus is not undone, and this is the single most important hardware rule in the design. An abort discards *internal* results; a store cycle that reached the memory with **RWB** low has physically written. So copy-on-write is achievable only if the not-present or read-only decision is available **before the write strobe and gates it** – in the same cycle, in Helium:

WE_to_memory = RWB low AND cycle permitted AND NOT aborting this cycle . Wire it as one equation in one place. Every other protection on sheet L is a permission bit; this one is the thing that makes a permission bit mean anything on a write.

E.18 WDC's caveats enumerate four cases where an abort corrupts state, and they persist in the current datasheet. Aborting a read-modify-write – **ASL** , **LSR** , **ROL** , **ROR** , **INC** , **DEC** , **TSB** , **TRB** – after its modify cycle corrupts **P** , and the write to the target has already happened, so a late abort leaves both a damaged flag register and a partial write. Aborting **RTI** after cycle 3 corrupts **P** . Aborting after cycle 2 of any interrupt, **BRK** , **COP** or abort sequence forces **PBR** – and, in emulation mode, **DBR** – to **\$00** . And the datasheet is explicit that asynchronous aborts may therefore produce undesirable results.

One rule covers all four: assert only on the memory cycle that generates the faulting address, at or before the instruction's last abortable cycle. One cycle late is corruption. The third case is the one hardware must forbid rather than software avoid, and the guard is E.15's: **Helium refuses to abort any cycle with VPB low**, which is exactly the window the abort, interrupt and **COP** sequences fetch their vectors in (L. 12 already watches that pin for privilege).

This is also the substance behind the community verdict that "ABORT is broken". The forum claim is an imprecise summary of the same cycle-level constraints; **where anecdote and the datasheet disagree, the datasheet's per-cycle statements are primary**.

E.19 Restartable is not the same as precise, and only the first is needed. *Precise* means the machine state at the fault is exactly the pre-instruction state; *restartable* means that after the handler fixes the fault, **RTI** re-runs the instruction and gets the right answer. The abort is **restartable for ordinary loads and stores when the write is gated and the fault is detected on the faulting cycle**, and **not precise** for the cases in E.18. Demand paging needs restartability on the instructions that touch pageable memory – which is obtained by constraining *which* instructions may touch it (E.21), not by hoping.

E.20 The abort sequence pushes to the stack, so an abort taken while the stack page is unmapped faults inside the fault and the machine deadlocks. L.11 already pins each process's stack pages for its lifetime, and E.12 already notes the pushes. **This section makes the set explicit and makes it the kernel's job to enforce**: the stack pages, the direct page, the vectors, the abort and interrupt handlers, and any page holding code that performs a read-modify-write or a block move against non-pinned data – all **PIN=1** , and the kernel **refuses to unmap them** rather than merely declining to choose them as victims (L.5, L.13).

E.21 MVN and MVP are forbidden against pageable memory, and the ban is categorical rather than careful. The block moves are designed to be *interruptible* – they decrement the accumulator and step the index registers per byte, so an interrupt resumes them – but that is exactly what makes them hostile here: a move straddles many pages, its per-byte state is mid-instruction, and E.18's corruption windows sit inside it. Large copies go through a software loop of ordinary loads and stores, each individually restartable, or through Helium's own engines.

The rest of the discipline follows the same instinct and belongs in the toolchain rather than in the hardware (sheet 0). **Prefer load → modify in a register → store over a read-modify-write on pageable data**, so a fault lands on a plain access rather than on a modify cycle; where a read-modify-write on pageable data is unavoidable, **pin the target or pre-touch it with a probe load first**. Keep the stack and direct page in pinned physical RAM, which L.11 already requires. And consider a **probe at function entry for large stack frames**, so the fault happens on a benign load rather than mid-instruction.

E.22 ABORTB cannot stop an unprivileged task executing WAI or STP, and that is a privilege gap rather than a paging one. Either untrusted code is not run, or Helium watches the instruction stream and aborts those opcodes at fetch – which is the one documented production use of the pin in another machine, where an FPGA traps opcodes it wants to emulate. **The cheap answer for v1 is a watchdog** (D.7 already holds one), since a halted CPU is indistinguishable from a hung one from outside (→ Q115).

Known ABORT limitations – the audit in one table.

LIMITATION	AFFECTED	SEVERITY	WHAT THIS DESIGN DOES
Abort after the modify cycle corrupts P , and the write has already committed	ASL LSR ROL ROR INC DEC TSB TRB	High	Assert only on the faulting cycle; gate WE ; avoid read-modify-write on pageable data or pin the target (E.21)
Abort after cycle 3 corrupts P	RTI	Medium	Stack pinned (E.20); the abort logic never re-fires during the return
Abort after cycle 2 forces PBR / DBR to \$00 – aborting the abort	IRQ NMI ABORT BRK COP sequences	High	Helium inhibits abort while VPB is low (E.18)
A push during the abort sequence can itself fault → deadlock	Any abort with the stack page unmapped	Critical	Stack, direct page, vectors and handlers permanently pinned, and unmapping them refused (E.20)
Committed bus writes are never rolled back	Any faulting store	Critical	The write-gate equation of E.17
Mid-instruction register state across many pages	MVN MVP	High	Forbidden on pageable memory (E.21)
WAI / STP by unprivileged code cannot be trapped	WAI STP	Medium	Watchdog, or opcode detection at fetch (E.22)

LIMITATION	AFFECTED	SEVERITY	WHAT THIS DESIGN DOES
The in-cycle decision is tight at the top of the speed range	every fault	Medium	8 MHz gives 125 ns (E.14); a registered TLB if that ever changes
The 65C802 has an abort vector but no abort <i>pin</i>	wrong part	–	W65C816S only, as E.1 specifies

E.23 Nobody has fielded this, and that is the honest headline. The Apple IIGS – the highest-volume 65816 machine – does not use the pin at all; its chipset does bank switching, shadowing and speed matching, and every OS on it relies on banking rather than paging. The SNES's CPU is a licensed core plus a DMA-capable memory controller, and abort is not exposed for demand paging. The CMD SuperCPU and the Foenix machines use slot-based bank switching with no paging. **The strongest real deployment of the pin is instruction trapping**, in an FPGA-based 65816 machine that raises abort to catch banned opcodes and emulate them in software – which confirms the pin works, for a different purpose. Everything else is datasheet reading and small experiments; the community's own summary is that the pin is largely unexplored in a practical system. Which is an argument for the bench work below rather than against the design. **The mechanism this machine needs is narrower than the one nobody has proved:** restartable aborts on plain loads and plain stores, with writes gated, and everything else forbidden by construction.

E.24 Three staged bench experiments, and the first is a go/no-go before the OS design is committed. With the board up, the TLB present and virtual memory disabled, put a logic analyser on **PHI2**, **VDA**, **VPA**, **RWB**, **ABORTB** and **VPB**, assert abort on a chosen load, and confirm **the correct address is stacked, RTI re-executes it, and A, X, Y, P, DB and DP are intact**. Repeat for a store and confirm the write was gated – memory unchanged. **If plain load and store aborts are clean and writes are suppressed, proceed; if not, the fallback is a stall-only design with statically mapped banks**, and it is better to know that before P3.k than after (Q116). **Then deliberately do the wrong thing, once, on purpose.** Assert abort one cycle late on a read-modify-write, on **RTI** after cycle 3, and after cycle 2 of an interrupt, and observe the corruption first-hand – to prove that the production logic cannot reach those cases rather than to believe that it does not. Then confirm the **VPB** inhibit holds under a fault taken during a vector pull. The questions only silicon can answer are worth listing where they will be found: whether aborts on plain accesses are precise **100 %** of the time on the actual parts; whether the write gate holds across temperature at the target clock with no partial or glitch write; **the exact last abortable cycle for each addressing mode**, especially the indexed long modes and the page-crossing case that takes an extra cycle; and whether second-source parts, if one is ever substituted, behave identically (E.1 specifies a WDC part for reasons that now include this one).

Candidate cores – should the slot ever be populated.

CORE	NATURE	STANDING
P65816 (srg320)	VHDL, microcoded, cycle-exact	The most complete open option, and the reference this project already names. Basis of the MiSTer SNES core and the Apple IIgs core derived from it. Would be cleaned up and given a flat memory interface.

CORE	NATURE	STANDING
MiSTer Apple IIgs	Adaptation of srg320	Validated in hardware at selectable speeds to 14.3 MHz. That is a <i>system compatibility</i> limit, not the isolated core's Fmax – it says nothing about what the core could reach.
FT816	Accelerator, not cycle-exact	Conceived for speed rather than fidelity, so a better base where cycle exactness does not matter. Needs an instruction-coverage audit before it can be taken seriously.
WDC soft IP	Commercial	Paid, and against the project's open-toolchain principle (A.6).

M Kernel–hardware interface

HELIUM'S CONTROL REGISTERS · COMMAND MODEL · INVARIANTS

SHEET 10

Helium owns three pieces of state ordinary memory accesses cannot reach: the TLB, the active context, and the write-back cache. This sheet is the interface the kernel drives them through – and it sits **strictly below the syscall layer**.

M.1 Three layers, kept deliberately apart: the **syscall ABI** (*COP*, visible to processes) · the **kernel's MMU and cache logic** (C and assembly in bank *\$01*, kernel only) · **hardware control** (stores to bank *\$FF*, this sheet, kernel only).

"Flush the TLB" and "flush a frame" must never become system calls: any process could flush in a loop and degrade the machine – a denial of service. They are steps *inside munmap*, inside the context switch, inside the page evictor. The command encodings therefore live in a numbering space entirely independent of the syscall numbers, and the two must never be aligned.

M.2 Address as opcode. Each command has its own address; the value written is the command's *argument*, not a command code – `sta f:$FF0010` with an ASID in A flushes that ASID's TLB entries. Against the alternative of one command register plus a separate argument register, this removes the value decoder inside Helium (leaving the address comparator that already exists), removes the argument register, reduces every command to a single bus cycle, and kills a whole bug class: there is no intermediate "argument written, command not yet triggered" state for an interrupt to split.

What it spends instead is address space, the cheapest thing this block owns. Every command is a distinct address inside a 256-byte window no user process can reach (M.4), so paying sixteen of them for sixteen commands buys the decoder's absence outright.

One address, one cycle: no half-issued command for an interrupt to split.

M.3 Why memory-mapped and not **WDM**. An earlier proposal captured **WDM** (**\$42**, reserved by WDC, executed as a two-byte two-cycle no-op) and had Helium decode its signature byte. Rejected on four counts: memory mapping **widens a comparator that already exists** for the fault registers, where a WDM snooper needs a new state machine tracking VPA/VDA to tell an opcode fetch from an operand; it **inherits the protection keystone** free, since **\$FF** is already unmappable to user space, where WDM would need its own privilege gate; it is **testable before a kernel exists**, with the RP2354B driving the bus in E1-E3; and it **keeps Argon honest** – a softcore need only implement the documented instruction set, not replicate a private decode table.

WDM stays a reserved no-op and Argon must implement it as one, decoding nothing. The single use it seemed to serve uniquely – a breakpoint insertable into user code, which cannot touch **\$FF** – is already covered by **BRK** and its own vector.

A fifth consequence appears later: memory-mapped, the block is reachable by the debug agent (M.10), where a **WDM** channel would have needed a running CPU.

M.4 The block lives at **\$FF:0000 – \$FF:00FF**, and it is no longer the bank's only tenant. Registers are 16 bits, accessed with **M=0**; byte-granular access is undefined this revision, and unlisted addresses read **\$00** and ignore writes – within Helium's half, **\$FF:0000 – \$FF:7FFF**, not across the bank. The upper half is Neon's, **\$FF:8000 – \$FF:80FF**, across the shared bus, in **\$FF** because it is the one bank a user process can never map and Neon's command processor must not be reachable from one (D36); sheet X claims a 256-byte window per slot at the top of Helium's half, Helium decoding the window and the card decoding inside it (X.7, X.22). Each is a **hole in Helium's decode**: it must not answer there, and on a read releases D0–D7 for the other device to drive – the turnaround an **\$FE** access already performs (→ T.21). Three neighbours keep their own conventions, and two are still unplaced: **power and battery** (sheet S) is byte-wide behind its own 256-byte wall and value-decoded rather than address-as-opcode, clear of the cache, the arbiter and SDRAM so the power path cannot depend on them (→ Q40); **storage** – where the SPI-SD controller becomes reachable from software – is 16-bit and follows this sheet, but is written as offsets until allocated (G.6, → Q22); and a **card's registers are byte-wide**, an eight-bit device on an eight-bit bus – the **M=0** convention above is this block's, not the bank's.

M.5 Two completion classes, split by whether the command touches SDRAM. **Synchronous** – every TLB and context operation finishes within the bus cycle of the write (a flush is clearing validity bits in EBR, a context update is a register load), so there is nothing to poll and the effect is visible to the very next cycle. **Asynchronous** – the cache flushes, which may run for microseconds.

- M.6** Asynchronous commands **must not be implemented by stalling PHI2**, even though a bounded hardware operation nominally qualifies (→ D16). Holding PHI2 low blocks interrupt service, which would inject microseconds of scheduler jitter on every page eviction. The kernel polls **BUSY** instead – the page evictor is already slow kernel code and the spin costs nothing there.
- One case does stall PHI2, and it is worth saying why it does not contradict this.** A CPU read of Neon's **\$FE** aperture holds the clock until **NEON_BUS_BSY** clears (T.18). The difference is the size and the shape of the cost: a cache flush runs for microseconds *on every page eviction*, in the middle of ordinary kernel work, where the jitter is unavoidable and repeated; a VRAM read costs a few hundred nanoseconds on an explicitly diagnostic path, and the alternative is not a **BUSY** poll but no read path at all. The rule this sheet states still holds – **do not stall PHI2 for something the kernel could poll instead** – and the one exception exists because there is nothing to poll (→ D37).
- Gateway rules: **BUSY** must read 1 on the *first* read after the triggering write: a kernel sampling a stale 0 proceeds against a half-flushed frame. A write to an async register while **BUSY** is set is ignored – neither queued nor aborted. Ordinary traffic, the timer and interrupts keep running throughout: slowed, never stalled.
- Nothing covers a flush that never finishes – the one gap this model leaves open.** F.11's watchdog resumes PHI2 and raises NMI or ABORT when a fill overruns, but an async flush deliberately does not stall PHI2: no clock to resume, so that watchdog never fires. A hung SDRAM controller strands the kernel in the **BUSY** spin with nothing stopped, raised or reported. What is wanted is a counter in Helium bounding a flush's run time, escalating to NMI or **ABORTB** with its own **FAULT_CAUSE** code (→ Q74).
-
- M.7** Ordering comes free. The W65C816S has **no prefetch queue and no store buffer**: every bus cycle is architectural and appears in program order, so a store to a command register takes effect strictly after everything before it and strictly before everything after. **No memory barrier is required, and none is provided.** A PTE written to SRAM before a **TLB_INVAL_PAGE** is guaranteed visible to the walker at the moment of invalidation.
-
- M.8** Two invariants the kernel must uphold, both of which fail silently. **Write CTX_SET_PTBASE before CTX_SET_ASID**: an interrupt taken between them still sees the old ASID, whose TLB entries remain consistent with the old table. Reversed, the new ASID is live while the walker still reads the old table – filling the TLB with wrong entries tagged as correct. **Flush an ASID before recycling it**: skip that and the previous process's translations leak into a fresh address space, a protection failure with no symptom until something is corrupted.
-
- M.9** One requirement on the gateway, not an optimisation: **the pinned region must be non-cacheable or write-through**. Page tables live there and the walker reads SRAM directly, so if CPU writes to table memory went through the write-back cache the walker could read a stale PTE. The **NC** flag exists in the PTE format for exactly this (→ F.5).
- It is a gateway rule because software cannot enforce it: a kernel that sets **NC** on every table page still loses if the pinned region is cacheable at all.
-
- M.10** This block has a **second, deliberately asymmetric user**. The debug agent's internal accesses land on the same **\$FF:00xx** address comparator a CPU store does, so every command below can be issued – and its **BUSY** discipline observed – with the 65816 held in reset and no kernel in existence; that is what makes the model testable rather than merely specified. The traffic is one-way: the agent's own register file is SPI-only and **is not mapped here**, because the block that can bypass the protection model must not be reachable through it (R.4).

- M.11 Reaching this block costs more than it looks, and the cheap form is a convention rather than an optimisation.** Absolute long addressing – `sta f: $FF0010` – is 4 bytes and 5 cycles, and the `f:` prefix is not decoration: without it `ca65` narrows the access to 16-bit absolute and the store lands in whatever bank `DBR` happens to hold. Any routine issuing two or more commands should set `DBR` to `$FF` on entry instead and use 16-bit absolute – `sta $0016` then `sta $0014` for a context switch – at 3 bytes and 4 cycles each.
- One of the few places the fixed-`DBR` small-data model (0.3) is deliberately repointed. it is confined to leaf assembly routines that push `DBR` on entry and restore it before returning. The MMU layer repoints it; nothing above the MMU layer does.
- M.12 Invalidation is a separate command from flushing, and it exists because Helium now has a bus master that is not the CPU.** The SPI-SD engine of sheet G writes frames into SDRAM behind the cache, and the CPU's view of those frames must be discarded before the transfer, not after: a *clean* line left behind merely serves stale data, but a *dirty* one is written back on top of the new contents at whatever later moment the cache chooses. `CACHE_INVAL_FRAME` drops both without writing either out. Sizing makes it as cheap as its sibling – F.7's line is exactly a frame, so it is one tag lookup and up to eight validity bits cleared, and it is synchronous for the same reason `TLB_FLUSH_ASID` is.
- It is not `CACHE_FLUSH_FRAME` with the write-back suppressed, and using one for the other corrupts in both directions.** Flushing before a DMA read spends an SDRAM burst writing out data that is about to be overwritten – wasteful but harmless. Invalidating before a DMA write to the card discards the dirty sub-blocks the engine was supposed to send, which is silent data loss. The two rules are stated as a pair in G.9: **card ← memory, flush then write; card → memory, invalidate then read.**
- This command has no user in the design as it stands other than DMA, and that is the point at which to note the general rule: **any future master inside Helium that writes memory without going through the cache needs the same discipline**, and the kernel-side cost of getting it wrong is a corruption that appears only under memory pressure.
- M.13 The fault registers are one set, latched together, and reading them is part of taking the fault.** `FAULT_ADDR_L/H`, `FAULT_CAUSE`, `CONTEXT` and `MODE` are captured on the aborting cycle and hold until the next ABORT overwrites them – no queue, no second bank. The handler's first act is therefore to copy all five into its own frame, before it touches a user pointer: a second fault inside the handler silently replaces the evidence of the first.
- Helium cannot tell a nested fault from an ordinary one, so this is a kernel rule, not a gateway one: no shadow bank, no capture-enable bit to leave set.
- M.14 open One name is doing two jobs.** `MODE` at `$FF:0008` is read-only here and witnesses the fault: bit 0 says the access was in user mode. L.12's `MODE` is live state – the privilege Helium raises on a `VPB` vector fetch and drops on return, writable only while privileged. A latched witness and a running mode bit cannot be one register; they need separate names and addresses.

Register map – base block `$FF:0000`. Helium owns `$FF:0000–$FF:7FFF`; the upper half is Neon's (T.21).

ADDRESS	NAME	ACCESS	MEANING
<code>\$FF:0000</code>	<code>FAULT_ADDR_L</code>	read	Faulting virtual address, bits [15:0]
<code>\$FF:0002</code>	<code>FAULT_ADDR_H</code>	read	Bits [23:16] in the low byte; upper byte reserved

ADDRESS	NAME	ACCESS	MEANING
\$FF:0004	FAULT_CAUSE	read	Bit 0 not present · 1 write to read-only · 2 execute from non-executable · 3 user access to supervisor page · 4 user access to bank \$FF · 5 malformed PTE / walker error
\$FF:0006	CONTEXT	read	ASID active at the time of the fault
\$FF:0008	MODE	read	Bit 0 set if the fault occurred in user mode
\$FF:0010	TLB_FLUSH_ASID	write	Invalidate every entry tagged with the ASID written. Other ASIDs untouched. Synchronous.
\$FF:0012	TLB_FLUSH_ALL	write	Invalidate everything regardless of tag. For global kernel-mapping changes and bring-up – on a hot path it is a design smell. Synchronous.
\$FF:0014	CTX_SET_ASID	write	Set the tag attached to new entries and matched on lookup. Does <i>not</i> flush. Synchronous.
\$FF:0016	CTX_SET_PTBASE	write	SDRAM frame number the walker reads PTEs from. Write this before CTX_SET_ASID. Synchronous.
\$FF:0018	CACHE_FLUSH_FRAME	write	Write back every dirty sub-block of the given 2 KB frame, then mark clean. Required before evicting a page to the card. Asynchronous.
\$FF:001A	TLB_INVAL_PAGE	write	Invalidate one translation for the active ASID – the cheap path for single-page munmap and for permission downgrades such as marking a page read-only for copy-on-write. Synchronous.
\$FF:001C	CACHE_FLUSH_ALL	write	Write back all dirty lines. Shutdown, suspend, bring-up. Asynchronous.
\$FF:001E	MMU_STATUS	read	Bit 0 BUSY – an asynchronous command is running · bit 1 WALK_ERR – the walker met a malformed PTE · 15:2 reserved
\$FF:0020	CACHE_INVAL_FRAME	write	Discard every sub-block of the given 2 KB frame without writing it back , dirty or not. Required before a DMA engine deposits data into that frame – see M.12. Synchronous.

F Physical memory structure

HOW CPU · FPGA-A (HELIUM) · SRAM · SDRAM RELATE

SHEET 11

All system memory lives behind FPGA-A (Helium): the CPU never sees SRAM or SDRAM directly, only virtual addresses translated by the MMU and served by the cache. The SRAM sits on the CPU's own nets, the SDRAM behind the FPGA – an asymmetry that falls out of one property of the page size.

F.1 Hierarchy: registers → **SRAM cache** → **SDRAM** (physical frames) → microSD (swap, post-v1). The SRAM is a hardware cache with tags, **not** a tier of allocatable frames – see L.3 for why that distinction is forced.

F.2 The key insight: with 2 KB pages, **A0–A10 are never translated** – the physical address is `FRAME : offset`, and the offset comes straight from the CPU. So both the data bus *and* the low address bus can be shared nets: the SRAM's address bus *is* the physical address bus, split at the page boundary, and Helium drives only the translated half.

D0–D7 → CPU net (0 pins) · A0–A10 → CPU net (0 pins) · A11–A19 → Helium (9) · /CE, /OE, /WE (3). Thirteen pins for 1 MB, against 41 for the ×16 part it replaces.

The same property bounds the page size from below. At 1 KB the offset would stop at A9, A10 would become a translated bit, and the SRAM's address bus would stop being the CPU's own – one more Helium pin, and the argument above stops being free.

F.3 What makes the sharing work is the half-cycle pipeline. **PHI2 low**: the CPU drives A0–A15 and the bank byte on D0–D7; Helium latches the bank **internally** – **no external '573 is needed**, decodes it, and – if it is not `$FF` – forms the 24-bit address, does the TLB lookup (~10 ns in EBR) and prepares A11–A19 and /CE. **The bank decode comes first, before any translation or tag lookup**, so an `$FF` access short-circuits straight to the I/O bus: it never evicts a hot line to read a status register, and its latency is fixed. **PHI2 high**: /CE asserts and the SRAM has the whole high phase (~62 ns at 8 MHz) to deliver data straight to the CPU, with no FPGA in the data path.

Two dividends: ABORTB gets half a cycle of margin, since translation and permission checks finish before the rising edge; and the SDRAM leaves the critical timing path altogether.

F.4 Price of the shared nets: Helium **cannot reach the SRAM on its own**. Page-table walks, SDRAM→SRAM fills and the RP2354B's BIOS preload all require taking the bus – RDY low to freeze the CPU, BE low to tristate it, then Helium drives A0–A10 and D0–D7 through pins it already has. No pin cost, real gateway cost. Also: three loads (CPU, Helium, SRAM) now hang off D0–D7 and A0–A10. This is the most layout-critical net on the board.

- F.5** SRAM 1 MB ×8 (AS7C38096A-10TIN, TSOP-II 44, 3.3V, industrial), partitioned: **256 KB fixed and uncached** – kernel, active page tables, bank \$00, interrupt vectors; directly addressed, never evicted, and where the PIN PTE bit applies – plus **768 KB of cache**: 384 lines of 2 KB, **4-way set associative**, tags (~1.5 KB) in EBR beside the TLB.
- 10 ns is mandatory: /OE must be deasserted while PHI2 is low, since the bus carries the bank byte then. Direct mapping was never an option – 32,768 physical lines competing for 384 positions is 85:1. open the 256 KB may not hold the resident tables K.2 promises (→ Q9).
- F.6** Fill granularity: a whole 2 KB line over the 8-bit shared bus costs ~41 µs, plus SDRAM read, plus write-back of a dirty victim – 50–100 µs with interrupts blocked, which is unacceptable. Fills therefore move **256 B sub-blocks**, with valid and dirty bits per sub-block and one tag per 2 KB line: ~6 µs per fill, healthy interrupt latency, and the part of the page never touched is never transferred.
- The ~6 µs is Helium's own clock, not PHI2's: the fill runs at core speed with the CPU stopped, so the figure scales with the shared bus's width, not the CPU's 8 MHz.
- F.7** One consequence of sizing the tagged line at 2 KB: **a cache line is exactly a frame**, and both are 2 KB-aligned. So the "flush everything cached for frame N" operation the kernel needs before paging a frame out (→ L.16) collapses to a single tag lookup and up to eight dirty sub-block write-backs.
- Helium must expose that operation. It is cheap now; when the line was 256 B the same job meant sweeping eight separate lines.
- And it must expose the same operation without the write-back**, for frames the SD engine is about to overwrite. Same tag lookup, same sizing argument, opposite treatment of the dirty bits (→ M.12, G.9).
- F.8** SDRAM 64 MB (AS4C32M16SB-7TIN, 32M×16, 512 Mbit, TSOP-II 54) – 32,768 frames of 2 KB, one device on Helium and an identical one on Neon. 512 Mbit is where SDR silicon tops out; no 1 Gbit part exists in volume.
- The 256 Mbit W9825G6KH is a drop-in fallback: same footprint, same 13 address lines, one column-address bit fewer – a controller parameter, not a board change. Alternates: IS42S16320F-7TLI, IME5116SDBETG-75IA.
- F.9** Off the critical path, the SDRAM controller stops being a low-latency backing store and becomes a **DMA and paging engine**. Substantial de-risking for bring-up: its timing no longer gates the CPU.
- Its latency is now paid once per miss instead of once per access, which is the whole of what moving it off the critical path bought.
- F.10** Architectural ceiling: 128 MB physical (16-bit frame in the PTE), **deliberately not populated**. A second device would double the RAM the 65816 can barely touch – at 8 MHz it sustains ~4 MB/s, so merely zeroing 64 MB takes ~16 s. A second /CS is routed to an unpopulated footprint and left there.
- The half that is not populated now has a claimant, and it is not memory.** X.8 gives a range of those unreachable frame numbers to the expansion slots, so a PTE can name a card instead of the SDRAM with the TLB and the permission bits still in front of it. The ceiling stops being purely theoretical without a single frame of DRAM being added – and the frames spent that way are frames the machine could never have populated anyway.

F.11 Bring-up requirements that come with the stall (→ L.4): SDRAM auto-refresh must be interleaved into the fill state machine; a **watchdog** in Helium must resume PHI2 and raise NMI or ABORT if a fill overruns, since a hung fill freezes the machine with no clock and no diagnostic path; clock gating must be glitch-free and the first pulse on resume must meet the minimum PHI2 high width; RDY is bidirectional – the CPU pulls it low on **WAI** – so open-drain with pull-up, never driven hard high.

Free bonus: a stopped clock is literally the low-power mode.

A third trigger arrives with the expansion slot. A card that asserts **/EXP_BSY** and then dies stalls the clock exactly as an unfitted Neon does, so the same counter must resume PHI2, return **\$00** and raise the fault – with a cause code naming the slot, since "something timed out" is not a diagnosis when the something is user-installed (X.12). One timeout, three clients, still no number (Q57).

That watchdog now covers a second client, and it is what makes Neon's read path safe to build. A CPU read of the **\$FE** aperture stops PHI2 until Neon deasserts **NEON_BUS_BSY** (T.18); if no answer arrives inside the window, Helium must resume the clock, return **\$FF** and raise the fault exactly as it does for an overrunning fill. Without it, a Neon that is unconfigured, held in reset, or **simply not fitted yet** – which E1.4's staged population guarantees – freezes the machine with no clock and no way to say why. Same timer, one more trigger, and the timeout value is still unassigned (→ Q57).

F.12 Neon's 64 MB = **chip RAM** (framebuffers, image atlases, pre-rendered level buffers, command lists and audio DMA buffers), outside the CPU hierarchy and reached through the **\$FE** window. At 103.125 MHz on a 16-bit bus that is **206 MB/s peak, ~150 MB/s effective**, of which scanout takes under 3 % – so **bandwidth is not a design constraint for Neon; logic is** (→ T.13, T.3).

The PSRAM is gone from the whole board, and with it every QSPI interface.

Neon has a second memory, and the division between them is strict. Its 32 EBR blocks – 16 KB, fully allocated – hold everything that must be reachable with no wait state and everything scanout needs at single-cycle latency: the 8 KB text buffer, the 4 KB font, the palette, the scanout line buffers, the blitter's burst buffers and the command FIFO. The SDRAM holds everything large and nothing latency-critical. That split is what lets Mode 0 exist with the memory controller unwritten (→ T.11).

F.13 Layout note: SRAM and SDRAM in a short comb hugging Helium – the SDRAM side runs at ~100 MHz and the shared side has three loads; neither tolerates long stubs.

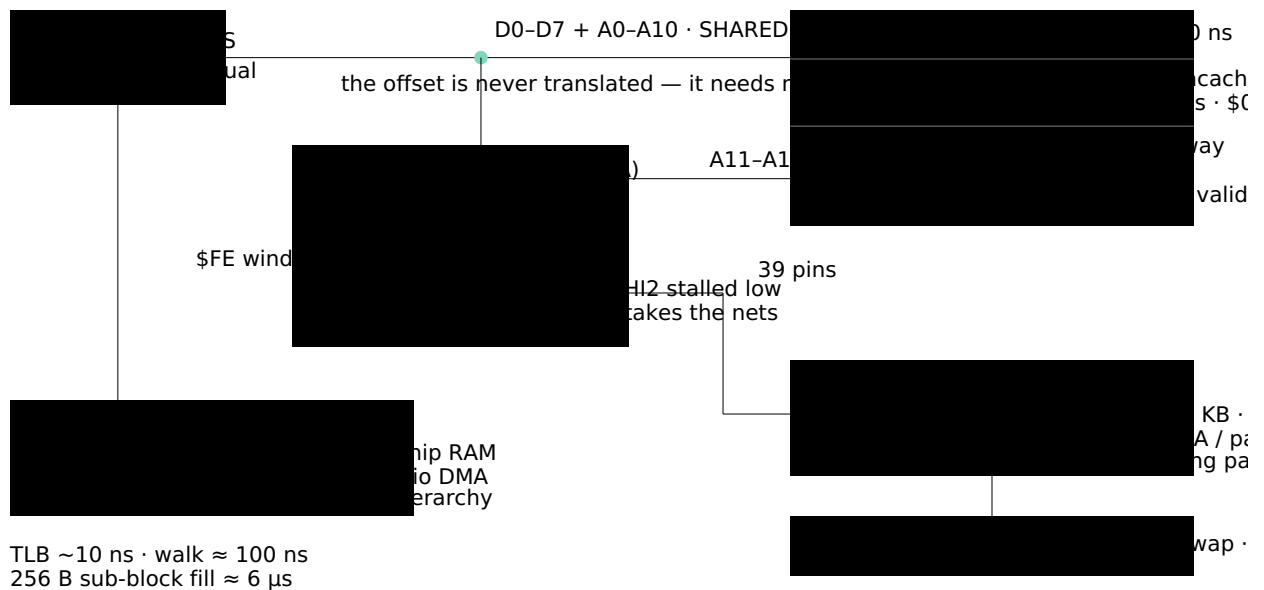


Fig. 4 – Memory structure. The SRAM hangs off the CPU's own data and low-address nets; Helium drives only the translated half, and owns the SDRAM alone.

Explanatory groundwork, not normative: it explains paging from first principles and gives the reasoning behind the field widths, so that the machinery in sheet L can be read without stopping to decode acronyms. Term-by-term definitions live in the glossary, sheet Z1. It introduces no design decision – where it disagrees with an architecture sheet, the architecture sheet wins.

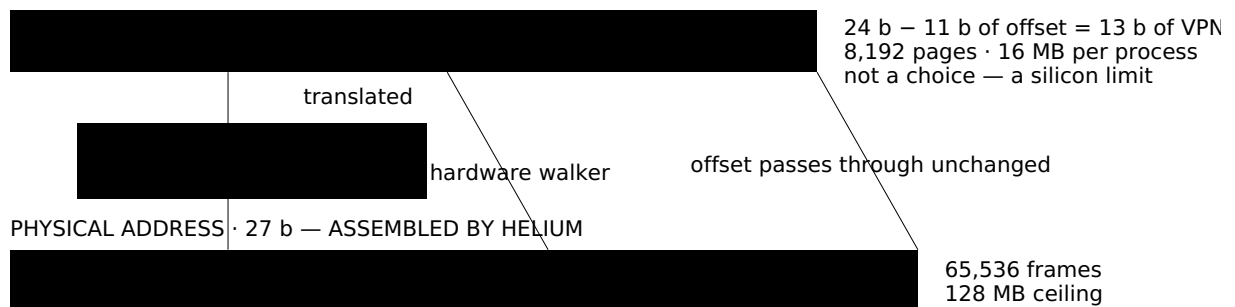
- K.1** Why it exists – three problems it solves at once: **relocation** (the program no longer needs to know what address it will be loaded at), **protection** (no process can touch another's memory or the kernel's) and **fragmentation** (free memory no longer breaks into holes nothing fits into). The process lives in a fiction: it believes it owns the full 16 MB, starting at zero.
-
- K.2** The core idea: the address space is cut into fixed-size chunks – **pages** on the virtual side, **frames** on the physical side, both 2 KB. Translation answers a single question: which physical frame holds virtual page N? The offset within the page is never translated; the low bits pass straight through.
Fixed chunks trade one waste for another: external fragmentation goes, internal arrives – 1 KB wasted per allocation on average, and no compaction is needed.
-
- K.3** The translation path in one line: virtual address → **TLB** lookup (ASID + VPN together, so another process's entry can never produce a false hit) → on a hit the frame is available immediately, on a miss the **hardware walker** reads the PTE from SRAM in ≈100 ns.
A TLB miss is not an error – it costs one table walk. Detail and figure in sheet L / Fig. 6.
-
- K.4** Why a TLB is not optional: translation is itself a memory lookup, so without a cache of translations every access would cost two – read the PTE, then do the access it was for. The TLB settles the common case in a cycle and only a miss pays the walk. What the page size buys is its **reach**, the working set one TLB-full of entries covers: 32 entries × 2 KB = **64 KB**, against 8 KB had pages stayed at 256 B.
Reach is entries × page size: 64 × 2 KB = 128 KB of working set before entries evict each other. Striding through more costs a ~100 ns walk per access.
-
- K.5** Only two situations are genuine failures: **P=0**, the page is not in physical memory – recoverable, the kernel allocates a frame, fills it, writes the PTE and retries (this is the *page fault*, deferred while v1 loads eagerly); and a **permission violation** – a write with W=0, a fetch with X=0, a user access with U=0 – which is a program error with no retry. Both ride the ABORTB pin.
Everything else that looks like a failure is routine: a TLB miss costs a walk, and a write to a copy-on-write page is a violation the kernel *resolves* rather than reports.
-
- K.6** Why FRAME is 16 bits against VPN's 13 – the two fields measure different things: **VPN measures how much memory one process sees** (set by the CPU), **FRAME measures how much memory the machine has** (set by the MMU design). There is no reason for them to match, and here they deliberately do not.

K.7 The virtual side is not a choice: the 65816 emits 24-bit addresses, so 24 – 11 of offset = 13 bits of VPN. The number falls out of a subtraction. The physical side is a choice: Helium assembles the physical address after translation and drives the memory bus itself, so only the FRAME width sets the ceiling. **The MMU breaks the 65816's architectural barrier at system level without breaking it at process level.**

K.8 The 3-bit margin is what makes the multitasking real: 128 MB ÷ 16 MB = up to **8 complete virtual address spaces resident simultaneously** in physical memory – exactly what preemptive multitasking with resident processes needs. Eight is a limit on how many address spaces are **resident**, not on how many processes exist. More can exist and be scheduled; they cannot all keep their pages in physical memory at once, which is what swap is for.

K.9 **Why 16 and not 15 – where the width stops being arithmetic and becomes a choice.** The PTE splits into exact half-words and the upper one was already full: the flags are eight (P W X U D A N C P I N) plus eight (SW[7:0]), sixteen bits precisely, so the lower half-word was going to be underused whatever FRAME did. Rounding it up to 16 costs nothing and buys the 128 MB ceiling. **Fifteen would not have been the cheaper answer but the tighter one:** 64 MB of SDRAM is 32,768 frames, exactly 2^{15} – the memory actually fitted lands precisely on the fifteen-bit ceiling, with nothing left over. The same split earns its keep a second time in the walker. With FRAME in the low half-word and the flags in the high one, Helium reads 32 bits from SRAM and sends the lower sixteen to the address bus while the upper sixteen go to the permission logic – in parallel, with no shifting and no masking cycle. L.5 fixes the format; this is why the cut falls where it does (→ L.8).

VIRTUAL ADDRESS · 24 b — EMITTED BY THE W65C816S



FRAME 16 b × 2 KB = 65,536 frames = 128 MB ceiling — a design choice, not a CPU limit.

Page table: 8,192 PTEs × 32 b = 32 KB per process · physical pool fitted today: 64 MB = 32,768 frames.

128 MB ÷ 16 MB = up to 8 complete virtual address spaces resident at once — the headroom preemptive multi

Fig. 5 – Field-width asymmetry. The virtual split falls out of the CPU's 24 bits; the physical one is chosen by the MMU, and the offset crosses untranslated.

Summary of numbers – every value the two virtual-memory sheets settle on, in one place.

PARAMETER	VALUE
Page size	2 KB
Virtual address	24 bits – 16 MB per process
VPN	13 bits – 8,192 pages

PARAMETER	VALUE
Offset	11 bits
PTE	32 bits
FRAME field	16 bits
Physical address	27 bits – 128 MB ceiling
Page table	32 KB per process (8,192 × 4 B)
Physical pool fitted	64 MB SDRAM = 32,768 frames
TLB entries	32–64, in Helium's EBR
TLB reach	64 KB with 32 entries · 128 KB with 64
Page walk cost	≈ 100 ns – 4 SRAM accesses at 8-bit width

L Virtual memory management

FULL PAGING · TLB · WALKER · STALL VS ABORT

SHEET 13

Every process sees a clean, contiguous 16 MB; Helium's MMU does the rest. The 65816's ABORTB pin turns this into real memory protection – but only for the faults that genuinely need the kernel; the frequent case never reaches software at all. Vocabulary and first-principles background in sheet K.

L.1 Virtual space: 16 MB per process (24-bit), full paging with 2 KB pages → 8,192 pages (13-bit page + 11-bit offset). Page size is decoupled from the **256 B fill granularity** – they are two separate knobs. The tagged cache line, at 2 KB, does coincide with the page, and that coincidence is worth something (→ F.7).

Three separate knobs: the page is what the MMU maps, the sub-block what a fill moves, the tagged line what the cache tracks. Only the first is written into the PTE.

L.2 Flat table: 8,192 PTEs × 32 bits = **32 KB per process**. Active tables live in the 256 KB fixed region of SRAM (→ F.5), alongside the kernel, bank \$00 and the vectors; inactive ones can live in the kernel's pageable memory. open
How many tables stay resident once the kernel and bank \$00 have taken their share is unresolved (→ Q9).

L.3 **The SRAM never migrates a frame.** Were a fill to move a page from an SDRAM frame into an SRAM frame, hardware would have to rewrite the PTE – which breaks `mshare` outright (a shared page has several PTEs, in different process tables, all pointing at the same frame; gateway cannot find and update them all) and leaves every other cached TLB entry holding a stale frame number. So the SRAM is a cache with tags, invisible to the page tables: **the PTE always names the SDRAM frame and never changes.**

- L.4** Fault taxonomy – the distinction the gateway must enforce. **Bounded:** page resident in SDRAM but not in the SRAM cache → hardware fill with PHI2 stalled, μ s range, no software involvement. **Unbounded:** page not present (zero-fill, swap from SD), permission violation, copy-on-write → **ABORT** to the kernel. Stalling PHI2 on a fault that needs the kernel deadlocks the machine instantly: the kernel is the only thing that could resolve it, and with the clock stopped it cannot run. The split is also a split of owners: the bounded case is resolved inside Helium and the kernel never learns it happened – bar the watchdog catching a fill that never finishes (F.11) – and only the unbounded case costs an exception.
-
- L.5** 32-bit PTE (finalized format): $\text{FRAME}[15:0] + P \cdot W \cdot X \cdot U \cdot D \cdot A \cdot \text{NC} \cdot \text{PIN} + \text{SW}[7:0]$. 27-bit physical address → 128 MB ceiling. Division of labour inside it: **P, W, X, U and NC are checked by hardware; D and A are set by hardware** – D by read-modify-write on the PTE in SRAM, A for the OS to read and clear when choosing swap victims; **PIN and SW[7:0] are software-only** and the walker ignores them. And one reuse worth knowing: **when P=0 the whole FRAME field is free**, since there is no frame to name – the OS stores the page's swap slot on the card there.
-
- L.6** Physical frame map – the address space the FRAME field indexes is not all SDRAM. The 32,768 SDRAM frames sit at the bottom ($\$0000\text{--}\$7FFF$) and the 128 frames of the pinned SRAM region at the very top ($\$FF80\text{--}\$FFFF$), with everything between reserved for expansion. So "pinned" is a physical address range, not a cache line marked non-evictable: an access there bypasses tag lookup entirely and completes in one cycle at fixed latency (→ F.5).
-
- L.7** TLB: 32–64 entries in EBR (the iCE40's 128 Kbit isn't enough for full on-chip tables), tagged by **ASID** → no flush on context switch. Reach: 64–128 KB of working set (→ K.4). Cache tags share the same EBR. ASID tagging removes the flush from the context switch but not from the *recycling* of an ASID: reusing a tag without flushing its entries hands the new process the old one's translations, silently (M.8).
-
- L.8** TLB miss → **hardware walker**, ≈ 100 ns: four SRAM accesses rather than two, since a 32-bit PTE is read eight bits at a time. Irrelevant against a 64-entry ASID-tagged TLB. Real faults → **ABORTB** with exact instruction retry; VPA validates X permission; VPB marks privilege transitions. **"Exact instruction retry" is a property the gateway has to earn, and sheet E is the audit of what it costs.** Three of its rules land here rather than there. The abort decision must be a same-cycle combinational function of the translated address, asserted only once **VDA** or **VPA** is high (E.15). **The memory's write enable must be gated in that same cycle**, because the CPU never rolls back a store that has already driven the bus – which is what makes copy-on-write possible at all (E.17). And **Helium must refuse to abort any cycle with VPB low**, or an abort taken during the abort's own vector fetch corrupts **PBR** (E.18). It is the same pin L.12 already watches, for a second reason. Removing the frequent case from the ABORT path is a large de-risking: the 65816's ABORT has a deserved reputation for being delicate around instruction restart, and it is now exercised only where the process may be terminated anyway. The walker reads SRAM directly, not through the cache, which is why the table region has to be uncached or write-through: a PTE the CPU wrote through a write-back cache would still be sitting there when the walker went looking for it (M.9).

- L.9** The kernel reads the fault in bank `$FF` registers: `FAULT_ADDR · FAULT_CAUSE · CONTEXT · MODE`.
They are a single set, latched together on the aborting cycle, so copying them is part of taking the fault rather than something the handler can defer (M.13).
-
- L.10** Virtual bank map – **architecture, not a proposal**: `$00` per-process stack + DP plus a minimal privileged zone (vectors, trampolines, kernel stack – a 65816 requirement: `PBR=0` on interrupt entry) · `$01` kernel · `$02-$FD` user · `$FE` VRAM window · `$FF` privileged I/O, **not mappable to user space** – the self-protecting key of the scheme.
That last clause is load-bearing beyond the MMU, and sheet T is where it earned its keep a second time. Neon's control registers were moved *out* of the `$FE` window and into `$FF:8000` for exactly this property: a process can legitimately be granted a framebuffer mapping, and if the command processor's registers sat inside that window it could aim an engine with no MMU in front of it at all 64 MB. **`$FF` being unmappable is what turns that from a rule the kernel must remember into something the architecture forbids** (→ T.21, D36). Bank `$FF` is consequently no longer Helium's alone – its upper half is physically on the other side of the shared bus.
Everything downstream links inside `$02-$FD` (→ N.4); the boundaries do not move to accommodate a toolchain. Stated as a rule the gateway can check in eight bits: **a user process may never hold `DBR = $FF` or `PBR = $FF`**.
-
- L.11** Bank `$00` in practice – why the kernel cannot leave it wholly to the process. Two hardware facts force a kernel footprint there. **Native-mode vectors sit at `$00:FFE4-$FFEE` and the CPU forces `PBR=0` on entry**, so every handler *begins* in bank `$00`: hence one or two privileged 2 KB pages holding the vectors plus short stubs that *JSL* into the real kernel in bank `$01`. And **the interrupt frame is pushed onto the current stack before the kernel can intervene**, which settles two things. The page *S* points into must be present unconditionally – an ABORT during interrupt entry would be a fault inside the fault mechanism, with no way back – so **each process's stack pages are pinned for its whole lifetime** and never swapped. And the stub must switch *S* to a kernel stack, also inside those privileged pages, before doing any real work, so the kernel neither depends on the user stack nor leaks its own data back on return.
-
- L.12** How privilege is acquired and lost – the piece that makes the rest enforceable. Entry to the kernel is always through *COP* (or an interrupt), which vectors to a known address, and the 65816 asserts *VPB* while fetching that vector. Helium takes *VPB* as the privilege transition: it raises privilege on the vector fetch and drops it again on the return to user code.
The *MODE* register that holds this state lives in `$FF` and is therefore writable only while already privileged. That circularity is deliberate and is the whole scheme: were *MODE* reachable from user space, a process could simply grant itself permissions, and every other protection in this sheet would be decoration.

- L.13** Physical allocator: frame bitmap over the SDRAM (32,768 × 2 KB). PIN marks non-evictable frames; NC marks uncached zones (I/O) – including the 256 KB fixed region, which is addressed directly and never cached.
- The bitmap costs 32,768 bits – 4 KB – which is small enough to live in the pinned region beside the tables it serves. PIN and NC are both software-only in the PTE (L.5): the walker ignores them, so nothing in hardware enforces what the allocator promises here.
- Which makes the pinned set a kernel invariant rather than a hardware one, and E.20 is the list it must hold.** Stack pages, direct page, vectors, and the abort and interrupt handlers are PIN=1 for the process's whole lifetime, and the kernel **refuses an unmap of them** rather than merely never choosing them as swap victims. The failure mode if it does not is not a corrupted page: it is an abort taken during the abort sequence's own stack push, with nowhere to go.
-
- L.14** ABORT handler: legitimate heap/stack growth → allocate a frame and retry; copy-on-write → duplicate and retry; anything else → kill the process. v1 uses eager loading; demand paging and swap to SD are post-v1.
- The handler saves its own registers.** The CPU pushes PB, the faulting address and SR, and nothing else – not A, not X, not Y, not DB, not DP (E.16) – so the abort path carries the same save-set question as the context switch (N.1) and had better answer it the same way.
- Copy-on-write is the case that depends entirely on E.17's write gate.** A page marked W=0 faults on the store, the gate suppresses the write, the handler copies the frame, sets W=1 and returns – and the retried store lands on the copy. Without the gate the original has already been modified before the handler runs, and the whole scheme silently does the opposite of what it promises.
-
- L.15** Consequence for the kernel: **once PHI2 can stop, cycle counts stop being time.** The scheduler timer must be a free-running counter in Helium on its own clock, never derived from CPU cycles, and any delay loop written as a cycle count becomes invalid.
- That applies to code written before the stall existed as much as to new code: a delay loop in the BIOS or a driver has to come from the timer, and anything measured by counting cycles is measuring a quantity the machine no longer has.
-
- L.16** Swap-out rule, for when demand paging arrives: the cache sits *in front of* the SDRAM, so a frame written straight to the SD card would be the stale copy.
- Evict a page → flush the frame's dirty sub-blocks to SDRAM → only then write it to the card.** Getting this order wrong corrupts silently, and only under memory pressure. The hardware side of it is F.7.
- The page-in direction has a mirror rule and it is not the same command.** A frame the SD engine is about to fill must be **invalidated** first, not flushed – a dirty line surviving over the destination is written back on top of the new page later (G.9, M. 12).

24-BIT VIRTUAL ADDRESS + ASID

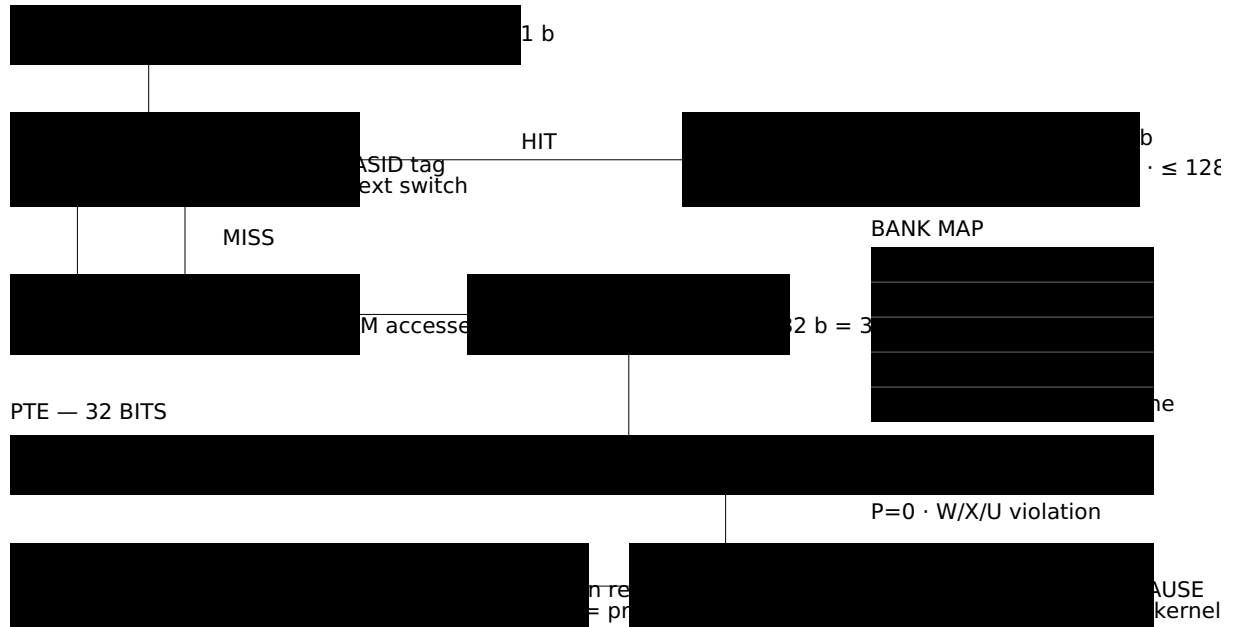


Fig. 6 – Address translation: TLB with ASID, hardware walker over the flat table in SRAM, and the ABORTB fault path.

A single slot, two owners at different times. It's also the only boot path: the dedicated BIOS flash was removed from the design.

G.1 Connection: the slot hangs off an **SN74CBTLV3257** mux between the RP2354B and Helium's SPI-SD controller, with a handoff handshake.

Why the card is switched rather than simply left with the EC, which is the cheaper arrangement and was examined as a way of recovering pins on Helium. Swap is what forbids it: the page evictor writes a frame out and the fault path reads one back, so the card needs a direct route into Helium, and giving it to the EC instead puts a microcontroller and its link in the middle of the page-fault path. The mux, the handshake and Helium's own SPI-SD controller are what that costs, and they were accepted on those grounds rather than on pin count (→ L.16).

G.2 At boot: the RP2354B owns the SD – it reads **BIOS.BIN** and preloads it into the pinned SRAM zone (steps 5–6 of Fig. 3). It then switches the mux to Helium before releasing reset.

The card's format is now specified and it is not FAT. The system volume is NVFS (sheet Y2) in an MBR partition of type **0x7F**, whose first eight blocks are reserved so that a boot image can sit at the front of the partition with no hole in the filesystem (Y2.5). The EC therefore needs only enough of a reader to find one file – or, if the boot image is placed in that reserved area, none at all.

G.3 In normal use: the 65816 accesses it through Helium's SPI-SD via the kernel's block driver. **The BIOS mounts NVFS read-only to load **KERNEL.BIN**** – a minimal reader that neither verifies checksums nor replays the journal, which Y2.20 deliberately makes possible by putting both of those in the read-only feature mask; the OS mounts it read-write behind sheet Y1, and adds ext2 for anything that arrived from a host (sheet Y3).

Read-only in the BIOS is deliberate: less code before a kernel exists, and no way for a half-built boot path to damage the card it booted from. **The format enforces it rather than trusting it:** a read-only mounter that meets a dirty volume or a non-empty journal must refuse, so a BIOS cannot quietly read a card that needs recovery (Y2.20).

G.4 Card contents: **/BIOS.BIN** · **/KERNEL.BIN** · user tree (programs, data) · future reserve for swap. All of it in the NVFS partition, whose reserved head also holds whatever the EC needs before a filesystem exists (Y2.5).

G.5 Recovery: the RP2354B can reclaim the SD at any time and be driven over its console UART (D.4) – update BIOS/kernel without removing the card.

G.6 **The card needs a register block in Helium, and it is the peripheral block Q22 has been holding space for.** Everything above describes a path from the slot to memory; none of it is reachable until the 65816 has a control surface for it, and the SPI-SD controller of B.2 is that block's first tenant. It sits behind its own 256-byte wall in **\$FF:0000–\$FF:7FFF** like every other block in the bank, registers 16 bits and accessed with **M=0**, following sheet M's conventions – **and it costs no pins:** the decode is a widening of a comparator that already exists on the CPU bus (M.3). The pins it does cost are the SPI-SD lines themselves, and those are already open and blocking as Q8. Its base address is unassigned along with the rest of the map, so the register table below is written as offsets (→ Q22, Q40).

G.7 Two data paths, both present, sharing one set of registers. The block exposes a **FIFO** – the CPU reads 512 bytes through `SD_DATA` – and a **DMA engine** that deposits blocks straight into SDRAM as a client of Helium's arbiter. This is not indecision: they arrive at different stages and answer different needs.

Why the FIFO is not enough on its own. At 8 MHz a copy loop costs 12–15 cycles per 16-bit word, so moving a 512-byte block through the accumulator is ~3,500 cycles – **roughly 440 µs with the CPU at 100 %**, against the ~164 µs the SPI link itself takes at 25 MHz. The transport costs more than the medium, and it costs it in the one resource the machine has least of. Worse, a FIFO-only design **retracts G.1**: the whole argument for the mux, the handshake and a controller inside Helium was that the page evictor and the fault path need a direct route between the card and a frame. If every byte has to pass through the CPU anyway, the card could have stayed with the EC and Helium could have kept the pins.

Why the FIFO exists at all, then. It touches neither the arbiter nor the cache, so it is a page of gateway rather than a subsystem, and it is what makes E6 and P5.j – mount the volume, load `KERNEL.BIN`, list a directory – reachable before the DMA engine is written or the coherence rule of G.9 is implemented. Bring-up uses it; demand paging cannot.

G.8 The DMA engine addresses physical memory, never virtual. It is not behind the MMU: no TLB lookup, no walker, no permission check. The destination is a frame number plus an offset, and the kernel – which owns the page tables – translates and pins the buffer before writing the registers. A 512-byte block never straddles a 2 KB frame provided the destination is block-aligned, which the block driver enforces rather than the hardware.

This is also the protection argument. An engine that took a virtual address would need an ASID, a walker port and a permission check to avoid becoming a way for a process to write outside its own address space; one that takes a physical frame is unreachable from user code by construction, because the registers live in bank `$FF` (M.1).

G.9 Coherence: the read direction needs the mirror of L.16, and it is the sharper of the two. The cache sits in front of the SDRAM, so an engine writing a frame behind it leaves the CPU's view stale – and a *dirty* line for that frame is worse than stale, because it will be written back on top of the DMA'd data at some arbitrary later moment. **The rule is: invalidate the destination frame, then start the transfer.** Not flush – the contents about to be overwritten are worthless, and writing them out first wastes an SDRAM burst to no purpose. M.9 already covers a destination in the pinned region, which is non-cacheable by requirement; this is the SDRAM case. The command is `CACHE_INVALID_FRAME` and it did not previously exist (→ sheet M's register map).

The two rules are worth stating together, because they are asymmetric and the asymmetry is the part that gets misremembered. **Card ← memory: flush the frame, then write (L.16).** **Card → memory: invalidate the frame, then read.** Both fail silently, both fail only under memory pressure, and F.7's line-equals-frame sizing makes each of them a single tag lookup.

G.10 Completion is a `BUSY` bit and an interrupt, and the transfer must not stall PHI2. M.6 forbids stalling the clock for something the kernel could poll, on the grounds that microseconds of scheduler jitter on every page eviction is not payable; an SD transfer is hundreds of microseconds, an order of magnitude further past the line. Unlike a cache flush, though, it should not be polled either: the block driver blocks a process for the duration anyway, so completion raises an IRQ through Helium's PIC and the scheduler runs somebody else. `BUSY` remains readable for the BIOS and for bring-up, where no scheduler exists to hand the time to.

Gateway rule, the same one M.6 states for asynchronous commands: `BUSY` must read 1 on the first read after the triggering write, never lag, and a trigger written while `BUSY` is set is ignored – it neither queues nor aborts.

Which is also the argument for the interrupt over the poll: a block is ~164 μ s of SPI, so a driver that spins burns a quantum watching a bit where one that blocks gives the time back (X.25 argues the same one tier down).

TEST ▶ a DMA read of 64 blocks completes while the CPU runs a counter loop throughout, and PHI2 never stops.

G.11 Card initialisation is software, not a state machine. `SD_CMD`, `SD_ARG_L` / `SD_ARG_H` and `SD_RESP` expose a raw SD command, and the `CMD0` · `CMD8` · `ACMD41` · `CMD58` sequence, the voltage negotiation and the SDHC-versus-SDSC determination are all driven by the kernel through them. Helium's gateway does the SPI shifting and the block engine, nothing above that – which is B.2's rule that intelligence lives in software applied to the one peripheral most tempting to violate it for.

It is also not optional. G.2 hands Helium a card the EC has already initialised, so the fast path can skip the sequence – but after a G.5 reclaim, or a card swap, or any resume in which the EC touched the slot, the 65816 has to be able to bring a cold card up with no EC involvement. The addressing mode matters beyond initialisation: SDSC takes a byte address and SDHC a block number, and `SD_CARD` publishes which, so the driver does not have to guess.

Sheet Y2 makes one request of this block and it is short: the natural transfer unit is four consecutive sectors – 2048 bytes, one NVFS block, one MMU page – into a 2048-byte-aligned buffer. `SD_DMA_READ` and `SD_DMA_WRITE` already take a block count, so satisfying it costs nothing but issuing a multi-block command rather than four single-sector ones; four transfers with per-command overhead is measurably worse than one burst, and the difference multiplies through every figure in Y2.23's cost model. It is much cheaper to design in before this map is frozen than to add afterwards (Y2.24).

`SD_CTRL` bit 3 is part of the same story: a cold card runs at ≤ 400 kHz until it has answered, then moves to full speed. A state machine would have hidden that transition; software owning it makes the change one register write.

TEST ▶ a cold card the EC has never touched, brought from power-on to `SD_CARD.initialised` by the 65816 alone.

Register map – the storage block. Offsets from a base still to be assigned within `$FF:0000 – $FF:7FFF` ($\rightarrow$ Q22).

OFFSET	NAME	ACCESS	MEANING
+\$00	SD_LBA_L	read/ write	Sector address, bits [15:0]
+\$02	SD_LBA_H	read/ write	Sector address, bits [31:16]
+\$04	SD_DST_FRAME	read/ write	Destination physical frame, DMA only

OFFSET	NAME	ACCESS	MEANING
+\$06	SD_DST_OFF	read/ write	Byte offset within the frame, DMA only. Block-aligned
+\$08	SD_DMA_READ	write	Card → memory. Argument: block count. The destination frame must have been invalidated first (G.9). Asynchronous
+\$0A	SD_DMA_WRITE	write	Memory → card. Argument: block count. The source frame must have been flushed first (L.16). Asynchronous
+\$0C	SD_PIO_READ	write	Card → <code>SD_DATA</code> . Argument: block count. Asynchronous
+\$0E	SD_PIO_WRITE	write	<code>SD_DATA</code> → card. Argument: block count. Asynchronous
+\$10	SD_DATA	read/ write	FIFO port for the PIO path. <code>SD_STATUS</code> carries the empty and full flags
+\$12	SD_STATUS	read	Bit 0 <code>BUSY</code> · 1 <code>FIFO_EMPTY</code> · 2 <code>FIFO_FULL</code> · 3 <code>ERR_CRC</code> · 4 <code>ERR_TIMEOUT</code> · 5 <code>ERR_ARB</code> · 6 <code>NO_CARD</code> · 7 <code>IRQ_PENDING</code>
+\$14	SD_CARD	read	Bit 0 card present · 1 initialised · 2 <code>SDHC</code> – block addressing rather than byte · 3 write-protected · 15:4 reserved
+\$16	SD_CTRL	write	Bit 0 IRQ enable · 1 clear <code>IRQ_PENDING</code> · 2 clear error bits · 3 SPI clock divisor select – the initialisation sequence runs at ≤ 400 kHz, transfers at full speed
+\$18	SD_CMD	write	Raw SD command index. Argument: the index; the argument registers must be loaded first
+\$1A	SD_ARG_L	read/ write	Raw command argument, bits [15:0]
+\$1C	SD_ARG_H	read/ write	Raw command argument, bits [31:16]
+\$1E	SD_RESP	read	Response to the last raw command. Successive reads walk a multi-byte R3/R7 response

Y1

Filesystems – the VFS layer

ONE SYSCALL PATH · THE VNODE MODEL · WHO OWNS WHAT

SHEET 15

The machine is about to have two filesystems – NVFS on the system volume (sheet Y2) and ext2 on whatever card arrived from a host (sheet Y3) – and one syscall table that has to reach both. This sheet is the layer that stops that from becoming two access paths and a branch on filesystem type at every call site. It is specified **before** the ext2 driver exists, and that ordering is the whole economy of it: introducing it now costs one module and a mechanical rework of NVFS's call surface; introducing it afterwards costs the same rework applied to two drivers, plus a

syscall layer that has already grown the branches. It supersedes the one-line sketch at J.6.

Y1.1 The layer exists for a second reason, and it is the one that survives the first. Some decisions cannot live inside a filesystem driver at all: mount-point traversal, symlink loop bounds, vnode identity, cross-mount rejection, and the locking discipline the Amiga milestone will need. Neither NVFS nor ext2 can see across a mount boundary, and neither can know that another volume has been grafted onto one of its own directories. **Those decisions have no correct home except here**, and that is true even in the counterfactual where the machine only ever has one filesystem.

Y1.2 It is not a Unix VFS, and the difference is a rule rather than a preference. Every operation in the tables below is one that *both* drivers implement without emulation. Nothing is added because Unix has it. The interface is cheap to extend and expensive to shrink, which is the asymmetry that decides where the default sits.

Y1.3 What is actually being unified is narrower than the two formats are, and the divergences below are what the interface may not expose. Block size differs – 2048 in NVFS because it equals the MMU page (L.1), 1024 in ext2's write path – so **block size is a mount property and never a constant**. File layout differs – extents against a triple-indirect tree – so `read` and `write` take byte offsets and nothing else. Directory position differs – a slot index against a byte offset – so the `readdir` cookie is an opaque `uint32_t` the VFS stores and hands back without interpreting. Metadata consistency differs – a redo journal against ordered writes – so writeback ordering stays under driver control (Y1.14).

One row of that comparison came out better than it had to. NVFS draft 3 adopted POSIX-identical `mode` and type encodings (Y2.9), so `vattr.mode` passes through both drivers unchanged and there is **no translation table in either direction** – not in `getattr`, not in `readdir`, not in `create`. Had the two formats disagreed on a single nibble, every one of those would carry a conversion, twice.

Y1.4 Four objects, with ownership stated rather than implied. `mount_t`, one per mounted volume, holding `vfs_ops` and the driver's private state – VFS-owned. `vnode_t`, one per *in-use file*, cached and refcounted, identity (`mount`, `file_id`) – VFS-owned. `file_t`, one per `open()`, holding an offset and flags – VFS-owned. And the descriptor table, index → `file_t*` – owned by the process.

Y1.5 `file_t` is separate from `vnode_t` because two `open()` calls need independent offsets and a `dup()` needs a shared one. Only the three-level structure gives both. It costs about ten bytes per open file to have from the start and an audit of every call site that assumed the offset lived in the vnode to add later.

Y1.6 The vnode cache is the VFS's and not each driver's, and the argument is a corruption rather than a tidiness one: if two open files on the same underlying file hold two copies of the inode, an append through one silently discards the size and the block pointers written by the other. Both drivers need an in-memory identity map for exactly this reason – **it is the same logic, it needs the same LRU pressure, and mount traversal has to inspect it**, so writing it twice is writing the same 300 lines twice and then keeping them in step.

Y1.7 `lookup` returns a `file_id`, never a `vnode`. This is the single load-bearing line of the interface. A driver cannot know whether a `vnode` is already cached and cannot know whether a mount is grafted onto the result; the VFS takes the identifier, consults its cache, and calls `vget` only on a miss. Getting it backwards leaks the `vnode` cache into both drivers and cannot be unpicked later.

Which puts a hard requirement on sheet Y2: **NVFS must expose a stable 32-bit file identifier that does not change when the file is modified**, because the cache is keyed on it. Its inode number satisfies this, and `i_generation` (Y2.8) makes the pair unique over the life of the volume (→ Q90).

Y1.8 Directory-mutating operations take a parent `vnode` plus a name, never a path. Resolution happens once, here. **A driver never sees a `/`.**

Y1.9 Write operations may be `NULL`, and the VFS answers `E_NOTSUP`. That is not defensive coding; it is what makes a read-only ext2 mount a genuinely smaller deliverable rather than a full driver with stubs in it (Y3.16).

Y1.10 There is no `open` or `close` in the per-`vnode` table. Opening is a VFS act – resolve, check the type, allocate a `file_t` – and neither driver has per-open state. A device filesystem would need them; adding two slots at that point is trivial, and the shape of that question is Q95.

Operation tables – split per mount and per `vnode`, which is the natural division of the operations rather than a convention borrowed.

TABLE	OPERATIONS
<code>vfs_ops</code> , per mount	<code>mount</code> · <code>unmount</code> · <code>root</code> · <code>sync</code> · <code>statfs</code> · <code>vget</code> (VFS wants a <code>vnode</code> it has not cached) · <code>vput</code> (last release; the driver frees its private state and, if the link count is zero, performs the deferred delete)
<code>vnode_ops</code> , read half	<code>lookup</code> · <code>read</code> · <code>readdir</code> · <code>getattr</code> · <code>readlink</code>
<code>vnode_ops</code> , write half – <code>NULL</code> on a read-only driver	<code>write</code> · <code>truncate</code> · <code>setattr</code> · <code>create</code> · <code>mkdir</code> · <code>rmdir</code> · <code>unlink</code> · <code>link</code> · <code>symlink</code> · <code>rename</code> · <code>fsync</code>

Y1.11 Path resolution is the VFS's, and drivers supply `lookup` on one component.

Absolute paths start at the root mount's root `vnode`, relative ones at the process's `cwd`; empty components and `.` are skipped. `..` at the root of a mount **steps out to the covering `vnode` first** and then resolves `..` there; at the root of the root mount, `..` is the root. A `vnode` carrying `VF_MOUNTED` is descended into. **Symlinks are followed here and are bounded at 8 traversals per call**, beyond which the answer is `E_LOOP`.

The two steps that could not be delegated are the mount ones, and a third is nearly as bad: a symlink stored on ext2 may name a path that resolves on NVFS. `readlink` is per-driver, because the two store short targets differently – ext2 packs a target under 60 bytes into `i_block[]`, NVFS stores one under 72 bytes inline in the inode (Y2.11) – but *following* is not.

`nameiparent` exists beside `namei` because every mutating call needs the containing directory plus the final component, and resolving the whole path first fails on `create` by construction.

Y1.12 Resolution is iterative, not recursive per component, and the reason is L.11 rather than taste: VFS operations run in process context on a per-process kernel stack, every one of which lives in bank \$00 and is addressed by a 16-bit `S`. That is the scarcest address space in the machine. **The budget is 512 bytes of kernel stack for any VFS operation**, the symlink bound of Y1.11 bounds its share, and ext2's indirect walkers – read *and* truncate – recurse at most three deep and no deeper (Y3.5).

The corollary is a rule that is easy to break by accident and produces an overflow into the neighbouring process's kernel stack, which is close to undebuggable: **the 262-byte directory-entry structure is always caller-provided and is never a stack local inside the VFS**. The syscall layer keeps one per open directory descriptor, not one per process.

Y1.13 Four mount slots: the root (NVFS), one removable ext2, two spare. The root mount is established at boot and cannot be unmounted. `mount` requires the mountpoint to resolve to a directory, sets `VF_MOUNTED` and pins that vnode for the volume's lifetime; `unmount` fails with `E_BUSY` if any vnode of that mount other than its root is still referenced – **which is what stops a card being pulled while a file on it is open**, and costs one scan of the vnode cache.

A requested read-write mount **may be downgraded by the driver**, which ext2 does whenever the feature policy of Y3.4 demands it. The VFS records the result and reports it through `statfs`, and the caller is told: a silent downgrade to read-only produces confusing failures much later and in a different part of the system.

Y1.14 One block cache below the VFS, not one per driver – and the ordering hook is the part worth arguing about. Two independent caches means two pools sized by guesswork, no global LRU, and one mount's buffers sitting idle while the other thrashes. The shared cache holds **uniform 2048-byte buffers**, of which an ext2 mount uses half; that wastes 50 % while an ext2 mount is active, and it is accepted because ext2 is a transfer medium and NVFS is the resident root, while a two-size slab allocator costs more code than the memory it saves at this scale.

The complication is that NVFS's journal forbids a naive global LRU. A dirty metadata block must not reach the card before its journal record has committed (Y2.15), and an eviction at an arbitrary moment breaks exactly that. The mechanism is **a `commit_seq` per buffer and a `flushed_seq` per mount**: the cache refuses to write back a buffer whose sequence exceeds its mount's, and calls `ops->sync` to advance it first. ext2 leaves the field at zero and is unaffected. Four bytes per buffer and one comparison in the eviction path.

It is still a hook whose semantics one driver uses, which is a smell, and the honest form of the trade is that **the hook's cost is one field and the alternative's cost is permanent** (→ Q91). Explicit `bpin / bunpin` exists beside it, used by both drivers for superblocks and descriptor tables.

What is **not** proposed: a page cache, read-ahead beyond the multi-block transfers G.7 already provides, and any unification with the SRAM. **The SRAM is a hardware-managed cache over SDRAM and must not be statically carved up for filesystem buffers** (F.5); all of this lives in SDRAM.

Y1.15 One error space, because these values reach userland through the COP ABI (J.3). Each driver keeps its internal enumeration – the detail is useful inside the driver and meaningless to a caller – and maps at its `vnode_ops` boundary. `E_XDEV` is checked here and cannot be checked anywhere else. `link` and `rename` compare the two directories' mounts before dispatching; a driver handed a vnode belonging to a different mount dereferences the wrong private pointer, which is not an error it can detect.

`E_CORRUPT` is sticky. When a driver reports it the VFS marks the mount read-only and latches the condition. Continuing to write to a filesystem that has already failed a consistency check is how a recoverable problem becomes an unrecoverable one – Y3.10 states it for ext2, and it belongs here so that it holds for both.

Unified error space – the values a process sees.

CODE	MEANING	CODE	MEANING
<code>E_OK</code> 0	success	<code>E_BUSY</code> -11	mount or vnode in use
<code>E_IO</code> -1	device failure	<code>E_LOOP</code> -12	symlink bound exceeded
<code>E_NOENT</code> -2	no such name	<code>E_NAMETOOLONG</code> -13	
<code>E_EXIST</code> -3		<code>E_MFILE</code> -14	descriptor table full
<code>E_NOTDIR</code> -4		<code>E_BADF</code> -15	
<code>E_ISDIR</code> -5		<code>E_XDEV</code> -16	cross-mount link or rename
<code>E_NOTEMPTY</code> -6		<code>E_CORRUPT</code> -17	structural inconsistency – sticky
<code>E_INVAL</code> -7		<code>E_FEATURE</code> -18	unsupported on-disk feature
<code>E_NOSPC</code> -8	no blocks	<code>E_NOTSUP</code> -19	operation <code>NULL</code> in this driver
<code>E_NOFILE</code> -9	no inodes	<code>E_FBIG</code> -20	
<code>E_ROFS</code> -10		<code>E_NOTMOUNTED</code> -21	

Y1.16 Permissions are read, preserved and reported – and never enforced. The machine has no user model, so there is nothing to enforce them against, and enforcing them with a single implicit root user produces obstacles and nothing else. Files created here take `uid = gid = 0`, mode `0644` for regular files and `0755` for directories. Revisit only if the machine ever grows multiple users, which is on no roadmap.

Y1.17 The VFS supplies the timestamp; a driver never calls the clock. `vfs_now()` is one function, and with no RTC on the board it returns zero (Y2.4). When an RTC arrives, one function changes – and **the absence of one is a hardware gap rather than a filesystem question**, which is why it is Q88 rather than an item on this sheet. `atime` is never updated on read in either driver, because doing so turns every read into a write.

Y1.18 Deferred delete, and it costs nothing to have from the start. `unlink` on a file with open descriptors decrements the link count and returns; the blocks are released in `vput`, when the last vnode reference goes and the link count is zero. Both drivers implement it in their `vput`, and the VFS guarantees the ordering by holding the vnode reference for as long as any `file_t` points at it. **This is what makes deleting an open file safe rather than catastrophic.**

Y1.19 The locking discipline is written now and the primitives are no-ops until the Amiga milestone. The Apple II milestone is single-threaded – one program, one call into the filesystem at a time – and every shared structure here becomes a race the moment sheet N's scheduler preempts. Retrofitting locking into a vnode cache is notoriously painful and fails intermittently rather than cleanly, whereas specifying it now costs a handful of macros that compile to nothing.

The rule that is expensive to discover late is the ordering one, so it is stated first: parent before child, and between siblings, lower `file_id` first. That is what makes cross-directory `rename`, which holds two directory locks, deadlock-free – and the deadlock it prevents appears only under concurrent renames in a running GUI. The rest: a vnode is locked across any operation that reads or writes its metadata; a vnode lock is never held across a block-device wait that is not part of the operation; and the reference count and the lock are independent – **holding a reference does not imply holding the lock.**

Once processes are preemptive, an SD read of several milliseconds must not spin: the caller sleeps and the scheduler runs somebody else (N.3). That is what puts VFS operations in process context and makes Y1.12's stack budget binding rather than advisory.

Y1.20 The syscall table does not change; what sits behind the dispatch does. `open` is `namei` – or `nameiparent` plus `create` for `O_CREAT` – then a `file_t` installed in the descriptor table. `read/write` are the driver's, at the file's offset, which is then advanced; `pread/pwrite` are the same without the advance. `lseek`, `chdir`, `dup` and `dup2` never reach a driver at all. `opendir/readdir` are a `file_t` with the directory flag and the cookie kept in its offset field.

`getcwd` deserves its own decision and gets the cheap answer. Reconstructing a path from a vnode means walking `..` upward and scanning each parent for the entry whose identifier matches the child – $O(\text{depth} \times \text{entries})$, with no cache, and it simply cannot resolve an unlinked working directory. **The current directory is stored as a string in the process control block** beside the vnode reference, updated on `chdir`, which costs 64–128 bytes per process and makes `getcwd` a copy (N.1).

Y1.21 The host-side F-series of sheet P is where this interface is discovered rather than designed, and it has one shape requirement attached. `libfuse` offers a path-based high-level API and an inode-based low-level one; **only the low-level shape transfers**, because path resolution, symlink following and mount crossing are all VFS responsibilities here and a driver never sees a path. The vtable that track builds must therefore key on inode number with `lookup(parent, name)`, and then a driver here is a thin adapter over the same functions rather than a parallel implementation (→ Q92).

Two further shapes must not leak back: FUSE supplies uid and gid and expects permission checks, which Y1.16 does not have; and `libfuse` is multi-threaded unless told otherwise, which would make its threading model a dependency of this one. **Run the host implementations single-threaded.**

Budgets – against roughly 19 KB for ext2 read-write and 9 KB read-only (Y3.13), plus NVFS.

CODE	ESTIMATE	RAM	SIZE
Vnode cache and lifecycle	1.5 KB	Vnode cache, 24×28 B	672 B
Path resolution, symlinks, mount crossing	1.5 KB	Mount table, 4×48 B	192 B

CODE	ESTIMATE	RAM	SIZE
Mount table	0.5 KB	Open file objects, 24 × 10 B	240 B
<code>file_t</code> , descriptor table, syscall glue	1.5 KB	Descriptor table, 16 × 3 B	48 B per process
Error mapping, <code>statfs</code> , miscellany	0.5 KB	cwd string	128 B per process
Total	≈ 5.5 KB	Path scratch	320 B
Total, excluding per-process			≈ 1.4 KB

Small against the block cache's ~20 KB (Y3.14). The vnode cache size of 24 is a guess and should exceed the maximum number of simultaneously open files by a margin, since directory vnodes along active paths are held too (→ Q94). **If the whole filesystem stack has to be resident, address space is the constraint rather than code size**, and it should be checked against the kernel's bank \$01 budget before the ext2 driver is started rather than after (J.1).

Phasing · V0–V5 – V6 is the Amiga milestone and needs no work before it.

- V0 Structures, error space and the header frozen.** `vfs.h` compiles under both host GCC and Calypsi, and the static assertions pass on both.
The identifier question of Y1.7 is answered before this gate, not during it.
- V1 Vnode cache, mount table and `namei`.** A stub in-memory filesystem mounts at / and resolves paths, on the host.
- V2 NVFS reworked behind `vnode_ops`.** Its existing test suite passes unchanged through the VFS.
This is the gate that must happen before the ext2 driver is started, and it is the one with the least visible payoff – a rework of code that already works. Which is precisely why it is easy to skip and expensive to have skipped.
- V3 `file_t`, the descriptor table and syscall dispatch.** `open / read / close` on NVFS through the real `COP` path.
- V4 ext2 read-only behind the same table** – the same gate as E2.1. Both filesystems mounted at once, and paths resolving across the mount point.
- V5 The shared block cache and its ordering rule.** The journal invariant holds under the shared cache and the crash-injection suite passes.
V5 may trail V4. ext2 on its own private cache is a valid intermediate state, because the ordering hook exists for NVFS's benefit alone.

Y2

Filesystems – NVFS, the native volume

THE ON-DISK FORMAT · THE JOURNAL · WHAT A CRASH CAN LEAVE

SHEET 16

NVFS is the filesystem the machine boots from, keeps its kernel and programs on, and will eventually page from. It is designed for this machine and attempts no generality: **recoverable without human intervention** first, then **implementable on a**

65816 – every hot-path index a shift or a mask, steady-state RAM in single-digit kilobytes – then **cheap in block transfers**, since transfers dominate and elegance that costs an extra read is a bad trade, then **verifiable**, then faithful to POSIX wherever that is free. Explicitly not goals: compression, encryption, snapshots, quotas, extended attributes, ACLs, multi-device volumes, online resize, big-endian hosts, and any block size but 2048. The layer above is sheet Y1; the foreign filesystem beside it is sheet Y3.

Y2.1 Draft 3 is a format break and there is no migration path inside the filesystem. A draft 2 volume is evacuated by copying its files out with the old tooling, reformatted, and copied back – which makes **archiving and tagging the draft 2 tooling the one action that must happen before any draft 3 code is written**, because step one depends on it and there will be no way to reconstruct it. Draft 3 refuses a draft 2 volume cleanly rather than as generic corruption: the superblock is at a different block and the magic check fails. `mkfs`, `nvfsck` and the 65816 read path are rewritten; the assembly CRC-32 is reusable unchanged; the conformance and crash-injection harnesses survive and their corpora do not.

The cost is real and is the main argument against the break. What buys it is five changes, each of which pays on this machine specifically: a journal in place of ordering discipline, a 2048-byte block equal to the MMU page, extents in place of indirect trees, fixed-size directory slots, and POSIX-identical mode bits (→ D51).

Y2.2 A journal replaces write ordering, and the argument is where the correctness lives. Ordering discipline is a correctness argument spread across every mutating operation – which is why the crash-injection harness found ordering bugs in draft 2, and why Y3.9 has to make the same argument again for ext2. A journal is a correctness argument in **one place**. On a machine with no on-target `fsck`, bounded idempotent single-pass recovery is worth what it costs.

Y2.3 The block is 2048 bytes, fixed, and that single number is the source of most of this sheet's arithmetic. It is exactly one virtual-memory page (L.1) and exactly four SD sectors. **Demand-paging a page out of an executable is therefore one block read** – worth more than any other optimisation here. And every index conversion becomes a shift on a CPU with no divide and a 16-bit ALU: `lba = partition_lba + (block << 2)`, `itable_block = start + ((ino-1) >> 4)`, `bitmap_block = start + (b >> 14)`. **There is no division anywhere in the NVFS data path.**

`s_block_size` exists in the superblock so that a later version can relax this, and in version 3.0 it MUST be 2048; a volume where it is not is refused at mount rather than worked around.

Fundamental parameters – the constants every formula above depends on.

PARAMETER	VALUE	WHY
Block size	2048 B, fixed	= MMU page · = 4 SD sectors · all index maths become shifts
Inode size	128 B, fixed	16 per block; <code>slot = (ino-1) & 15</code>
Directory slot	64 B, fixed	30 slots per block after the header
Maximum name	56 bytes	fits the slot with no spill mechanism
Block and inode numbers	<code>u32</code>	same width as the SD LBA; two 16-bit adds

PARAMETER	VALUE	WHY
File size	u32	4 GiB – 1, against a 16 MB address space
Maximum extent	65,535 blocks	128 MiB in one entry
Maximum extent depth	3	sufficient for a maximum-size file of single-block extents
Blocks per journal transaction	507	the 65816 driver caps itself at 64 (Y2.17)

Y2.4 Conventions, and two of them are load-bearing. Every multi-byte field is **little-endian**, which matches both the 65816 and every intended host: **no implementation performs a byte swap**, and one that finds itself wanting a swap accessor has a bug elsewhere. Every field is naturally aligned, which the 65816 does not require but which lets a host map structures directly and lets a static assertion catch layout drift. Reserved fields are written zero and **ignored** on read – forward compatibility is governed by the feature flags of Y2.20 and by nothing else. Names are byte strings, 1 to 56 bytes, no NUL and no /, with no encoding, no normalisation and no case folding: **two names differing in any byte are different names**.

Time is u32 seconds since 1970 and the board has no source for it. A host with no reliable wall clock writes **zero**, meaning "unknown", and `nvfsck` must not report zero as an error. That is a hardware gap rather than a filesystem question and it should be closed before the board is frozen (→ Q88).

One checksum algorithm appears in this sheet and no other: CRC-32, reflected, polynomial `0xEDB88320`, init and final XOR `0xFFFFFFFF` – the CRC-32 of IEEE 802.3, zlib and gzip, check value `0xCB43926`. Every 2048-byte metadata block that carries a block checksum stores it at offset `0x7FC`, computed over the whole block with those four bytes zeroed, so **there is exactly one such routine in any implementation**. The directory name hash is derived from the same pass (Y2.13).

Volume layout – one MBR partition, everything numbered from its first block.

REGION	EXTENT	SUPERBLOCK FIELDS
Reserved, including the primary superblock at block 1	blocks 0–7, fixed	–
Journal superblock + ring		<code>s_journal_start</code> · <code>s_journal_blocks</code>
Block bitmap		<code>s_bitmap_start</code> · <code>s_bitmap_blocks</code>
Free-count summary		<code>s_summary_start</code> · <code>s_summary_blocks</code>
Inode bitmap		<code>s_ibitmap_start</code> · <code>s_ibitmap_blocks</code>
Inode table		<code>s_itable_start</code> · <code>s_itable_blocks</code>
Data	<code>s_first_data_block</code> ... <code>s_total_blocks</code> – 2	

REGION	EXTENT	SUPERBLOCK FIELDS
Backup superblock	last block	s_backup_sb

Y2.5 The order is fixed and **nvfsck** verifies it. The 8-block reserved area exists so a boot loader can live at the front of the partition without leaving a hole in the filesystem; blocks 0 and 2–7 are never read or written by NVFS itself. Everything from block 0 to `s_first_data_block - 1`, plus the backup superblock, is marked allocated at `mkfs` time, and **bits beyond `s_total_blocks` in the last bitmap block are set to 1 so the allocator can never return an out-of-range block.**

`mkfs` sizing, informative rather than normative: journal ring `clamp(total/64, 256, 8192)`, one inode per 16 KiB, and the bitmap, summary and table lengths that follow. On a 64 MiB volume that is 782 blocks of metadata out of 32,768 – 2.4 %, of which the journal is 1.6 % – and the journal fraction stays constant as the volume grows, which is the argument for capping it (→ Q89).

Y2.6 The backup superblock is written by `mkfs` and **nvfsck** and by nothing else. It records the immutable layout, which never changes; its counters and timestamps are those from format time and are ignored on recovery. **An implementation must not silently fall back to it:** if block 1 fails its magic or CRC, the mount fails, and recovering from the backup is an explicit `nvfsck` operation. `s_partition_lba` in the superblock is informative – the implementation uses the LBA it got from the partition table and warns if the two disagree. `s_free_blocks` and `s_free_inodes` are advisory, so that `statfs` need not scan the bitmaps, and they are **maintained inside the same transaction as the bitmap change that alters them** so a crash cannot desynchronise them. `nvfsck` recomputes them and corrects silently.

Y2.7 A flat bitmap with a free-count summary in front of it, and no block groups. Bit n is block n , 1 means allocated, 16,384 bits per block – which is why the index is a shift. There are no groups because at the volume sizes this machine will see, 8–32 GiB and so 256–1024 bitmap blocks, the locality a group buys does not pay for its bookkeeping. The summary is one `u16` per bitmap block holding its free count, so the allocator skips a full block without reading it; on a 1024-block bitmap the whole summary is one block. 16,384 does not fit in 16 bits, so **0xFFFF encodes it and is reserved for that** – a completely free bitmap block only occurs on a fresh volume and the special case is two instructions, against four times the summary for a `u32`.

Bitmap and summary blocks carry no checksum, and this is documented debt rather than an oversight. A 4-byte CRC would leave 16,352 bits per block, which is not a power of two, and `bitmap_block_of()` would become a division by 16,352 in the hottest path in the allocator on a CPU with no divide. The accepted consequence: a torn bitmap block is not detected at mount. It is detected by `nvfsck`, which recomputes the bitmap from the extents – and the journal already guarantees that a bitmap block is only ever written from a CRC-protected transaction. **Revisit it if the block size ever changes.**

Y2.8 Inodes are 1-based, 16 to a block, and identity outlives them. Inode 1 is the root directory, always allocated and always a directory; 2–7 are reserved and marked allocated with type 0; 8 upward are allocatable. Each new inode takes `i_generation` from a superblock counter, so **the pair (number, generation) uniquely identifies an inode over the life of the volume** – which is what a host binding needs for handle validity and what Y1.7's vnode cache will need if it ever holds a handle across an unlink.

A free inode has `i_mode` and `i_links` zero and a valid `i_crc32`, so that a free inode is distinguishable from a corrupt one.

Y2.9 The inode is 128 bytes and `i_mode` is bit-identical to POSIX, which is what makes Y1.3's translation table unnecessary in both directions. Type nibble: `0x1` FIFO, `0x2` character device, `0x4` directory, `0x6` block device, `0x8` regular, `0xA` symlink; any other value makes the inode invalid. **Bits 11–9 – `setuid`, `setgid`, `sticky` – are reserved and must be zero**: this is a single-user machine, they have no meaning here, and accepting them would create a security surface with no matching benefit. The kernel interprets only the owner triplet; group and other exist so a host binding round-trips and so a multi-user story is not foreclosed.

`i_blocks` counts **NVFS blocks, not 512-byte units** – deliberately unlike ext2, whose 512-byte convention is a well-known source of bugs and which there is no reason to inherit. It counts extent-tree nodes and excludes holes.

Three inode flags are defined and the rest are reserved: `IMMUTABLE`, `APPEND`, and `SYNC` – the last making every mutation on that inode commit its transaction before returning.

`i_atime` **is not maintained on read**. Doing so turns every read into a write, which this machine cannot afford. It is set by `mkfs`, on creation, and on an explicit `utimes`; the format supports relative-atime behaviour and nothing here depends on it.

Inode – 128 bytes, the fields that matter.

OFFSET	FIELD	MEANING
0x00	<code>i_mode</code> · <code>i_links</code>	POSIX type and permissions · hard link count, 0 = free or orphaned
0x04	<code>i_size</code> · <code>i_blocks</code>	bytes · NVFS blocks charged to the file
0x0C	<code>i_mtime</code> · <code>i_ctime</code> · <code>i_atime</code>	
0x18	<code>i_generation</code>	Y2.8
0x1C	<code>i_flags</code> · <code>i_ext_count</code> · <code>i_ext_depth</code>	flags · valid entries in <code>i_ext[]</code> , 0–6 · tree depth, 0–3
0x20	<code>i_ext[6]</code>	six 12-byte extent entries – 72 bytes, also the inline symlink area
0x68	<code>i_uid</code> · <code>i_gid</code> · <code>i_rdev</code>	preserved, never enforced · device number or 0
0x70	<code>i_orphan_next</code> · <code>i_dirhash_order</code>	Y2.19 · Y2.14

OFFSET	FIELD	MEANING
0x7C	i_crc32	over bytes 0x00 – 0x7B

Y2.10 Extents, not an indirect tree, because at ~2.5 ms per block transfer an indirect read is a visible cost. A leaf entry is 12 bytes – logical block, physical block, length in blocks, flags – and an index entry is the same shape pointing at a child node. The root is the inode's six entries; a node is 2048 bytes holding **exactly 169 entries**, since $0x010 + 169 \times 12 = 0x7FC$ leaves precisely the checksum. Depth 0 gives 6 extents, depth 1 gives 1,014, depth 2 gives 171,366 and **depth 3 gives 28,960,854 – enough for a maximum-size file in which every extent is a single block**, so depth 3 is the cap and it is sufficient by construction.

A logical block covered by no extent is a hole, read as zeros. Holes are the absence of coverage and never a special extent value. Entries at every level are sorted ascending and must not overlap, which is what makes lookup a search and lets `nvfsck` validate a tree in one pass.

The lookup's structural check is mandatory, not advisory: a child's depth must equal its parent's minus one. That is what stops a corrupted index pointer from causing an unbounded walk, and combined with the depth cap it bounds the walk by construction.

Merging on insert is required and merging on delete is not. An implementation must merge an extent with its predecessor when they are contiguous in both spaces with identical flags and the result fits – without it, sequential writes produce one extent per block and defeat the entire design. A tree left sparse by truncation is acceptable and `nvfsck` must not report it.

`UNWRITTEN` marks preallocated space: charged to the file, unavailable to anything else, contents undefined, **read as zeros**. Writing into one splits it. A 3.0 implementation need not create them but **must handle them on read**, because the host library creates them for `fallocate`.

Y2.11 A symlink whose target is 72 bytes or shorter is stored inline in the extent area, with no extents and no blocks charged, and it is detected purely by the size – **no flag bit is needed and none is defined**. Longer targets are ordinary file data. Directories are files whose size is always a whole number of blocks and which may contain **no holes**.

Y2.12 Directory slots are fixed at 64 bytes, and this is the change that removes a whole class of corruption. Variable-length records with an in-band length field are the single largest source of directory corruption in ext2-class filesystems: one bad length desynchronises a block, and Y3.10 has to spend sanity checks defending against exactly that. Here there is no length-linked chain: a slot holds inode number, a 16-bit name hash, name length, type, and 56 name bytes. **A free slot is one whose inode number is zero**, deletion is a single-field write followed by zeroing the rest, and slots within a block are unordered. Thirty slots fit a block after its header.

The hash is the low 16 bits of the same CRC-32, and it exists so that **a lookup compares two bytes before it compares up to 56**. It is a fast reject only: a match must still be confirmed against the length and the bytes.

The type byte matches the Linux `DT_*` constants so a host binding passes it through, and it **must agree with the target inode's type nibble**; `nvfsck` reports a disagreement and repairs it from the inode, which is authoritative.

Y2.13 Neither `.` nor `..` occupies a slot; both are synthesised. `.` is the directory itself, and the parent inode number is not stored – it is supplied by the caller, which necessarily knows it because that is how it reached the directory. This removes two slots per directory and, far more importantly, removes the `..` link-count bookkeeping that is a persistent source of `fsck` errors.

The cost is stated rather than hidden: a directory found by inode number alone has no path back to its parent, so `nvfsck` reconstructs parenthood by full traversal – which it does anyway. And a directory's link count is therefore always exactly 2, one for its entry in the parent and one for itself, with subdirectories *not* incrementing the parent. That differs from the POSIX convention of 2 + subdirectories, so a host binding must synthesise the POSIX value if any tool depends on it. There are 8 reserved bytes in the inode if a stored parent is ever wanted instead (→ Q93).

Y2.14 Lookup is a linear scan and the cost is stated honestly: one block read per 30 entries, so a 300-entry directory is 10 reads, about 25 ms. Insertion takes the first free slot in the first block with room, or appends a block. Deletion never frees a block and never shrinks the directory – a directory shrinks only when it is removed. The `readdir` cookie is `block × 30 + slot + 1`, with 0 meaning start, stable for the directory's lifetime as long as no block is added.

DIRHASH is defined so that it can be added later without a format change, and is not required in 3.0 – `mkfs` must not set the bit by default, and the recommendation is to ship linear-only. When enabled, a directory's blocks become 2^{order} buckets selected by the upper 16 bits of the same CRC whose lower 16 are the slot hash, so one CRC pass still serves both. Growth is a bulk rebuild rather than incremental splitting, because a rebuild is easier to make crash-safe and easier to verify, and directories large enough to need it are rare here.

Y2.15 The journal is physical, redo-only, metadata-only and circular, and three properties follow that are the entire reason for that shape. **Replay is idempotent** – it copies whole blocks to fixed destinations, so running it twice equals running it once and a crash *during* recovery is harmless. **Replay is order-only** – no undo, no dependency graph, no compensating records, so recovery is a forward scan and a sequence of block copies, a few hundred bytes of 65816 code. And a transaction needs one block of RAM: metadata blocks are streamed into the journal as they are modified rather than buffered, and a block logged twice in one transaction is simply logged twice, with the later copy winning at replay.

File data is never journaled, and Y2.18 is the ordering rule that replaces it.

There is no escape mechanism and none is needed, unlike ext3. Recovery never has to guess where a block boundary falls: the descriptor states the count, recovery reads exactly that many blocks and expects the commit block at a computed position, and it never interprets a logged block. The scan for the next descriptor resumes arithmetically, never by searching for a magic. Which is why the count is validated against 507 *before* it is used, and why an out-of-range count terminates recovery rather than being skipped.

Y2.16 Commit is seven steps that must not be reordered, and a checkpoint that may be deferred. Log each modified metadata block into the ring, recording its home destination; **ensure every file data block this transaction wrote is durable**; write the descriptor; barrier; write the commit block carrying a CRC over the descriptor and all logged blocks; barrier – **the transaction is durable here**. Then checkpoint: write each logged block to its home, barrier, advance the journal superblock's checkpoint and sequence, barrier. The last three may be batched across several transactions.

Recovery is the mirror and stops at the first thing that does not check out: a bad descriptor magic, CRC, sequence or count; a bad commit block; a payload CRC that disagrees. **Destinations are validated before use** – in range and outside the journal region – and a violation aborts recovery and fails the mount. Entries are applied in ascending order, because that is what makes the later copy of a twice-logged block win. **The checkpoint is advanced only after the replayed blocks are durable**: advancing it first is the one way to lose data in this design.

"Barrier" means every write issued before it is durable before any issued after. On SD that means waiting for the card to leave busy and confirming with `CMD13` – and **whether that is actually a durability guarantee is an open assumption, not a fact** (→ Q96).

Y2.17 A freed block may not be reused until the transaction that freed it is checkpointed. Without that rule a replay can write stale metadata over freshly written file data. ext3 solves the same problem with revoke records; those were considered and rejected, because they add a second record type, a second scan pass and a hash table to recovery – all in the one piece of code that must stay under a few hundred bytes on the 65816. The implementation is a small in-memory set cleared at every checkpoint, and an implementation may simply refuse to reuse *any* freed block until the next checkpoint at the cost of some transient under-reporting.

A transaction that does not fit the free ring forces a checkpoint until it does. **The 65816 driver caps transactions at 64 blocks**, which bounds the in-RAM destination array to 256 bytes; a single transaction larger than the ring is a bug, and the general cap is `min(507, ring/2)`.

Y2.18 One rule governs data, and it is `data=ordered`: every file data block a transaction wrote must be durable before that transaction's commit block is written. This guarantees that metadata never references a block whose contents are stale – that **a crash can never expose the previous occupant of a reallocated block through a newly written file**. `data=writeback` would be faster and is not permitted: on a single-user machine with no other protection, stale-data exposure is both a correctness and a privacy failure. If the barrier per transaction proves intolerable, the alternative is journaling small writes with the metadata, not relaxing this (→ Q97).

The complete list of post-crash states is three, and there is no fourth. Not committed: nothing of it is visible, its data blocks are unreferenced and their bits are clear – no leak. Committed, not checkpointed: replayed, fully visible. Committed and checkpointed but the checkpoint not advanced: replayed again, idempotently, fully visible. In particular **there is no state in which a directory entry points at a free inode, or a bitmap bit disagrees with an extent**, because those pairs are never split across transactions.

`fsync` flushes the file's data, commits the transaction holding its metadata, and returns – it need not checkpoint. **`fsync` on a directory commits that directory's modifications**, which is what makes "create a file, `fsync` the directory" durable, and the layer above must expose it (Y1.20).

Transaction atomicity – the normative checklist the crash-injection harness tests against.

OPERATION	WHAT MUST COMMIT TOGETHER
Block allocate / free	bitmap block · summary entry · superblock counters · the metadata block that now does or no longer does reference it
Inode allocate	inode bitmap · inode table block · superblock counters
Create	the inode-allocation group · the directory block · the directory inode · any new directory block's allocation
Unlink	directory block · target inode's link count · orphan-list insertion if it reaches zero
Link	directory block · target inode
Rename	both directory blocks · both directory inodes · the replaced target inode if any
Write extending a file	inode · every modified extent node · an allocation group per new block
Truncate	inode · every modified extent node · a free group per released block
Directory grow	directory inode · the new directory block · the extent change · the allocation group

A `rename` spanning two directory blocks is the largest common transaction and fits the 64-block cap comfortably.

Y2.19 The one thing a crash can leak is an orphan, and the orphan list is what collects it. An inode whose link count reached zero while it was still open is threaded onto a list headed in the superblock – inserted in the same transaction as the final `unlink`, removed and freed when the last handle closes. **At mount, after recovery, the list is walked and every inode on it freed.** That is the mechanism that keeps crashes from leaking space, and it is why NVFS needs no on-target `fsck` for the common case.

The walk **bounds itself**: it stops after `s_total_inodes` steps and rejects an out-of-range inode number. A cycle is corruption and fails the mount rather than looping.

Y2.20 Three feature masks, and the policy is enforced at mount before any other validation. An unknown `incompat` bit refuses the mount; an unknown `ro_compat` bit mounts read-only; an unknown `compat` bit is ignored. **The implementation must name the bit it did not recognise** – a mount failure that says only "unsupported features" is not acceptable, and this is the single most useful diagnostic a filesystem can emit.

The two `ro_compat` bits defined – metadata checksums present, journal present – are **always set in 3.0**, and putting them in the read-only mask rather than the incompatible one is deliberate: a minimal read-only reader, a boot loader for instance, that neither verifies checksums nor replays the journal can still mount a **cleanly unmounted** volume. One that meets a dirty state or a non-empty journal must refuse. `DIRHASH` is the only `incompat` bit defined.

Y2.21 Mount is eight steps and the order is the safety. Locate the partition; read block 1 and verify magic and CRC; check version and block size; **apply the feature policy**; sanity-check the layout – every region in range, correctly ordered, non-overlapping; read the journal superblock and run recovery; if read-write, walk and free the orphan list, then mark the volume dirty and count the mount; finally read inode 1 and verify it is a directory. A volume in the error state mounts read-only and reports that `nvfsck` is required.

Y2.22 **nvfsck** is host-side, there is no on-target **fsck**, and none is planned. In a single pass it must verify the superblock and its agreement with the backup on immutable fields; the journal's validity and **emptiness**, replaying first or refusing; every inode's CRC, type and flags; every extent tree's magic, CRC, back-reference, depth consistency, sorted non-overlapping entries and in-range physical blocks; **a reconstructed block bitmap against the stored one**; a reconstructed inode bitmap and per-inode link count; every directory block's magic, CRC, back-reference, used count, name lengths, hashes and types, with no duplicates; **full connectivity from the root**, no cycles, no directory with two parents; the orphan list; and the advisory counters. Counters and bitmap leaks are repaired silently; everything else asks.

The severity ordering in the bitmap check is the part worth remembering: **allocated but referenced by nothing is a leak and correctable**; **referenced by two inodes is a cross-link and serious**; **referenced but marked free is the fatal case**.

Y2.23 **Steady-state RAM for a single-mount, single-open-file driver is about 4.7 KiB**: two 2048-byte buffers, one for metadata and one for data, a 256-byte journal destination array, a 128-byte inode working copy, 64 bytes of cached layout constants – **not the whole superblock block** – and scratch for a name and a path component. That fits comfortably in the kernel's bank **\$01** allocation (J.1). A second data buffer doubles sequential throughput for 2 KiB and is an implementation choice, not a format requirement.

The 2048-byte buffers must not straddle a 64 KB bank boundary. Aligning each to 2048 satisfies that automatically and also makes the SD destination address trivially computable (G.8). The same rule holds for ext2 (Y3.12).

Under Calypsi, **int** is 16 bits: every on-disk field and every counter that can exceed 65,535 takes an explicit fixed-width type, and block and inode arithmetic is two 16-bit operations. **A static assertion on every structure size must be present in both the host and the Calypsi build** – a structure that silently gains padding on one of the two is exactly the bug that produces a volume readable by one implementation and not the other (0.2).

Cost model – budget targets extrapolated from ~8–10 K cycles per KiB at 8 MHz, **not measurements**. They must be measured on real hardware before this format is frozen.

	BLOCKS	ESTIMATE
One block transfer	1	≈ 2.5 ms
Clean mount	3	≈ 8 ms
Path lookup, two components, small directories	4	≈ 10 ms
Demand-page one 2 KiB page of an open executable	1	≈ 2.5 ms
Create a file in a directory with a free slot	≈ 13	≈ 33 ms
Sequential write of 64 KiB into one extent	≈ 44	≈ 110 ms · ≈ 580 KiB/s

The journal roughly doubles metadata write cost. For unpacking an archive that is a real slowdown; for bulk data it is negligible, because data is not journaled.

Batching several operations into one commit recovers most of it, and the driver should do so with an interval of about a second plus an immediate commit on `fsync`.

Y2.24 What this sheet needs from sheet G's register map is one sentence, and it is on the critical path. The natural transfer unit is **four consecutive 512-byte sectors into a 2048-byte-aligned buffer**, and the map should support issuing that as a single multi-block command with one burst into SDRAM rather than four single-sector transfers. Four transfers with per-command overhead is measurably worse than one burst, and the difference multiplies through every figure above. **It is much cheaper to design in now than to add later**, and it is the same request Y3.12 makes without a number attached (→ Q22, G.7).

Y2.25 The volume lives in an MBR primary partition of type **0x7F**, and the partition must start at an LBA that is a multiple of 4. The type byte is the one reserved for local and experimental use by the Alternative OS Development Partition Standard; **0x7A** and **0x7B**, which an earlier draft proposed, are not safe – they are the hidden-variant aliases of FAT16 LBA and FAT16B under the +0x10 hidden-FAT convention, so some host tools will silently treat an NVFS partition as a hidden FAT. GPT is not supported. `mkfs` refuses an unaligned partition rather than working around it, because working around it turns every block transfer into two, and refuses volumes below 4096 blocks (8 MiB). NVFS makes no attempt at wear levelling or erase-block alignment and issues no discard – an SD card contains a flash translation layer that does both (→ D52).

Magic number registry – the prefix **NV** is reserved at offset 0 of any metadata block. Bitmap, summary, inode-table and data blocks carry no magic.

MAGIC	STRUCTURE	MAGIC	STRUCTURE
NVFS	Superblock	NVJC	Journal commit block
NVJS	Journal superblock	NVDB	Directory block
NVJD	Journal descriptor block	NVXN	Extent node

Y3

Filesystems – ext2, the transfer medium

READING A HOST'S CARD · THE FEATURE GATE · WHAT WRITING WOULD COST SHEET 17

The second filesystem exists for one purpose: moving files between this machine and a Linux or macOS host by carrying the card, with **no custom tooling on the host side**. It is not, and is not intended to become, the system volume – sheet Y2 is specified

for this machine, and this one is a transfer medium. Both sit behind sheet Y1's single table.

Y3.1 ext2 rather than FAT32, and the reasons are all about the 65816 rather than about licensing. It is **little-endian on disk**, so every field is a direct load and there is no byte-swap in any structure access – which is not a small advantage when the alternative adds one to every access in the driver. It has a clean, small, well-documented structure set – superblock, group descriptor, inode, directory entry – with **no FAT chain to walk**. It carries a real directory tree, permissions, hard links and symlinks, which map onto sheet Y1 without loss. And it has no long-filename encoding layer and no patent ambiguity.

The cost is stated plainly: **ext2 has no journal**, so its crash consistency comes entirely from write ordering – the argument Y2.2 spent a format break to stop making. Y3.9 is that argument, made again, and it is the reason Y3.16 recommends what it recommends.

Y3.2 The target is revision 1, 1024-byte blocks, 128-byte inodes. In scope: read/write mount of one partition, full path resolution, file read, write, extend and truncate, directory create and remove, hard links, rename, symlink read, an MBR partition scan and a write-back buffer cache. Out of scope and stated so: every ext3/ext4 feature – journals, extents, 64-bit block numbers, htree *maintenance*, metadata checksums – plus extended attributes, ACLs, quotas, resize, files above 2 GiB, and **an on-target fsck**. Recovery is the host's job and the tool is **e2fsck**.

Y3.3 With 1 KiB blocks the geometry is fixed and small: 8 inodes per block, 32 group descriptors per block, 8192 blocks per group because one bitmap block covers exactly 1024×8 bits, the superblock at byte 1024, and **s_first_data_block = 1**. **That last constant is why the write path is restricted to 1 KiB** – it becomes 0 at any larger block size, and the difference propagates into every group and bitmap index in the driver. Cache footprint scales directly with block size, and a 4 KiB read-modify-write costs four times the CPU on a machine where block copying is already the dominant cost. **The read path handles 1024, 2048 and 4096; the write path refuses anything but 1024**, and that is a restriction on this implementation rather than on the format.

Y3.4 The feature-flag gate is the most safety-critical part of the mount path, because mounting a volume whose features are not understood and then writing to it is the primary route to silent corruption. The rule is the standard one and the only safe one: **an unknown INCOMPAT bit refuses the mount, an unknown RO_COMPAT bit mounts read-only, COMPAT bits may be ignored for reading**. The downgrade is reported upward, never swallowed (Y1.13).

Feature acceptance policy – the table the mount path implements literally.

SET	FLAG	POLICY HERE
COMPAT	DIR_PREALLOC · IMAGIC_INODES · RESIZE_INODE	Ignore. Nothing is preallocated and inode 7 is never touched
COMPAT	HAS_JOURNAL	Force read-only. This is an ext3 volume; mounting it read-write as ext2 is legal only if the journal is clean, and verifying that is out of scope

SET	FLAG	POLICY HERE
COMPAT	EXT_ATTR	Ignore on read. On write, a modified inode's <code>i_file_acl</code> is preserved untouched
COMPAT	DIR_INDEX	Read-safe. On any modification of that directory, clear its index flag – see Y3.6
INCOMPAT	FILETYPE	Supported , and set by every modern <code>mke2fs</code> . Decides whether the directory entry's type byte means anything
INCOMPAT	COMPRESSION · RECOVER · JOURNAL_DEV · META_BG	Refuse the mount. <code>RECOVER</code> needs a journal replay only the host can do; <code>META_BG</code> moves the descriptor table
RO_COMPAT	SPARSE_SUPER	Accept read-write. It affects only where backup superblocks live, and this driver never writes backups and never resizes
RO_COMPAT	LARGE_FILE	Accept read-write with a guard: refuse to open for writing any regular file whose size-high field is non-zero
RO_COMPAT	BTREE_DIR	Force read-only

Y3.5 Twelve direct pointers and three levels of indirection, and the level that looks optional is not. Twelve direct blocks reach 12 KiB, single indirect reaches 268 KiB, double indirect reaches 64.26 MiB and triple reaches 16.06 GiB. **A single-indirect implementation covers 268 KiB and is not sufficient; double indirect is mandatory.** Triple is needed only above 64 MiB and is implemented anyway, because the recursive walker handles all three levels with the same code and **omitting level 3 saves nothing.**

That walker is where Y1.12's stack budget is spent, and the bound is explicit: **the read walker recurses at most three deep, and the truncate walker must be written to the same bound and not deeper.** It is easy to write truncation as the naturally deeper recursion and hard to notice until a process's kernel stack has already overflowed into its neighbour's.

Y3.6 DIR_INDEX is the dangerous one, and the danger is that reading works. An htree directory is deliberately backward-compatible for reading: the index root in block 0 hides inside a fake entry whose length spans the whole block, and interior nodes hide the same way, so a linear scan sees `.`, `..` and every real entry and sees the index blocks as empty space. **Writing is not backward-compatible.** Insert an entry into a leaf without updating the hash index and the host will subsequently fail to find that file – it exists, it occupies space, it appears in `readdir`, and it is not openable by name.

The fix is one field: **clear the directory's index flag on its first modification**, after which the host falls back to a linear scan for that directory. The orphaned index blocks stay allocated and harmless, and `e2fsck` reports and reclaims them.

The problem is removable at source, and the recommended host-side formatting is `mke2fs -t ext2 -b 1024 -I 128 -O ^dir_index,^resize_inode,^ext_attr`. Whether the driver is required to handle arbitrary host-created volumes or only that one invocation decides how much of the table above must actually be implemented (→ Q98).

Layer architecture – each layer calls only the one below it, with one deliberate exception: L3 reads the superblock through L0 directly, before the cache's geometry is known.

	LAYER	CONTENTS
	L12	syscall / VFS
	L11	vnode operations
	L10	path resolution
	L9	directory operations
	L8	file I/O
	L7	allocators
	L6	block map
	L5	inode layer
	L4	group descriptors
	L3	superblock and mount
	L2	buffer cache
	L1	primitives
	L0	block device

Y3.7 The read-only subset is exactly L0–L6, L8 read, L9 iterate and lookup, and L10 – gates E2.0 and E2.1. It is well bounded, low risk, and it makes any Linux-formatted card readable. The write path adds the allocators, truncation, record-length manipulation, the counter invariants, the whole ordering discipline of Y3.9 and the crash campaign that validates it – realistically three to four times the work, and carrying the risk of destroying data rather than merely failing to read it.

Y3.8 The buffer cache and the in-memory inode table are both mandatory, and for different reasons. Without a cache a single file creation is 15–20 device transactions. Without a refcounted inode table, two descriptors on one file diverge silently – the same failure Y1.6 moved into the VFS, which is where it now lives for both drivers.

Y3.9 With no journal, consistency is entirely a property of the order blocks reach the card, and one asymmetry generates every rule. A block or inode may be allocated but unreferenced – that leaks space and `e2fsck` reclaims it. A block or inode must never be free while still referenced – that is corruption, and `e2fsck` cannot always repair it without data loss. In one line: **detach before freeing, allocate before referencing.**

Write ordering – the flush points are where the guarantees live.

OPERATION ORDER

Create	allocate the inode → write the inode → flush → add the directory entry → flush . A crash between leaves an orphan <code>e2fsck</code> moves to <code>lost+found</code> ; the reverse order leaves an entry pointing at an uninitialised inode, which is unrecoverable garbage
Extend	allocate the block → write the data → flush → store the pointer → update size and count, write the inode → flush . A crash before the pointer leaks a block; between pointer and size leaves a block past the end, which <code>e2fsck</code> corrects. Neither loses existing data. An indirect block is zeroed and written before the pointer to it is stored
Truncate	clear the pointer → write that inode or indirect block → flush → clear the bitmap bits and counters. Detach first . The reverse order leaves a live pointer to a block the allocator may immediately hand to another file – two files sharing a block, the worst outcome in the whole failure space
Delete	remove the directory entry → flush → decrement links, write the inode → flush → if zero, set the deletion time, truncate, free the inode
mkdir	allocate the inode → allocate the first block → write <code>.</code> and <code>..</code> → flush → write the inode with two links → flush → increment the parent's links → count the directory in the group → add the entry → flush

The volume is marked dirty for the entire duration of a read-write mount, deliberately. Any crash therefore leaves it dirty and the host runs `e2fsck` at its next mount. That is the recovery mechanism; writing an on-target one would be a larger project than this driver. **If the card's write cache cannot be reliably flushed, none of the above guarantees anything.** That is a hardware property, it is the same assumption Y2.16 makes for the journal's barriers, and it should be **verified during SD bring-up and recorded, not assumed** (→ Q96).

Y3.10 On a structural inconsistency or a device error in a write path, the mount is downgraded to read-only immediately and the condition is latched, and the console says so loudly. This is Y1.15 stated where it is first needed. The cheap sanity checks that must be present, because without them the failure mode is silent corruption rather than a clean error: the superblock magic; the block-size and per-group counts non-zero; **the group count derived from blocks agreeing with the one derived from inodes**; each group's bitmap and table blocks inside that group's range; every block pointer in range; every inode number in range; and a directory's size a whole number of blocks. **The directory-entry checks matter most of all** – a record length of zero turns directory iteration into an infinite loop, and one larger than the block turns it into an out-of-bounds read. Length at least 8, four-byte aligned, not running past the end of the block, and name length plus 8 within it. **This is the class of bug Y2.12 designed away with fixed-size slots,** and having to defend against it here is the clearest measurement of what that decision was worth.

Y3.11 Semantics the driver does not get to choose. uid, gid and mode are read, preserved and never enforced (Y1.16). Timestamps come from `vfs_now()`, which is zero until the board has a clock (Y1.17, Q88). Names are byte strings compared bitwise, with no encoding assumption. `i_atime` is never updated on read. **Backup superblocks are never written** – `e2fsck` reconciles them.

Y3.12 Three 65816 consequences, and only the third is a request on somebody else's sheet. Endianness costs nothing, as Y3.1 said. **A 1 KiB cache block must not straddle a bank boundary**, or the copy and the directory walk have to handle the wrap: align every cache block to 1 KiB out of a bank-aligned region, the same rule Y2.23 states at 2 KiB, and let the assembly bitmap scans take a 24-bit pointer and use indirect-long indexed addressing rather than assuming a data bank.

Block transfer is the dominant performance term and it is not close. A 1 KiB block moved byte by byte through the accumulator at 8 MHz costs roughly 8–10 K cycles, about 1 ms; a 1 MB file copy is then 1000 blocks × 2 = **two seconds of pure copying** before any filesystem overhead. So sheet G's register map must move a block into SDRAM **without CPU involvement**, or at minimum through a burst FIFO wide enough to avoid a per-byte handshake – which is the request Y2.24 makes with a concrete number attached, and which G.7's DMA path already answers if it survives to implementation.

Budgets – Calypsi C with hand-written assembly at L1.

LAYER	CODE	RAM ITEM	SIZE
L0–L1	1.5 KB	Buffer cache, 8 × 1 KiB	8 KB
L2 buffer cache	1.5 KB	Buffer headers	128 B
L3–L5	3 KB	Pinned superblock	1 KB
L6 block map	2 KB	Pinned group descriptors, 2 GiB volume	8 KB
L7 allocators	2 KB	In-memory inodes, 16 × 144 B	2.3 KB
L8 file I/O and truncate	3 KB	Mount state, path and entry scratch	1 KB
L9 directories	4 KB		
L10 path resolution	1.5 KB		
Total read-write	≈ 18–19 KB	Total	≈ 20.5 KB
Read-only subset	≈ 9 KB		

Y3.13 The code budget is the term to check early. If the driver is resident it competes with the kernel for a bank (J.1); if it is not, it needs a loadable-module story that does not exist. That question should be answered before E2.1 rather than after (→ Q99).

Y3.14 The RAM budget has one term that scales badly and it is the pinned descriptor table. A 2 GiB volume has 256 groups and needs 8 KB; **a 32 GiB card has 4096 groups and needs 128 KB**, which is not payable. Either an LRU sub-cache over the descriptors or a stated maximum supported volume size, and the choice has to be made before the write path exists (→ Q100). All of this lives in SDRAM: **the SRAM is a hardware-managed cache over SDRAM and is not carved up for filesystem buffers** (F.5).

Y3.15 Testing is differential against the kernel's own driver, and that is what makes this tractable at all. Every operation is performed twice – once through this driver and once through a reference mount – and the results compared, with `e2fsck -fn` run on the image after every gate. **Crash injection hooks a specific write number** and truncates the image there, then checks that what remains is either clean or trivially recoverable. The host track of the host track is where all of it is built, under a debugger and sanitizers rather than over a serial console.

Y3.16 The recommendation is read-only, and it is a recommendation rather than a decision. F0 and F1 give the interoperability the machine actually needs – getting files from a host onto this machine – at roughly a quarter of the cost and with none of the corruption risk. The write path is specified here in full so that the choice stays open and so that nothing has to be redesigned if it is taken, but **it should be revisited only when a concrete need for on-target writing to ext2 appears** (→ Q101).

Whichever path is taken, one operational rule holds. During bring-up this machine will hang frequently, and a half-debugged read-write ext2 driver pointed at a card holding real data is a reliable way to lose it. **Use a dedicated card with nothing of value on it, and run `e2fsck` on the host after every session.**

Phasing · E2.0–E2.7 – the gates are lettered for the filesystem, not for sheet E. E2.0 to E2.5 are almost entirely host-side and are **not** blocked by the SD register map; only the on-target validation from E2.1 onward is.

□ **E2.0 L0–L1, the MBR scan, the host image harness.** An arbitrary LBA reads on host and target; the division and bitmap routines pass unit tests.

□ **E2.1 L2–L6, read, directory iteration and lookup, path resolution.** Any file on an `mke2fs` image reads correctly and a tree walk matches the host's.

□ **E2.2 Buffer cache write-back, superblock state, sync.** Mount read-write, touch nothing, unmount; `e2fsck -fn` clean and the state transitions correct.

□ **E2.3 The allocators, allocating `bmap`, `write`.** Files of every size up to double indirect created and written; `e2fsck -fn` clean after each.

□ **E2.4 Truncate, create, unlink, directory entry insert and remove.** A thousand operations of create/delete/truncate churn; `e2fsck -fn` clean.

□ **E2.5 `mkdir`, `rmdir`, `link`, `rename`, `symlink` read.** Full tree manipulation, and a host mount agrees with it.

□ **E2.6 Crash injection.** Every truncation point recoverable per Y3.9.

□ **E2.7 Unification with NVFS behind sheet Y1** – both filesystems on one syscall path. This is V4 seen from the other side.

T Neon – the GPU

Neon (FPGA-B) is the graphics half of the machine: a text mode that works before any software exists, two bitmap modes, a drawing engine the CPU commands rather than feeds, and a command processor that renders whole frames with the 65816 doing nothing at all. This sheet is the whole of it. Audio shares the device, the memory arbiter and the register base, and is otherwise out of scope. It is at REV B, and REV B exists because REV A was written against assumptions this board does not meet – in four places, every one of them consequential.

Three principles – the rest of the sheet is derived from these, and they are worth more than any individual number in it.

T.1 Text mode depends on nothing. Mode 0 uses no SDRAM, no arbiter, no FIFO and no inter-FPGA signalling: its buffers live in Neon's own block RAM and are initialised from the bitstream. A console therefore exists *before* the SDRAM controller – the most defect-prone block in Neon – has been written, which makes the console the instrument for debugging everything that comes after it. **This ordering is deliberate and must survive every later revision of this sheet.**

T.2 The CPU must not draw. The 65816 at 8 MHz moves ~1.3 MB/s through the \$FE aperture, ~1.6 MB/s at best; a full 320×200 screen costs it ~40 ms, more than two frames. The drawing engine does the same fill in **0.31 ms**. Every decision below moves work off the CPU, ending in the autonomous command lists of T.39, where per-frame cost is ~180 bytes of stores.

T.3 Logic is the constraint, not bandwidth. With its own SDRAM (F.12), scanout consumes under 3 % of Neon's memory system. What limits Neon is the HX8K's **7,680 LUT and 32 EBR blocks**. Every feature is judged on LUT cost against what the blitter can already do – which is what removed the tile engine and the sprite engine (T.44), and it is the only reason the budget closes.

Four reconciliations – where REV A and this board disagree. Read these before trusting any figure below.

T.4 The reference clock is 25 MHz, not 12. REV A derived a single 102.000 MHz domain from a 12 MHz crystal. This board has **one 25 MHz oscillator** starred to all three FPGAs (B.7), and 102.000 MHz is not reachable from it: the iCE40 PLL needs ≥10 MHz at its phase detector, allowing only **DIVR=0** (25 MHz) or **DIVR=1** (12.5 MHz), and neither lands on 102. Two settings bracket it.

SETTING	PFD	VCO	CORE	PIXEL	AT 1344×635	AT 1344×640
DIVR=0 DIVF=32 DIVQ=3	25 MHz	825 MHz	103.125 MHz	51.5625 MHz	60.42 Hz	59.95 Hz

SETTING	PFD	VCO	CORE	PIXEL	AT 1344×635	AT 1344×640
DIVR=1 DIVF=64 DIVQ=3	12.5 MHz	812.5 MHz	101.5625 MHz	50.78125 MHz	59.50 Hz	59.04 Hz

T.5 Take the first, and move the vertical total from 635 to 640. The horizontal set is untouched – 1024 active + 24 front + 136 sync + 160 back = 1344 – and the vertical becomes 600 active + 3 front + 6 sync + 31 back = 640, giving **59.95 Hz**. Every derived figure in this sheet is computed at 103.125 MHz core, 51.5625 MHz pixel, 16.68 ms per frame. **This costs nothing to change later:** the blanking values were provisional at REV A and remain provisional here, because the module datasheet is still missing (→ Q1, Q43). What will not change is the pixel clock and the active geometry.

The 60 Hz target is physics, not preference – the panel's polarity inversion becomes visible flicker as the rate drops (sheet H) – but 59.95 against 60.00 is four hundredths of a percent and is invisible. Do not spend a fractional PLL on it.

T.6 Neon's SDRAM is 64 MB, not 32. REV A assumed a W9825G6KH; D14 put an AS4C32M16SB-7TIN on both FPGAs, with the W9825G6KH a drop-in fallback on the same footprint. Bandwidth is unaffected – 16 bits either way – and every capacity argument gets easier: the pre-rendered level buffer of T.60 is 1.64 MB, **2.5 % of memory rather than 5 %**, and a dozen levels could be resident at once.

T.7 The wait line REV A wished for already exists. Its write-only graphics aperture, with a slow **PEEK** register pair for diagnostic reads, rested on the premise that Neon cannot stall the CPU because the inter-FPGA link was removed. It was not removed entirely: B.6 keeps **NEON_BUS_BSY (Neon → Helium)** alive for exactly this, and sheet H already routes Neon's VSync interrupt into Helium's PIC. Both wires are in the pin budget. **So the aperture can be readable, and PEEK is redundant and is dropped** (→ D32, T.15).

REV A would have had that line stretch RDY; it must gate **PHI2** instead, for D16's reason – RDY is reported ignored during write cycles on some W65C816S revisions, and gating the clock sidesteps the question entirely. The wire is now named **NEON_BUS_SEL / NEON_BUS_BSY** and its protocol is T.18; the philosophy is in the names, **we select Neon's bus, and while that bus is busy Helium holds the CPU** (→ B.6, D37).

Clocking – one domain, and a second PLL nobody has claimed.

T.8 A single clock domain at 103.125 MHz. The pixel stage advances on a clock enable asserted every second cycle, giving 51.5625 MHz; **PCLK** leaves through a **DDR output register**, so the panel gets a clean clock with no combinational clock path and no divided-clock domain. SDRAM runs at the full rate, exactly 2× the pixel rate. The consequence is the one that matters: **there is no clock-domain crossing anywhere inside Neon** – no synchroniser FIFO between scanout and memory – which eliminates an entire class of intermittent, hard-to-diagnose failure.

The panel latches on the falling DCLK edge, so the gateway changes data on the rising one – E0.4.

T.9 The second PLL is free and has two claimants. REV A spent both, one on pixels and one on SDRAM; the single-domain decision returns one. Audio wants a master clock the I2S divider can hit cleanly, and the VGA bring-up path of sheet H wants 25.175 MHz for 640×480 – which the video domain cannot supply, since $103.125 / 4$ is 25.78 MHz, 2.4 % fast. Most monitors tolerate that; the panel is the product and VGA is scaffolding, so this is not worth a fight (→ Q54). **Audio's claim is softer than this item assumes, and one strapping decision is why.** With *SCK* tied to GND the PCM5102A runs its own internal PLL off *BCK* (sheet H), so Neon never owes it a dedicated master clock – it owes it a *BCK* at an exact rate, which is a weaker requirement, and one the part's PLL is designed to clean up. That does not settle Q54, whose prior question is whether VGA is worth existing at all; but the two claimants are not evenly matched and the item should stop implying they are. It also returns a pin: three driven I2S signals rather than four.

T.10 blocking 103 MHz is demanding for an iCE40 HX and closure is unproven until synthesis. SDRAM controllers in this range are documented on the family; that is not the same as having one placed and routed with a blitter beside it. **Fallback:** run everything single-rate at 51.5625 MHz, halving peak SDRAM bandwidth to ~103 MB/s. Every performance figure below halves and still beats CPU-driven rendering by more than an order of magnitude. **The architecture does not depend on 103 MHz; only the margins do** (→ Q44). The fallback keeps the property T.8 was bought for. At a single rate the clock enable simply disappears and pixel and memory stages run together, so there is still no crossing anywhere inside Neon – what halves is bandwidth, not the architecture.

Memory – two of them, with a strict division of responsibility.

T.11 EBR holds everything that must be reachable without a wait state, and everything scanout needs at single-cycle latency. It is genuinely dual-port: port A serves scanout, port B serves the CPU aperture. At 103 MHz fabric against 8 MHz PHI2 every CPU access to EBR – read *or* write – completes inside its bus cycle. No write FIFO, no shadow copy in system SRAM, no arbiter, and no wait line asserted. Glyph fetch is one cycle; in SDRAM the same random access would pay a row activation (~60 ns) for no benefit. The slack is 103.125 MHz against 8 MHz PHI2 – about 12.9 core cycles per bus cycle – which is what lets "no wait line" be a statement rather than a hope. Even at T.10's halved rate it is still six, so the fallback does not put this claim at risk.

STRUCTURE	SIZE	BLOCKS	USED IN
Text buffer	8 KB	16	Mode 0
Font	4 KB	8	Mode 0
Palette – 256 × 18 bit	1 KB	2	all modes
Scanout line buffer, double	1 KB	2	Modes 1, 2
Blitter source/destination burst buffers	1 KB	2	Modes 1, 2
Command FIFO – 128 × 32 bit	512 B	1	Modes 1, 2
Cursor pattern	512 B	1	all modes

STRUCTURE	SIZE	BLOCKS	USED IN
Total	16 KB	32 of 32	

T.12 Fully allocated, zero margin, and it closes only because T.44 removed the tile store and the sprite pattern cache. Two reliefs exist, insurance rather than slack: **reduce the font to 128 glyphs** – full ASCII plus box-drawing, freeing 4 blocks, and it should be a synthesis parameter from day one, not a later edit; or **multiplex the 16 text-buffer blocks into graphics use**, freeing 8 KB in graphics modes at the cost of address multiplexers and of destroying screen contents across a mode switch. The second is acceptable on its own terms – returning to text mode implies a redraw – but it has a software consequence that must be designed in rather than retrofitted (T.55).

The font blocks must not be reclaimed under any scheme. Their bitstream initialisation is the entire mechanism by which the machine prints before software exists (T.28); reusing them destroys it. Which relief to take is decided once the blitter's real EBR demand is known from synthesis, and not before (→ Q45).

T.13 SDRAM holds framebuffers, image assets and command lists – everything large, nothing latency-critical. At 103.125 MHz on a 16-bit bus, peak is **206 MB/s**. Row activation costs ~20 clocks per 512-word row, about 4 % on sequential access; auto-refresh ~0.2 %. Sequential throughput is therefore ~187 MB/s, and the figure to budget against for mixed work is **~150 MB/s effective**.

CONSUMER	MODE 2A	MODE 1
Scanout	3.8 MB/s	4.6 MB/s
Hardware cursor	0.1 MB/s	0.1 MB/s
Audio DMA	0.7 MB/s	0.7 MB/s
CPU aperture traffic, worst case	1.6 MB/s	1.6 MB/s
Left to the drawing engine	~144 MB/s	~143 MB/s

T.14 Scanout consumes under 3 % of the memory system: **memory bandwidth is not a design constraint for Neon**, and any argument that reaches for it has gone wrong somewhere else. One layout convention does matter – put the source and the destination of a copy in **different SDRAM banks** where possible, so the controller interleaves them instead of paying a row activation on every alternation. The hardware does not enforce it; it is a rule for the kernel's VRAM allocator and the asset loader (→ Q50).

CPU access – what the aperture is, and the one thing software should still not do with it.

T.15 Mode 0 is fully read/write with zero wait states, as T.11 requires – its buffers are in EBR and `NEON_BUS_BSY` is never asserted for them, so the console path stays free of everything below and T.1 is untouched. **Modes 1 and 2 are readable too**, through the stall protocol of T.20. REV A's `PEEK` path cost ~20 CPU cycles per byte plus a register file and a state machine to drive it; this costs a wire already in the budget and is about six times faster. **The slow mechanism is deleted and its address space returned.**

T.16 The software model should nevertheless be written as though the aperture were write-only. Asset flow is one-way – microSD → system memory → aperture → VRAM – and once the drawing engine is running the CPU has no reason to read the framebuffer at all. The two operations that historically want reads are screen capture and a colour-picker tool; both now simply read, at a speed neither cares about. **Any design in which the CPU reads pixels in a loop has taken a wrong turn** and will run at aperture speed instead of the blitter's.

This is a stronger position than REV A's, not a weaker one. A GUI cannot implement save-under by pulling the screen into system memory and pushing it back – not because it is forbidden, but because it is ~60× slower than telling the blitter to move the same rectangle inside VRAM, where the pixels never travel over the bus at all (T.58).

T.17 Writes are absorbed by a **32-entry FIFO** and retired to SDRAM by the arbiter, so a write never stalls the CPU even when a burst is in flight. The register file is in fabric registers, not memory, and is readable and writable in every mode with no latency at all – which is what allows it to live in the privileged bank rather than in the aperture (T.23).

T.18 The read protocol: Helium stalls first and asks afterwards. The obvious arrangement – Neon detects the read and asserts a wait line in time to hold off the rising edge of PHI2 – is a race nobody should sign up for. The low phase is ~62 ns at 8 MHz, and inside it Neon would have to see the address settle, decode it, and get a signal across an inter-FPGA crossing that is asynchronous in practice, since Helium's core clock and Neon's 103.125 MHz come from different PLL outputs of the same oscillator. **The race disappears once you notice Helium does not need to be told.** Helium decodes the bank itself (F.3) and is the party that asserts `NEON_BUS_SEL`; it therefore knows a read of `$FE` is in progress at the same instant Neon does. So:

STEP	WHO	WHAT
1	Helium	Decodes bank <code>\$FE</code> with <code>RWB</code> high and asserts <code>NEON_BUS_SEL</code> . Holds PHI2 low unconditionally – it does not wait to observe anything
2	Neon	Latches the address, requests the read through its arbiter, and holds <code>NEON_BUS_BSY</code> asserted while it works
3	Neon	Places the byte in an output register and deasserts <code>NEON_BUS_BSY</code> – the meaningful edge of the whole protocol
4	Helium	Sees busy clear, raises PHI2. Neon drives D0–D7 through the high phase; the CPU latches on the falling edge

Stalling PHI2 low rather than stretching the high phase is deliberate and it is what keeps this cheap. Extending the high phase would need a fresh reading of the W65C816S datasheet on holding a static core with the clock high, which nobody has done; stalling low reuses the exact clock gate Helium already owns for cache fills and the exact datasheet permission D16 already established. It also falls out correctly on the bus: D0–D7 carries the bank byte while PHI2 is low (F.3), so Neon *must not* drive during the low phase, and in this scheme it never does. **`NEON_BUS_BSY` is busy by default, and that is a safety property, not a convenience.** Were it idle-low, Helium could stall and immediately sample "not busy" before Neon had even noticed the cycle, and resume onto a bus nobody was driving. Busy-by-default also makes an absent, unconfigured or reset Neon read as *busy* – so the machine stalls and the watchdog reports it, instead of silently returning whatever a floating bus

settles at. That matters because E1.4's staged-population rule guarantees there will be stages where **Neon is not on the board at all**.

T.19 Four requirements, and the first is the one that would otherwise ship as an intermittent bug.

#	REQUIREMENT	WHY
1	A read drains the write FIFO before it is issued	Writes are posted (T.17). A read that overtakes a pending write returns the <i>previous</i> contents – and it would fail intermittently, according to arbiter load, so "read back what was written" would pass almost every time. Draining is simpler than store-to-load forwarding and this path is slow by design
2	The arbiter bounds the wait	A row activation is ~60 ns and irrelevant. A read that queues behind a full blitter operation is up to 1.5 ms with the CPU clock stopped. The CPU read port needs high priority and the blitter must be preemptible at a bounded burst boundary (→ Q29)
3	Helium's fill watchdog covers this path	No answer within its window ⇒ resume PHI2, return \$FF , raise the fault. Without it an unconfigured Neon freezes the machine with no clock and no diagnostic path – the failure F.11 already demanded a watchdog for
4	The software model stays write-only	The restriction was doing useful work: it is <i>why</i> save-under is a blitter copy and why assets flow one way. With reads available someone will use them, and the symptom is a slow GUI rather than a compile error (T.16)

It is far less slow than "slow" suggests, and the numbers argue for deleting **PEEK** rather than keeping both paths. A **LDA \$FE0000,x** is 5 cycles, 625 ns at 8 MHz; the stall adds ~300–400 ns. Inside a copy loop of ~10 cycles that is **+30 %** – a VRAM read loop runs at roughly 70 % of the speed of the same loop against SRAM, because **the bottleneck is the 65816, not the SDRAM**. Capturing the 64 KB of Mode 2a takes ~26 ms, under two frames, against ~160 ms through **PEEK**. The same wire serves a second requester with no clock involved. When **Helium** reads VRAM – the debug agent issuing an external cycle with **BE** low and the CPU off the bus (R.5) – there is no PHI2 to stall; Helium waits for busy to clear before sampling D0–D7. **NEON_BUS_BSY** is a ready line that Helium happens to use to gate PHI2 when the requester is the CPU, which is the cleanest way to hold the mechanism in mind.

Two windows, and the division between them is the protection model – **\$FE** is Neon's data, **\$FF:8xxx** is Neon's control.

RANGE	SIZE	MODE 0	MODES 1 / 2
\$FE:0000 – \$FE:1FFF	8 KB	Text buffer, R/W	VRAM window
\$FE:2000 – \$FE:2FFF	4 KB	Font, R/W	VRAM window
\$FE:3000 – \$FE:7FFF	20 KB	unmapped, reads \$00	VRAM window
\$FE:8000 – \$FE:83FF	1 KB	Palette, R/W	Palette, R/W
\$FE:8400 – \$FE:FFFF	31 KB	unmapped, reads \$00 – and nothing in it is privileged	unmapped

RANGE	SIZE	MODE 0	MODES 1 / 2
\$FF:8000 – \$FF:80FF	256 B	Register file	Register file
\$FF:8100 – \$FF:FFFF	–	reserved to Neon, reads \$00	reserved to Neon

T.20 In graphics modes **\$FE:0000 – \$FE:7FFF** is a **32 KB sliding window** into SDRAM at byte offset **VRAM_PAGE × 32768**. The window is a power of two, so page arithmetic is a shift. It supersedes an earlier claim that the Mode 2a framebuffer fits one aperture page – it spans two – and the point is moot in practice, because with the blitter drawing, the CPU walks the framebuffer only during asset upload, where paging costs one register write per 32 KB. The upper half of the aperture is now wholly free, which is headroom for a larger window later and is deliberately left unclaimed.

T.21 **Neon's registers live in bank \$FF, and this is what closes the hole an earlier draft could only paper over.** That draft put them at **\$FE:FF00**, inside the aperture – and J.5 lets a process map aperture pages into its own address space, because **\$FE** is user-mappable by L.10. A process holding a mapping that reached the register file could write **LIST_BASE** and **CMD_CTRL.LIST_RUN**, and the command processor then reads and writes **anywhere in the 64 MB with no MMU between it and memory** – a complete escape from the protection model, arrived at through a device node rather than through a bug. The fix was a rule the kernel had to remember. **Moving the file into \$FF makes it a property of the architecture instead:** bank **\$FF** is unmappable to user space by construction, the self-protecting keystone of the whole scheme, so the command processor's control surface now sits behind the same wall as the MMU's.

This costs nothing in pins and introduces no new mechanism. Neon already taps A0–A15 and already captures the bank byte off D0–D7 during the low phase of PHI2 (T.51) – that is how it recognises **\$FE**. Recognising a second bank is the same comparator with a different constant. **NEON_BUS_SEL** means **"Helium has validated this cycle for Neon"** and covers both windows – for **\$FE**, after translation and the permission check; here, after the bank decode and the privilege check. That generality is why the net is named for Neon's bus rather than for video memory (B.6, Q56).

T.22 **The base is \$FF:8000 so that Helium's decode is one address bit.** Helium's rule becomes "A15 set in bank **\$FF** ⇒ not mine" – a single-bit test, not a range comparator – and it keeps **\$FF:0000 – \$FF:7FFF** entirely for its own blocks, so the allocation Q22 still owes for the timer, the interrupt controller, the peripheral block and the power block proceeds with **no risk of ever colliding with Neon**. Neon's file needs 256 bytes and fits one of Q22's walls exactly.

T.23 Three requirements on the gateway, and the first is the only genuinely new work on the board. Helium must release D0–D7 on a read in Neon's window and let Neon drive them; it is the same turnaround an `$FE` read already performs, so it is a reuse rather than a new kind of thing, but it is one more case to get right and it belongs in the E5 test list. **Neon must gate on the strobe, not on the address alone** – without it, a user-mode store to `$FF:8000` reaches the register before `ABORTB` lands, and `ABORT` cancels the instruction on the CPU without side effects; it cannot un-write a register on the far side of the bus. And **no stall is needed**: the file is fabric registers, so Neon drives data inside the `PHI2` high phase – ~62 ns, about six fabric clocks at 103 MHz – exactly as the SRAM does in F.3, and `NEON_BUS_BSY` stays deasserted. Register latency is fixed and identical to Helium's own `$FF` blocks.

One convention differs and should be stated rather than discovered. M.4 makes Helium's MMU registers 16-bit, accessed with `M=0`; Neon's are byte-wide with 16- and 24-bit values latching on their high byte (T.51). That is allowed – Q22's point is that walls let each block keep its own rules, and the power block of sheet S departs further – but a kernel header assuming one convention across bank `$FF` will be wrong.

What does not change is the property the sheet is built on. The service port reaches the register file directly (T.48), so the EC still drives Neon with Helium unconfigured and the CPU in reset. Had the registers been reachable only through Helium's decode, this move would have broken T.55 – the whole boot display – and it does not.

Mode 0 – text, and the reason Neon is specified first.

PARAMETER	VALUE	DERIVATION
Cell	8 × 16 px	1024 / 8 = 128 exactly
Grid	128 × 32	32 × 16 = 512 lines
Cells	4096	2 ¹² , exactly 12-bit addressable
Text area	1024 × 512 px	panel lines 44–555
Border	44 lines top and bottom	(600 – 512) / 2
Text buffer	8 KB in EBR	4096 cells × 2 B
Font	4 KB in EBR	256 glyphs × 16 B

T.24 The 8 × 16 cell is the standard VGA text cell, and every part of that is load-bearing. Any existing 8 × 16 VGA font drops in unmodified; glyph row lookup is a 4-bit shift rather than a multiply by 15; and **descenders are not truncated** – a 15-line cell clips `g j p q y` and forces a bespoke font nobody wants to draw. The 4096-cell total being an exact power of two is what makes the scroll of T.27 free: the ring wrap is a bit mask, not a comparator.

T.25 Character generation is fully real-time and needs no line buffer. Per cell the pipeline wants one text-buffer read and one font read every 8 pixel clocks, which at 51.5625 MHz against single-cycle EBR is 8× slack. For panel active line `L` in 44..555: `text_row = (L-44) >> 4` · `glyph_row = (L-44) & 15` · `cell_index = (TEXT_START + text_row*128 + column) & $0FFF` · `code = buf[cell_index<<1]` · `attribute = buf[(cell_index<<1)+1]` · `glyph_bits = font[(code<<4) | glyph_row]`. Each of the 8 bits selects foreground or background, MSB leftmost. The pipeline is three stages deep, so cell fetch starts three clocks early – absorbed in the back porch. Outside the text area and during horizontal blanking the output is `BORDER_COLOR`.

T.26 Cell format: two bytes, little-endian, naturally aligned. Byte 0 is the glyph code; byte 1 is `bg[3:0]` in bits 7:4 and `fg[3:0]` in bits 3:0, both indices into the first 16 palette entries. A 65816 in 16-bit accumulator mode writes a whole cell with one **STA – one instruction but two bus cycles**, because the data bus is 8 bits wide; the saving is instruction overhead, not bus traffic, and an earlier note claiming otherwise is wrong. A full-screen fill is 8,192 bytes, about 5 ms at 8 MHz with an unrolled loop.

Deferred: there is no per-character blink attribute. The classic VGA approach reallocates attribute bit 7 and cuts the background palette to 8 entries; keeping full 16/16 colour is judged more valuable. If blink is wanted later it arrives as a *global* mode bit that reinterprets bit 7 – never as a silent change to the format.

T.27 Scrolling is a pointer move. The buffer is a ring of 4096 cells and `TEXT_START` is the cell index shown at the top-left corner; to scroll one line, add 128 modulo 4096 and clear the 128 cells newly exposed at the bottom.

	BYTES MOVED	TIME AT 8 MHZ
Naive block move	8192	~5 ms
<code>TEXT_START</code> + clear one row	256	~160 µs

A factor of 32, and because the ring wraps at exactly 4096 the software needs no wrap-around case – the hardware masks the index. On a console scrolling continuously through boot, that is the difference between visible cost and free.

T.28 Two-tier font residency. The **default font is initialised in the bitstream**, so the system prints from the instant `CDONE` rises – before the microSD is mounted, before a kernel exists, before the 65816 has left reset. This is not a convenience; it is the only mechanism by which boot and bring-up diagnostics reach a screen. The font is then **replaceable at runtime** by writing `$FE:2000 – $FE:2FFF`, which goes to EBR port B while scanout reads port A.

Writing the font outside vertical blanking may produce a single-frame artefact on the glyph row being fetched at that instant. Harmless, but a kernel loading a full font should wait for `STATUS.VBLANK` or accept one torn frame.

T.29 Power-on state is a correct screen, not a random one. The text buffer EBR is likewise initialised at synthesis – every cell code `$20`, attribute `$0F`, light grey on black – and the palette to the standard 16-colour CGA/EGA set. The panel therefore shows a blank, correctly bordered field the moment configuration completes, with no register written by anybody.

The hardware cursor – the one place dedicated hardware beat the blitter.

T.30 In Mode 0 the cursor is rendered by **swapping the foreground and background palette indices** for the affected pixels of the cell at `CURSOR_POS`. It consumes no buffer storage and needs no save/restore of the underlying cell by software – a persistent source of complexity in software-drawn consoles. `CURSOR_CTRL` selects shape (block · underline, rows 14–15 · half block, rows 8–15 · vertical bar, column 0), blink (steady · 1 Hz · 2 Hz, derived from the vertical sync counter) and enable. Cost: ~60 LUT.

T.31 `CURSOR_POS` is a **buffer-absolute** cell index, not a screen-relative one, so it scrolls with the text it is attached to, which is the correct behaviour. A terminal that wants the cursor pinned to the last visible row updates it when it scrolls: one register write. In Modes 1 and 2 the cursor becomes a 16×16 or 32×32 , 2 bpp pattern held in EBR and positioned by pixel coordinate, composited at scanout. **It survived the cull of T.44 for one reason: a GUI pointer must track the mouse independently of the rendering loop.**

Mode 1 – hires monochrome, and the GUI's mode.

PARAMETER	VALUE
Resolution	1024 × 600 – native, no replication, square pixels
Depth	1 bpp, MSB leftmost
Framebuffer	128 B/line × 600 = 76,800 B
Colours	<code>MONO_FG</code> , <code>MONO_BG</code> – two palette indices
Border	none, full panel
Scanout	4.6 MB/s

T.32 Full panel resolution at 1:1 is what makes readable text and precise window furniture possible, and nothing else about Mode 1 is expensive. open **The expensive part is arbitrary-alignment blitting, not the framebuffer.** Placing an image at any horizontal position in a packed 1-bpp buffer means shifting each source word by 0–15 bits, merging across word boundaries and masking both edges – the classic Amiga blitter problem, estimated at **~400 LUT** for the barrel shifter and mask generator. It is not optional *for a 1-bpp destination*, and that qualification turns out to matter: at **8 bpp on a 16-bit bus the shifter degenerates to a byte swap**, one multiplexer, so a $1024 \times 600 \times 8$ bpp GUI mode would delete these 400 LUT and buy 256 colours instead of two, for a framebuffer of 614,400 B and 36.8 MB/s of scanout – both comfortable. **That mode does not exist in this sheet and sheet U assumes it throughout** (→ U.10, Q59).

Mode 2 – multicolour, in two sub-modes that share all their logic.

	MODE 2A	MODE 2B
Logical resolution	320×200	512×300
Replication	3×3	2×2
Displayed area	960×600	1024 × 600 – exact
Horizontal border	32 px each side	none
Framebuffer	64,000 B	153,600 B
Scanout	3.8 MB/s	9.2 MB/s

	MODE 2A	MODE 2B
Double-buffered	128 KB	300 KB

T.33 Both are 8 bpp against the 256-entry palette, and the sub-mode is a register field driving the replication counters and the line stride – **supporting 2b costs approximately nothing in logic**. Mode 2a is the recommended default for games: familiar geometry, and pixels that are exactly square because the panel's are. Mode 2b fills the panel with no side bars, costing only memory and bandwidth, both of which are abundant.

This is where D06's px-doubling finally lands. Recorded as a global policy – draw at 512×300, emit each pixel twice – it was superseded when the PSRAM that motivated it disappeared. It returns as **Mode 2b**, a mode among three rather than a property of the machine, the right shape for it: the GUI gets native 1024×600 in Mode 1, games get replication in Mode 2, and no subsystem pays for another's compromise.

The drawing engine – eight operations, all clipped, all strided. contradiction Sheet U specifies a different engine. **What follows is the operation-based engine.** A later design note specifies the Amiga four-channel blitter instead – 256 bitwise logic functions, pointer-and-modulo addressing, mask surfaces rather than a colour key. They are different machines and the choice is open and blocking (Q58, U.1). Sheet U also settles three things this sheet left open – the bank partition, the arbiter's worst case and descending mode – none of which depends on how that choice goes.

OPERATION	DESCRIPTION
FILL_RECT	Solid colour fill
FILL_PATTERN	Fill with the 8 × 8 pattern register
COPY_RECT	Opaque rectangular copy, overlap-safe – direction chosen from the sign of the displacement
COPY_KEYED	Copy skipping pixels equal to the colour key
COPY_TILED	Copy with the source wrapped modulo its dimensions – texture fill
COPY_EXPAND	1-bpp source expanded to two colours – glyphs and icons into Modes 1 and 2
DRAW_LINE	Bresenham, arbitrary endpoints
DRAW_HLINE DRAW_VLINE	Optimised axis-aligned cases

T.34 All eight respect the clip rectangle set by **SET_CLIP**, and all take **independent source and destination strides**, so any operation may address a sub-rectangle of a larger image. That one property is what makes the atlas and the pre-rendered level buffer of T.60 work at all.

COPY_TILED is what "apply a texture" means here – filling a region by repeating a bitmap, in the QuickDraw and Amiga-blitter sense. **Perspective-correct 3D texture mapping is out of scope and not implementable here:** it needs a per-pixel divide, and the iCE40 has neither hardware multipliers nor the budget to build them.

FULL-SCREEN OPERATION, MODE 2A – 64,000 B	TRAFFIC	TIME	VS. THE 65816
FILL_RECT	64 KB write	0.31 ms	129×

FULL-SCREEN OPERATION, MODE 2A – 64,000 B	TRAFFIC	TIME	VS. THE 65816
COPY_RECT	64 KB read + 64 KB write	~1.0 ms	40×
COPY_KEYED	read src + read dst + write dst	~1.5 ms	27×
DRAW_LINE	~1 px/clock	1024 px in 10 μs	–

T.35 One frame is 16.68 ms; after scanout, cursor, audio and CPU traffic, roughly **15.5 ms** is left for drawing. That is about 50 full-screen fills, or 15 full-screen opaque copies, or 10 keyed copies – per frame. The reference on the right-hand column is the 65816 needing ~40 ms for a single full-screen rewrite through the aperture, which is the whole argument of T.2 stated as a ratio.

T.36 **Double buffering should be the default for anything animated.** **DISPLAY_BASE** holds the 24-bit SDRAM address being scanned out and **DRAW_BASE** the one the engine writes; **SWAP_BUFFERS** exchanges them **at the next vertical blanking interval, never mid-frame**, so a partially drawn frame is never displayed. The command completes immediately and the exchange is latched and applied by the timing generator. The cost is 128 KB against 64 MB.

The command interface – three layers, each usable without the ones above it.

T.37 **Layer 1, register-driven.** Parameters into the blitter registers, **BLIT_CTRL.GO** to execute, **STATUS.BLIT_BUSY** polled for completion. It needs no FIFO and no decoder, which is exactly why it exists: it is how the blitter is tested in isolation at stage N1. **It is not for production use** – the CPU is occupied for the duration of every operation.

T.38 **Layer 2, the command FIFO.** The CPU writes 32-bit command words to **CMD_PORT**, a single aperture address; Neon buffers 128 of them in EBR and executes them in order, asynchronously. Software checks **CMD_FIFO_FREE** before writing a whole command's worth of words, so the engine never begins decoding a half-written command. **The FIFO does not fill under realistic load:** the CPU produces ~1.6 MB/s while a full-screen fill is four words in and 0.31 ms of work. This layer alone decouples the CPU from drawing and is sufficient for a GUI.

T.39 **Layer 3, command lists in SDRAM – the autonomous mode, and what makes the command processor's 800 LUT worth it.** **LIST_BASE** plus **CMD_CTRL.LIST_RUN** points the processor at an instruction stream in memory rather than the FIFO.

COMMAND	EFFECT
JUMP addr	Continue at addr
CALL addr · RET	One level of subroutine nesting
WAIT_VBLANK	Suspend until the next vertical blanking interval
WAIT_LINE n	Suspend until panel line n – raster-synchronised effects
SWAP_BUFFERS	Exchange front and back buffers at the next vblank
SET_PALETTE i, rgb	Palette write; legal mid-frame after a WAIT_LINE
SET_CLIP	Set the clip rectangle for everything after it

COMMAND	EFFECT
SIGNAL <i>n</i>	Raise an interrupt to the CPU with tag <i>n</i> , readable in IRQ_TAG
END	Return to FIFO execution

T.40 A game builds its display list once, ends it with **SWAP_BUFFERS** and a **JUMP** back to the top, and **the CPU is then free: Neon renders every frame indefinitely until the list is modified**. Per-frame work reduces to patching coordinates inside a list that already exists – a handful of stores. The model is the ANTIC display list and the Amiga Copper, generalised to include the blitter, and it is the mechanism the whole sheet has been building toward.

T.41 Self-modifying lists are the intended usage pattern, not an abuse of one. Because the CPU can write into a list while Neon executes it, moving an object is a store to a parameter word of an existing **COPY_KEYED** . Two mechanisms guard against the engine reading a half-updated command, and they are for different jobs. **CMD_CTRL.LIST_LOCK** makes the engine finish its current command and stall before fetching the next; the kernel patches, clears the lock, execution resumes – but holding the lock across a vblank costs a whole frame, so patches under it must be short. **SIGNAL plus the frame gap** puts a **SIGNAL *n*** at the end of the list, immediately before it loops back to **WAIT_VBLANK** , and the CPU patches in the interval between the signal and the next frame's drawing; no lock, no stall, and a patch that arrives late affects the following frame rather than tearing the current one. **The second is for steady-state animation, the first for structural changes to the list.**

Command word format – and the revision the programming guide forced back into it.

T.42 Commands carry coordinates, not addresses, and this is a change from REV A, made while writing the game example and worth more than it first appears. Computing $dst = base + y \times stride + x$ on the 65816 is a shift-and-add sequence per object per frame – the CPU has no multiplier – and it is precisely the class of work this architecture exists to eliminate. Neon computes the address instead, from **DRAW_BASE** and **LINE_STRIDE** , at the cost of **one sequential multiply per command, ~16 clocks and ~80 LUT – 160 ns** against a command that takes tens of microseconds. The larger consequence follows immediately: because a destination is a coordinate pair packed into one 32-bit word, **moving an object is a single 32-bit patch to one word of a list**, which is what makes T.62 cost 180 µs a frame instead of several milliseconds.

WORD 0 FIELD	BITS	CONTENTS
opcode	31:24	See the operation tables above
flags	23:16	Clip enable · key enable · direction · ...
count	15:0	Parameter words following

COPY_KEYED – 6 WORDS	CONTENTS	PATCHED PER FRAME
w0	Header, count = 5	no
w1	$XY(src_x, src_y)$ – offset +4	yes, animation frame
w2	$XY(dst_x, dst_y)$ – offset +8	yes, position
w3	$WH(w, h)$	no

w4	Source base, 24-bit	no
w5	Key in 31:24, source stride in 15:0	no

T.43 **FILL_RECT** is 4 words, **COPY_RECT** and **COPY_KEYED** 6, **COPY_EXPAND** 6 with foreground and background colours in w5. Destination base and stride come from the registers; source base and stride are per-command, because atlases and level buffers have their own geometry. **An explicit word count was chosen over a fixed length per opcode so that an unknown opcode can be skipped rather than desynchronising the stream** – which matters exactly when gateway and kernel versions drift during development, and they will (→ Q51).

Two requirements on the clip unit come with this format and are not optional. It must **sign-extend coordinates and clip negatives**, so an object can hang off the left or top edge; and it must **reject a fully off-screen rectangle before issuing any memory traffic**, because the supported idiom for an unused list entry is to park it off-screen: removing it would mean rewriting the list every time an enemy dies (→ Q48).

Removed on purpose – the tile engine and the sprite engine, recorded so the argument is not had twice.

T.44 **Tile mode.** The blitter redraws a full 320 × 200 tilemap – 260 tiles of 16 × 16, 66,560 B of opaque copy – in **~1.3 ms**, under 8 % of a frame. It can therefore rebuild the entire tiled background every frame from scratch and leave 92 % for everything else, with pixel-smooth scrolling free by offsetting destination coordinates. A hardware tile engine costs ~800 LUT to accelerate that, and fragments rendering into two incompatible paths. **Sprite engine.** Blitting 32 masked 16 × 16 sprites is ~8 KB of keyed transfer, **~0.12 ms**, 0.7 % of a frame. Hardware sprites exist to avoid save/restore of background on machines that cannot afford to redraw; with double buffering and 64 MB of SDRAM that problem does not arise, because every frame is drawn fresh into the back buffer.

T.45 Combined saving **~1,650 LUT, about 21 % of the device** – which takes Neon from 95 % occupancy, not routable once audio grows, to 77 %. What is given up is real: sprites composited at scanout cost no framebuffer bandwidth and cannot be erased by drawing, which simplifies certain effects. What is gained is one rendering model, and the budget to fit the blitter, the command processor and audio on one device. **The hardware cursor is retained** (T.31).

The service port – four pins that were already there.

T.46 The RP2354B configures FPGA-B over SPI. **After configuration completes the same four pins are reused as a control channel into Neon**, costing no additional pins and roughly 200 LUT. It reaches the register file, text buffer, font and palette in EBR, and SDRAM through the same arbiter port the CPU aperture uses. Three things follow, none available any other way. **A display before the CPU exists** – the EC writes boot progress to the text buffer while Helium is still being configured and the 65816 is still in reset, exactly when a display is most needed and latest to arrive. **A debug channel independent of the CPU bus** – if the 65816 hangs, the EC can still read Neon's registers and write to the screen. And **bulk asset loading that bypasses the CPU** – at 20 MHz SPI the port sustains ~2.5 MB/s from microSD into VRAM against the CPU's ~1.3 MB/s through the aperture, and costs the 65816 nothing (→ Q52).

This is the second independent debug path on the board and it is worth seeing them as a pair. Sheet R's agent gives the EC memory and bus visibility from *inside Helium*; the service port gives it a screen and Neon's state from inside Neon. **Between them, a machine with a dead CPU still has a way to say what is wrong** – and the two paths share no logic, so a fault in one does not take the other.

Interrupts – three sources, one wire, and it goes to the PIC.

BIT	SOURCE
0	Vertical blank
1	Raster line compare against <code>RASTER_CMP</code>
2	<code>SIGNAL</code> executed from a command list; tag in <code>IRQ_TAG</code>

T.47 REV A left open whether Neon's interrupt goes straight to the 65816's `IRQB` or through FPGA-A. **This board already answered it**: sheet H routes VSync into Helium's prioritized interrupt controller, which is the right answer for three reasons – masking and priority come free from a PIC that exists, the CPU keeps a single interrupt entry path, and it costs one pin on each FPGA rather than a wire into a socket. The three sources above are aggregated inside Neon into one line and demultiplexed by the kernel reading `IRQ_STATUS`, which is `W1C`. Until the gateway exists, software polls `STATUS.VBLANK`; every usage pattern in this sheet is written to work either way. That is not a fallback, it is the bring-up path – the console at T.53 runs with no interrupt controller in existence.

Register map – base `$FF:8000`, privileged and unmappable (T.21). All registers byte-wide; reserved bits read zero and must be written zero.

OFFSET	NAME	ACCESS	DESCRIPTION
<code>\$00</code>	<code>NEON_ID</code>	R	<code>\$4E</code> , the letter <code>N</code>
<code>\$01</code>	<code>NEON_VER</code>	R	Gateway version, BCD
<code>\$02</code>	<code>NEON_MODE</code>	R/W	0 text · 1 hires mono · 2 = 320×200 · 3 = 512×300
<code>\$03</code>	<code>NEON_CTRL</code>	R/W	b0 <code>DISPLAY_EN</code> – clear paints border colour over the whole screen

OFFSET	NAME	ACCESS	DESCRIPTION
\$04	STATUS	R	b0 VBLANK · b1 HBLANK · b2 BLIT_BUSY · b3 CMD_BUSY · b6 FIFO_FULL · b7 LIST_ACTIVE
\$05	BORDER_COLOR	R/W	Palette index
\$06 · \$07	RASTER	R	Current panel line, 10-bit
\$08 · \$09	RASTER_CMP	R/W	Compare line
\$0A	IRQ_EN	R/W	Bits as the interrupt table above
\$0B	IRQ_STATUS	R/W1C	Same bits; write 1 to clear
\$0C	IRQ_TAG	R	Tag from the last SIGNAL
\$10 · \$11	TEXT_START	R/W	Top-left cell index, 12-bit
\$12 · \$13	CURSOR_POS	R/W	Cell index, 12-bit
\$14	CURSOR_CTRL	R/W	b1:0 shape · b3:2 blink · b4 enable
\$18 – \$1A	DISPLAY_BASE	R/W	24-bit SDRAM address of the displayed framebuffer
\$1B – \$1D	DRAW_BASE	R/W	24-bit SDRAM address of the draw target
\$1E · \$1F	MONO_FG · MONO_BG	R/W	Mode 1 foreground and background palette indices
\$20	VRAM_PAGE	R/W	Aperture window select; base = value × 32768
\$21 · \$22	LINE_STRIDE	R/W	Bytes per framebuffer line
\$28 – \$2D	BLIT_DST · BLIT_SRC	R/W	Destination and source base addresses
\$2E – \$31	BLIT_W · BLIT_H	R/W	Width and height in pixels
\$32 – \$35	BLIT_DST_STRIDE · BLIT_SRC_STRIDE	R/W	Bytes per line, each side
\$36 – \$38	BLIT_COLOR · BLIT_COLOR2 · BLIT_KEY	R/W	Fill colour · COPY_EXPAND background · colour key
\$39 · \$3A	BLIT_OP · BLIT_CTRL	R/W	Operation code · b0 G0 , b1 key enable, b2 clip enable
\$3C – \$43	CLIP_X0 ... CLIP_Y1	R/W	Clip rectangle, inclusive
\$48 – \$4F	PATTERN	R/W	8 × 8 bit pattern, one byte per row
\$50 – \$53	CMD_PORT	W	32-bit command word, latched on the write to \$53
\$54 · \$55	CMD_FIFO_FREE · CMD_CTRL	R · R/W	Free entries · b0 LIST_LOCK , b1 LIST_RUN , b2 FIFO_RESET
\$56 – \$5B	LIST_BASE · LIST_PC	R/W · R	List start address · current fetch address, for debugging

OFFSET	NAME	ACCESS	DESCRIPTION
\$60 – \$68	CURSOR_X · CURSOR_Y · CURSOR_GFX_CTRL · CURSOR_COLORS	R/W	Graphics-mode cursor
\$70 – \$73	–	–	Free. Was PEEK_ADDR / PEEK_DATA , deleted by T.15

T.48 TEXT_START , CURSOR_POS , RASTER_CMP and LINE_STRIDE latch on the write to their high byte, so a 16-bit STA applies them atomically with no explicit sequencing – which is what a 16-bit store does naturally, low byte first. The 24-bit address registers take effect on the write to their highest byte, three stores, same rule. **Software must never write these high-to-low**, and the gateway must not make the low-byte write visible on its own.

T.49 The palette is 256 entries of 18-bit RGB666 at **\$FE:8000** , four bytes each – R, G, B in bits 5:0, fourth byte reserved – so the address arithmetic is a shift. It wastes one byte in four and buys 262,144 selectable colours per entry, reaching the panel unmodified. The bitstream initialises the first 16 entries to the standard CGA/EGA set, which is what makes T.29 true.

This supersedes REV A, which put a 16-entry palette at **\$FE:3000** . **The palette stays in the aperture while the registers do not**, and the split is the one T.21 draws: the palette is bulk data written a kilobyte at a time and nothing about it is privileged, so it belongs with the framebuffer. The offset **\$8000** recurring in both banks is a mnemonic worth keeping – **\$FE:8000** is Neon's palette, **\$FF:8000** is Neon's control – but it is two decodes, not one, and a kernel header should not treat them as related.

Resource budget – provisional until synthesis, which is the only thing that settles any of it.

BLOCK	LUT (EST.)	BLOCK	LUT (EST.)
Timing generator	120	Blitter – 8-bpp path	700
Text character generator	200	Blitter – 1-bpp barrel shift and edge masks	400
Text cursor	60	Line drawing	250
Palette and output stage	150	Command processor – FIFO, fetch, decode, flow	800
Aperture / CPU bus interface	250	Coordinate multiplier (T.42)	80
Register file and interrupt logic	120	Graphics cursor	150
SDRAM controller and 4-way arbiter	1,200	Service port	200
Graphics scanout – Modes 1, 2a, 2b	400	Audio, reserved	800

BLOCK	LUT (EST.)	BLOCK	LUT (EST.)
		Total	~5,880 of 7,680 – 77 %

T.50 That leaves ~1,800 LUT of margin. With the tile and sprite engines it came to ~7,300 – 95 %, not routable once audio grows; the removal of T.44 is the whole difference. **Mode 0 alone is ~900 LUT, about 12 % of the device**, which is the number that matters for T.53: the console is cheap enough to synthesise on its own, long before anything else in Neon exists.

Pin budget – the recount Q12 has been asking for, and it closes, barely.

BLOCK	PINS	
Panel RGB666 + PCLK HSYNC VSYNC DE	22	18 bpp is the recommendation; 24 bpp would not fit. 33 Ω in series on every one of the 22 , placed against the FPGA – parallel edges at 51.5625 MHz into an FPC, which is the same argument as B.7's clock star and a worse case of it
SDRAM – 13 addr + 2 bank + 16 data + 8 control	39	
CPU bus tap – A0–A15, D0–D7, PHI2, RWB	26	Bank byte captured off D0–D7 during PHI2 low
I2S to the PCM5102A	4	Three if SCK is grounded – see T.9
SSPI configuration, reused as the service port	4	CRESET_B and CDONE are dedicated pins, not I/O
NEON_BUS_SEL in · NEON_BUS_BSY out · NEON_IRQ out	3	The three surviving inter-FPGA control lines. NEON_BUS_SEL validates both of Neon's windows (T.21) – moving the registers into \$FF cost no pin
VGA bring-up sync	2	The R-2R ladders hang off the panel's own RGB bus
Total	100 of ~107	

T.51 Two corrections to the old estimate, and they pull in opposite directions. It reserved **16 pins for an inter-FPGA link that D04 deleted**, and it never counted the CPU bus tap that replaced it – which is 26 pins, so the substitution costs 10 and the budget was wrong in the dangerous direction. Against that, **the bank byte is captured from D0–D7 during the low phase of PHI2**, as the 65816 emits it there, which avoids eight dedicated bank lines and is what keeps the count inside the TQ144 at all. **VDA / VPA** are not tapped: Neon has no reason to distinguish an opcode fetch from a data access.

T.52 **Backlight EN and PWM are not on this list, and that is a decision.** REV A gave them to Neon and put **BACKLIGHT_EN** in **NEON_CTRL**; they belong to the EC, which already owns every rail enable through its expander (D.6) and is the only party awake both before Neon is configured and while the switched domain is

coming down. Neon recovers two pins it needs and the bit leaves the register map. **This closes Q6** (→ D33).

What the software actually does – five scenarios, in the order the machine meets them.

T.53 Power-on: nothing configures Neon for display, because it is already displaying. When `CDONE` rises the timing generator scans and the panel gets valid video – 128 × 32 spaces, grey on black, correct borders – no register written, no mode selected, no PLL wait. The EC then switches its SPI to service-port mode and writes a banner, so every boot step after that is **visible as it happens**: Helium configuring, the card mounting, the BIOS loading, the CPU released. **That ordering – Neon useful before anything else works – should be preserved.**

T.54 The BIOS inherits a working display and initialises nothing – no probe, no mode, no font: it stores bytes to `$FE:0000` and characters appear. It reads `NEON_ID` once for `$4E`, to tell a fitted Neon from an absent one, continuing either way. `putchar` is a 16-bit `STA` to `$FE0000,x` – **absolute-long indexed: the bank byte is in the instruction, so it runs from any bank** – then a 16-bit store to `CURSOR_POS`, latching on its high byte. Scrolling is T.27's pointer move.

T.55 The kernel's mechanism is identical to the BIOS's – same stores, same addresses – and the four differences are all layering: the console becomes a kernel driver behind J.4's five-function interface, as a device node; bank `$FE` stays privileged where it matters (T.21); a custom font can be loaded into `$FE:2000` in vblank; and the display tick can come from the vblank IRQ. **One constraint must be built in from the start.** Graphics modes may reclaim the text buffer's EBR (T.12), so a mode switch and back can destroy the screen: **the console driver must hold its own scrollbar and repaint from it** (→ Q47).

T.56 The GUI, in Mode 1. Assets – the UI font as a 1-bpp glyph atlas, icons, patterns – go to VRAM once at boot, by the CPU through the aperture or by the service port at twice the rate. **Generated content never becomes pixel data in system memory**: a line of text is one `COPY_EXPAND` per glyph from the resident atlas, selected by a width table, so proportional fonts cost nothing extra. A window is a dozen commands plus one per title character – `SET_CLIP`, two `FILL_RECT`, four lines, a keyed icon, the title – ~1.2 KB of command words and **~1 ms of blitter time**. The CPU writes pixels once per asset. **The software built on this is sheet V**, whose first correction is to the line above: emitting 1.2 KB through the aperture at ~1.3 MB/s is **~0.9 ms of CPU** against that ~1 ms of blitter, so the CPU does not return to the event loop as promptly as this reads. The ratio holds only once commands live in a list that is re-executed rather than re-emitted (T.39) – which is why the GUI wants N5 for client repaint (→ V.24, V.35).

T.57 A window is not a rectangle that moves, and getting this wrong produces a design that cannot work. There are no windows in VRAM: there is one flat framebuffer of pixels, and a window is a structure in system memory – coordinates, title, widgets, z-order – with **no association in hardware between the pixels and the window that produced them**. If A sits over B and A moves away, B's pixels are not underneath; they were overwritten when A was drawn. A single `COPY_RECT` drag works, ~0.3 ms for 400 × 300 at 1 bpp, but leaves the exposed region behind it and whatever it now covers: **it is correct only over a uniform background**. Every windowing system has a damage model for this reason. **Neon makes the damage model cheap, and that is the useful part**. Repainting a whole window costs ~1 ms, so the compositor can skip precise damage-region arithmetic – the most defect-prone part of any window manager – and simply repaint every window intersecting the dirty area, back to front. **That is the recommended first implementation**; optimise to precise regions only if a case demands it. On machines of this class the trade was not available; here it is. [open](#) **A later design note reverses this**, recommending per-window backing stores with damage-limited recomposition, on the arithmetic that the budget carries ~90 full-screen composites a second – which makes full recomposition a correct fallback and damage tracking a pure optimisation. The disagreement is downstream of the mode question, not a matter of taste (→ U.31, Q60). **Sheet V then writes the OS against backing stores** and the system gets smaller for it: no refresh events, no cliprect arithmetic, ten event types, a hung client that still displays (→ V.6).

T.58 Save-under works, and it is worth spelling out why. The restriction that made it look impossible was on the CPU reading VRAM; the blitter reads VRAM without restriction. A `COPY_RECT` moves the region about to be occluded into a scratch pool in VRAM and another moves it back, and the saved pixels never leave VRAM or touch the bus. It remains a bounded special case – a menu over a known rectangle, restored before anything else can draw there – and it does not substitute for the damage model. **Per-window VRAM buffers with compositing** would remove the damage problem entirely, since occluded content would still exist and moving a window would be a recomposite rather than a repaint. With 64 MB and ~143 MB/s it is entirely affordable – ~15 KB per window on top of the 75 KB framebuffer – which is not true of most systems this size. **It is nonetheless deferred, not rejected**: it is materially more gateware and more software, and it must not be attempted before the flat model works (→ Q55).

T.59 Games, in Mode 2a, and the one place where the display list earns its logic cost. The obvious method – a tilemap in system memory redrawn per frame, one `COPY_RECT` per tile – fails, and it fails for an instructive reason. The blitter handles 260 tiles in ~1.3 ms without difficulty; **the CPU cannot feed it**. 260 commands × 6 words × 4 bytes is 6,240 bytes through the aperture at ~1.3 MB/s = **4.8 ms per frame**, nearly a third of the budget spent emitting commands rather than drawing. The bottleneck moved from the blitter to the command stream, which is the trap Layer 3 exists to avoid.

T.60 The approach that works: pre-render the level once, scroll it with one command. At level load, render the whole background into a wide SDRAM buffer – 8,192 px by 200 at 8 bpp is 1.64 MB, 2.5 % of memory, and the tile-by-tile cost is irrelevant because it happens behind a load screen. Per frame, the entire scrolling background is **one `COPY_RECT` with a source stride of 8,192**, windowing a 320-wide viewport out of the level; scrolling is a change to `scroll_x`, **one 16-bit store**. Parallax is a second such command from a second buffer with its own scroll rate, drawn first and keyed over.

T.61 Sprite frames live in a VRAM atlas loaded at level start, and each character is one `COPY_KEYED` selecting a source rectangle. The frame list – `WAIT_VBLANK`, `SWAP_BUFFERS`, the two background copies, twenty-four keyed sprites, the HUD, `SIGNAL 1`, `JUMP` – is built once and then **executed forever**; unused sprite slots are parked off-screen rather than removed, where the clip unit rejects them before any memory traffic (T.43). The CPU writes `LIST_BASE`, sets `LIST_RUN`, and from that point **Neon renders continuously with no CPU involvement at all**: if the game logic stopped entirely the display would keep refreshing at 60 Hz with the last-patched contents.

PER FRAME	BLITTER		CPU
Parallax sky	~1.0 ms	Background <code>scroll_x</code>	1 word
Level background, keyed	~1.4 ms	24 sprite positions and source frames	48 words
24 sprites at 32 × 32	~0.4 ms	HUD when it changes	~4 words
Total	~2.8 ms of 15.5 ms	~180 bytes	~140 μs of 16.68 ms

T.62 Roughly 18 % of the drawing budget and 1 % of the CPU. What is left is enough for a third parallax layer, a full-screen palette effect, or several times as many actors – and, more to the point, essentially the entire 65816 for game logic, which is what it should be doing. Synchronisation is the `SIGNAL 1` of T.41: the CPU patches between the signal and the next `WAIT_VBLANK`, no lock is taken and the engine never stalls.

The per-frame patch loop is 48 32-bit stores, the natural assembly candidate if the budget tightens – a hand-written version with absolute-long indexed addressing and a fixed unroll roughly halves it. It is not needed at 140 μs, and depends on the Calypsi addressing question that already blocks the rest of the C here (→ Q19, Q53).

Staging – N0 to N6, and where they land on the board's own build.

STAGE	CONTENT	ACCEPTANCE	BOARD STAGE
N0	Mode 0 text; service port	Console at power-on, EC only, no CPU	E1.7
N1	SDRAM controller; Mode 1 scanout; register-driven fill and copy	Bitmap displayed; blitter fills a rectangle	E5
N2	Modes 2a/2b; 256-colour palette; 8-bpp blitter; <code>COPY_KEYED</code> ; double buffering	Animated multicolour scene at 60 Hz	E5
N3	1-bpp barrel-shift blitter; <code>COPY_EXPAND</code> ; graphics cursor	GUI primitives – the Amiga milestone becomes reachable	E8
N4	Command FIFO	The CPU stops polling <code>BLIT_BUSY</code>	E8
N5	SDRAM command lists; flow control; <code>WAIT_LINE</code> ; <code>SWAP_BUFFERS</code>	Autonomous frame rendering – the games target	E8 · +

STAGE	CONTENT	ACCEPTANCE	BOARD STAGE
N6	DRAW_LINE , FILL_PATTERN , COPY_TILED	–	+
T.63	Each stage is a usable machine, and if work stops at any of them nothing is stranded. It is the layering of T.37–T.39 again: a failure in the command processor still leaves a GUI-capable blitter, a failure in SDRAM a working boot console. N3 delivers the primary milestone and N5 the games capability, in that order and separably.		
T.64	The right-hand column is the finding, not the table. Sheet P currently puts everything about video at E5, three stages after the FPGAs are first configured. N0 needs no SDRAM, no CPU, no Helium and no bus – only Neon, the panel and the EC's SPI – so it belongs at E1.7, beside the blinky that proves configuration, and it is a far better proof: a bitstream that puts 128 × 32 correct characters on a panel has demonstrated the PLL, the timing generator, EBR initialisation, the RGB pins, the FPC and the backlight in one shot. Bringing it forward gives the whole project a screen three stages earlier than planned, and every stage after it is easier to debug for having one (→ Q43). Start below the gateway even so: the panel's own BIST mode generates test patterns with no external clock and no bitstream, which separates a panel, FPC or backlight fault from a logic fault before either can be blamed on the other (E5).		
Verification – stage N0, and four of these need no CPU.			
T.65	The panel syncs and shows a stable image at 59.95 Hz with no visible tearing or jitter.		
T.66	At power-on, before any CPU activity, the screen shows a blank field in the default background colour with correct borders.		
T.67	The EC writes glyphs through the service port and they appear at the correct positions.		
T.68	RASTER tracks the display and STATUS.VBLANK sets once per frame. Those four are achievable before the 65816 is populated, which is the point of the whole arrangement and the intended E1.7 path. The four below need the CPU or a test harness driving the aperture.		
T.69	Reading the text buffer back through the aperture returns exactly what was written.		
T.70	Advancing TEXT_START by 128 scrolls the display by exactly one row, with no artefacts.		
T.71	The hardware cursor appears at CURSOR_POS , in the selected shape, blinking at the selected rate.		
T.72	A font written through the aperture takes effect on the following frame.		

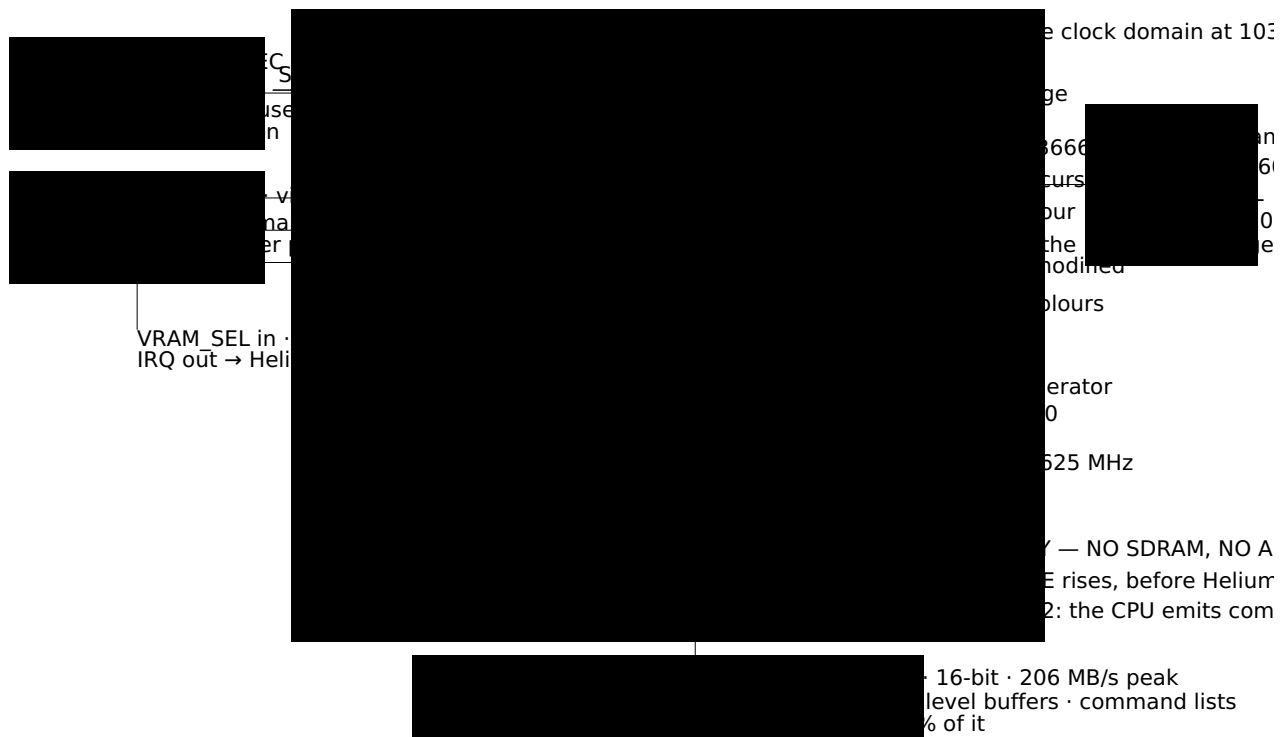


Fig. 10 – Neon. The text path on the left depends on nothing but the bitstream and reaches the panel without touching SDRAM; the graphics path on the right is fed by commands, not pixels, and the CPU's only per-frame work is patching a list it built once.

Trace legend: mint = pixel and command paths · gold = the EC's service port and bitstream initialisation · dashed = interrupts and the paths software should not use.

U Neon – blitter and compositor

FOUR CHANNELS · 256 LOGIC FUNCTIONS · BANK PARTITIONING · BACKING STORES

SHEET 19

Sheet T specifies Neon as a whole and says what the drawing engine *does*; this sheet is the engine itself – its datapath, its arbitration policy, and the window compositor built on it. It does two things at once: **it closes three open questions with real numbers, and it proposes a blitter that is not the one sheet T describes.** That second half is stated first, because everything after it is conditional on how it is resolved.

The conflict – two blitters, and they are different machines, not different register maps.

U.1 blocking Sheet T's engine is operation-based; this sheet's is channel-based. T's blitter takes FILL_RECT, COPY_KEYED, COPY_EXPAND and the rest, in commands carrying (x, y) coordinates, fed from a 128-entry FIFO or a display list in SDRAM (T.32, D30). This one is the Amiga blitter: **four memory channels – A, B, C read, D write – combined by any of 256 bitwise logic functions**, addressed by raw pointers and per-line modulus, with edge masks and a descending mode, fed from an 8-deep queue of register snapshots. Both are good designs. They are not variants of one design.

U.2 The capability difference is real and cuts both ways. The minterm LUT is bitwise, so a **colour key cannot be expressed in it** – an 8-bit equality test is not a boolean function of three bits. The channel model does transparency with a **mask surface** on channel A and logic function `$CA`, which means every sprite carries a second bitmap that something must generate and keep in step. Conversely T's fixed operation set has **no XOR**, so no rubber-band outline and no reversible cursor, and no arbitrary bitwise compositing at all.

	SHEET T – OPERATIONS	THIS SHEET – CHANNELS
Transparency	<code>COPY_KEYED</code> , one designated index, no extra memory	Mask surface plus <code>LF = \$CA</code>
Glyphs	<code>COPY_EXPAND</code> , 1-bpp atlas expanded on the fly	Mask-and-copy from an 8-bpp atlas, or a pre-expanded one
Arbitrary bitwise logic	none	256 functions , XOR included
Pattern fill	<code>FILL_PATTERN</code> , 8 × 8 register	<code>LF = \$F0</code> from channel A with a modulo of <code>-stride</code>
Addressing	<code>(x, y)</code> ; Neon computes <code>base + y × stride + x</code>	Four pointers and four modulus, computed by the CPU
Setup per blit	4–6 command words	~30 byte stores
Clipping	clip rectangle in hardware	expressed through pointer, width, height and modulo

U.3 The decisive number is setup cost, and it falls out of D30 rather than from either document's advocacy. At ~30 byte stores per blit, the channel model costs about **19 µs of CPU per blit** at 8 MHz. For the GUI that is irrelevant: a composite pass is a handful of large blits and the setup amortises over hundreds of kilobytes. **For the game scenario of T.59 it is fatal** – 24 sprites is ~450 µs per frame of pointer arithmetic, against the 140 µs that sheet T budgets for *the entire frame*, and the display list cannot help because a list of raw pointers has to be recomputed every time anything moves. This is exactly the work D30 removed by making commands carry coordinates. The 8-deep command queue of U.24 exists to mitigate this, and the case for it was originally argued against a **65816 at 14 MHz**. This board runs 8 MHz (B.1), so the figure is worse here than that argument assumed, not better.

U.4 There is a third option and it is probably the right one: keep this datapath and put T's command layer on top of it. The command processor already has to exist for display lists, and it already has the multiplier D30 budgeted; having it generate four pointers and four modulus from a coordinate pair is arithmetic it is already doing. That buys the 256 logic functions, the mask channel and the edge masks **and** coordinate commands, display lists and a bounded per-frame CPU cost. `COPY_KEYED` and `COPY_EXPAND` remain genuinely extra hardware – a comparator and a bit expander respectively – because no minterm can express either.

It is roughly LUT-neutral, which is the surprising part. This datapath is 900–1200 against sheet T's 700 for the 8-bpp path plus 400 for the 1-bpp barrel shifter – and U.10 deletes that shifter outright. The command processor's 800 and the multiplier's 80 sit on top either way (→ U.37).

U.5 Recommendation: adopt this sheet's datapath, bank policy and arbiter; keep the command model from sheet T. But this is an architectural choice with consequences for the kernel, the toolchain and both milestones, and it should be made deliberately rather than settled by whichever sheet was read last (→ Q58).

What this sheet settles, and it is more than it overturns.

U.6 The SDRAM bank partition (U.16) is a concrete policy where T.14 had only a convention, and it closes Q50. The difference it makes is a factor of two in delivered bandwidth, which is not a tuning detail.

U.7 The arbiter's worst case is proven, not asserted (U.28): 23 clocks, **223 ns** at 103.125 MHz, with preemption at burst boundaries and the CPU placed above the blitter. That is precisely the bound Q57 asked for and the priority order Q29 needs on Neon's side, and it independently confirms the read-stall protocol of D37 – the analysis arrives at "a CPU access stalls PHI2 until the controller services it" from the other direction.

U.8 Descending mode is required, not optional. The note lists it as an open question; sheet T already answered it by specifying `COPY_RECT` as overlap-safe with the direction chosen automatically (T.32), which is descending mode by another name. A window drag is the case that needs it.

U.9 Its own first open item is already closed. The note proposes `$FE:FF00` for the register window and asks for it to be reconciled against the aperture decode; D36 moved the registers to `$FF:8000` instead, because `$FE` is user-mappable and `$FF` is not.

Pixel format – the strongest new argument on this sheet, and it reaches back into Mode 1.

U.10 8 bits per pixel, palettised, because of the shifter – not because of memory. A blit whose destination does not start on a word boundary must shift its source before combining. On a 16-bit bus the shifter width is set by how many pixels share a word.

DEPTH	PIXELS PER WORD	SHIFTER POSITIONS	COST
1 bpp	16	16	The Amiga's barrel shifter – ~400 LUT (T.30)
4 bpp	4	4	Four-way mux per channel
8 bpp	2	2	A byte swap. One mux per channel

DEPTH	PIXELS PER WORD	SHIFTER POSITIONS	COST
16 bpp	1	none	Free, but see U.15

U.11 This undercuts T.30, which calls the 1-bpp barrel shifter "not optional" for the GUI. It is not optional for Mode 1, and Mode 1 is 1 bpp only because an earlier draft assumed the GUI had to be cheap. At $1024 \times 600 \times 8$ bpp the framebuffer is 614,400 B against Mode 1's 76,800 – irrelevant against 64 MB – and scanout is **36.8 MB/s against 4.6**, still under a quarter of the effective budget. What is bought: **256 colours instead of two, and ~400 LUT deleted**. `COPY_EXPAND` still needs a bit-granular source read for 1-bpp glyph atlases – that is Q49 – but it never needs a 1-bpp destination, which is what the barrel shifter was for.

U.12 So a fourth mode is implied and sheet T does not have it: **$1024 \times 600 \times 8$ bpp, the GUI mode this sheet assumes throughout**. Every bandwidth figure below is computed for it. Whether it replaces Mode 1 or joins it is not a detail – it decides whether the 400 LUT come back, and it decides what the window server is written against (→ Q59).

Memory baseline – the same part as F.8, and two consequences of its geometry.

PARAMETER	VALUE
Part	AS4C32M16SB-7TIN, 32M × 16, four banks, 64 MB (D14)
Rows per bank · columns per row	8192 · 1024 words = 2048 B
Controller clock	103.125 MHz (T.8); the original figures assumed 100
CAS latency	2
Peak · usable	206 MB/s · ~150 MB/s budgeted for mixed work

U.13 A row holds exactly 2 KB, which is two complete 1024-pixel scanlines at 8 bpp. Row management in the scanout path is therefore about one activation per two lines against 512 words of payload – negligible, and it is a property of the geometry rather than of the controller.

U.14 Refresh costs 0.84 %. One AUTO REFRESH every 7.8125 μs, each occupying `tRFC` ≈ 66 ns. Budgeted, not ignored, and not a constraint.

CONSUMER AT $1024 \times 600 \times 8$ BPP	DEMAND
Scanout, 59.95 Hz	36.8 MB/s
Audio DMA, stereo 44.1 kHz 16-bit plus mixing reads	< 1 MB/s
CPU aperture, worst case	1.6 MB/s

Left to the blitter

~111 MB/s

- U.15** A full-screen composite is 1.23 MB – read every source pixel once, write every destination pixel once – so the budget carries about 90 of them per second. The design target is 60. **This is the finding that settles the compositor model (U.31):** full recomposition is affordable, and damage tracking becomes an optimisation rather than a correctness requirement. It is also why **16 bpp is specified as a non-composited mode only** – 73.7 MB/s of scanout and double the composite traffic exceeds the budget at 60 Hz, so 16 bpp suits a single full-screen application that owns the framebuffer, never the windowed GUI. The note reaches ~100 composites per second against a 160 MB/s usable figure; the 90 above uses sheet T's more conservative ~150 MB/s and its 59.95 Hz. The conclusion is the same and does not depend on which figure is right, which is the useful thing about a margin of this size.

Bank allocation – a first-order decision, not an implementation detail.

- U.16** The four banks each carry their own open-row register, and a bank can be activated while another transfers. **Exploiting that is the difference between roughly 80 and roughly 160 MB/s delivered.** The failure being avoided is specific: a 2D blit advances by one stride at every line boundary, and if source, destination and scanout share a bank, each boundary forces a PRECHARGE and ACTIVATE that overlaps with nothing – about 40 ns, and over a 200-line blit, 8 µs of pure stall.

BANK	ADDRESS RANGE	CONTENTS
0	\$000000 – \$3FFFFFFF	Scanout framebuffer, front
1	\$400000 – \$7FFFFFFF	Scanout framebuffer, back
2	\$800000 – \$BFFFFFFF	Window backing stores, even-numbered windows
3	\$C00000 – \$FFFFFFF	Backing stores, odd windows · font atlas · icon sheets · audio buffers

- U.17** The invariant: **during composition the blitter's reads (banks 2/3), its writes (bank 0 or 1) and scanout (the other of 0/1) are always in different banks.** Every ACTIVATE is issued inside another bank's data phase. Expressing it as an address partition works only if **the controller maps the bank bits to the high address bits**, which is a requirement on the controller and not a property of the device – worth stating because the obvious alternative, interleaving banks on low bits to spread sequential traffic, would destroy the whole policy.

- U.18** **Minimum blitter burst is 8 words, 16 bytes;** shorter fails to amortise `tRCD` and the arbitration handover. This has a consequence the window manager can feel: **wide rectangles are cheaper per pixel than narrow ones.** A 16 × 400 blit and a 400 × 16 blit move identical pixel counts, but the first pays 400 line-boundary costs against 8 words of payload each. Vertical scrollbars, window borders and similar narrow furniture should be batched, or drawn by the CPU where that is practical. The kernel's VRAM allocator is not free to place a surface wherever it likes, and that obligation now has a specific shape rather than a general one (→ Q50, U.6).

The blitter datapath.

U.19 Three read channels and one write channel converge on a minterm LUT, then edge masks, then the write burst buffer. **Each channel carries a 24-bit byte pointer, a 16-bit signed modulo added at the end of every line, and an enable bit** – and a *disabled* read channel supplies a constant from its data register instead of consuming bandwidth, which is what makes a solid fill cost one write stream and no reads.

U.20 Shared across channels: width in words, height in lines, first-word and last-word masks applied to channel A for non-aligned left and right edges, descending mode for overlapping copies that move down and right, and the logic function byte.

U.21 **The modulo is what makes the transfer two-dimensional.** It is the difference between the source bitmap's stride and the width of the rectangle being moved, so the same registers that walk a rectangle also express a clip against it: clipping needs no separate hardware, which is the same economy that lets one `COPY_RECT` window a viewport out of a much wider level buffer in T.58.

U.22 **The logic function byte is indexed by {A, B, C} read as a three-bit number; bit *n* of LF is the output for input *n*.** Eight rows of a truth table, 256 functions.

LF	FUNCTION	USE
\$00	0	Clear
\$0C	B	Straight copy
\$3C	B XOR C	Rubber-band outlines, reversible cursors
\$CA	(A AND B) OR (NOT A AND C)	Cookie-cut – masked sprite over background
\$F0	A	Pattern fill from channel A
\$FA	B OR C	Additive overlay
\$FF	1	Set

U.23 `$CA` is the one worth being able to derive, because it is what every non-rectangular or partially transparent window uses. With the mask on A, the window's pixels on B and the destination read back on C, the output takes B where the mask is 1 and C where it is 0.

A	B	C	OUT	BIT
1	1	1	1	7
1	1	0	1	6
1	0	1	0	5
1	0	0	0	4
0	1	1	1	3
0	1	0	0	2
0	0	1	1	1
0	0	0	0	0

1100 1010 = \$CA . For a fully opaque rectangular window the compositor uses \$0C with A and C disabled instead – one read stream, one write stream, **half the traffic**, and it is the common case.

Register map – offsets within Neon's register page, wherever D36 puts it.

OFFSET	NAME	WIDTH	ACCESS	DESCRIPTION
\$00 – \$02 \$04 – \$06	BLT_APT · BLT_BPT	24	W	Channel A and B pointers
\$08 – \$0A \$0C – \$0E	BLT_CPT · BLT_DPT	24	W	Channel C and D pointers
\$10 – \$17	BLT_AMOD ... BLT_DMOD	16 each	W	Per-channel modulo, signed, added at end of line
\$18 – \$19 \$1A – \$1B	BLT_SIZH · BLT_SIZV	16	W	Width in words · height in lines
\$1C	BLT_LF	8	W	Logic function minterm byte
\$1D	BLT_CON	8	W	SHA SHB shift · ENA – END enables · DESC · MASK
\$1E – \$21	BLT_AFWM · BLT_ALWM	16	W	First-word and last-word masks for channel A
\$22 – \$27	BLT_ADAT · BLT_BDAT · BLT_CDAT	16 each	W	Constants supplied when a channel is disabled
\$28	BLT_START	8	W	Any write enqueues a snapshot of the register set
\$29	BLT_STATUS	8	R	b0 BUSY · b1 QFULL · b2 QEMPTY · b3 IRQ , cleared on read
\$2A	BLT_QUEUE	8	R	Queue occupancy, 0–8
\$2B	BLT_CTRL	8	W	b0 abort current · b1 flush queue · b2 IRQ enable
\$30 – \$3F	–	–	–	Reserved: line mode, area fill

U.24 BLT_START pushes a snapshot of the whole register file into an 8-deep FIFO in block RAM, so the CPU can set up the next blit while the current one runs. The completion interrupt is raised when the queue *drains*, not per blit. The queue exists because setup is ~30 stores and without it the engine idles through every one of them – which is the mitigation U.3 weighs, and it is why the queue disappears if the command layer of sheet T is adopted instead.

Multi-byte registers are little-endian and **latch on the write to their highest byte**, so a half-written pointer can never be acted on – the same discipline as T.47 and worth keeping identical across every block in Neon.

Line mode – Bresenham in hardware – and **area fill** – a single bit propagating along each line to fill polygon interiors – are both deferred to a second RTL pass, with register space reserved. Sheet T places **DRAW_LINE** at stage N6 (T.61), which agrees.

Arbitration – five clients, and one of them cannot be allowed to fail.

PRIORITY	CLIENT	DEADLINE CHARACTER
0	Refresh	Hard, but schedulable with 7.8 μ s of slack
1	Scanout FIFO refill below watermark	Hard real-time. Cannot fail
2	Audio DMA	Soft; a deep FIFO absorbs jitter
3	CPU aperture access	Stalls PHI2 while pending (D37)
4	Blitter	Best effort

U.25 Scanout is not merely high priority. If its FIFO underruns mid-line the display shows corruption for that line: there is no recovery and no degradation mode. The arbiter must therefore be *shown* to satisfy it in the worst case, not merely favour it. **This is the inverse of the Amiga's BLTPRI bit**, which let the blitter claim every free slot and starve the CPU; here the blitter must never be able to starve the display.

U.26 The CPU sits above the blitter, and that placement is what bounds D37's read stall. Preemption happens only at burst boundaries, and the minimum burst is 8 words – so a CPU read waits for one burst, never for a whole blit. That is the "bounded burst boundary" requirement of T.19 stated as an arbiter policy, and it is what stops a full-screen keyed copy from adding 1.5 ms to interrupt latency.

WORST-CASE SCANOUT STALL	CLOCKS
In-flight blitter burst cannot be preempted	8
Refresh already granted	7
PRECHARGE of the scanout bank	2
ACTIVATE	2
CAS latency	2
Arbitration and handover	2
Total	23 – 223 ns at 103.125 MHz

U.27 FIFO sizing follows from that number and has enormous margin. During the active region the FIFO drains one byte per pixel clock – 51.56 MB/s, about 25.8 Mwords/s – so a 223 ns stall drains **5.8 words**. A 256-word FIFO would already give forty times the worst case; **512 words, two block RAMs, is specified anyway** because the second also absorbs the burst-granularity mismatch during horizontal-blanking prefetch. Refill is requested at half depth.

U.28 That margin is the point rather than the number: **the guarantee survives every open item on this sheet.** If the clock drops to 51.5625 MHz under Q44's fallback, if the burst grows, if audio's demand doubles – the stall stays two orders of magnitude inside the FIFO. **A real-time guarantee that depends on an unverified estimate is not a guarantee**, and this one does not.

Hardware cursor – justified by bandwidth, not by convenience.

U.29 A 32×32 cursor in block RAM with index 0 transparent, composited into the pixel stream at scanout, positioned by `CURS_X` and `CURS_Y` – signed, so the pointer may hang off any edge – with an auto-incrementing `CURS_DATA` port for bitmap upload and a three-entry `CURS_PAL`. **Pointer motion is the highest-frequency event in a GUI:** without an overlay every mouse movement damages two rectangles, the old position and the new, and forces a composite pass. With one it costs two register writes and **zero memory traffic**. This is the same conclusion T.43 reached from the opposite direction – it is why the cursor survived when the sprite engine did not.

One reconciliation. The note specifies 4 bpp, sheet T 2 bpp; both fit 512 bytes at 32×32 only if the depth is 4, and both describe three colours plus transparent, which needs 2. **2 bpp at 32×32 is 256 bytes and leaves half a block spare** – take that, and say so, rather than carrying two numbers.

Double buffering.

U.30 Front and back buffers occupy banks 0 and 1; the compositor writes the back, scanout reads the front, and they exchange at `VBLANK` by swapping the base address register. **The swap also exchanges which bank holds which role, and the separation invariant of U.17 holds either way round** – which is not luck, it is why the partition was drawn with two framebuffer banks rather than one. A `VBLANK` interrupt and a `FB_SWAP_PENDING` status bit let the kernel avoid starting a composite it cannot finish before the next swap point.

The compositor – and this reverses a deferral in sheet T.

MODEL	BANDWIDTH	MEMORY	KERNEL COMPLEXITY	HIDDEN CONTENT
A – single framebuffer, damage list, clipping (Intuition, X11)	Minimal	Minimal	High – clip-region arithmetic in every drawing path	Lost; the application must redraw
B – backing stores, full recomposition (Quartz, Wayland)	High	High	Low	Preserved
C – backing stores, recomposition of damaged rectangles only	Moderate	High	Moderate	Preserved

U.31 **Model C, implemented strictly as an optimisation of Model B.** The compositor is written so that a full-screen composite is always correct and always affordable; damage tracking then restricts the work actually done. **A bug in damage accounting therefore costs performance, not visible corruption**, and a `COMPOSITE_ALL` path is always available as a fallback. That inversion – correctness in the slow path, speed in the fast one – is the whole value of the model, and it is only reachable because U.15 showed full recomposition fits.

Sheet V is what this model looks like as an operating system, and it makes one correction to the margin quoted here. Full recomposition fits *once* a frame at 8 bpp, not repeatedly: a typical desktop at 1.8× overdraw is ~2.2 MB and consumes essentially the whole 15.5 ms. So the "optimisation" half of Model C has to be written from the first day rather than held in reserve – the fallback stays correct, but it stops being the routine path (→ V.11).

U.32 Sheet T recommends Model A and defers backing stores to Q55; this sheet recommends C and gives the arithmetic. The disagreement is not really about compositing – it is about *which framebuffer*. T argued Model A against a flat 1-bpp Mode 1 where a repaint is cheap and memory was assumed tight; this sheet argues C against 8-bpp backing stores with 64 MB behind them. **Both are right about their own premise**, and the premise is Q59's mode question, not a matter of taste (→ Q60).

The consequences for the kernel are opposite and neither is retrofittable cheaply. Under A, occluded pixels are **gone** and the window manager must repaint from application state – T.55 spells out why that forces a damage model. Under C they are **preserved**, and moving a window is a recomposite. Writing the window server against one and switching to the other is close to a rewrite.

ITEM AT 1024 × 600 × 8 BPP	SIZE
Front buffer · back buffer	600 KB each
Backing store, full-screen window	600 KB
Backing store, typical 640 × 400 window	250 KB
Font atlas, one face	~64 KB

U.33 Against 64 MB, a generous allowance for backing stores is a few percent of memory. **Memory is not a constraint on this decision** – which is the same thing T.3 says about Neon generally, and it is why the decision can be made on kernel complexity instead.

U.34 The composite pass, for each damaged rectangle, back to front over the window stack: clip the window's visible region against the damaged rectangle · skip if empty · **if opaque and rectangular, one blit with LF = \$0C, channels B and D only** · otherwise one blit with LF = \$CA, the mask on A, the window's pixels on B and the destination read back on C. Clipping is expressed entirely through the pointer, width, height and modulo registers (U.21).

U.35 Two drawing paths into a backing store coexist and both are needed. The **blitter path** covers rectangle fills, glyph blits from the font atlas, widget chrome from an icon sheet, and scrolling a text region – and text is the case that matters: with the font resident in bank 3, drawing a character is an 8 × 16 blit and the CPU issues only the command. **That is the difference between a usable terminal and one that visibly redraws.** The **CPU aperture path** covers irregular geometry the blitter cannot express; it stalls PHI2 (D37) and is reserved for cases with no alternative.

The aperture exists for irregular drawing, **not for bulk transfer** – which restates T.16 from the compositor's side and is the same rule the whole of Neon is arranged around.

Resource cost – and the block RAM does not close.

BLOCK	LUT4	BLOCKS
SDRAM controller with bank interleaving	700–900	–
Arbiter	200–300	–
Blitter datapath and address generators	900–1200	–
Blitter command queue, 8 deep	150	2
Scanout FIFO	100	2

BLOCK	LUT4	BLOCKS
Palette RAM	50	2
Cursor overlay	200	1
Timing generator	200	—
Audio DMA and mixer	400	2
CPU bus interface and register file	400	—
Total	~3300–3900	9

U.36 The conclusion these figures were first drawn to support – that this is at or above the ceiling – was true of an HX4K and is not true here. They were estimated against 3520 usable LUT4, which made the risk look genuine; D01 put HX8K in every socket, so the same figures land at about half the device and that risk is closed before it was raised. What the estimate could not account for is what else shares the part: Mode 0's text generator, the command processor, and everything else in T.48.

U.37 Folded into sheet T's budget the totals barely move, which is U.4's point in numbers: **this datapath at 900–1200 replaces T's 700 for the 8-bpp path plus 400 for the 1-bpp barrel shifter, and U.11 deletes the shifter anyway.** Neon stays near 6,000 of 7,680 – about 78 % either way. **Logic is not what decides Q58.**

U.38 blocking Block RAM is what decides it, and it does not fit. Sheet T is at 32 of 32 with zero margin (T.12). This sheet's 9 blocks include **2 for audio that T never counted and 2 for the 8-deep snapshot queue against T's 1 for the command FIFO** – so the combined design wants 34 or 35 blocks, before the blitter's own burst buffers. The relief exists and is already identified: **a 128-glyph font frees 4 blocks (Q45), and the cursor at 2 bpp rather than 4 gives back half of one (U.29).** **What was insurance is now spent,** and it must be decided before RTL rather than after synthesis (→ Q61).

Stale context these figures were first drawn against, recorded so it is not carried forward.

ASSUMED THEN	SUPERSEDED BY
ice40 HX4K, LUT budget at the ceiling	HX8K in all three sockets (D01)
ANX6345 bridge in the scanout pipeline	Direct RGB-TTL to the panel (D05); the bridge is a future variant only (Q23)
Working resolution open, 14" panel under consideration	10.1", 1024 × 600, fixed (D05)
Register window proposed at \$FE:FF00	\$FF:8000 , out of the user-mappable bank (D36)
65816 at 14 MHz	8 MHz (B.1) – setup costs are worse here, not better
SDRAM controller at 100 MHz	103.125 MHz, from the 25 MHz oscillator (T.4)

ASSUMED THEN

SUPERSEDED BY

CPU aperture writes stall PHI2

Writes are posted through a 32-entry FIFO and never stall; only reads stall (T.17, D37)

APS6404L PSRAM, ~66 MB/s QSPI ceiling

Already removed board-wide (D13)

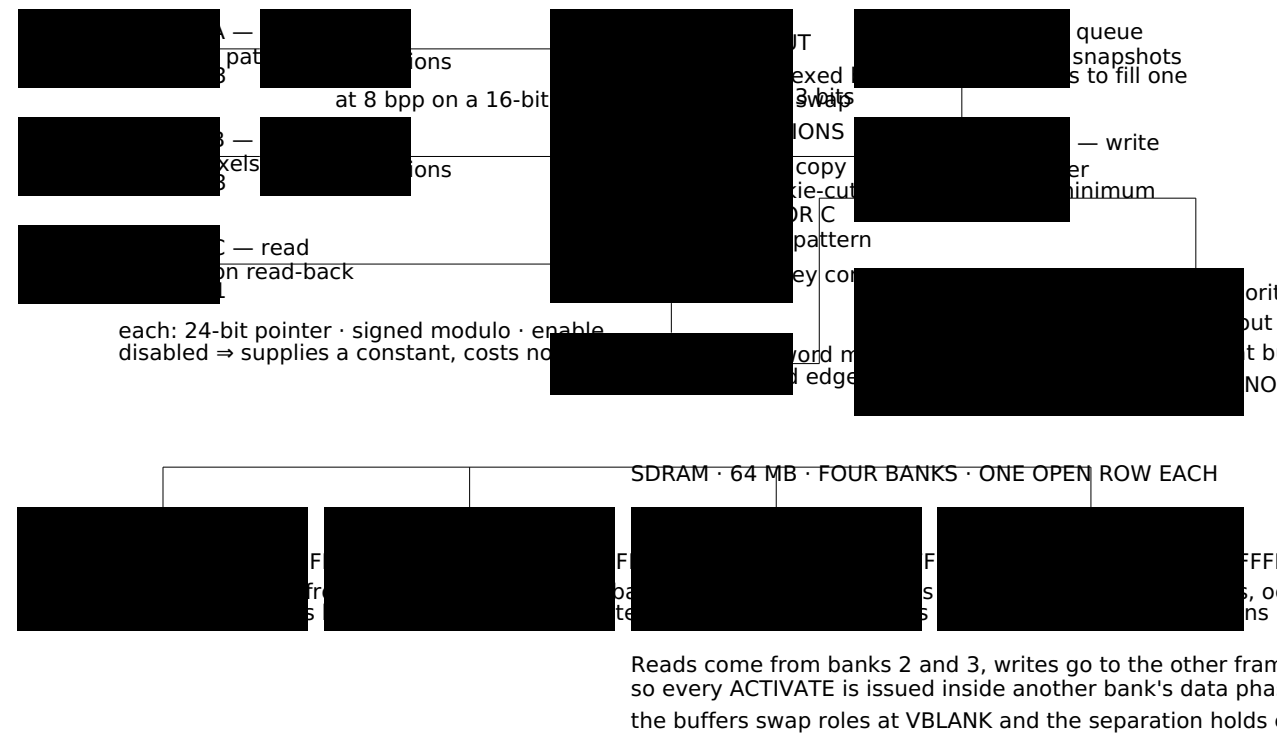


Fig. 11 – The blitter and the bank partition. Three read channels and one write channel meet in the minterm LUT; the partition below is what keeps every ACTIVATE inside another bank's data phase, and it is worth a factor of two in delivered bandwidth.

Trace legend: mint = channel and pixel data · gold = the bank partition and the arbiter's hard deadline · dashed = the paths that exist only under the channel model of U.1.

Audio is the last block of Neon with no sheet of its own, and it has been specified by accident: a row in sheet H, a line of the pin budget in sheet T, a reserved figure in two resource tables, a priority level in an arbiter, a bank in a memory partition, and four checkboxes in sheet P. None of it is wrong. But nothing anywhere says how many channels there are, what the sample rate is, what a buffer looks like, what happens when one runs dry, or what `/dev/audio` accepts – and one of those questions turns out not to have the answer everybody assumed. This sheet is the whole of it, at REV A. It shares Neon's device, its memory arbiter and its register page with sheets T and U, and specifies nothing about video.

Three properties this sheet is built on, and the third is the one that pays for the design.

W.1 The CPU never touches a sample. It is the same rule the whole of Neon is arranged around (T.2, J.5), and here it is not a preference but arithmetic. At 8 MHz the 65816 has **181 cycles per stereo frame** at this sheet's rate; mixing four streams costs eight fetches, eight sign extensions, eight volume multiplies on a machine with no multiplier, four saturating adds and two stores. Software mixing is not a slow option on this board, it is **not an available one**, and everything below follows from that.

W.2 A tone depends on nothing. The I2S serialiser and its divider need no SDRAM, no arbiter, no CPU, no Helium and no bus – only the bitstream, three pins and the DAC. `AUD_TEST` (W.30) generates a square wave inside the serialiser with no memory access of any kind, which makes audio's first bring-up stage as independent as Mode 0's (T.1) and puts it in the same place on the build.

W.3 Audio is the cheapest client in the machine, by two orders of magnitude. 0.7 MB/s against ~150 MB/s of delivered bandwidth, ~500 LUT of 7,680, one block RAM, three pins. Every trade below is decided on logic and on block RAM, never on bandwidth – which is T.3 restated for a block that is smaller than the cursor's neighbours.

Nothing on this sheet depends on Q58. The audio path reaches memory through the arbiter port of P.15, not through the blitter, so whichever engine wins – sheet T's operations or sheet U's channels – this block is unchanged. It is worth saying because almost everything else about Neon is currently conditional on that question.

The part – the PCM5102A, and its strapping is half the specification.

PIN	STRAP	WHAT IT DECIDES
FLT	GND	Normal latency filter rather than low latency
DEMP	GND	De-emphasis off – it is a 44.1 kHz-only legacy filter and would colour everything else
FMT	GND	I2S format – data delayed one <code>BCK</code> after the <code>LRCK</code> edge, MSB first
XSMT	3V3 through an RC	Soft un-mute, delayed past the rail's edge. The delay is the anti-pop

PIN	STRAP	WHAT IT DECIDES
SCK	GND	The part runs its own internal PLL off BCK and needs no master clock
W.4	SCK to GND is the decision the pin count rests on.	With it grounded the PCM5102A synthesises its own internal clocks from BCK alone, so Neon owes the DAC three driven signals – BCK , LRCK , DIN – and not four (P.09). Sheet T's pin budget still carries the four-pin figure with a conditional beside it; sheet H has already grounded the pin, so the condition is met and the budget should say three (W.38).
W.5	It also decides what the sample rate is, which is the part nobody has written down.	There is no master clock against which BCK is a ratio; the DAC's PLL locks to BCK , and the sample rate is therefore exactly BCK divided by the frame length , whatever Neon happens to emit. Software cannot ask for 44.1 kHz. It gets whatever the divider produces, and the machine's only honest move is to tell it what that is (W.29).
W.6	The frame is 32 BCK per stereo pair – 32fs – and that is a specification, not an economy.	Two 16-bit slots is exactly the payload with no padding bits, and W.11 shows it is also the only ratio from which an integer divider lands anywhere near a standard rate. The part's PLL supports 32fs, 48fs and 64fs; 64fs is the more common choice elsewhere and is the wrong one here. Checked against the datasheet, and it holds with one condition attached that this design already meets. 32fs is the <i>minimum</i> ratio the part accepts, valid in PLL mode at every rate from 32 kHz upward – and it is legal only with 16-bit audio data : 16-bit takes 32, 48 or 64fs, 24-bit takes 48 or 64, 32-bit takes 64 alone. The mixer emits 16-bit (W.16, W.27), so the condition is satisfied by construction rather than by care – but it means the frame length and the sample width are now coupled , and widening the output later would cost the pin W.4 saved. 8 kHz supports neither 32fs nor 64fs and 16 kHz supports only 64fs. Both are unreachable here and neither is wanted ; W.12's writable divisor must not be swept into them during bring-up.
W.7	XSMT 's RC is sized for the rise of 3V3_MAIN, and the part is on that rail with the panel logic, the touch controller and the card (C.10).	Its share of the budget is already counted – 95 mA typical in the 130 mA group of sheet C – and audio adds nothing to the power path that is not already there. The strap should not be a strap at all, and the datasheet is blunter about it than this sheet was: XSMT must be a driven pin. An RC can only delay un-mute past a <i>rising</i> edge; it can do nothing about a falling one, which is exactly the power-down click of W.40. The general rule it is an instance of: no functional strap belongs on a passive-only net (→ Q113). Which party drives it is the open half, and the answer is probably not the one W.40 proposed. That item offered a Neon pin driven by AUD_CTRL.MUTE , which is the natural place while the machine is running and the wrong place at both edges that matter: at power-up Neon is not configured yet, and at shutdown the switched domain is coming down around it. That is T.52's argument for the backlight, verbatim – the EC is the only party awake before Neon exists and while the rails are dropping – and it points the same way here, at one GPIO out of the 48 D.6 now has spare. AUD_CTRL.MUTE then stays a <i>mixer</i> mute, which is what software wants between tracks, and XSMT becomes a power-sequencing signal owned by whoever owns the sequence.

W.7a blocking **BCK** and **LRCK** must be continuous and glitch-free whenever the DAC is powered, including when nothing is playing. The part re-initialises itself if the relationship between the two is invalid for more than four **LRCK** periods, and forces its outputs to bipolar zero until they resynchronise – so gating the clocks to save power, or stopping them between tracks, produces a mute of up to a sample period and a re-lock of the internal PLL every time. The serialiser therefore runs from reset and emits zero PCM when idle; it never stops.

This is not in tension with W.22's underrun behaviour, it is the same instinct one level down: a channel holds its last sample rather than emitting zero because a step to zero is a click, and the serialiser emits zeros rather than nothing because stopping is worse than silence. It also means W.2's "a tone depends on nothing" has a companion obligation – the clocks depend on nothing either, and must survive every mode change, every blit storm and every idle period.

TEST ▶ scope **BCK** and **LRCK** continuously across a mode switch, a blitter saturation run and ten minutes of silence – no gap, no phase discontinuity, and **LRCK** exactly **BCK** / 32 throughout.

The sample rate does not fall out of the clock, and this is the finding the sheet exists for.

Neon is one clock domain at **103.125 MHz** (D28, T.8), derived from the board's single 25 MHz oscillator (B.7) as $25 \times 33 / 8$. A **BCK** divider off that domain is an integer.

TARGET	FRAME	EXACT DIVISOR	NEAREST INTEGER	RATE OBTAINED	ERROR
44.1 kHz	32fs	73.078	73	44,146.0 Hz	+0.104 %
48 kHz	32fs	67.139	67	48,099.3 Hz	+0.207 %
44.1 kHz	64fs	36.539	36 · 37	44,758.7 · 43,549.4 Hz	+1.49 % · -1.25 %
48 kHz	64fs	33.569	33 · 34	48,828.1 · 47,391.8 Hz	+1.73 % · -1.27 %

W.8 At 64fs nothing lands within 1.2 %, and at 32fs both standard rates land within 0.21 %. That is the whole argument for W.6, and it is a property of 103.125 MHz rather than of the audio design – the divider is small, so the quantisation is coarse, and halving the frame halves the divisor's granularity in the only direction that matters.

W.9 Specified: divisor 73, **BCK** = 1.412671 MHz, fs = 44,146.0 Hz. The error is +1.8 cents of pitch and 3.75 seconds per hour of drift. The first is below the threshold at which a trained listener identifies a mistuning in isolation; the second matters only against an external time reference, and this machine has none – there is no video playback to sync to and no capture path to sync with.

W.10 The frame is exactly 2,336 clocks, by construction – 32×73 – so every downstream number on this sheet is an integer and the mixer's schedule (W.26) has no ragged edge to design around. Choosing the divisor first and reporting the rate afterwards is what buys that; choosing the rate first would have bought a fractional divider and a jittered **BCK**.

- W.11** **blocking** **Exact 44.1 or 48 kHz is unreachable on this board by any PLL configuration, and that closes half of Q54.** The iCE40 PLL is integer-only – $f_{out} = f_{in} \times (DIVF+1) / ((DIVR+1) \times 2^{DIVQ})$ – so from 25 MHz the achievable denominators are $(DIVR+1) \leq 16$ times a power of two. **44.1 kHz needs a factor of 441 = 3² × 7²** in the denominator and **48 kHz needs 3,125 = 5⁵**; neither is expressible in that form at any setting. The second PLL cannot produce an exact audio clock, so **audio has no claim on it** – not a weaker claim than VGA's, as T.9 says, but none at all. Q54 reduces to the question it should always have been: whether VGA is worth its two pins. **The two exact routes both cost a part, and both are refused here.** A second oscillator at 11.2896 or 24.576 MHz gives 44.1 or 48 kHz exactly and breaks B.7's one-oscillator star, which exists so that no cross-chip skew has to be reasoned about. A fractional NCO gives an exact average rate with one 103.125 MHz period – 9.7 ns – of cycle-to-cycle jitter on **BCK**, fed to a PLL whose datasheet expects a clean one. **Neither is worth 1.8 cents**, and the decision should be recorded rather than rediscovered (→ Q77).
- W.12** **AUD_DIV is writable anyway**, defaulting to 73. It costs eight flip-flops and buys three things: a bring-up sweep that finds the DAC's actual lock range instead of trusting it, a path to 48 kHz for material that arrives at 48, and the ability to slew the rate deliberately if a resampler is ever written. **AUD_RATE** tracks it, so software is never told a rate the hardware is not producing.
- W.13** **Resampling is a software problem and this sheet does not solve it.** Content authored at exactly 44.1 kHz plays 0.104 % sharp, which is correct behaviour for a machine with one clock: the alternative is an interpolator in the mixer, and an interpolator needs a multiplier per channel per sample rather than the one time-multiplexed unit of W.26. **Tracker and sample playback are unaffected in principle** – a sample's pitch is set by its own step rate, not by fs – and that is what this machine is actually for.
- What plays** – four channels, and the count is derived rather than chosen.
- W.14** **Four, because that is what T.13's 0.7 MB/s already is.** One stereo stream at 16 bits and this sheet's rate is $44,146 \times 4 = 176,584$ B/s; four of them is **706 KB/s**, which is the figure sheet T has carried since REV B with nothing behind it. The number was right and its derivation was missing. U.14's "< 1 MB/s plus mixing reads" is the same figure with the mixing reads counted twice – in hardware the mixed result never goes to memory at all.
- W.15** **Mixing in hardware costs reads only, and that is not a small distinction.** A software mixer reads N streams and writes one buffer; this one reads N streams and writes nothing, because the accumulator's output goes straight into the serialiser. **There is no audio write traffic anywhere in the machine.**
- W.16** Each channel is independently **16-bit or 8-bit, mono or stereo**. A mono channel costs half the bandwidth and is the common case – a tracker's voices, a UI click, a game's effects are all mono, and stereo is for streamed music. Four stereo 16-bit channels is therefore the worst case rather than the expected one; four mono 16-bit channels is 353 KB/s.
- W.17** **Per-channel VOL_L and VOL_R, 8-bit linear, and pan is what falls out.** Two volumes rather than a volume and a pan costs one more register and no more hardware – the multiplier runs twice per frame per channel either way – and it expresses hard-panned, centred and anything between without a pan law having to be agreed on.

W.18 Why not eight channels, and why not one. Eight doubles the ring generators and the multiplier's schedule, still fits the bandwidth trivially, and does not fit the block RAM (W.36) – which is the constrained resource on this device and the subject of a blocking question already. One channel puts mixing back on the 65816 and is excluded by W.1. Four is the number at which a GUI's system sounds, a game's music and two effect voices coexist without the kernel having to steal one.

The DMA engine – one ring per channel, and the underrun behaviour is a decision.

W.19 Each channel is a **ring buffer in Neon's SDRAM**: a 24-bit base, 16-byte aligned, and a length in 16-byte units, giving 16 B to 1 MB. Buffers live in **bank 3** with the font atlas and the icon sheets (D38, U.16) – the bank that carries everything read at low rate, which is exactly what audio is, and which keeps the framebuffer banks free for the invariant of U.17.

W.20 LOOP distinguishes the two things a ring is used for. Set, the pointer wraps and the channel plays until stopped – streaming refilled by software, or a looped instrument. Clear, the channel plays **LEN** once, stops, clears its own **EN** and raises an end interrupt – a fire-and-forget sound effect, which is what a GUI and a game mostly want and which costs no further CPU attention at all.

W.21 The fetch engine requests 8 words, 16 bytes, and never less. That is U.18's minimum burst, obeyed here for the same reason: shorter fails to amortise **tRCD** and the arbitration handover. At one stereo channel's rate a 16 B burst covers four frames – 90.6 μ s – so four active channels average **one burst per sample frame**, about 28 of the frame's 2,336 clocks.

W.22 On underrun the channel holds its last sample and sets a sticky UNDERRUN bit. The alternative – emitting zero – is worse and obviously so: a jump from wherever the waveform was to zero is a step discontinuity, which is a click, and a stream that stutters would click twice per gap. Holding is a DC offset for the duration, which is inaudible for the milliseconds an underrun lasts and audible only as the silence it already is. **The flag is what matters:** an underrun that cannot be observed is a bug report that says "it crackles sometimes."

W.23 AUD_POS reports the read pointer, latched atomically. Writing **AUD_POS_SEL** captures the selected channel's 24-bit pointer into three registers that the CPU then reads at its leisure – the same latch-then-read discipline S.16 uses for telemetry, and for the same reason: a 24-bit value moving 44,146 times a second, read a byte at a time by an 8-bit CPU, otherwise produces pointers that were never real.

W.24 The refill interrupt fires at the half-way point and at the wrap, which gives software a whole half-buffer of slack. A 4 KB stereo ring is 23.2 ms of audio, so the interrupt arrives every **11.6 ms** – a comfortable multiple of the 100 Hz scheduler tick (N.3), and a refill that misses one deadline still has the other half of the ring behind it. P.11's 128 KB of audio buffers on the prototype is four channels of 32 KB, which is eight times this and never a constraint.

The mixer – one serial multiplier, time-multiplexed, and the schedule is not tight.

W.25 Per frame: for each enabled channel, fetch its next sample from the channel FIFO, sign-extend 8-bit sources, multiply by `VOL_L` and by `VOL_R`, and accumulate into two 24-bit sums. Then **saturate** to 16 bits and hand both to the serialiser.

W.26 **Eight multiplies per frame against 2,336 clocks, so one shift-add multiplier does the whole job.** A 16×8 serial multiplier is eight cycles; eight of them is 64 clocks, **2.7 % of the frame**. Building four parallel multipliers would cost roughly 600 LUT and buy nothing – this is the same economy the coordinate multiplier of T.42 applies to the command processor, one device over.

W.27 **The accumulator saturates rather than wraps, and this is not a detail.** Four channels at full volume sum to four times full scale; a wrapping accumulator turns that into a sign flip, which is full-amplitude noise – the loudest possible output at the exact moment the material is loudest. Saturation is a comparator and a mux per rail. **Wrapping is the single easiest way to destroy a speaker or a listener on this machine**, and it must be in the RTL from the first line rather than added after the first time somebody hears it.

Register map – Neon's page at `$FF:8000`, offsets `$80–$AF`, privileged and unmapable (T.21).

Channel n occupies eight bytes at `$80 + 8n`, for $n = 0 \dots 3$.

OFFSET	NAME	ACCESS	DESCRIPTION
+0 – +2	<code>AUDn_BASE</code>	W	Ring base, 24-bit, 16-byte aligned; low 4 bits ignored
+3 · +4	<code>AUDn_LEN</code>	W	Ring length in 16-byte units, 1–65535
+5 · +6	<code>AUDn_VOL_L</code> · <code>AUDn_VOL_R</code>	R/W	Linear volume per output rail, 0–255
+7	<code>AUDn_CTRL</code>	R/W	b0 <code>EN</code> · b1 <code>LOOP</code> · b2 <code>FMT8</code> · b3 <code>STEREO</code> · b4 <code>IRQ_HALF</code> · b5 <code>IRQ_END</code>
\$A0	<code>AUD_CTRL</code>	R/W	b0 <code>MASTER_EN</code> · b1 <code>MUTE</code> – drives <code>XSMT</code> low · b2 <code>RESET</code> , clears every pointer and the FIFO
\$A1	<code>AUD_STATUS</code>	R	b3:0 channel active · b4 <code>UNDERRUN</code> , sticky, cleared on read · b5 <code>XSMT</code> state as driven
\$A2	<code>AUD_IRQ_STATUS</code>	R/W1C	b3:0 half-buffer per channel · b7:4 wrap or end per channel
\$A3	<code>AUD_IRQ_EN</code>	R/W	Same bits
\$A4 · \$A5	<code>AUD_RATE</code>	R	Actual sample rate in Hz. <code>\$AC72</code> = 44,146 at the default divisor
\$A6	<code>AUD_DIV</code>	R/W	<code>BCK</code> divisor from the 103.125 MHz domain. Default <code>\$49</code> = 73
\$A7	<code>AUD_POS_SEL</code>	W	Channel select; the write latches that channel's pointer into <code>AUD_POS</code>
\$A8 – \$AA	<code>AUD_POS</code>	R	Latched 24-bit read pointer

OFFSET	NAME	ACCESS	DESCRIPTION
\$AB	AUD_TEST	R/W	b0 enable · b3:1 tone divisor · b7:4 amplitude. Square wave into the serialiser, no memory touched
\$AC – \$AF	–	–	Reserved

W.28 **\$80 – \$AF collides with nothing.** Sheet T's map ends at \$68 with \$70 – \$73 free, and sheet U's proposed blitter map occupies \$00 – \$3F of the same page under either outcome of Q58. Audio sits above both with 32 bytes of room to grow, which is what the page was left with space for.

W.29 **AUD_RATE is read-only and exists so that no piece of software ever has to assume a rate.** It is the whole software consequence of W.5: a mixer, a tracker replayer or a WAV player computes its step rate from what the register says rather than from a constant in a header, and the day **AUD_DIV** changes, nothing downstream is wrong. **A machine that knows its own sample rate costs two bytes of register file.**

W.30 **AUDn_BASE and AUDn_LEN latch on the write to their highest byte,** so a partially written pointer can never be acted on – the same rule as T.48 and U.24, and it is worth being identical across every block in Neon rather than nearly identical.

Interrupts – a fourth source on a wire that carries three.

W.31 Sheet T's interrupt table has bits 0, 1 and 2 – vblank, raster compare, **SIGNAL** – aggregated into the single **NEON_IRQ** line into Helium's PIC (T.47). **Audio takes bit 3,** and **AUD_IRQ_STATUS** demultiplexes which of the eight conditions raised it. No new pin, no new wire, and the kernel's single interrupt entry path is untouched.

W.32 **The rate is per streaming channel, and only streamed channels have one.** With a 4 KB ring one channel interrupts **86 times a second**; four of them is 345, against a 100 Hz scheduler tick. A channel playing a **resident sample** – one-shot or looped, already in SDRAM – interrupts once at the end or never, because there is nothing to refill. **The interrupt rate is a property of how many streams are running, not of how many channels exist,** and the ring length trades it away directly: 32 KB gives 10.8 a second.

Bring-up needs no interrupt at all. A driver can poll **AUD_POS** against the half-way mark, which is what stages A0 and A1 do – exactly as T.47's note describes for vblank, and for the same reason: the gateway arrives before the PIC does.

Arbitration and bandwidth – the cheapest client in the machine.

CLIENT	PRIORITY (D39)	DEMAND	SLACK AGAINST THE ARBITER'S 223 NS WORST CASE
Scanout FIFO	1	36.8 MB/s	5.8 words of a 512-word FIFO (U.27)
Audio DMA	2	0.7 MB/s worst case	0.72 ms of buffered audio – about 3,200×
CPU aperture	3	1.6 MB/s	Stalls PHI2 (D37)

CLIENT	PRIORITY (D39)	DEMAND	SLACK AGAINST THE ARBITER'S 223 NS WORST CASE
Blitter	4	the rest	Best effort

W.33 Audio's real-time claim is real but enormously over-served. A 128-byte per-channel FIFO holds 32 stereo frames – 0.72 ms – against an arbiter whose proven worst case is 223 ns. U.28 says the scanout guarantee survives audio's demand doubling; the inverse is worth stating too, which is that audio's guarantee survives essentially anything the rest of the machine can do to it.

W.34 Which means audio is the cheapest thing in the arbiter to demote, if the CPU's latency is ever measured as a problem. Its placement above the CPU costs a CPU access at most one extra burst – about 78 ns – and buys jitter immunity it does not need. The recommendation is to leave D39 exactly as it stands: 78 ns is not worth an argument, and a fixed priority order that has been reasoned through once is worth more than a marginally better one that has not. It is recorded here so that if that latency is ever contested, the cheapest concession is already identified.

Resource cost – and it gives a block back to Q61.

BLOCK	LUT4	EBR
I2S serialiser, BCK / LRCK divider, test tone	60	–
Ring address generators, four channels	140	–
Fetch engine and memory port	90	–
Channel FIFOs, 128 B each	–	1
Serial multiplier, time-multiplexed	60	–
Accumulator with saturation	50	–
Register file, IRQ logic, AUD_POS latch	100	–
Total	~500	1

W.35 T.50 reserves 800 LUT for audio and this is the first time that reservation has had anything behind it. ~500 returns roughly 300 LUT to Neon's margin, which moves the device from ~77 % to about 73 %. That is not slack to spend – it is the difference between a routable design and an argument with the place-and-route tool, and T.45 is on record that the whole point of removing the sprite engine was to fit audio at all.

W.36 One block RAM, not two, and that is a real relief on a blocking question. iCE40 EBR is 4 kbit – 512 B – and four channel FIFOs of 128 B fit one block exactly, written on port A by the fetch engine and read on port B by the mixer. U.36 budgeted 2 blocks for audio with no channel count behind the figure; Q61 then counted those two against a design already at 32 of 32. This returns one of them. The question does not close – it wants 33 or 34 blocks against 32, and the 128-glyph font of Q45 is still the relief that decides it – but it is one block less bad than it was, and the figure it replaces was an estimate rather than a design.

W.37 **Eight channels would want a second block and are refused on exactly that ground** (W.18). It is the clearest case on the whole device of block RAM, not logic and not bandwidth, deciding a feature – which is what T.3 claims generally and this sheet demonstrates specifically.

Pins – three, and sheet T's budget closes with one more than it claims.

W.38 **BCK , LRCK , DIN .** Sheet T's pin budget lists **4** for I2S with "three if **SCK** is grounded" beside it, and sheet H grounds **SCK**; the condition is met and has been since the strapping was written down, so **Neon's total is 99 of ~107, not 100**. **XSMT** is not a fourth pin **provided it is the EC that drives it** (W.7); the count only rises to 100 if the pin is taken from Neon instead, which is the reason the ownership question in W.40 is a pin-budget question as well as an audio one.

The analogue side – the jack now, the speaker later, and a click nobody has budgeted for.

W.39 **Output is a 3.5 mm jack today.** P.21 already places a **PAM8302 and a pair of speaker pads, unpopulated, behind a jumper** on schematic sheet 1.5, against the laptop build at L4 – a square centimetre now against a respin later. This sheet owns that footprint: the DAC's line output feeds the amplifier input, the jumper selects jack or amplifier, and nothing about the gateway changes when it is populated.

W.40 **open Shutdown has no mute step, and it should.** The RC on **XSMT** exists because a rail edge through an un-muted DAC is a click in the headphones (sheet H) – and S.13's shutdown sequence takes the machine all the way to **POWER_CTRL** with nothing anywhere muting the output first. The power-up click was designed out and **the power-down click was never considered**, which is the more audible of the two because the listener is wearing the headphones by then. The fix is two things: **XSMT** driven from **AUD_CTRL.MUTE** rather than strapped, at the cost of one Neon pin, and **a mute step inserted into S.13 between stopping the window server and flushing the filesystem** (→ Q76). The ordering matters and is not obvious. Muting must happen **before** the switched domain begins to come down and **after** the last sound the session wants to make – which is exactly where the window server stops, because a shutdown chime, if there ever is one, is the window server's to play. On the EC's side nothing changes: it already sequences the rails and already tri-states its boundary signals (C.13). **Half of it is answered and the half that remains is ownership.** The RC goes either way – W.7 – and if the EC drives **XSMT** rather than Neon, the mute step is a GPIO write inside a sequence S.13 already runs, needing no new ordering, no Neon pin, and nothing from a subsystem that may already be unpowered by then. The alternative costs a Neon pin and inherits Neon's power state, which is the thing being muted against.

W.41 **Headphone detection is not specified and is not wanted for v1.** The jack's switch contact would cost a pin and an EC input to route the output between jack and speaker automatically; the jumper does it manually until a laptop exists, and by then the choice belongs with the rest of the EC's expander I/O rather than with Neon.

`/dev/audio` – and it is the one bulk path where the aperture is the right answer.

W.42 The driver implements J.4's five functions and **adds nothing to the dispatch table**, for the same reason `/dev/power` does not (S.11): `open` allocates a channel, `write` copies PCM into that channel's ring, `read` returns status, `ioctl` carries volume, rate query, loop and stop, `irq_handler` services the half-buffer interrupt.

W.43 `write()` through the aperture is affordable here, and this is the exact inverse of the framebuffer argument. J.5 forbids per-pixel `write()` because a process walking the framebuffer runs at the aperture's ~1.3 MB/s while the blitter beside it runs at ~100. Audio's entire worst-case demand is **0.7 MB/s, about half the aperture** – and one stereo stream is 176 KB/s, **14 %**. There is no faster path to compete with, because the alternative is the CPU generating samples anyway. **The rule was never "the aperture is slow"; it was "the aperture is slow compared to the engine beside it", and for audio there is no engine beside it.**

The worst case is nevertheless expensive, and it is worth knowing which case it is. Refilling four streamed channels means copying 706 KB/s through the aperture – **about 54 % of the CPU**, because a refill copies every byte it plays. **Four simultaneous streams is not a shape anybody builds**: the normal one is a single streamed channel for music at 14 % and three resident sample channels at nothing, because a sample loaded once and triggered by DMA costs no refill traffic at all (W.20). A larger ring cuts the interrupt rate and **not** the copy, which is irreducible for streamed data – the only way to spend less is to stream less.

Bulk sample assets take the other path when they are large: the EC's service port loads from microSD into VRAM at ~2.5 MB/s with no CPU involvement (T.46), which is the same trick as icon and font upload and costs the 65816 nothing.

W.44 `open` **Four channels are a resource, so somebody has to allocate them.** The recommendation is the simplest one that does not surprise anybody: **channel 3 is reserved for the session's system sounds**, channels 0–2 are allocated to `open` in order, and the fourth caller gets `ENODEV` rather than being silently mixed into somebody else's channel. A software mixer behind a virtual channel is the general answer and is excluded by W.1 – there are no cycles for it – so the honest interface is a finite one that says so (→ Q78).

W.45 **The window server plays the system sounds, not the kernel** – the same split as Q62 draws for the compositor. A click, an alert and an error are session policy; the kernel owns the device node and the ring, and the server owns which sound and when, reaching it through the ordinary `/dev/audio` contract with no privileged path and no new mechanism (V.28).

Staging – A0 to A4, and the first of them belongs three stages earlier than the build currently puts it.

STAGE	CONTENT	ACCEPTANCE	BOARD STAGE
A0	I2S serialiser, divider, <code>AUD_TEST</code>	A tone at the jack – no SDRAM, no CPU, no Helium	E1.7
A1	One channel, ring DMA, arbiter port	Continuous playback from SDRAM, no underrun	E5 · P5.a
A2	Four channels, volumes, mixer, saturation	Four streams mixed with no clipping	E8

STAGE	CONTENT	ACCEPTANCE	BOARD STAGE
A3	Interrupts, <code>/dev/audio</code> , IRQ-driven refill	A user program makes sound (P5.l)	E8 · G-series
A4	One-shot and loop modes, system-sound reservation	GUI sounds under the window server	+

W.46 **A0 needs nothing but the bitstream, three pins and the DAC**, which is the same claim T.64 makes for Mode 0 and it lands in the same place. Sheet P currently has audio appearing at **E5**, as "tone over I2S" bundled with the whole of video; it belongs at **E1.7** beside the console, and for the same reason – a bitstream that produces a clean 1 kHz tone has demonstrated the PLL, the divider, three I/O pins, the DAC's strapping, its internal PLL lock, the RC on `XSMT` and the jack, in one shot and before any of it can be blamed on the SDRAM controller. **The pop, if there is one, is heard at E1.7 rather than at E5**, which is the difference between a strapping fix and a respin.

W.47 Each stage is a usable machine and nothing is stranded if work stops at one of them, which is T.63's property applied to a much smaller block: A1 alone plays music, A2 alone runs a tracker, and A3 is the first stage at which a user process is involved at all.

Verification – and the first four need no CPU.

☐ **W.48** **A 1 kHz tone from `AUD_TEST`** is audible at the jack with the 65816 unpopulated, driven entirely from the EC's service port.

☐ **W.49** **`BCK` measures 1.4127 MHz ±100 ppm** on a scope, and `LRCK` is exactly `BCK` / 32 with the correct phase for I2S.

☐ **W.50** **No click at power-up**. Scope `XSMT` against 3V3_MAIN and confirm the RC holds the DAC muted past the rail's edge.

☐ **W.51** **The DAC's internal PLL locks and stays locked** across the full writable range of `AUD_DIV`, and the range at which it does not is recorded rather than assumed.

☐ **W.52** **Continuous single-channel playback for ten minutes** from SDRAM with `UNDERRUN` never setting.

☐ **W.53** **A deliberately induced underrun** – refill interrupt masked – holds the last sample, sets the flag, and recovers cleanly when refill resumes.

☐ **W.54** **Four channels at full volume on correlated material saturate rather than wrap**, verified by ear and on a scope: the output clips flat, and no sign inversion appears anywhere.

☐ **W.55** **`AUD_POS` never reports a pointer outside its ring**, sampled continuously across a wrap, which is the test that the latch of W.23 actually latches.

☐ **W.56** **Audio survives a full-screen blit storm**: playback with the blitter saturating memory shows no underrun and no audible artefact, which is the arbiter claim of W.33 as an experiment rather than a calculation.

What this sheet changes in other sheets, recorded so the edits are not lost.

SHEET	CHANGE
T.9	Audio has no claim on the second PLL – not a weaker one. No PLL setting reaches 44.1 or 48 kHz from 25 MHz (W.11)
T.47	The interrupt table gains bit 3, audio (W.31)
T.51	Pin budget: I2S is 3 , total 99 of ~107 , not 100 (W.38)
T.50	Audio's 800 LUT reservation measures at ~500 (W.35)
U.36	Audio is 1 EBR block, not 2 , which improves Q61 by one block (W.36)
U.14	"< 1 MB/s plus mixing reads" double-counts: hardware mixing writes nothing (W.15)
S.13	The shutdown sequence needs a mute step before the rails drop (W.40, Q76)
P.05 · E5	Stage A0 moves from E5 to E1.7 (W.46)
Q54	Reduces to "is VGA worth two pins", with audio's half of it closed (W.11)

I BIOS – what it provides

RESIDENCY · SERVICES · BOUNDARY WITH THE EC

SHEET 21

There's no BIOS chip: it's a binary on the SD that the RP2354B preloads into SRAM. The 65816 comes to life with the `$FFFC` vector pointing inside that zone.

I.1 Residency: `BIOS.BIN` on the SD → preloaded by the RP2354B into the pinned SRAM (step 5 of Fig. 3). No flash.

I.2 At reset it provides: CPU init (native mode `CLC/XCE`, stack, direct page), MMU in supervisor identity mapping, cache init, quick memory test, UART console, and boot screen.

The boot screen costs the BIOS nothing to produce. Neon is already displaying when the first instruction runs – it does not have to be probed, put into a mode, or given a font – so `putchar` is a 16-bit `STA` to `$FE0000,x`, absolute-long indexed so the routine runs from any bank, and scrolling is one store to `TEXT_START` rather than 8 KB of block move (T.54, T.27). The one sensible probe is reading `NEON_ID` for `$4E` once, purely to tell "no Neon fitted" from "Neon fitted", and continuing either way.

"Quick" is doing real work in that list: the BIOS is already fetching its own instructions out of the memory it would be testing, so it can only check what is not under it. The march test that has no such problem runs at E2, before any CPU exists (R.22).

I.3 Machine monitor: peek/poke, dumps, serial loading – the working tool for stages E2–E4.

A user button wired to Helium raises NMI, which is how you drop into the monitor on a machine that is already running rather than resetting it to get there.

I.4 OS loader: **mounts NVFS read-only** (Y2.20), locates `KERNEL.BIN`, loads it, and jumps to it passing an **info block**: RAM size, device map, battery state, and RTC time (via EC). `open`

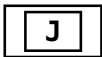
"Read-only" is a specific and cheap thing here rather than a caveat: the minimal reader neither verifies checksums nor replays the journal, and **must refuse a volume that is dirty or whose journal is non-empty** – so a card needing recovery fails the boot loudly instead of being read past (Y2.20, G.3).

The info block's `RTC time` field is currently unfulfillable: **no part of this board holds a wall clock**, which is why every timestamp the filesystem writes is zero (Y1.17, Q88).

Exact block format yet to be finalized (→ N).

I.5 Residual services to the OS: a small `JSL` jump table (`putchar`, `getchar`, `read sector`) during the transition; once the kernel drivers take over, the BIOS stops being used.

I.6 Boundary of responsibilities: the RP2354B is the "board BIOS" (everything pre-CPU); this binary is the "system BIOS" (everything post-reset). Together they cover what a single firmware does in a PC.



Small monolithic kernel, written in C with assembly only where it hurts. It consumes the BIOS for a few milliseconds and then provides everything else.

J.1 Model: preemptive kernel resident in virtual bank `$01` (64 KB, pinned in SRAM), mapped identically and privileged in every process's page table. Mapped identically in every process's table means a syscall changes privilege, not address space – the kernel is already resident at those addresses when the `COP` vector is fetched (L.12).

J.2 How it uses the BIOS: only the info block and the `JSL` table during the transition; its drivers replace the console and SD, and from then on the BIOS goes inert.

J.3 Syscall mechanism: `COP`, with the service number **in the accumulator**, loaded by the libc stub immediately before the instruction. The handler indexes its dispatch table straight from A; arguments stay wherever the C compiler's calling convention left them. So the stable ABI is exactly **"calling convention + a number in A"** – nothing is marshalled, and the stubs that carry it are three lines each (→ 0.7).

The inline signature byte is still emitted, but no longer read: it survives for disassembly, tracing and static verification of a binary. Taking the number from A instead of walking the stack for it saves 20–30 cycles on every single call.

One constant needs verifying against the WDC manual before anything is built on it.

The assumption recorded earlier – that the signature byte sits at stacked `PC-2` – appears to be off by one: `COP` increments PC past the signature before pushing, placing it at `PC-1`. Irrelevant to the handler, which no longer reads it, but a tracer or a static verifier built on the wrong constant misdispatches silently rather than failing (→ Q75).

The 65816's other software interrupt, `BRK`, is reserved for the debugger and must stay that way – two software traps with separate vectors is exactly enough to keep breakpoints from colliding with system calls.

The width of `A` at the `COP` decides how many service numbers can exist – 256 with an 8-bit accumulator, and the libc stub is what sets the width it means. Fixing that convention belongs to the ABI audit of E0.8, not to the first stub that needs it.

J.4 Internal drivers with a fixed 5-function interface – `init · read · write · ioctl · irq_handler` – reached through a device table indexed by device number, and exposed as `/dev/*`. A minimal VFS decides only whether an `open` routes to the filesystem or to that table.

Drivers expose **no syscalls of their own**, and that is the point: per-driver calls would break the ABI every time a chip changed. Swapping hardware means rewriting one driver against the same internal interface, while libc and the syscall numbering stay put.

The screen console is one of these and needs **no initialisation at all** – the hardware mechanism is identical to the BIOS's, the same stores to the same addresses, wrapped so `write()` is a loop over `putchar`. One requirement is not free and cannot be retrofitted: **the console driver must hold its own scrollbar in system memory and be able to repaint from it**, because the text buffer occupies block RAM a graphics mode may reclaim, so a mode switch and back can destroy the screen contents (→ T.55, Q45).

J.5 Framebuffer: the preferred path is an `ioctl` that maps VRAM pages (`$FE`) into the process's address space – no per-pixel `write()` . **Two things now qualify that**, both from sheet T. The mapping is for **asset upload and command emission, not for painting**: a process that walks the framebuffer runs at the ~1.3 MB/s aperture rate while the blitter beside it runs at ~100, and the whole of Neon is arranged so the CPU writes pixels once per asset and never again (T.2, T.56). And **the aperture is data all the way through, deliberately**: Neon's control registers were moved out of it into `$FF:8000` , so a process holding a mapping cannot reach `LIST_BASE` and aim the command processor – which reads and writes all 64 MB with no MMU in front of it – wherever it likes. That escape was real in an earlier draft, and the fix is now structural rather than a rule the mapping `ioctl` has to remember, because bank `$FF` is unmappable to user space by construction (T.21, D36). **Power is the mirror case and now the proof of the rule**: shutting the machine down is as privileged an act as exists, and it still adds nothing to the dispatch table. It is `/dev/power` – an `ioctl` for poweroff and reboot, a `read` for battery state – and the privilege check comes free from the node's permissions rather than from a bespoke gate inside a `sys_poweroff()` (→ S.11).

The kernel's own shutdown path is not a driver concern and is spelled out in S.13. One step of it belongs here: **quiescing the MMU before halting**. The cache is write-back, so a shutdown that flushes the filesystem but not the cache loses exactly the writes the filesystem just made.

J.6 **Filesystem: specified, and it is three sheets rather than a line**. Sheet Y1 is the VFS – the vnode model, path resolution, the mount table, one error space and the concurrency contract; sheet Y2 is NVFS, the native journalled volume the machine boots from and will page from; sheet Y3 is ext2, read-only by recommendation, for cards that arrived from a host. Per-device blocking queues and basic IPC as before.

This supersedes the sketch that stood here – "FAT over the SD + minimal devfs" – in three ways worth naming. The card is **not FAT** (G.3). The VFS is **not minimal**: J.4's claim that it "decides only whether an `open` routes to the filesystem or to that table" is exactly the shape Y1.1 rejects, because mount traversal, symlink bounds and vnode identity have no home inside a driver. And whether the device table lives inside the namespace as a `devfs` – which decides whether `open` and `close` slots are needed per vnode – is open rather than assumed (Q95).

The one thing the sketch had right is the one thing that does not change: **`/dev/*` and the five-function driver interface of J.4 stay**, and no filesystem adds a syscall of its own (D23, Y1.20).

J.7 The windowing OS is specified in sheet V, and it lands on this sheet in three places rather than beside it. **The compositor is kernel code and the window server is not** – not by preference but because D36 put `CMD_PORT`, `LIST_BASE` and `VRAM_PAGE` in bank `$FF`, so **command emission is privileged** and a user task cannot drive Neon however many aperture pages it holds. **The client drawing protocol needs no new mechanism**: the per-client command buffer is `mshare`, a 4 KB page writable in the client and readable in the server, flushed with one syscall for 5–200 drawing ops. And the hardware is reached through a `/dev/neon` node and an `ioctl` – two calls a frame, to start a list and to swap the buffers – which keeps D23's rule that no subsystem adds a syscall of its own (→ V.16, Q62).

One genuine addition to the dispatch table is proposed and it is the first since the table was written: ports, signals and 32-byte messages – `port_create` · `port_send` · `port_recv` · `task_wait` · `task_signal`. They are general primitives rather than one subsystem's verbs, which is the test D23 set. The condition attached is that **there must be one blocking primitive, not two**: N.3's per-device blocked queues have to fold onto signal bits, or a process waiting on a device and a message has nowhere to wait (→ Q63).

v1 dispatch table – Unix-shaped, deliberately small. Numbering the `SYS_*` constants is still pending.

GROUP	CALLS	NOTES
Processes	<code>exec(path)</code> · <code>exit(code)</code> · <code>wait(pid)</code> · <code>getpid()</code> · <code>yield()</code> · <code>sleep(ms)</code> · <code>kill(pid,sig)</code>	<code>exec</code> loads the binary from the card (→ N.6). <code>sleep</code> is backed by Helium's free-running counter, never by cycle counting. Signals stay minimal: KILL, TERM, perhaps one user-defined.
Memory	<code>sbrk(n)</code> · <code>mmap(addr,n,prot)</code> · <code>munmap</code> · <code>mshare(pid,addr)</code>	<code>sbrk</code> only moves the heap end; the frames arrive later by ABORT. <code>prot</code> maps straight onto the PTE R/W/X bits. <code>mshare</code> is optional – shared-memory IPC, and the reason the cache may never migrate a frame (→ L.3).
Files and devices	<code>open(path,flags)</code> · <code>close(fd)</code> · <code>read</code> · <code>write</code> · <code>seek</code> · <code>stat</code> · <code>unlink</code> · <code>rename</code> · <code>mkdir</code> · <code>readdir</code> · <code>ioctl(fd,op,arg)</code>	One unified surface: a path resolves either to the filesystem or to the device table, and everything past that point is the same seven calls. <code>ioctl</code> is the escape valve for whatever does not fit.
Time and IPC	<code>time()</code> · <code>pipe()</code>	Little more than this in v1.

The piece that turns the machine into "an Amiga": several isolated processes taking turns without knowing it, with the MMU eliminating relocation at the root.

N.1 PCB per process: registers (A · X · Y · S · D · DBR · PBR · P · PC), ASID, pointer to its page table, state, and queues. [open](#)

The save set is not ours to choose: it must cover exactly what the C compiler treats as live state, direct-page pseudo-registers included. That audit has not been done (→ Q19).

N.2 Context switch: save state to the PCB → write CONTEXT/ASID to the MMU → restore the next one. Tens of μs; no TLB flush thanks to ASID.

Order matters and is not optional: write `CTX_SET_PTBASE` before `CTX_SET_ASID`, or an interrupt landing between the two leaves the new ASID live while the walker still reads the old table (→ M.8). The bank \$00 question that used to block this is settled in L.11; only sizing remains (→ Q18).

N.3 Scheduler: preemptive round-robin with a 100 Hz timer tick; per-device blocked queues; 2–3 priority levels as a later option. `sleep()` is backed by Helium's free-running counter, never by cycle counting (→ L.15).

N.4 One canonical virtual layout, identical for every process: stack and direct page in virtual bank \$00 (a hardware requirement of the 65816), then `crt0` and `.text` from \$02:0000, then `.data`, then `.bss`, then a heap growing upward – `sbrk` merely moves a pointer and the frames arrive lazily by ABORT.

User code starts at \$02, not \$01: the bank map of L.10 is architecture and does not move for the loader. Region addresses within \$02–\$FD, stack size and heap start are still to be frozen (→ Q17).

N.5 Custom binary format: header (magic, entry point) + segments with `vaddr · filesz · memsz · R/W/X` permissions matching the PTE bits. `.bss` occupies no bytes on disk, only a size in the header.

Field layout, magic value and versioning still to be specified (→ Q20).

N.6 Loading (`exec`): create the page table, copy segments from the card into frames and map them with their permissions, zero `.bss`, map the stack, place the arguments where `crt0` expects them, and `RTI` into the entry point **in user mode**. From there `crt0` finishes initialisation, calls `main()`, and issues `exit()` when it returns. Every process sees the same addresses: the MMU makes relocation unnecessary.

N.7 Lifecycle: `exec` → `ready` → `running` ⇌ `blocked` → `zombie` → `wait`. Isolation is demonstrated in E7: one process dies and the other keeps running.

Boot is assembly-first, but the compiler question is settled: C arrives as soon as the kernel has firm footing. The useful framing is that **the compiler is a solved**

problem and the runtime around it is not – what this project has to build is the board support, not a code generator.

0.1 Assemblers: **ca65** (cc65 suite) and **64tass** – the base for the BIOS, monitor, and kernel boot.

0.2 C compiler: **Calypsi C**, WDC65816 target – selected, not merely shortlisted. C99, fully re-entrant code model, integers to 64 bits, IEEE-754 floats, large code and data models with far pointers and JSL/RTL cross-bank calls; actively maintained and in real use by the Foenix community, on new-build 65816 machines comparable to this one.

The pointer width was recorded wrongly and the correction changes the ABI, not just a sentence. This document said "24-bit pointers", which is what the machine's address space is; **Calypsi's default far pointer is 32 bits – four bytes.** The qualifiers are `__tiny` for 8-bit direct-page pointers, `__near` for 16-bit within the current data bank, `__far24` for the 24-bit pointer this document assumed by default, and `__far` for the 32-bit one that actually is the default. **That decides struct layout, the calling convention and every kernel and driver bank-crossing convention**, so the qualifier is chosen per object and written down rather than inherited – and E0.8's audit is where it is checked against compiled output rather than against the manual (→ Q114).

Supersedes the earlier WDCtools / LCC-816 shortlist; WDC's own C survives as a dated fallback. Ruled out: cc65 and vbcc (6502 only, no 65816 codegen) and llvm-mos – it accepts `-mcpu=mosw65816`, but native 16-bit codegen is immature and the target was still at the RFC stage. Worth re-checking later.

0.3 Memory model: **large code** (24-bit, JSL/RTL) + **small data** pinned to one bank with a fixed DBR, reaching for far pointers only where genuinely needed – dereferencing one is expensive on the 65816, whether through long addressing or DBR reloads, and **__far24 should be preferred to the 32-bit default wherever a far pointer is unavoidable**: the fourth byte is dead weight in a 24-bit address space, and it costs a load, a store and four bytes of every structure that holds one.

0.4 The BSP – five things nobody else will write for us. **crt0**: set up stack, direct page and DBR, initialise `.data` and `.bss`, call `main()`, and turn its return into `exit()`. **A user linker script**, one canonical copy for every user binary. **A kernel linker script**, separate, linking against the privileged space rather than the user 16 MB map. **The syscall stubs**. And **an audit of the compiler's calling convention**, including how much direct page it claims as pseudo-registers – that audit defines the context-switch save set (→ N.1).

0.5 A consequence of full paging that lands squarely on the toolchain: because every process sees an identical virtual layout, **all user binaries link at fixed virtual addresses**. Zero relocation, no load-time fixups, no relocation records in the format – one linker script serves every program (→ N.4).

0.6 Kernel: C with assembly only in vectors, context switch, critical MMU/cache routines, and boot (~hundreds of lines of asm total).

0.7 libc: about twenty three-line syscall stubs – `COP #SYS_n + RTL` – with Calypsi's low-level libc hooks (`open`, `read`, `write`, `sbrk` ...) pointed at them. Each stub loads the service number into A, then `COP`; the compiler emits an ordinary JSL and the `COP` never leaves the stub.

Which makes the stable ABI literally "the compiler's calling convention plus a number in A": arguments sit wherever Calypsi put them. Numbering the `SYS_*` constants is a prerequisite for writing any of it.

0.8 Development flow: compile on the PC (Makefile) → custom binary → transfer over the EC's console UART or SD → run; debug via serial console + Helium debug port.

0.9 Gateware: Verilog or VHDL (decision pending, it constrains the P65816 reference softcore) with Yosys + nextpnr-ice40 + IceStorm; simulation always before the board.

V The windowing OS – wserver and the toolkit

BACKING STORES · A CLIENT PROTOCOL · WHERE THE PRIVILEGE BOUNDARY FALLS

SHEET 25

Sheet T specifies Neon and sheet U the engine that draws; this sheet is the software that runs on them – the window server, the compositor, the client drawing protocol and the widget toolkit. **The first pass at it described the Amiga milestone as software rather than as a capability, and it was right about the shape of the thing** – but it was drawn against a machine with a 12 MHz CPU, 32 MB of unpartitioned video memory and a write-only aperture, none of which is this board. So every figure below is corrected, most of them in the same direction, and one of that pass's architectural claims does not hold at all.

Five corrections to its premises – read these before trusting any number it quotes.

V.1 The CPU is 8 MHz, not 12, and the aperture is slower than the first pass modelled it. Its whole argument rested on command emission being the binding constraint, and that argument gets stronger here, not weaker.

	ASSUMED THEN	THIS BOARD
CPU	12 MHz	8 MHz (B.1)
Aperture throughput	~4 B/μs implied	~1.3 B/μs in practice, 1.6 at best (T. 2)
A 12-byte command	4.2 μs	~9 μs
Sheet T's 6-word COPY_RECT	–	~18 μs (T.43)
Commands per millisecond of CPU	~240	~110, or ~55

V.2 Neon's memory is 64 MB and it is partitioned by function, not 32 MB of free space (D14, D38). The capacity correction is welcome and changes nothing; the partition is a constraint the first pass's memory map violates outright, because everything in that map lands inside what U.16 reserves for the front framebuffer. Redrawn below (→ V.31).

V.3 The CPU can read VRAM (D32, D37); the "aperture is write-only" premise is two revisions old. **Every conclusion it draws from that premise survives anyway** – no software rasterisation into VRAM, no `GetPixel`, save-under as a blitter copy – because T.16 keeps the software model write-only for the reason that outlasts the hardware: a read runs at aperture speed while the blitter beside it runs at a hundred times that. What does change is the failure mode. Writes are posted through a 32-entry FIFO and never stall; **reads stall PHI2**, so a client library that reads back a pixel does not merely run slowly, it adds to interrupt latency (T.19).

V.4 `blocking` The aperture is no longer the whole protection story, and a user-mode `wserver` cannot drive Neon at all. The first pass mapped bank `$FE` into exactly one address space and called that the protection model. That half still works – J.5 already provides the mapping `ioctl` and L.10 makes `$FE` mappable. But D36 moved Neon's control surface to `$FF:8000`, which is **unmappable to user space by construction**, and `LIST_BASE`, `CMD_CTRL.LIST_RUN`, `CMD_PORT`, `VRAM_PAGE`, `DISPLAY_BASE` and the cursor position are all in it. **Command emission is therefore a privileged act on this machine**, and where the user/kernel split falls is decided by that fact rather than by taste (→ V.15, Q62).

V.5 The vblank interrupt exists, and the blocking open item N-6 is closed. D34 routes Neon's three interrupt sources – vblank, raster compare and the list's `SIGNAL` – into Helium's PIC as one line. The compositor is interrupt-driven and does not poll.
Until the PIC's gateway exists, T.47's rule applies: software polls `STATUS.VBLANK` and every pattern here works either way.

What this sheet settles, and it is more than it disturbs.

V.6 It is the software half of Q60, and it argues the same way U.31 did from the hardware side. U.31 showed backing stores with damage-limited recomposition are affordable; this sheet shows what the OS looks like once you have them, and the finding is that **the system gets smaller, not larger**. `layers.library` – cliprect arithmetic, `SIMPLE_REFRESH`, `SMART_REFRESH`, `SUPER_BITMAP` – exists to answer one question, *what is behind this window and who repaints it*, and per-window backing stores delete the question. That is a real argument for Model C and it did not exist when T.57 recommended the flat framebuffer.

V.7 Three consequences worth stating in the affirmative, because they are what the milestone actually buys. **Clients never receive refresh events** – no `REFRESHWINDOW`, no `BeginRefresh` / `EndRefresh`, and the single largest source of bugs in Amiga application code disappears. **A hung client still displays**, because its backing store is intact and the compositor keeps drawing it, where a hung `SIMPLE_REFRESH` app left a blank hole in the Workbench. And **live window drag is the default rather than an XOR outline**, because a recomposite costs one frame's worth of blitter time and dragging at 60 Hz costs nothing beyond it.

V.8 The event set collapses with it. Ten events – mouse down/up/move, key, resized, activate, deactivate, close, menupick, timer – against Intuition's IDCMP, most of whose extra traffic exists to service the refresh model or to move gadget handling into the server. **Gadgets are entirely client-side here**, so `EV_MOUSEDOWN` with window-relative coordinates is enough. Resize is the one

event a client is obliged to act on, and it is obliged because the server has reallocated and cleared the backing store.

V.9 Menus are windows. Borderless, always-front, composited like anything else, closed and recomposited. No save-under path, no special case – which is worth recording next to T.58, where save-under was specified as a bounded special case for the flat model. **Under backing stores it is not needed at all**, and the scratch pool T.58 reserves for it can be spent elsewhere.

The bit-depth fork reaches the toolkit, and the first pass assumed 1 bpp throughout.

V.10 Every visual and every budget in it assumes Mode 1 – 1024 × 600 × 1 bpp, 75 KB, dithered patterns, Susan Kare and Workbench 1.3. Q59 may replace that mode with 8 bpp, which U.10 argues for on **logic** rather than on colour. The arithmetic is not close:

PER COMPOSITE PASS	MODE 1, 1 BPP	1024 × 600 × 8 BPP
Clear the back buffer	76.8 KB W · 0.51 ms	614 KB W · 4.1 ms
One full-screen surface copy	153.6 KB · 1.02 ms	1.23 MB · 8.2 ms
Typical desktop, overdraw 1.8×	~276 KB · 1.8 ms	~2.2 MB · ~15 ms
Full recomposites in the 15.5 ms left after scanout	~15	~1.4–1.9
400 × 300 backing store	15 KB	120 KB
A hundred of them	1.5 MB	12 MB

V.11 At 1 bpp "recomposite, don't repair" is a luxury; at 8 bpp it is a bare fit, and that changes the advice rather than the model. The first pass said not to implement dirty-rect union in v1 and to measure first. That is correct at 1 bpp, where the whole screen can be rebuilt fifteen times a frame. At 8 bpp a typical desktop at 1.8× overdraw consumes **essentially the whole frame – ~15 ms of the 15.5 available** – and U.15's more conservative 111 MB/s puts it over; the heavy case of 3× overdraw does not fit under either figure. So damage-limited recomposition stops being an optimisation held in reserve and becomes the routine path from the first day. U.31's inversion still holds and is what makes it safe – full recomposition stays correct and always available, so a bug in damage accounting costs frames rather than pixels – but the fast path has to exist at 8 bpp, and the first pass did not budget for writing it.

V.12 The client-rendering budget the first pass advertised mostly evaporates too. It left ~12 ms of the frame for client drawing; at 8 bpp, after one composite pass, there is closer to 5–8 ms. This is not fatal – client drawing is blitter work into backing stores, and a window repaint is a handful of wide blits – but a design that assumes three quarters of the frame is free will be surprised.

V.13 The 1-bit visual language is not portable across this decision, and it is the expensive part to redo. Gray-as-pattern, selection-as-invert, depth-as-lpx-frame-plus-shadow: every icon, every widget state and every dither pattern is drawn against a two-colour screen. **Redrawing them for 256 colours is weeks of art, not an afternoon of code.** Open item N-4 said decide before drawing a single icon; that item is not separate from Q59, it is Q59 seen from the toolkit, and it is the strongest reason to answer it early.

N-4 proposes 2 bpp as a middle path – four colours, the Workbench look, 150 KB. **On this device it is the worst of the three.** U.10's shifter argument is not linear: at 8 bpp on a 16-bit bus the shifter degenerates to a byte swap, at 2 bpp it is an eight-position shifter and the ~400 LUT largely come back. 2 bpp pays most of the logic cost of 1 bpp and buys four colours instead of 256.

V.14 The toolkit needs XOR, and that is a third argument in Q58 arriving from software. Selection highlight, pressed states, the text caret, the focus rectangle and any rubber-band outline are all invert operations. Sheet T's operation set has none (T.34); sheet U's minterm LUT has \$3C and 255 others (U.22). Without a raster op the toolkit's §8.3 has to be rewritten around explicit two-pass masked drawing – every selection becomes a fill plus a keyed copy from an inverted duplicate of the content, at twice the traffic and twice the commands. **The two earlier arguments for U.4's hybrid were about logic cost and setup cost; this one is about whether the widget set can be drawn at all.**

Where the window server lives – the one place the first pass's architecture has to change.

V.15 What wserver needs to touch, and which side of the wall each thing is on.
This is the whole of the problem in one table.

IT NEEDS	WHERE IT LIVES	REACHABLE FROM USER SPACE
Backing stores, framebuffers, atlases	Neon SDRAM, through the \$FE window	Yes – the mapping ioctl of J.5
The palette	\$FE:8000 (T.49)	Yes, same mapping
CMD_PORT – FIFO command emission	\$FF:8050	No
LIST_BASE · CMD_CTRL.LIST_RUN	\$FF:8056 , \$FF:8055	No
DISPLAY_BASE · SWAP_BUFFERS	\$FF:8018 and the command stream	No
VRAM_PAGE – which 32 KB the aperture shows	\$FF:8020	No
CURSOR_X · CURSOR_Y	\$FF:8060	No

IT NEEDS	WHERE IT LIVES	REACHABLE FROM USER SPACE
Vblank interrupt	Helium's PIC	No

- V.16** So the split falls on command emission, and the recommendation is that the compositor is kernel code and the window server is not. The glossary already draws this line without having been asked to – *Compositor: the kernel code that assembles the visible framebuffer from the backing stores, driving the blitter. Distinct from the window manager, which decides what should be where* (Z1) – and the register map is what makes it true rather than tidy. **The compositor is small:** a vblank handler, a z-ordered list of surfaces, a top-level command list it rebuilds when geometry changes, and a swap. **The window server is not small** – hit-testing, focus, drag, menus, decoration policy, client validation – and none of it needs a privileged instruction.
-
- V.17** `wserver` in user space then builds command lists in VRAM through its aperture mapping and asks the kernel to run them, which costs two syscalls a frame – start the list, swap the buffers – against a `COP` entry that is tens of cycles (J.3). That is free. It also keeps D23's rule intact: **no new syscalls**, a `/dev/neon` node and an `ioctl`, with the privilege check coming from the node's permissions exactly as `/dev/power` does.
-
- V.18** `open` `VRAM_PAGE` is the wrinkle in that arrangement and it is open item N-2 answered with a twist the first pass did not anticipate. The aperture shows 32 KB at a time (T.20) and the page register is privileged, so a **user-space server pays a syscall every time its writes cross a 32 KB boundary**. Reprogramming costs one byte store and no stall, so the cost is the syscall, not the hardware. Two mitigations, and they are cheap: keep the list arena, the string cache and the decoration assets **inside as few pages as possible**, and let the `ioctl` take *page and offset* so a page change and a burst of list writes are one call. If measurement shows the boundary crossings dominating, the compositor takes the list building with it into the kernel; that is a move of one module, not a redesign.
-
- V.19** Client validation is unaffected by any of this and remains the security property that matters. The shared page is client-writable and must be treated as hostile (V.20); `wserver` bounds-checks every op against the window's own surface and rejects any surface handle the client does not own. **A user-space server that is compromised can corrupt what is on screen and nothing else** – it cannot aim the command processor at system memory, because it cannot write `LIST_BASE`. Under the first pass's model, where the server owned the registers, a compromised server owned the machine: the command processor reads and writes all of VRAM with no MMU in front of it. **D36 turned that from a rule into a wall, and this is the first design that stands behind it.**

The client protocol – and it needs no new kernel mechanism.

- V.20** The shared command buffer is `mshare`, which J.5's table already has. A 4 KB page mapped writable in the client and readable in the server; the client's `gcl` library appends compact ops and flushes once. A widget repaint is 5–20 ops in one syscall, a full window repaint 50–200. **The alternative rejected here is worth recording** – a syscall per primitive costs ~200 cycles of kernel entry against a primitive the blitter finishes in microseconds, and it would put the drawing API on the wrong side of the ratio the whole machine is arranged around (T.2).

- V.21 The op set maps one-to-one onto Neon primitives, and it is worth saying plainly what that forbids.** `rect_fill` · `frame` · `line` · `blit` · `blit_masked` · `tile` · `text` · `clip`, and no arcs, no curves, no polygons, no floods. Clients can only produce imagery Neon's primitive set can produce, because a client cannot write SDRAM and cannot rasterise into it. The escape hatch – `upload(src_sysram, surf, x, y, w, h)` – is worse than the first pass assumed and should be named as such. The client rasterises on an 8 MHz 65816 with no multiplier, the result crosses into the server, and the server pushes it through the aperture at ~1.3 MB/s: a 200 × 100 patch at 8 bpp is 20 KB and ~15 ms, a whole frame, most of it CPU. It is correct to provide it and correct to discourage it; an application whose visual identity depends on it has chosen the wrong machine.
- V.22 Decoration is composed into the window's own backing store, and this is quietly the right answer to U.18.** The server draws the frame, title bar and gadgets into the surface above the content area and clips the client out of that band. So the per-frame composite of a window is **one wide blit**, not a wide blit plus four narrow ones – and U.18 is explicit that a 16 × 400 blit pays 400 line-boundary costs against 8 words of payload each. Window furniture is exactly the narrow-rectangle case, and this arrangement pays for it once per resize instead of once per frame.
- Text** – where the compositor above already answers most of its worst problem.
- V.23 The per-glyph arithmetic first, corrected to this board.** A text-filled screen at the 8 × 16 cell is 128 × 37 ≈ 4,700 glyphs. That was costed at ~20 ms of emission at 12 MHz; here it is **44 ms at 12 bytes per command and ~88 ms at sheet T's 6-word COPY_EXPAND** – three to five frames to emit commands for one screen of text, against ~0.23 ms of blitter traffic to draw it at 1 bpp. The mismatch is not 20:1, it is closer to **200:1**, and the conclusion – naive per-glyph blitting is unusable – is understated rather than wrong.
- V.24 But the emission happens once, not once a frame, and per-window command sublists are what make that true.** They live in SDRAM and are `CALL` ed, which is T.39's Layer 3. Two kinds of list follow and they are worth naming separately, because the first pass ran them together: a **content list**, which draws into a backing store and is re-executed only when that window's content changes, and a **composite list**, one or two commands per window, executed every frame. **Text lives in the content list.** A window whose text has not changed costs zero commands per frame, no cache of any kind involved.
- V.25 What is left is repaint latency, and there the string cache earns its place.** When a window's content does change, the server rebuilds its content list, and that rebuild is emission through the aperture at 0.77 μs a byte. A 20-line text pane is ~1,200 glyphs – **~22 ms of emission**, more than a frame, for one repaint. Composed as cached runs it is 20 `COPY_RECT` s, **~0.4 ms**. So the cache is not what makes text possible per frame – the content list is – **it is what makes a repaint feel instant**, and hit rates in a desktop are high because menu labels, button labels, filenames and column headers all repeat. Proportional fonts fall out of it free, since it stores composed runs and not glyphs, and the caret and selection stay separate `rect_fill` s rather than being baked into a cached run. Composing a run into the strip still places glyphs at arbitrary x, which is Q49 exactly: whether `COPY_EXPAND` reads its 1-bpp source at a bit-granular offset decides whether the atlas can be packed tight or must be padded to 8 pixels. The cache does not remove that question, it concentrates it in one place.

V.26 `DRAW_GLYPHS` – open item N-3 – is not a substitute for the cache; it is the complement of it, and costing them together is what shows why. The proposal is one command taking (`font_base`, `glyph_w`, `glyph_h`, `string_ptr_in_sdram`, `count`, `dx`, `dy`) .

DRAWING AN 80-CHARACTER LINE	CPU EMISSION	FIRST TIME
Per glyph, <code>COPY_EXPAND</code>	80 × 24 B ≈ 1.5 ms	same
Cached string, one <code>COPY_RECT</code>	24 B ≈ 18 μs	~1.5 ms to compose the strip
<code>DRAW_GLYPHS</code>	80 B of text + the command ≈ 80 μs	same – there is no first time

V.27 So the cache wins for stable labels and `DRAW_GLYPHS` wins for text that changes, which is to say for the editor, the terminal and the file listing – the applications a desktop is *for*. A 128 × 32 console repaint is ~3.2 ms with `DRAW_GLYPHS` against ~76 ms per glyph, and a single line update is 100 μs. It also removes 256 KB of strip and all the LRU machinery. **The reason to hesitate is Q61 and nothing else:** block RAM is at 32 of 32 before this feature is asked for, and a glyph sequencer wants a source buffer. Cost it in LUT and EBR against the 128-glyph font relief (Q45) before committing to either (→ Q64).

Kernel additions – ports, signals and messages.

V.28 Fixed 32-byte messages copied by the kernel, bulk payloads through `mshare` . The model is Exec's and it is the right one: each port bound to one bit of the owning task's 32-bit signal mask, `task_wait(sigmask)` as the **single blocking primitive**. Two numbers matter on this CPU. A 32-byte copy through the kernel is ~100 cycles, ~12 μs at 8 MHz, which is fine for input events and window notifications and is why the message is fixed-size rather than variable. And **motion coalescing is not optional** – at 12 μs a message, an uncoalesced mouse stream is a measurable tax on a machine that also has to composite; never deliver more than one `EV_MOUSEMOVE` per frame per client.

V.29 open This is the first addition to J.3's dispatch table since it was written, and it deserves the same scrutiny D23 applied to power. Five calls – `port_create` , `port_send` , `port_recv` , `task_wait` , `task_signal` – and they pass the test that `sys_poweroff()` failed: they are general primitives usable by anything, not one subsystem's verb wearing a syscall's clothes. **What must not happen is two blocking mechanisms.** N.3 already has per-device blocked queues; if `task_wait` is added beside them, a process waiting on a device and a message has no single place to wait. **Fold the device queues onto signal bits** so there is one wait primitive, or do not add this one (→ Q63).

V.30 Input arrives RP2354B → HID driver → kernel input queue → `wserver` , and that path puts one requirement on Q4, which still owes the HID event format: **an event must fit in 32 bytes and must carry enough state to be coalesced** – absolute position rather than deltas, button state as a bitmap rather than as edges. A format that forces the server to replay every intermediate motion to know where the pointer is cannot be coalesced at all, and that is discovered late and expensively.

The memory map, redrawn against the bank partition.

V.31 The first pass's memory map is a flat allocation from `$000000` upward: two screen buffers, command lists, font atlases, string cache, assets, then backing stores. **Under D38 all of it lands in bank 0, which is the front framebuffer**, and the invariant that every ACTIVATE is issued inside another bank's data phase collapses – worth a factor of two in delivered bandwidth (U.16). The map is not wrong so much as written before the partition existed. Redrawn, keeping the original sizes at 1 bpp:

BANK	CONTENTS	GUI USE
0	Framebuffer, front	Scanout reads it
1	Framebuffer, back	The compositor writes it; they exchange at vblank
2	Backing stores, even-numbered windows	Composite source
3	Backing stores, odd windows · font atlas · string cache strip · icon and pattern assets · command lists · audio	Composite source, and everything the content lists read

V.32 [open](#) **Putting the command lists in bank 3 gives the arbiter a fifth stream that D38 never counted**, and it is the one place this sheet adds a hardware question rather than answering one. During a composite pass the command processor is fetching from SDRAM while the blitter reads a backing store and writes the back buffer. For an even-numbered window the fetch and the pixel read are in different banks and nothing is lost; **for an odd-numbered window they are both in bank 3** and each alternation costs a row activation. The traffic is small – a 20-window composite list is a few hundred bytes – but the stall is per crossing, not per byte. Either give the lists their own bank, or interleave window parity so that a window's backing store and the list describing it are never in the same bank, or measure and decide it is noise. **It is cheap to decide now and awkward to discover at G2 (→ Q65).**

V.33 [open](#) **And the map cannot exceed 16 MB whatever the device holds, which is worth confirming rather than assuming.** `DISPLAY_BASE`, `DRAW_BASE`, `LIST_BASE` and sheet U's four channel pointers are all **24-bit** (T.48, U.19), and U.16's partition is written as four 4 MB ranges from `$000000` to `$FFFFFF` – a 16 MB space on a 64 MB part. Nothing in the GUI needs more: at 8 bpp a hundred backing stores is 12 MB and the two framebuffers are 1.2 MB. But it means **the partition is a 16 MB window into the device, not a division of it**, and whether the address registers widen to 26 bits or the upper 48 MB is simply unaddressed by the blitter should be stated in sheet T rather than left to be inferred here (→ Q66).

Mode switching – open item N-7, and this board answers most of it.

V.34 The video mode is global, so a colour application takes the whole display and the desktop is suspended – there is no Amiga-style multi-Screen coexistence, and that is a consequence of the hardware rather than a simplification. What survives a switch and back is the useful part: **backing stores are in SDRAM and survive untouched**, so the desktop is restored by one full recomposite – 1 ms at 1 bpp, ~8 ms at 8 bpp, one frame either way. Three things do not look after themselves. **The palette is shared state** at `$FE:8000` and the returning application's is not the desktop's, so the compositor saves and restores it. **The framebuffers change size** between modes – 75 KB at 1 bpp against 600 KB at 8 – so banks 0 and 1 are reallocated rather than reused. And **the text console's block RAM may have been reclaimed** by the graphics mode (T.12, Q45), which is why J.4 already requires the console driver to hold its own scrollbar and repaint from it – the GUI inherits that requirement rather than adding one.

Staging – G0 to G8, and where they land on the board's own build.

STAGE	GOAL	NEEDS FROM NEON	BOARD STAGE
G0	neon driver, aperture, command-list builder	N1	E5
G1	Compositor, N static surfaces, vblank swap, cursor tracking	N2 · N3 · the PIC	E8
G2	wserver: window create/destroy/move/raise, live drag	N3 · N4	E8
G3	Ports, signals, input routing, focus	kernel only	E7 · E8
G4	Shared command buffer, validation, op set	mshare	E8
G5	Font atlas, string cache, text()	N3 COPY_EXPAND	E8
G6	tk: tree, layout, button, label, checkbox	–	E8 · +
G7	Menus, listbox, scrollbar, textfield	–	+
G8	Desktop shell – icons, drag-and-drop, launch	–	+

V.35 The compositor needs N4, not N5, and that is the scheduling finding. A 20-window desktop composited from the FIFO is ~40 commands – **960 bytes, ~0.7 ms of emission, about 4 % of a frame** – so full command lists in SDRAM are not on the critical path for the Amiga milestone. Sheet P already puts N3 and N4 at E8 and N5 after it, and that ordering survives this sheet unchanged. **What N5 buys is client repaint, not compositing**: it is what turns a content list into something re-executed rather than re-emitted (V.24), and until it exists every window repaint is paid in full at 0.77 µs a byte.

V.36 G0 to G2 are the stages that prove the architecture, and the gate is one measurement. If a full-screen recomposite does not come in near 1.0 ms at 1 bpp – or near 8 ms at 8 bpp – then V.6 through V.11 need revisiting before anything above G2 is written. Instrument it deliberately at G2 rather than inferring it from frame rate: **the number wanted is the blitter's own time for the pass**, separable from emission and from whatever the client tasks are doing.

What the first pass left open, and what became of it.

LEFT OPEN	HERE
N-1 Raster operations – does the blitter have XOR?	Sheet T's op set does not, sheet U's minterm LUT does. This is Q58 , and the toolkit is now its third argument (V.14)
N-2 Aperture addressing and the cost of paging	<code>VRAM_PAGE</code> , 32 KB window, base = value × 32768 (T.20). One byte store, no stall – but it is privileged, so the cost is a syscall (V.18)
N-3 A <code>DRAW_GLYPHS</code> command	Complementary to the string cache rather than a replacement; blocked on EBR, not on merit (V.26, Q61)
N-4 A 2-bpp hires mode	Folded into Q59. 2 bpp is the worst of the three on this device (V.13)
N-5 Blitter arbitration	Closed by D39 – fixed priority, blitter lowest, preemption at burst boundaries, 223 ns worst case
N-6 Interrupt line to the CPU	Closed by D34 – vblank into Helium's PIC. Not blocking
N-7 Mode switch protocol	Answered in V.34; the palette and the framebuffer reallocation are the parts that need code
N-8 Timing closure and EBR at 32/32	Q44 and Q61, both already open and both already blocking
12 MHz CPU · 32 MB · write-only aperture · unpartitioned VRAM	V.1–V.3
<code>wserver</code> owns the aperture and that is the protection model	Half true. Command emission is privileged; the compositor is kernel code (V.4, V.16)

Verification – the G2 gate, and none of it needs a client.

- **V.37** A full-screen recomposite is measured, blitter time separated from emission time, against 1.0 ms at 1 bpp or ~8 ms at 8 bpp.

- **V.38** A window drags at 60 Hz with no tearing and no visible cost, over a non-uniform background.

- **V.39** A window raised from the back of the z-order shows content that was never repainted – the proof that backing stores are doing what they exist for.

- **V.40** A task killed mid-frame leaves its window on screen, intact and composited, until the server closes it (V.7).

- **V.41** A client cannot draw outside its own window, with a deliberately corrupted shared buffer as the test vector rather than a well-behaved one (V.19).

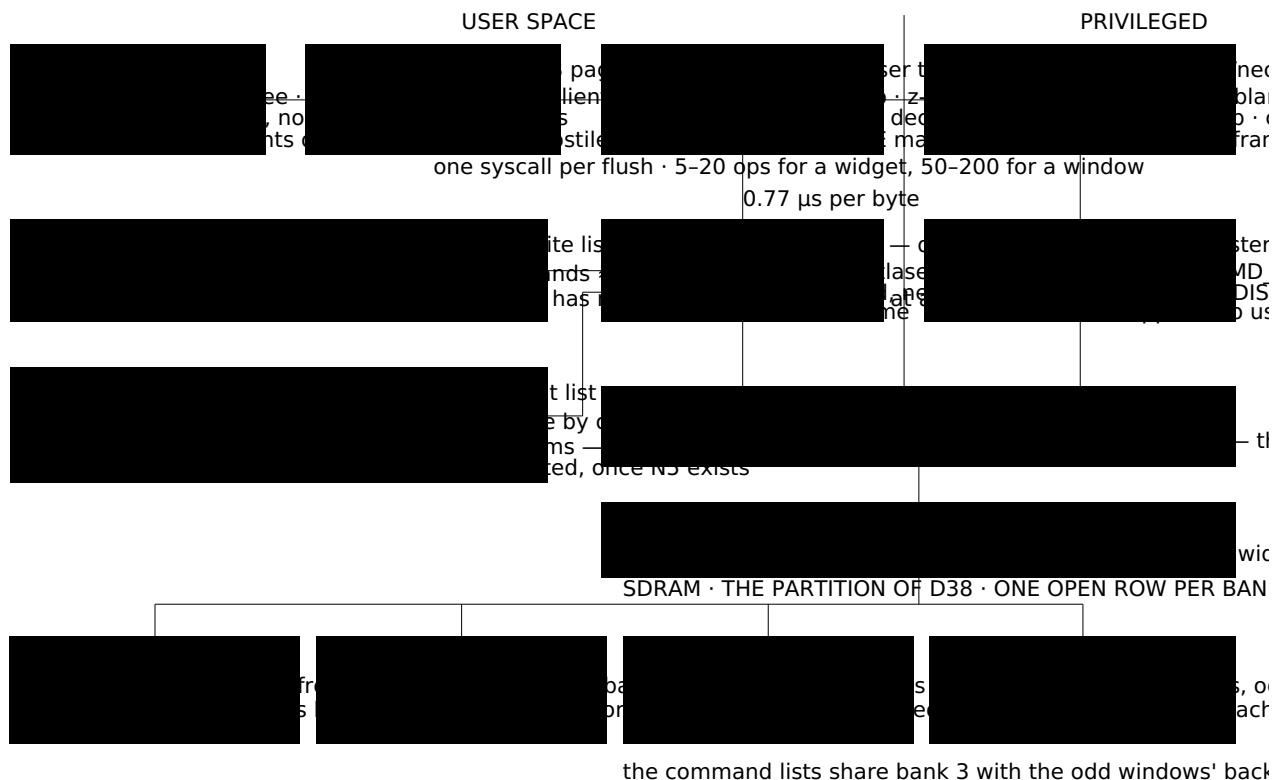


Fig. 12 – The windowing stack. Pixels and command lists cross into VRAM through the mappable `$FE` aperture; starting a list crosses into `$FF:8000`, which no user task can reach – so the privilege boundary falls on command emission rather than on the framebuffer.

Trace legend: `mint` = pixel and command data · `gold` = the privileged path and the wall it crosses · dashed = what a repaint costs when nothing is cached.

P

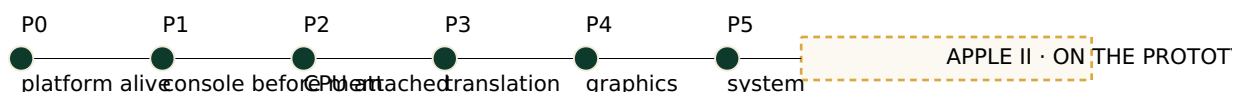
Step-by-step build · P0–P5 · E0–E8 · L1–L8 · F0–F9

TWO BOARDS · SMALL STEPS · VERIFIABLE TESTS · GOLDEN MILESTONES

SHEET 26

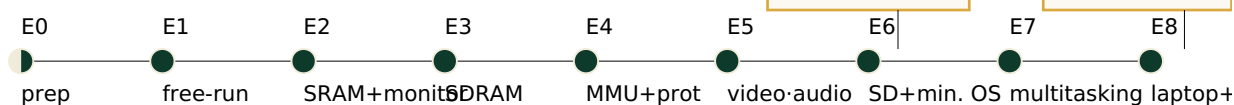
Two tracks now, not one. The **P-series** builds a prototype carrier around a commercial Colorlight i9 module, with Helium and Neon merged into a single ECP5 and everything else – CPU, SRAM, EC, video, audio – outside it; the **E-series** builds the machine this document specifies, three iCE40 parts on a board of our own. The P-series exists because the entire software roadmap was queued behind hardware that has not been drawn yet, and because gateware debugged on a bitstream costs a rebuild where the same bug on a PCB costs a respin. It is temporary packaging of the design, not a variant of it (→ D40). A third series sits at the foot of this sheet and touches no hardware at all: the **F-series** builds the filesystem stack of sheets Y1–Y3 on a development PC, ahead of the machine that will run it (P.26).

PROTOTYPE · ONE ECP5 · P0–P5



the prototype validates the gateware the E-series then com

TARGET BOARD · THREE iCE40 · E0–E8



■ done ■ in progress ■ pending ■ optional

Fig. 7 – Two build tracks: the prototype path above, the target path below with both milestones at their exact point – Apple II on closing E6, Amiga on closing E8.

The prototype board – one ECP5 carrying two devices

P.01 What it is. One Colorlight i9 v7.2 module – Lattice LFE5U-45F-6BG381C, 44,000 LUT4, 108 × 18 kbit EBR ≈ 243 KB, 8 MB SDRAM (M12L64322A, 512K × 32 × 4 banks), 8 MB SPI flash, 25 MHz oscillator on ball P3 – on a four-layer carrier of our own holding the W65C816S in its PLCC-44 socket, the SRAM, the RP2354B, a VGA output and the PCM5102A. Power is USB. The toolchain is the same open one, with `nextpnr-ecp5` beside the `nextpnr-ice40` that E0.1 already requires. The module's two Broadcom B50612D Ethernet PHYs are never used and **share a reset line on ball P4**: assert it at configuration and leave it asserted. It reclaims a few hundred milliamps and removes a switching-noise source next to the CPU bus.

P.02 What it deliberately is not. No battery, no charger, no fuel gauge, no rail sequencing, no eDP, no touch, no 10.1" panel. Sheets C and S are simply not exercised here, and that is a **gap to note rather than to close** – the RP2354B's power-sequencing role, the whole of the `$FF` power block and the shutdown path get their first hardware on the target board (→ Q73).

P.03 Logic budget – comfortable, and the slack is the instrument.

BLOCK	LUT4
Helium: MMU, TLB, cache controller, arbiter, peripherals	3,000 – 4,000
Neon: scanout, blitter, compositor, audio DMA	3,300 – 3,900
Shared SDRAM controller and QoS arbiter	1,200 – 1,800
VGA timing generator and palette lookup	150 – 250
Debug agent, trace buffer, instrumentation	800 – 1,200
Clock domain crossing, glue	400 – 600
Total	8,850 – 11,750

20 – 27 % of 44,000. The unused capacity buys what will never fit in an HX8K: wide trace buffers, bus transaction counters, cache hit/miss histograms, an embedded logic analyser on the 65816 bus. Build it now – it is free exactly once.

P.04 EBR is where the prototype lies to you, and it is the most dangerous number on this sheet. The ECP5 has ~243 KB. **Each target iCE40 HX8K has 16 KB** – 32 blocks of 4 kbit (D01, T.3) – and Neon's text buffer and font already spend 12 KB of Neon's 16 (T.12 calls the budget fully allocated with zero margin). A 16 KB trace buffer, which is nothing here, is *larger than the whole of Helium's block RAM there*. Every EBR consumer therefore carries a synthesis parameter whose **default is the target value**, and the prototype only ever raises it for instrumentation, never for function (→ P.18).

P.05 Pin budget. Counted from the wuxx i9 v7.2 pinout, excluding GND, 5 V, NC and the Ethernet pairs, **106 FPGA balls** reach the SODIMM connector – raw balls, no buffers on the module. None of them collides with the SDRAM (52), the PHYs (~27), the SPI flash (4) or the clock on P3.

BLOCK	SODIMM	CONTENTS	PINS	FREE
A	57 – 104, dense	65816 bus	38	10
B	109 – 156, dense	SRAM 13, VGA 20, I2S 3	36	12
C	41 – 54, scattered	RP2354B EC link	8	2
Total			82 / 106	24

The CPU bus occupies one contiguous block and must not be split. Synchronous bus signals spread across distant connector regions and different ECP5 banks is where skew problems are manufactured. The 10 spare pins in block A become a probe header beside the CPU – the place they will actually be wanted. Block C's scattering is harmless for SPI at a few MHz and fatal for anything parallel, which is why the EC link is the correct occupant.

P.06 The 65816 bus, 38 pins. A0–A15 (16) · D0–D7 multiplexed with the bank byte (8) · FPGA outputs PHI2, RESB, ABORTB, IRQB, NMIB, BE (6) · FPGA inputs RWB, VPA, VDA, VPB, MLB, E, MX (7) · RDY bidirectional (1).

A0–A10 and D0–D7 must be bidirectional FPGA pins, and this has to be settled before ball assignment. During CPU cycles the FPGA observes them; during a walk, a fill or the EC's preload it drives them with **BE** low to tristate the CPU. That is F.4 verbatim, and it is the single most layout-critical net here as it is there.

P.07 SRAM – the same 1 MB ×8 part as the target board, AS7C38096A-10TIN (D15, F.5): 13 pins, A11–A19 (9) plus CEB, OEB, WEB (3) and the shared-net arrangement of F.2, where A0–A10 and D0–D7 cost nothing because they are already on the bus. **A ×16 part would be a mistake dressed as convenience** – it would hand the prototype a byte-lane topology the target does not have, and the whole point of the carrier is to rehearse the topology that does (→ D43).

P.08 Video – VGA through an R-2R ladder, 18 bits. The ECP5 has no DAC; conversion is 6 bits per channel into 1 % ladders terminated against the monitor's 75 Ω, plus HSYNC and VSYNC, into a DE-15. Six bits is not a compromise: the target panel is driven at **18 bpp over direct RGB-TTL (D05, E0.4)**, so the palette entry is 18 bits on both boards and there is no truncation, no format conversion and no software difference. RGB444 would save six pins that are not needed and buy a divergence.

Bring-up runs **640 × 480 at 25.175 MHz**, the most tolerant mode in existence and the safest way to prove a scanout path. Move to 1024 × 600 at ~50 MHz once the timing generator is trusted, and develop the compositor there. 1024 × 600 is also where the console geometry lands exactly: 128 columns × 8 px = 1024, with the 32 rows of T.25 centred inside 600 lines and a border above and below.

P.09 Audio – 3 pins. BCLK, LRCK, DATA into the PCM5102A, which synthesises its own internal clock and needs no MCLK.

P.10 The EC – 8 pins, SCK, MOSI, MISO, DBG_CSN, IRQ, READY, DEBUG_ENABLE, FPGA_READY. It keeps its specified role minus power sequencing: debug-agent SPI master driven from its **J-DBG** console UART, BIOS and kernel preload from microSD, USB HID through the native host controller (D.5), and release of the 65816 from reset. It **gains ECP5 JTAG over four GPIO**, because a carrier hosting the SODIMM socket directly has no DAPLink on it – a natural extension of the configuration role it already owns (→ sheet D).

The debug agent's two access axes – physical against virtual, internal against external bus cycle – partly collapse on one die, since Neon's memory is reachable through the shared arbiter. **Implement both anyway.** The external-bus-cycle path of R.5 must exist and be tested here, or the target board discovers it missing at the worst possible time.

P.11 Memory – 8 MB shared, and the scarcity is a feature.

Read P.11a before believing the allocation below: the module's SDRAM may not be usable for most of it.

ALLOCATION	SIZE
Framebuffer, front (1024 × 600 × 8 bpp)	600 KB
Framebuffer, back	600 KB
Window backing stores	2 MB
Physical page pool	4 MB
Audio DMA buffers	128 KB
Reserve	~700 KB

Enough for the whole G0–G8 roadmap, and the 16 MB per-process virtual space exercised against a small physical pool is *preferable* during development: paging and eviction run constantly instead of lying dormant behind abundant memory. Bandwidth: ×32 SDR at 100 MHz peaks at 400 MB/s and sustains 250–300; scanout 37, blitter 60–100, Helium fills 25, audio negligible – 122–162 MB/s, roughly 50–65 % utilisation.

None of those figures transfer: the target has two independent ×16 devices with separate controllers (D13, D14). Arbitration is fixed-priority with a starvation guard – scanout, audio, blitter, Helium fills, bulk – and the guard promotes anything waiting past a programmable threshold. Instrument it with per-requester wait-time histograms in EBR, readable over the debug channel. It is the same shape as D39 with one more claimant, so what is learned here is worth carrying.

P.11a blocking The module's SDRAM has **CKE**, **CS#** and all four **DQM** lines tied off to the rails rather than driven by the FPGA, and if that holds, most of P.11 cannot run on it. Two consequences, and the first is the fatal one: **there is no per-byte write masking**, so every write is a full 32-bit word – and the 65816 writes bytes. A page pool behind an unmaskable memory needs a read-modify-write on every byte store, which is two SDRAM accesses where the design assumes one, on the machine's hottest path. The second is that **CS#** and **CKE** can never be deasserted, so the part can neither be idled nor put in self-refresh.

The remedy is the one the target board already specifies: put the real SDRAM on the carrier, wired to spare FPGA I/O with **DQM**, **CS#** and **CKE** under gateway control, exactly as sheet F has it. The module's own 8 MB then survives as coarse scratch and an early framebuffer during bring-up – where a whole-word write is harmless – and nothing above it has to know. **This makes P.19's Stage A less attractive rather than more:** a wire-wrapped stage cannot rehearse a memory that has to be on the PCB anyway. **The finding comes from community reverse-engineering of the module, not from a Colorlight document, so confirm it with a continuity meter on the specific board on arrival** – **CKE** to 3V3, **CS#** and the **DQM** lines to ground – before committing the carrier's SDRAM footprint. It is cheap to check and expensive to discover after layout (→ Q112).

The direction of the surprise is worth noting: it **reduces** the risk of skipping Stage A rather than raising it, because it forces the memory interface onto the carrier where **DQM** and **CS#** are ours to drive.

Partitioning discipline – the section the plan succeeds or fails on

The prototype merges two devices the target separates. Every shortcut taken because Helium and Neon happen to share a die is a rewrite later, and the rules below are what stop that. Review them at every major merge, not only at the transition.

P.12 Rule 1 – two top-level modules, always. **helium.v** and **neon.v** are instantiated side by side in a top level whose only job is to wire them to each other and to the pads. Neither module references a signal inside the other. There is no shared-internals file.

The rule is mechanically checkable, which is the point of stating it this way: a reference from one module into the other's hierarchy is a grep, not a judgement.

P.13 Rule 2 – the interface is the shared bus and three control nets, and nothing else. This is the rule an earlier proposal for the carrier got wrong, and it is worth saying why. It wanted a dedicated 56-signal command/peek/event link between the two devices, multiplexed down to 32, and reported the pin cost as a finding for the package decision. **That link does not exist:** D04 deleted it. Neon taps the CPU bus directly and the only dedicated wires are **NEON_BUS_SEL**, **NEON_BUS_BSY** and **NEON_IRQ** (B.6). So the prototype's **neon.v** takes A0–A15, D0–D7, PHI2, RWB and **NEON_BUS_SEL**, and drives **NEON_BUS_BSY** and **NEON_IRQ** – every one of which is a PCB net on the target board. A signal added to that list is a cost paid twice; a signal smuggled around it is a defect.

- P.14 Rule 3 – separate clock domains from day one.** Both modules could run from one PLL output. **Do not.** Derive two clocks from separate outputs and do proper CDC on every crossing: async FIFOs for data, two-flop synchronisers plus handshake for control. On the target the two devices take the same 25 MHz star (B.7) into different PLLs, which is asynchronous in practice – T.18 already reasons that way. This costs a little latency now and buys the certainty that metastability does not surface after the PCB exists.
- One domain would also entangle timing closure: Neon's pixel clock would constrain Helium's paths, and a route that failed would present as a video defect rather than as a timing failure.
- P.15 Rule 4 – the memory port is an abstraction.** Neon's accesses leave through a memory port on Neon's boundary – `mem_addr[22:0]`, `mem_wdata / mem_rdata[31:0]`, `mem_we`, `mem_be[3:0]`, `mem_req`, `mem_ack`, `mem_prio[1:0]`. Here it routes to the shared arbiter; on the target it routes to Neon's own 64 MB device. **Neon contains no knowledge of which** – no arbitration logic inside it, no awareness that Helium exists as a competing requester. `mem_prio` keeps its meaning in both: 3 scanout, 2 audio, 1 blitter, 0 bulk.
- P.16 Rule 5 – palette entries are 18 bits, always.** Six bits per channel on the VGA ladder is the same depth the target panel takes over parallel RGB-TTL, so the palette format is byte-identical on both boards and no conversion layer ever exists (→ P.08, D05).
- Six bits per channel is also what the R-2R ladder can express, so the prototype's VGA is not a reduced-colour path – the same palette on a different connector.
- P.17 Rule 6 – what the aperture must preserve is the stall protocol, not a fake restriction.** REV A protected a write-only aperture with a slow `PEEK` register path; T.15 deleted that mechanism and returned its address space, because `NEON_BUS_BSY` already stalls PHI2 and makes `$FE` genuinely readable at about six times the speed. So on one die, where a read could trivially answer in zero wait states, **implement the four-step protocol of T.18 anyway**: busy asserted by default, the read issued after the stall rather than racing it, the meaningful edge being the *deassertion*, and Helium's watchdog turning a hung or absent Neon into a fault instead of a frozen machine. All four requirements of T.19 are testable here, including the write-FIFO drain – the one that otherwise ships as an intermittent bug – and requirement 4 still holds: **the software model stays write-only** even though the hardware answers reads.
- P.18 Rule 7 – the EBR abundance is the leak nobody catches.** Every buffer – text, font, palette, cursor, scanout FIFO, blitter line buffers, cache tags, TLB, walker cache, command FIFO, trace – is declared with a size parameter defaulting to what fits an HX8K, and a build with the target defaults must synthesise and pass the same tests before any merge. A hardcoded size anywhere is a defect, and this applies to the cache first: **the cache controller is parameterised by size, line count and associativity from its first line of code**, because the target is 768 KB of 4-way cache in 2 KB lines with tags in EBR (F.5) and this board carries whatever the carrier carries.

Construction

P.19 Stage A – extension board plus wire-wrap. 65816 in a PLCC-44 socket on perfboard, SRAM adjacent, ribbon to the extension board's PMOD headers. **Run PHI2 at 1–2 MHz:** flying leads will not meet the 65816's data setup requirement before the falling edge any faster, and chasing intermittent faults on a wire-wrapped bus is a poor use of a week. Purpose: prove the concept and reach P2 without waiting for a PCB.

blocking The PMOD headers must be verified as direct, unbuffered, bidirectional FPGA I/O before anything is soldered. Several Colorlight products route GPIO through unidirectional 74HC245 buffers; the module itself does not, so this concerns the extension board only, and its described layout – I/O in groups of eight, each with its own 3V3 and GND – suggests direct connection. Confirm with a continuity meter on arrival, or skip Stage A entirely and the question is moot (→ Q67).

A fault found at 1–2 MHz is real; a fault *absent* there says nothing about 8 MHz on a board with a ground plane. Stage A validates function, Stage B answers timing.

P.20 Stage B – the carrier PCB, and this is the real deliverable. Four layers: DDR2 SODIMM 200-pin socket, PLCC-44 socket and W65C816S, AS7C38096A SRAM, RP2354B and microSD, R-2R ladders and DE-15, PCM5102A and 3.5 mm, USB-C, probe header on the ten spare block-A pins. Four rather than two because the CPU bus needs a ground plane and the ladder outputs need a clean return. Hosting the socket directly bypasses the buffering question, reaches all 106 pins, and gives controlled impedance on the bus.

PHI2 targets 8 MHz here, the target board's figure (B.1). Every stall, fill and cache measurement taken at 1–2 MHz is an existence proof and nothing more; the numbers that matter are the ones taken at the real clock (→ Q72).

Capture it in KiCad with the symbols and net names intended for the target board. The CPU-to-SRAM-to-FPGA section is the most timing-sensitive part of the eventual design and this is the chance to validate it where a mistake costs a cheap respin. USB-C at 5 V / 3 A or a barrel jack; a legacy 500 mA USB-A port is marginal and may not start the board. **Measure the module's actual draw before finalising the schematic** – 400–600 mA for the module, ~500–700 mA total with the PHYs held down, ~700 mA–1.1 A with them running, and all of those are estimates (→ Q70).

The probe header on the ten spare block-A pins is this board's version of C.19's rule for test points: free at fabrication, unbuyable afterwards.

P.21 The order of the two stages, which is the one thing easy to get backwards.

Stage A exists *because* the carrier does not, so it comes first or not at all: wire-wrap at 1–2 MHz reaches P2 while the PCB is being drawn, and if the decision is to go straight to the carrier then Q67 is moot and Stage A is simply skipped. The carrier's own sequence is then fixed – **schematic captured with the target's symbols and net names → four-layer layout with controlled impedance on the CPU bus → DRC clean and fabrication outputs → assembly → P0 and P1 re-validated on the board before P2 runs on it at 8 MHz.**

Re-running two stages that already passed on temporary wiring is not ceremony. It is what separates a carrier defect from a gateway defect for the rest of the project, and it costs an afternoon.

E0.3's symbol-by-symbol pass covers every part on this board and happens **before** the schematic, not alongside it. A footprint error found by a printed sheet of paper costs nothing; the same error found by a delivered board costs the board.

P.22 Design-for-debug is a layout pass in its own right, and it is what buys back the validation P.19's Stage A would have provided. Skipping wire-wrap means debugging a first-spin board with no prior proof of anything, and the trade that makes that survivable is provisioning, not luck. Six things, all free at fabrication and unbuyable afterwards (C.19 states the same rule for the target board): **a series 0 Ω or jumper option in every clock and every inter-domain bus · test points on every rail and on PHI2, RDY, RESB, ABORTB, VDA, VPA, BE, every SD and SPI line, the I2S trio and the whole CPU bus · sockets for the CPU and, if the layout allows, the SRAM · a current-sense shunt or 0 Ω link in each switchable rail · a strap that brings the rails up with the FPGA module absent · and rail voltages on the silkscreen.**

The test-point list is written against P.23 rather than in the abstract: **every step of the bring-up procedure has to name a physical point or a strap it is measured at**, and a step that cannot is a step that will be guessed at.

The module makes this sharper rather than softer. Its FPGA is a caBGA part somebody else soldered, so **no ball is ever directly probeable** – everything observable has to come out on a SODIMM pin. P.05's ten spare block-A pins become the probe header for exactly that reason.

P.23 The first-spin bring-up runs in eight stages and the order is the safety, not the convenience. (1) **Bare board:** visual and continuity, every rail net checked for a short to ground before any power. (2) **Rails only, current-limited,** with the module and the CPU absent – bring up the always-on rail first, verify, then each gated rail one at a time by asserting its enable by hand, checking voltage and inrush on its shunt. (3) **The EC alone:** confirm it boots, takes firmware over SWD, answers on J-DBG, and can toggle every enable and read every I2C device (D.13). (4) **FPGA configuration:** insert the module, configure a blinky, confirm the device is detected and its ID is what it should be. (5) **Clocks:** load a PLL test bitstream and scope every derived clock – confirm each was promoted to a global net rather than routed as logic, and that BCK and LRCK are continuous (W.7a). (6) **Memory:** the SRAM first, then the carrier SDRAM, with march, walking-ones and address-in-address patterns – and **verify that DQM, CS# and CKE actually toggle**, which is the entire reason the SDRAM is on the carrier (P.11a). (7) **The CPU last,** socketed, with PHI2 from a jumper-selectable low clock at 2–4 MHz, the bus timing scoped, and control handed between the CPU and the debug agent over BE. (8) **Video:** the R-2R output and text Mode 0.

The CPU goes in last and it goes in socketed, which is the one irreversible ordering decision in the list: a bus fault with the CPU present looks like a CPU fault, and every stage above it is cheaper to diagnose with the socket empty.

P.24 The instruments are part of the plan, because a first-spin-only strategy is exactly the one that cannot be executed by inference. A 16-channel logic analyser with buffered capture at several hundred MSa/s is necessary and an eight-channel toy is not adequate – this is a 100 MHz-class board and the bank-byte window is tens of nanoseconds wide. A four-channel scope at 100 MHz or better, with 12-bit resolution earning its keep twice on this board: the R-2R ladder's steps and rail ringing. A bench supply with adjustable current limit, without which stage 2 of P.23 cannot be run at all. A JTAG probe for the module with the open loader, plus a couple of 3.3 V serial adapters. This is where the money saved by cancelling Stage A should go, and it is a better trade than it looks: wire-wrap validates function once, and an instrument validates every stage of the project after it.

P.25 Three toolchain traps on this device family, all of which produce a build that passes and a board that does not. Tristate and bidirectional nets must be resolved only at the top level – a Z assignment buried in a submodule is a synthesis-time surprise on this backend, which matters immediately because P.06 makes A0–A10 and D0–D7 bidirectional. PLL outputs are not always promoted to global clock nets automatically: check the place-and-route log and force the promotion rather than assuming it, which is what stage 5 of P.23 verifies on hardware. And the module's part is the slowest speed bin of its family, so static-timing margin has to be budgeted rather than discovered – the 103.125 MHz Neon domain (W.11) is reachable on it, and should be closed in place-and-route against real constraints early rather than after the gateway is written.

If that domain will not close after pipelining, the ranked answers are: pipeline harder, then drop the pixel clock, then – last – cross-check the timing in the vendor's own tool. The third is worth naming precisely because A.6 makes the open toolchain a principle rather than a preference: using a proprietary analyser to *check* a design the open flow builds is a smaller concession than using it to *build* one, and it is still a concession. The open flow stays primary in either case (→ Q117).

What must not leak to the target board

Each row is a convenience here that becomes a defect there.

PROPERTY OF THE PROTOTYPE	WHY IT MUST NOT SURVIVE
Shared SDRAM controller	The target has two independent memories
A single clock domain	The target has genuinely separate PLL domains
243 KB of EBR	Each HX8K has 16 KB, and Neon's is already fully allocated
Zero-wait access to Neon	The target stalls PHI2 and the model stays write-only
A ×16 SRAM, or any unshared byte lane	The target shares D0–D7 and A0–A10 with the CPU
Hardcoded cache or buffer sizes	Every one of them is a synthesis parameter
PHI2 at 1–2 MHz	The target runs 8 MHz and the margins are different
VGA timing assumptions	The target drives one fixed-resolution panel
Any Helium–Neon signal outside P.13	The target has three control nets and a shared bus

PROPERTY OF THE PROTOTYPE	WHY IT MUST NOT SURVIVE
Bandwidth figures from x32 shared memory	The target's memory topology differs
Debug reaching Neon by the internal path only	The target needs the external bus cycle path
The absence of Argon	The softcore option stays untested until the target board
The absence of a power path	The target sequences rails before configuration

Prototype stages · P0–P5

- **P0** **Platform alive.** Bitstream through openFPGALoader; blinky proving clocking and PLL lock; the EC's **J-DBG** console echoing.

TEST ▶ SDRAM march across all 8 MB – walking ones and address-in-address · Ethernet PHYs confirmed held in reset on P4 · current draw measured both ways and written into Q70.

- **P0.a** **BOM ordered first**, because the lead times are the only part of this stage nobody can compress: Colorlight i9 v7.2, W65C816S and PLCC-44 socket, the AS7C38096A-10TIN of P.07, an RP2354B board, DE-15 and 1 % ladder resistors, PCM5102A, microSD holder, passives and headers.

- **P0.b** **Blinky through openFPGALoader**, with PLL lock observable on a pin rather than inferred.

- **P0.c** **PHY reset asserted at configuration** and left asserted; module draw measured with the PHYs both ways and the numbers written down.

- **P0.d** **EC bring-up** – LED, build-and-flash loop over SWD, console echo on **J-DBG** . A host console attached to the system.

- **P0.e** **EC ↔ FPGA SPI link** with a scratch register file in the fabric, giving host-side read and write of FPGA state. This is the debug backbone and every stage after it is easier for existing.

- **P0.f** **SDRAM controller exercised** across all 8 MB – main memory proven and characterised, not assumed.

- **P0.g** **SRAM on temporary interconnect**, validated by the same march. What is being proved is the asynchronous parallel interface and its pin mapping, not the part.

- **P1** **Console before memory.** Text Mode 0 rendering a character grid over VGA at 640 × 480 from EBR alone, accepting console input.

This is T.1 and E1.7's N0 on different silicon, and it earns its place for the same reason: a bitstream putting correct characters on a screen has proved the PLL, the timing generator, EBR initialisation and the output pins in one shot, and every stage after it is easier to debug for having a display. **The geometry does not become a parameter** – one console driver serves both boards (→ D44).

TEST ▶ no SDRAM access anywhere in the path · the buffer is **128 × 32 with T.25's arithmetic unchanged**, displayed as an 80 × 30 window at 640 × 480 and in full at 1024 × 600.

- ☐ **P1.a** R-2R ladders assembled and the timing generator producing colour bars at 640 × 480 / 25.175 MHz – a monitor displaying output from your own gateway.
- ☐ **P1.b** Mode 0 text engine in EBR with the font ROM and the 128 × 32 buffer, T.25's `TEXT_START + text_row*128 + column` unchanged (D44).
- ☐ **P1.c** The buffer written from the EC over SPI, with console input echoing to the screen – a working terminal that never touches the memory controller.
- ☐ **P2** CPU attached. 65816 in its socket, released from reset, free-running NOP test executing.

TEST ▶ bus cycles captured in the trace buffer and readable over the debug channel
 · VPA/VDA decoding confirmed against expected fetch patterns · native mode entry verified.
- ☐ **P2.a** Two top-level modules from the first commit – `helium.v` and `neon.v` side by side, separate PLL outputs, CDC on every crossing, the interface of P.13 and nothing else. This is decided at the start or paid for later as a rewrite (P.12–P.15).
- ☐ **P2.b** Helium skeleton – PLL, reset sequencer, PHI2 generation, bus-cycle FSM, bank-byte latch, VPA/VDA decode. Every CPU bus cycle correctly framed and classified.
- ☐ **P2.c** Trace buffer in EBR with a raw debug path – direct access to EBR and SRAM, no cache anywhere in it, so cycles are capturable the moment the CPU is attached. It is re-routed through the cache at P3.f, once there is a cache to be coherent with.
- ☐ **P2.d** CPU boots from EBR-hosted memory with real reset vectors, and enters native mode – the CPU executing your code out of memory you control.
- ☐ **P2.e** Bank \$FF I/O decode with a write-only LED register. The privileged bank exists and the CPU can command a peripheral.
- ☐ **P2.f** Neon's text aperture written by the CPU across the shared bus, with the four-step stall protocol of P.17 implemented rather than shortcut. The CPU prints to the screen.
- ☐ **P2.g** CPU-visible UART in bank \$FF bridged to the EC's console UART – bidirectional interactive I/O between CPU code and your terminal.
- ☐ **P2.h** crt0, user and kernel linker scripts and a minimal C runtime; a C hello world printing through the UART register. The convention audited at E0.8 now reaches real hardware.
- ☐ **P2.i** Native 65816 monitor resident in SRAM – examine, deposit, go, disassemble. The machine is self-hosting for inspection, with no host attached.
- ☐ **P2.j** microSD boot path – the EC reads an image from the card into memory and releases the CPU from reset.
- ☐ **P2.k** USB HID keyboard on the EC, delivered to a CPU-visible register.

- **P2.1 Text console driver** – Mode 0 plus the keyboard, wired to the monitor. Power on and the board reaches its own prompt on its own display.
P2.h–P2.l are a checkpoint, not a milestone. Text mode, no MMU, no graphics – worth having for the debugging and for the morale, and every piece of it is needed by P3 and P5 anyway, so nothing here is built twice. It is not E6 and must not be recorded as it.
- **P3 Translation active.** Helium's MMU, TLB and walker operational with the CPU executing through translation; cache controller against the external SRAM with plausible hit/miss counters.
Pass condition: the virtual memory system is real, not simulated. This is the stage the whole prototype was worth building for – E4 arrives with its hardest gateware already debugged.
TEST ▶ PHI2 stalling on a fill, bounded and measured · ABORTB raised on an unmapped page, handled, instruction resumed · context switch between two ASIDs with correct CTX_SET_PTBASE / CTX_SET_ASID ordering · the fill watchdog tripped deliberately instead of freezing the board.
- **P3.a SRAM exposed to the CPU as raw uncached memory** – real off-chip memory usable by CPU code.
- **P3.b The shared SDRAM controller and its arbiter**, fixed priority with the starvation guard of P.11 and **per-requester wait-time histograms instrumented from the first version**, not added later when a number is finally wanted.
- **P3.c SDRAM reachable by the CPU through the arbiter**, uncached and slow – large main memory addressable before it is fast.
- **P3.d Cache controller against the SRAM** – 2 KB tagged lines, 4-way, 256 B fill sub-blocks with valid and dirty bits per sub-block (F.5, F.6), write-back, PHI2 stalled with BE=0 for bounded fills, hit and miss counters. **Size, line, sub-block and associativity are synthesis parameters from the first line of code (P.18).**
- **P3.e Flush and invalidate with the BUSY polling protocol** – what software needs to enforce coherence, and the prerequisite for both DMA and paging.
- **P3.f Debug agent re-routed through the cache controller**, with a per-transaction watchdog and the DEBUG_ENABLE strap. Host memory access is now coherent with the CPU's own view.
- **P3.g Halt, single-step, resume and register readback**, with the fill watchdog suppressed while the CPU is halted – otherwise every session ends in a spurious abort (R.17).
- **P3.h EC monitor front end** over the console UART – **load, go, dump, halt, step, trace** (R.22). A complete bring-up workbench: binaries load and run in seconds.
- **P3.i ASID-tagged TLB in EBR with a single identity mapping** – every access translated and behaviour unchanged, which is precisely what proves the datapath before anything depends on it.
- **P3.j Hardware page-table walker** reading 32-bit PTEs from the fixed, uncached SRAM region of F.5.

- **P3.k** **ABORTB on unmapped pages and permission violations**, an assembly handler that reports, and **instruction resumption verified correct** – faults detected, delivered to software, and recoverable.
- **P3.l** **Context registers** with the `CTX_SET_PTBASE` → `CTX_SET_ASID` ordering invariant enforced in hardware; two contexts switched from the monitor, two distinct 16 MB spaces coexisting.
- **P3.m** **Protection policy enforced** – bank `$FF` privileged and never user-mappable, bank `$00`'s vector and stub pages pinned in every context (L.11), bank `$01` the resident kernel (J.1), vectors mapped identically everywhere.
- **P4** **Graphics**. Framebuffer scanout from SDRAM stable at 1024 × 600, 18-bit palette lookup, blitter, hardware cursor.

TEST ▶ minterm logic verified against a reference implementation · cursor overlay tracking without involving the composite path · arbiter wait-time histograms showing no scanout underrun under blitter load · **a CPU read of the framebuffer returning exactly what was written, including immediately after a write** – the write-FIFO drain of T.19 – and **a read with Neon held in reset timing out into \$FF plus a fault, not into a frozen machine**.
- **P4.a** **Framebuffer scanout from SDRAM at 1024 × 600, 8 bpp**, through the 18-bit palette of P.16 – Neon stage N1 (T.63), in the same pixel format the production RGB path uses.
- **P4.b** **Hardware cursor overlay in EBR** – pointer motion costs nothing in the composite path.
- **P4.c** **Blitter, register-driven, straight copy only** (T.37) – rectangles move without the CPU.
- **P4.d** **The full blitter, its raster-operation logic verified against a reference software model** – barrel shifter, edge masks, descending mode, queue. **Which engine is built is Q58**, still open, and this is the board where trying it both ways costs a rebuild instead of a respin.
- **P4.e** **Command FIFO** – Neon stage N4, and the CPU stops polling `BLIT_BUSY`. SDRAM command lists are N5 and the compositor does not need them (V.35).
- **P4.f** **Arbiter wait-time histograms captured under sustained blitter load** – no scanout underrun, with evidence rather than assumption.
- **P5** **System**. Audio DMA to the PCM5102A without underrun; compositor with damage-limited recomposition; kernel booting to a prompt.

The milestone is reached on temporary hardware, which is the point and also the caveat: it proves the software and the gateway, not the machine. **E6 is still the milestone**, and closing it costs the P-series work plus a board that exists.

TEST ▶ an hour of audio with the underrun flag still clear · a window dragged across two others and every exposed region correct afterwards · the kernel reaching its prompt from a cold card with no host attached.
- **P5.a** **Audio DMA to the PCM5102A over I2S**, continuous playback with no underrun – the machine makes sound reliably.
- **P5.b** **Kernel skeleton in C** – startup, vector table, direct-page conventions, IRQ dispatch. A kernel image that owns the machine.
- **P5.c** **COP syscall dispatcher** with `write` and `exit`, against the ABI documented at E0.8 – the user–kernel boundary as a contract rather than a habit.

- **P5.d Physical frame allocator and page-table construction** – the kernel can build an address space.
- **P5.e First user process** loaded into its own space at reduced privilege, demonstrably unable to reach bank \$FF or the kernel.
- **P5.f Timer IRQ, scheduler and the assembly context switch** – two user processes running preemptively and concurrently.
- **P5.g Demand paging and copy-on-write** on top of ABORT – lazy allocation and cheap sharing, against the deliberately small physical pool of P.11.
- **P5.h Ports, signals and messages** – the IPC substrate sheet V is written against.
- **P5.i Driver framework** – init/read/write/ioctl/irq with fd-based device paths, and `/dev/tty`, which turns the console into a driver like any other.
- **P5.j microSD block driver and a read-only NVFS mount behind sheet Y1** (V2, V3) – files on the card visible to the OS through the real syscall path rather than a bespoke one.
The FIFO path is enough for this stage and the DMA engine is not needed yet – the driver is written against the register names of G.6 from the start, so the engine lands underneath it later with nothing above changing. What it cannot reach on the FIFO alone is paging (→ D45, Q79).
- **P5.k Program loader and a shell** – type a name and a program runs in its own address space.
- **P5.l `/dev/fb` with an ioctl mapping framebuffer pages into a process, plus `/dev/audio`** – a user program draws at memory speed and makes sound through the ordinary device contract.
The compositor's cost model is measured at G2, not here (V.36, V.37). The number wanted is the blitter's own time for a full-screen pass, separated from emission and from whatever the client tasks are doing – and it is the project's first real software gate, so it is instrumented deliberately rather than inferred from a frame rate.

■ **APPLE II · ON THE PROTOTYPE** – the software boots to a prompt on borrowed silicon.

Beyond P5 the G-series of sheet V runs entirely on this board while the target hardware proceeds in parallel – with the exception of everything resting on the \$FF power block, which has no prototype (P.02).

Target board · E0–E8

Nine stages, each with concrete hardware/gateway/software and a verification criterion before moving on. E0 and E1 are broken out below, because everything before a working board exists is where the project actually is today: the pin budget is closed, all six schematic sheets are designed *as documents*, and nothing has been drawn in KiCad. **E0 is also where the project-wide work lives** – the toolchains, the audited 65816 ABI, the simulator and CI – together with the two things that have to

be finished on borrowed hardware before a layout is committed: the iCE40 resource proof and the power path on eval boards.

□ **E0.1 Toolchain.** KiCad 8, oss-cad-suite (Yosys + nextpnr-ice40 + IceStorm), pico-sdk, and a simulator matching the HDL choice. **The P-series adds nextpnr-ecp5 and prjtrellis to the same suite** – one more target for the same flow, no proprietary tools on either board (P.01).

TEST ▶ a simulated iCE40 blinky and a compiled RP2354B "hello".

□ **E0.2 HDL decision** – Verilog or VHDL. Not a preference: it decides which reference softcore is usable at all (→ Q2).

This now blocks the P-series rather than the schematic, and it is the first thing the prototype makes urgent: every line of gateware written for the carrier is the same line that ships on the target board, so the choice is made before P0 rather than before E1.7.

TEST ▶ the decision written into Q2, with the reference cores it admits named beside it, before the first module of the carrier is committed.

□ **E0.3 Project KiCad library.** Symbol and footprint for every component, verified one at a time.

The parts the prototype never carries are the ones with nothing to catch an error first – **the power devices, the panel FPC, the touch connector, the audio DAC and the iCE40 TQ144** – so those get the paper pass twice, on different days. The carrier's own parts are checked here too, and that check blocks its schematic (P.21).

TEST ▶ each footprint printed 1:1 on paper and laid against the physical part.

□ **E0.4 Panel.** Candidate fixed and its controller verified – HX8282, 24-bit TTL at 3.3V with no level shifters, driven at 18 bpp on pin budget, DE mode by default, data latched on the *falling* DCLK edge so gateware changes it on the rising one. **The modeline has moved:** not ~51.2 MHz for 1344×635 but **51.5625 MHz for 1344×640 → 59.95 Hz**, which is what the 25 MHz oscillator can actually reach through one iCE40 PLL (T.4). Still missing the *module* datasheet, and the blanking values remain provisional until it arrives (→ Q1, Q43).

TEST ▶ the datasheet's DE-mode timing transcribed into the timing generator's own parameters, and the falling-edge latch confirmed against the part's diagram rather than from memory.

■ **E0.5 Pin budget across the three TQ-144 parts** – closed, it fits. This is what proved the architecture buildable, and what forced the shared-bus topology when it did not.

□ **E0.6 Freeze the block diagram** as the schematic's hierarchical reference.

This is not Fig. 1, and the distinction is what the stage is for. Sheet B's figure is the *architecture* – who inhabits the shared bus and who governs it from outside – and it is current. What E0.6 owes is the *schematic's* hierarchy: the same machine cut along the six sheets of E1.1, so that every block on the drawing is a page in KiCad and nothing on the board sits outside one. The REV B drawing this used to track has been retired rather than brought forward.

TEST ▶ every block on the frozen diagram maps to exactly one of E1.1's six sheets, and nothing on the board sits outside one.

- **E0.7 Repository and documentation skeleton** – gateway, firmware, kernel, hardware, docs, with the decision log at the centre of it. One place where every artifact lands, which is cheap on day one and expensive on day two hundred.
- TEST ▶ a fresh clone builds the gateway, the firmware and the 65816 binaries with no path pointing outside the repository.
-
- **E0.8 The 65816 toolchain audited, not merely installed.** Calypsi C and ca65, with the calling convention and the direct-page pseudo-register usage checked against actual compiled output and written down as the ABI the kernel is built on.
- blocking** No kernel work starts before this exists. A calling convention discovered by experiment halfway through P5 is the kind of blocker that stops a project for a fortnight, and it is the one known blocker on the software side (→ sheet 0).
- TEST ▶ a C function calling an assembly one and the reverse, with 8-, 16- and 24-bit arguments, disassembled and matched line by line against the written ABI.
-
- **E0.9 Simulator harness** for the 65816, with an I/O stub at the real bank \$FF UART address so the same binary runs on the simulator and on hardware unchanged.
- The point is discrimination rather than convenience. From here on a kernel bug is separable from a gateway bug – which is exactly the confusion that consumes weeks during bring-up.
- TEST ▶ one unmodified binary prints through the bank \$FF UART address in the simulator and on hardware, with the same output.
-
- **E0.10 CI building both gateway targets, the RP2354B firmware and the 65816 binaries, and running the simulator suite on every commit.**
- TEST ▶ a deliberately broken commit is rejected by the pipeline, not by a person.
-
- **E0.11 Gateway split into two independent builds.** `helium.v` and `neon.v` leave the shared top level and become separate projects communicating over the shared CPU bus and the three control nets of P.13 – every one of which stops being a wire inside a die and becomes a PCB net.
- There is no serializer and no dedicated inter-FPGA link to design here: D04 deleted that link, and P.13 is written the way it is precisely so that this step costs a build-system change. **If it costs more than that, the discipline section was not enforced** and the cost is being paid now instead of then.
- TEST ▶ each project synthesises, times and loads on its own, and the only signals crossing between them are the CPU bus and the three nets of P.13.
-
- **E0.12 Both halves ported to iCE40 HX8K on dev boards, and measured.** Argon's slot stays empty (B.4), so what has to fit is Helium and Neon: LUT, EBR and Fmax against the estimates of P.03 and P.04, built with the target EBR defaults and none of the prototype's instrumentation.
- blocking** This blocks E1.1. The one question the prototype cannot answer is whether the target devices are large enough and fast enough, because it has neither the fabric nor the block RAM of the parts that ship. Resizing a part here costs a line in the BOM; discovering it after a layout costs the layout.
- TEST ▶ **103.125 MHz closing on Neon's half** with the blitter placed beside the SDRAM controller, or the single-rate fallback taken deliberately and its bandwidth consequence accepted (→ Q44) · Helium fitting with the 768 KB cache's tags inside 16 KB of EBR (F.5).

□ **E0.13 The full software stack run on the two-device configuration** – kernel, drivers and the G-series as far as it has reached – before a board is drawn.

TEST ▶ nothing in the stack turns out to have depended on the single-FPGA prototype.

□ **E0.14 Power subsystem validated on eval hardware.** CH224K PD sink, BQ25896 charger, TPS63020 and the dedicated core buck, each rail brought up on the vendor's own board into resistive loads at target current; then the EC sequencer firmware written against them – always-on domain, staged enables, fault handling, MAX17048 readout.

This is the part of sheet C and sheet S the carrier deliberately does not exercise (P.02), so without this stage its first hardware would be REV A itself. Proving the iCE40's core-before-I/O order and a monotonic ramp on an eval rig costs days; discovering a violation on a populated board costs a respin and looks, for months, like an FPGA that configures unreliably (C.8).

TEST ▶ every rail brought to target current into a resistive load, with the sequencing order and each PG interlock observed on a scope rather than assumed.

□ **E0.15 Battery pack characterised on eval hardware** – protection, charge and discharge cycles, fuel-gauge learning – so the energy path is understood before it is designed into a layout rather than after.

The gauge is the reason this cannot be deferred to the board. A fuel gauge reports from a model that only settles after a cycle or two, so the figure the GUI shows at L3 is worth exactly what the characterisation here was.

TEST ▶ a full charge and discharge cycle logged, and the gauge's reported state of charge compared against the energy actually measured.

□ **E1.1 Six hierarchical schematic sheets**, designed simplest to hardest: 1.1 power · 1.2 RP2354B / EC · 1.3 FPGA configuration · 1.4 Helium, bus and memories · 1.5 Neon, panel and audio · 1.6 Argon and connectors.

All six exist as design documents; none is drawn in KiCad yet. Sheet 1.5 is held by the panel datasheet.

Capture them in the order they are designed, not the order they are numbered, and review the BOM and the DNP list against the pin-budget sheets as each one closes. The DNP that matters most is Argon: its 3V3 and 1V2 sit behind their own 00 jumpers so that an empty footprint cannot load the bus (E.5). A second one is worth placing on sheet 1.5 and is easy to forget because nothing before L4 needs it: **a PAM8302 and a pair of speaker pads, unpopulated, behind a jumper**. Audio ends at a 3.5 mm jack today and a laptop wants a speaker; the footprint costs a square centimetre now and a respin later.

TEST ▶ each sheet reviewed against the pin-budget tables as it closes, with its own DNP list written on the sheet rather than kept elsewhere.

□ **E1.2 Cross-review before layout.**

TEST ▶ clean ERC, plus a manual pin-by-pin pass of every TQFP against its datasheet – twice, on different days.

□ **E1.3 Layout.** Stackup decided against the router's difficulty, PLCC socket centred with the FPGAs around it and bus traces under ~10 cm, decoupling hard against the pins, continuous ground plane.

Route in order of difficulty, not in order of the netlist. The CPU bus first, then both SDRAM interfaces, then the parallel RGB to the panel FPC – and only once those are done, power distribution and everything that is left. The hard nets get the room while there is still room to give them.

TEST ▶ clean DRC and gerbers checked in an external viewer, not in KiCad's own.

□ **E1.4 Staged population – the golden rule:** never populate a block until the previous one works. The board is designed for this, with a test point and an LED per rail and 0Ω jumpers to isolate every branch.

Staged population also fixes the order the errata arrive in. A defect found with one block populated is attributable to that block; the same defect found on a fully assembled board is a search.

The rails are not the only thing worth probing, and the bus has no provision at all today. PHI2 , RESB , ABORTB , RDY , BE and the three Neon control nets each need a labelled test point – as do DCLK and DE , one bit of each colour channel, and audio's BCK and DIN , spaced to land two probes without shorting – they are precisely the signals E1.8 and E5 are debugged through, and a scope lead soldered to a TQFP pin during bring-up is how boards get killed.

Open the errata list at this stage and keep it as a document, not as a memory.

Every stage from here to E8 adds to it, and L7 is built from nothing else. A defect noticed during bring-up and not written down is a defect rediscovered on REV B.

TEST ► every rail measured at its test point before the next block is populated, and the isolation jumpers left open until it is.

□ **E1.5 Power alone**, in the order of C.19: charger and SYS with no downstream jumper fitted, then each rail into a resistive load, then the jumpers one at a time.

This stage also has to prove the **order**, not just the levels: 1V2 before 3V3_MAIN before VPP_2V5, each ramp monotonic, on a two-channel capture. A rail-order violation does not show up as a bad voltage – it shows up months later as an FPGA that configures unreliably (→ C.8).

TEST ► 3V3_AON and 3V3_MAIN within ±2% and 1V2 within ±3% under dummy load · PD contract measured at 9 V on VBUS · SYS holding with the battery disconnected entirely · charging with correct STAT lines · draw at each jumper checked against the budget table · rails driven by hand from the EN header, with no EC firmware in the picture.

□ **E1.6 RP2354B alone.**

The **crossings audit** of C.14 belongs before this stage, not during it – a net-by-net list of every AON↔switched connection checked against R1–R3. It is the one review pass on the board that cannot be recovered by rework if it is skipped.

TEST ► answers on the J-DBG console and takes firmware over SWD · mounts the microSD and lists files · reads the gauge and charger over I2C-AON and reports converted units · generates a measurable backlight PWM, with no FPGA populated · sequences every rail up and down under firmware control, including the R1 tri-state pass, measured with the switched domain dead.

□ **E1.7 The three FPGAs – and the screen, which arrives here rather than at E5.**

The debug agent's `DBG_ID` reading `$6516` belongs to this stage too – one SPI frame that proves link, bitstream and Helium clock at once (→ R.8).

N0 needs no SDRAM, no CPU, no Helium and no bus – only Neon, the panel and the EC's SPI – so there is nothing to gain by holding it to E5 and a great deal to lose. It is also a far better acceptance test than a blinky: a bitstream putting correct characters on the panel has demonstrated the PLL, the timing generator, block-RAM initialisation, the RGB pins, the FPC and the backlight in one shot.

Bringing it forward gives the project a screen three stages earlier than planned, and every stage after it is easier to debug for having one (→ T.64).

Inherited from P1, where the same text mode ran against a VGA monitor. What is new here is everything the prototype could not carry: the panel's own timing, the FPC, the backlight and the 51.5625 MHz modeline of T.4. The character generator, the buffer geometry and the console driver arrive already debugged, so a failure at this stage is a *board* failure – which is exactly the discrimination the P-series was built to buy.

TEST ▶ the EC configures each one separately with a blinky · CDONE high on all three · reconfiguring Neon without disturbing Helium demonstrated · **Neon stage N0**: a correct 128 × 32 blank text screen the instant Neon's CDONE rises, then glyphs written from the EC over the service port, then `TEXT_START` scrolling by exactly one row.

□ **E1.8 CPU alive – free-run. Bus forced to NOP, PHI2 from Helium.**

TEST ▶ the address counter advances consistently on the analyser · an LED blinks from a decoded address.

□ **E2 SRAM + serial monitor. SRAM on the bus; BIOS preload by the RP2354B; UART console; peek/poke monitor.**

The peek/poke does not have to wait for a BIOS: the debug agent gives physical read/write and a march test with the CPU still in reset, which is where most of its value is (→ R.22).

Inherited from P2 – the monitor, the linker scripts, the C runtime and the microSD load path all arrive already debugged, so a failure at this stage is a board failure or a timing failure and cannot be anything else. That discrimination is what the carrier was built to buy.

TEST ▶ console echo · read/write arbitrary memory · checksum of the loaded BIOS.

□ **E3 SDRAM. Controller (adapted open-source candidate) as a DMA/paging engine behind Helium, with auto-refresh interleaved into the fill state machine.**

TEST ▶ pseudorandom pattern over the full 64 MB with no errors for hours · refresh holds during sustained fills.

□ **E4** **MMU + cache + protection.** Walker, TLB with ASID, 4-way cache with tags in EBR, PHI2 stall with BE=0 and its watchdog, ABORTB, FAULT_* registers at \$FF.

The debug agent reaches its full extent here – halt, step and trace over the BE handoff, virtual-mode access, and TLB_PROBE against PTWALK to catch a stale entry. The same stage owes it one negative test: the fill watchdog must be suppressed while the CPU is halted, or every session ends in a spurious abort (→ R.17).

Inherited from P3 – walker, TLB, ASID switching, the PHI2 stall and its watchdog all arrive as working gateware. Two things do not transfer and have to be re-earned: the **cache size**, which was a synthesis parameter on the prototype and is 768 KB of 4-way here (P.18, F.5), and the **timing**, because the prototype's shared nets sat on a carrier at a lower PHI2 and this board runs 8 MHz with three loads (F.4).

TEST ▶ an illegal access triggers ABORT with correct FAULT_ADDR/CAUSE · a cache miss stalls and resumes cleanly · a deliberately hung fill trips the watchdog instead of freezing the board · measured cache hit rate.

□ **E5** **Video + audio.** Neon on the bus with its own SDRAM, \$FE window; VGA pattern → framebuffer; tone over I2S. This is now **Neon stages N1 and N2** (T.63): the SDRAM controller and arbiter, Mode 1 scanout, the register-driven blitter, then Modes 2a/2b with the 256-entry palette, COPY_KEYED and double buffering. The text console of E1.7 is the instrument the whole stage is debugged with.

Start below the gateware: the panel's own **BIST** mode generates test patterns with no external clock and no bitstream, validating panel, FPC and backlight in isolation before anything can be blamed on logic.

The two riskiest items are here and neither is a gateware detail. **103.125 MHz must close on the HX8K** with the blitter placed beside the SDRAM controller, and the fallback is single-rate at half the bandwidth (→ Q44). And the **aperture read path** is exercised for the first time: three of the four requirements of T.19 are only observable here, and the write-FIFO one fails *intermittently* if it is wrong – which is exactly the kind of defect that gets attributed to the blitter for a week (→ D37, Q57).

P4 moves that sentence. The four requirements of T.19 are observable on the prototype and are a pass condition there, so the write-FIFO drain is not seen for the first time on a board that costs a respin. What remains genuinely first-time here is the **103.125 MHz closure on the HX8K** and Neon's own SDRAM – neither of which the merged die can rehearse, since it has one memory and a different fabric (P.13).

TEST ▶ BIST patterns on the panel · stable VGA image · a CPU write shows up on screen · a CPU read of the framebuffer returning exactly what was written, including immediately after a write, which is the write-FIFO drain of T.19 · a read with Neon deliberately held in reset timing out into \$FF plus a fault, not into a frozen machine · worst-case read latency measured with the blitter running a full-screen keyed copy · a register-driven FILL_RECT covering the screen in 0.31 ms measured against BLIT_BUSY · an animated multicolour scene holding 60 Hz with SWAP_BUFFERS and no tearing · clean 440 Hz tone.

□ **E6** **SD + minimal OS + HID.** SD handoff, block driver, an NVFS read path (sheet Y2), kernel load, serial shell, basic USB keyboard.

The software reached this point at P5, on borrowed silicon. That is not this milestone and must not be recorded as it: the machine booting on its own is the claim, and it needs a board. What the prototype removes is the risk that the *software* is what fails here.

TEST ▶ power on → prompt, with no PC connected.

■ APPLE II MILESTONE – the machine boots on its own to a prompt.

□ **E7 Multitasking.** Scheduler with a 100 Hz tick, syscalls via COP, two processes with MMU isolation.

TEST ► two concurrent processes; one dies from a violation and the other stays intact.

□ **E8 Laptop + GUI.** 10.1" panel + GT911 + backlight, battery + charging, chassis; window compositor over `/dev/fb`; `/dev/power` with the battery indicator and the shutdown dialogue. On Neon's side this is **N3 and N4** – the 1-bpp barrel-shift blitter, `COPY_EXPAND` for glyphs and icons, the graphics cursor, and the command FIFO that stops the CPU polling `BLIT_BUSY` (T.63). N3 is what makes the milestone reachable; nothing above it is required for it.

`blocking` Which blitter is built here is not settled – the operation-based engine of sheet T or the four-channel engine of sheet U – and the same answer decides whether the compositor keeps per-window backing stores and whether the GUI runs at 1 or 8 bpp (→ Q58, Q59, Q60).

The software side of this stage is sheet V, staged G0–G8, and two things in it belong here. "Compositor over `/dev/fb`" is the flat-framebuffer phrasing and survives only if Q60 goes that way; under backing stores the framebuffer is not what a client draws into at all. And the split is not the obvious one – **the compositor is kernel code and the window server is a user task**, because command emission lives in bank `$FF` and no user task can reach it (Q62). G0–G4 fall inside this stage; G5–G8 – string cache, toolkit, menus, desktop shell – fall after it, and **G2 carries the project's first real software gate**: a measured full-screen recomposite (→ V.36).

The compositor's first implementation should not do precise damage regions.

Repainting a whole window costs ~1 ms, so repainting every window that intersects the dirty area, back to front, is affordable – and it skips the most defect-prone part of any window manager. Optimise only if a specific case demands it (→ T.57).

TEST ► touch GUI session running on battery · a window drawn as a dozen commands, with the CPU back in its event loop before the blitter finishes · a window dragged over a second one and both repainted correctly from application state · a short press raises a dialogue and a cancel actually cancels · `poweroff` from the shell flushes and drops the rails · a deliberately hung kernel is recovered by the 4-second press, and a deliberately hung EC by `QON` · the indicator reports *unknown* rather than a number when telemetry is stale.

□ **E8.1 GT911 touch on its dedicated I2C bus**, read by the EC and delivered into `wserver` through the same input queue as HID – absolute position, button state as a bitmap, an event that fits in 32 bytes and can be coalesced (V.30).

TEST ► a touch and a drag arrive at `wserver` as events indistinguishable from the HID path's, coalesced to no more than one per frame.

■ AMIGA MILESTONE – preemptive multitasking with a windowed GUI.

⌘ + Optional extensions outside the critical path: populating Argon (FPGA-C) with the softcore · demand paging + swap to SD · eDP panel variant (ANX6345) as a future revision · a writable filesystem once the read-only path is stable · networking on the driver framework of P5.i · **DRAW_GLYPHS** if EBR headroom ever appears, as a complement to the string cache rather than a replacement for it (V.26, Q61) · **Neon N5 and N6** – display lists in SDRAM with flow control, **WAIT_LINE** and **SWAP_BUFFERS**, which is the games target, then **DRAW_LINE**, **FILL_PATTERN** and **COPY_TILED** (T.63).
N5 sits here not because it is unimportant but because it is **separable**: N3 delivers the Amiga milestone and N5 delivers autonomous rendering, in that order, and neither depends on the other. A machine that stops after N4 has a complete GUI.

Beyond E8 · L1–L8 – the portable machine

E8 closes with a working noVa64 on a board that runs on battery and shows a windowed desktop. What follows turns that into something you pick up, open and close, and none of it is on the critical path for either milestone – which is exactly why it is listed separately rather than folded into E8 and allowed to delay it.

□ **L1** **Internal keyboard matrix**, designed and scanned by the EC firmware, delivered through the existing input path of V.30 so that nothing above the HID driver knows the difference. A built-in keyboard replaces USB HID. D.6 moved a matrix *off* the EC's pins onto USB HID so its budget would close; this stage brings one back, so where it scans is that budget reopened.
TEST ▶ every key and every two-key combination scanned without ghosting, with the HID driver unmodified above it.

□ **L2** **Integrated pointing device** selected and driven into **wserver** through the same queue – pointer input with no external peripheral attached.
TEST ▶ the pointer crosses the full screen with no external peripheral attached, through the same queue and with no new event type.

□ **L3** **Software power management** – rail gating, sleep and wake, battery status surfaced in the GUI through the **\$FF** power block of sheet S.
This is the stage where Q73 stops being theoretical. Everything resting on the power block had no prototype hardware at all (P.02), so the battery indicator, the shutdown dialogue and the stale-telemetry behaviour get their first real exercise between E8 and here.
TEST ▶ sleep and wake with the display, the card and the audio path all restored, and the battery indicator agreeing with the gauge after the resume.

□ **L4** **Mechanical CAD** – lid and hinge, mainboard mounting, battery bay, port cutouts, keyboard tray. A complete enclosure model before anything is printed.
TEST ▶ the model checked against the assembled board's measured outline, with every connector reachable and every fastener placed before anything is printed.

□ **L5** **Printed enclosure prototype** assembled, with the fit and cable-routing revisions applied. Everything physically fits and the lid closes.
The revisions this stage produces are the ones CAD cannot predict – cable bend radius, connector access with a lid at 100°, and where a hand actually holds the machine.
TEST ▶ the lid closes, every port is usable with the case shut, and no cable is under strain in either lid position.

□ **L6** **Thermal profiling and battery-life characterisation** under sustained GUI load, inside the enclosure rather than on a bench.
Inside the case is the only measurement that counts. A board that is comfortable in open air and a board in a sealed printed shell are different thermal problems, and the second one is the product.
TEST ▶ sustained GUI load to thermal equilibrium inside the closed case, with the battery figure taken on the same run rather than a separate one.

□ **L7** **REV B**, correcting the errata accumulated from E1.4 onward, and final assembly into the enclosure.
TEST ▶ the errata list closed item by item – each one either fixed in REV B or recorded as accepted, with nothing left unclassified.

□ **L8** **Release** – schematics, gateware, kernel, SDK, disk image, build instructions, user manual.
That test is also this document's: A.7 claims nothing here is written anywhere else, and a stranger building the machine is how that gets checked.
TEST ▶ someone who is not you builds it from what is published.

■ **PORTABLE** – a noVa64 you can pick up, open and use.

The host filesystem track · F0–F9 – off the critical path, and the only track that needs no hardware at all.

Ten gates, 107 sequential steps, run entirely on a development PC. It produces three host filesystem drivers under FUSE – FAT32, ext2 and NVFS – and it is **not** part of the P, E or L series: it blocks no milestone and no milestone blocks it. It exists for three reasons, and only the first is the obvious one. **Understanding:** implementing a filesystem from the on-disk bytes upward is the only reliable way to know one rather than to have read about one. **De-risking:** gate F4 is a host-side prototype of sheet Y3's L0–L8, executed where a debugger, sanitizers and `e2fsck -fn` are all available – and **a bug found there is a bug not found on a 65816 with a serial console**. And **specification:** three independent filesystems behind one internal

table is the cheapest possible way to discover what sheet Y1's interface actually has to be, rather than designing it and finding out later.

P.26 The order is FAT32, then ext2, then NVFS, and it is chosen so that exactly one thing is hard at a time. FAT32 first because its model is a reserved region, N copies of one allocation table and a data region of fixed clusters – no inodes, no permissions, no ownership, no hard links, no sparse files. The only genuinely new material in its gate is FUSE itself plus long-filename reassembly, which makes it the cheapest way to get a real filesystem mounted. **ext2 second** because it introduces the whole classical Unix vocabulary at once – inodes, bitmaps, block groups, the indirect tree, permissions, hard links, sparse files, fast and slow symlinks – and by then the FUSE side is free, so the attention goes to the format. **NVFS last** because by that point there are two reference designs to judge it against and its format work is already done: that gate is purely about the *binding* problem, which is a different skill and deserves not to be tangled up with learning what a superblock is. **Every read path before any write path**, which is the second ordering decision and the one that pays off least visibly. Write support is where the interesting failure modes live – ordering, crash consistency, allocation policy – and doing all three reads first means the write gates can be compared against each other while the read code is still fresh, and that **a bad decision in the shared layers is found before it has been baked into three write paths.**

P.27 Four layers, and the rules that keep the split real are enforceable rather than aspirational. L0 is the block device and is the only layer that knows the backing store is a file – on this machine it is replaced wholesale by sheet G's SD block driver and **nothing above it changes**, which is why its sector size is a runtime field rather than a constant. **L1** decodes on-disk structures and **performs no I/O at all**: it takes a byte buffer and an offset and returns values, which makes it unit-testable against a static array with no filesystem and no mount. **L2** is the filesystem logic and **never touches a FUSE type**. **L3** is the FUSE binding and **contains no arithmetic** – a cluster-to-address calculation appearing there means it is in the wrong layer.

Do not cast a packed structure over a sector buffer, however tempting it is on a little-endian host reading a little-endian format. Three reasons and the third is the one that matters here: taking the address of a packed member yields an unaligned pointer and some of these offsets genuinely are unaligned; **the eventual target is a different compiler with 16-bit int**, and **explicit accessors port where struct casts do not**; and an explicit read at a named offset beside a table of offsets is checkable against the specification line by line, where a structure definition hides a padding bug behind a `sizeof` that happens to be right.

Four portability rules to follow from the first step rather than retrofit: **never rely on int being 32 bits** – block, cluster and inode numbers take explicit widths; **no recursion whose depth is a function of on-disk data**, so directory descent and indirect traversal are iterative or depth-bounded (Y1.12); allocation confined to one wrapper so it can become a pool allocator; and no `long` or `size_t` in on-disk arithmetic.

P.28 The internal table is inode-keyed, not path-keyed, and that decision is the whole point of running this track before sheet Y1 is frozen. Path-keyed is simpler for FAT and does not survive hard links, and the kernel wants inode identity anyway – so FAT32 has to **synthesise** inode numbers, ownership and mode bits, which is the right forcing function: it makes explicit, early, what a `stat` structure actually means to a caller. `readdir` gets an opaque cursor rather than an offset, because both formats want to keep decoding state between calls. And the attribute structure is POSIX-shaped even where FAT has to invent most of it, because the alternative pushes the synthesis into every caller.

This is the shape Y1.21 requires, and it is the one thing on this sheet that is cheaper to get right at one implementation than at three.

GATE	CONTENT	EXIT
F0	Environment and lab	Images build, a kernel reference mount works, the build system runs
F1	FUSE mechanics	An in-memory filesystem of one's own mounts read-only; both libfuse APIs understood, and the low-level one chosen (P.28)
F2	Shared infrastructure – block layer, accessors, logging, the vtable header, the differential harness, the corruption corpus	All of it tested, with no filesystem logic written yet
F3	FAT32 read-only	Differential match against the kernel driver; survives the corruption corpus
F4	ext2 read-only – the host-side prototype of sheet Y3's read path	Differential match against the kernel driver; survives the corruption corpus
F5	NVFS read-only binding	The conformance read subset and the cross-implementation agreement harness pass
F6	FAT32 write	<code>fsx</code> clean, <code>fsck</code> clean, crash injection clean
F7	ext2 write – the prototype of everything Y3.9 specifies	<code>fsx</code> clean, <code>e2fsck -fn</code> clean, crash injection clean
F8	NVFS write and harness completion	Full conformance suite; a third participant in the agreement harness
F9	Windows and macOS – optional	FAT32 read-only mounts on Windows through a third-party FUSE layer

P.29 The chain is strictly linear and there is no parallelism in it, which matches this project's preference for one clear thread and accepts the calendar cost of that preference openly.

Run the host implementations single-threaded. libfuse is multi-threaded unless told otherwise, and letting that stand quietly makes its threading model a dependency of a design whose concurrency contract is Y1.19's and not libfuse's.

The track is done when the vtable is stable enough to be lifted into sheet Y1 unchanged – not when three daemons mount. That is the artefact; the daemons are how it was discovered.

The decisions shaping the current design, with their reasoning in one line – and the honest list of what's still open.

- D01** ECP5 → 3× iCE40 TQ144 – BGA isn't hand-solderable; 0.5mm TQFP is. Accepted consequence: 128 Kbit EBR → HW walker + small TLB instead of on-chip tables.
-
- D02** superseded by D13/D15 Main PSRAM **removed**; the 32 MB SDRAM is *the* memory – the QSPI gateway (the trickiest part) disappears, and a cache miss drops from ~μs to ~200–300 ns.
The removal stands and now covers the whole board, but its headline no longer holds: with the SRAM narrowed to 8 bits a fill is back in the μs range (D15). The decision survives on the gateway simplification alone.
-
- D03** superseded by D15 External table SRAM **removed**; a single 2 MB SRAM (cache + tables + pinned) – ~35 pins that didn't fit in the TQ144.
-
- D04** Amiga-style shared bus; out goes the dedicated FPGA-to-FPGA link and the eDP bridge – Neon taps the bus with the \$FE window.
-
- D05** Direct RGB-TTL 10.1" panel (ER-TFT101-1) – no bridge chip, no NDA (RTD2556 ruled out due to Realtek firmware). eDP/ANX6345 remains a future variant.
This choice is size-bound, and that is the thing to know about it. Parallel RGB-TTL exists only up to roughly 10 inches; from 14 inches upward every panel is a laptop panel speaking LVDS or eDP, and the direct path is simply gone (→ Q23).
-
- D06** superseded by D13/D35 512×300 → 1024×600 px-doubling – the framebuffer fits within the video PSRAM's budget and its QSPI bandwidth.
Closed by D35. Px-doubling is no longer a property of the machine: it is **Mode 2b**, one of three video modes, with 2×2 replication filling the panel exactly. The GUI gets native 1024×600 in Mode 1 and games get replication in Mode 2, instead of every subsystem paying for one compromise (→ T.33).
-
- D07** USB-only HID – out go PS/2 and keyboard matrix. Capacitive GT911 touch: the resistive option needed 4 ADCs the EC doesn't have free.
The mechanism underneath it changed and the decision did not. REV A put the host on the EC's PIO blocks because the RP2040 had no hardware host controller; the RP2354B has one, and the bit-banged stack leaves the critical path along with the pin-placement rule it imposed (D.8, D.9).
-
- D08** SD as the only boot path; dedicated BIOS flash **removed** – incompatible with the pin budget after adding the SDRAM.
-
- D09** Boosts (backlight, host VBUS) hanging off SYS, not VBAT – the machine runs on USB with the battery depleted. VPP 2.5V via BAT54: out goes the dedicated rail.
Both halves survive REV C, the second with its footing changed: the BAT54 now hangs off **3V3_MAIN**, which rises after the cores, so the iCE40's **VPP_2V5** ordering is satisfied by construction instead of being violated by it (→ D20).
-
- D10** Full 2 KB paging per process (out goes the "one bank per process" model) – a tractable 32 KB flat table, decoupled from the cache's 256 B fill granularity.

- D11** Hardware walker instead of a software trap on TLB miss – with 10 ns SRAM it costs ≈ 100 ns and simplifies the kernel.
- D12** 32-bit PTE with a 16-bit frame from day one – 128 MB ceiling with no format migrations. 16 rather than the 15 bits the pool needs is free: the flags occupy exactly 16 bits, so the other half-word was going unused, and the clean split lets the walker read 32 bits and take the low half as address and the high half as permissions with no shifting or masking ($\rightarrow$ sheet K).
- D13** PSRAM eliminated from the **entire board**, Neon's framebuffer included – every QSPI interface disappears with it. Neon gets its own 64 MB SDRAM instead: same part, same footprint, same controller as Helium's.
Streaming pixels across a link would need ~ 26 MB/s sustained with hard deadlines and would make Helium the bottleneck for the whole machine. A second SDRAM device costs the same pins as any smaller memory and buys double buffering at 16 bpp.
- D14** SDRAM sized at **64 MB per device** (AS4C32M16SB-7TIN, 512 Mbit) – the ceiling of SDR silicon in volume; no 1 Gbit part exists. The 256 Mbit W9825G6KH stays as a drop-in fallback on the same footprint. **The second device is deliberately not populated**: at 8 MHz the 65816 sustains ~ 4 MB/s, so merely zeroing 64 MB takes ~ 16 s. A second /CS is routed to an empty footprint.
- D15** SRAM narrowed to **1 MB $\times 8$** with D0–D7 and A0–A10 shared with the CPU nets – 13 pins instead of 41, which is what makes the budget close at 90/107. Possible only because A0–A10 are the untranslated page offset ($\rightarrow$ F.2). Accepted costs: Helium can no longer reach the SRAM alone, walks cost four accesses instead of two, and the shared nets carry three loads.
- D16** Cache misses handled by **stalling PHI2 low**, not by ABORT – the W65C816S core is fully static and the WDC datasheet sanctions holding the clock indefinitely. ABORTB is retained for genuine protection faults only, which removes the frequent case from the most delicate path on the chip.
Clock gating was also preferred over the RDY pin for a specific reason: RDY is reported to be ignored during write cycles on some revisions, a 6502 NMOS inheritance. Gating the clock sidesteps the question – with PHI2 stopped the CPU is not in any state at all, rather than in one whose rules depend on the cycle type.
- D17** The SRAM is a **hardware cache with tags**, never a tier of allocatable frames – migrating a page would force hardware to rewrite PTEs, which `mshare` and TLB coherence make impossible ($\rightarrow$ L.3). 768 KB of 4-way cache in 2 KB lines with 256 B sub-blocks, plus 256 KB fixed and uncached.
- D18** Debug access goes through a **debug agent inside Helium**, commanded by the RP2354B over a dedicated SPI chip select – the EC never masters the system bus. Pins (mastering needs ~ 32 lines the EC does not have), reuse (Helium already owns every memory controller) and coherence (an external master reads stale data past a dirty sub-block) all point the same way, and the agent's own registers stay **out of bank \$FF** so the privileged path cannot be reached by the software it can bypass ($\rightarrow$ sheet R).
Helium also drives `BE` as part of this: the bus handoff has to be a synchronous transition inside one clock domain, not a race across the SPI link. Closes Q25.
- D19** **3V3 split into an always-on rail and a switched one** – the decision the rest of the power revision hangs from. A controller cannot switch the rail that powers it, so REV B's single 3V3 made EC power management impossible in principle rather than merely unimplemented. Accepted costs: one extra converter, and a domain boundary that has to be honoured net by net ($\rightarrow$ C.2, C.13).

- D20** **1.2 V becomes a buck sourced from SYS**, not an LDO off 3V3. Two problems fall out together: the iCE40 wants VCC before VCCIO and an LDO off 3V3 guarantees the opposite, and dropping 3.3 → 1.2 V linearly at 250 mA burned ~0.5 W as heat in a SOT-23-5 on a battery machine. ~0.6 W recovered, and the ordering becomes structural rather than a firmware promise (→ C.8).
The rejected mitigation is worth recording so it is not proposed again: **holding `CRESET_B` low does not make a rail-order violation safe**. It prevents configuration, not conduction through the I/O clamps into an unpowered core rail.
- D21** **MCP73871 → BQ25896 + CH224K**, and the pack to 2× 5000 mAh in parallel. The pack grew to ~10 Ah and linear charging at that current is not viable; a switching charger at 90–92 % with a programmable SYS floor and an integrated ADC also hands the EC rail telemetry for free. The CH224K negotiates **9 V** – not 12 V, which leaves no headroom under the charger's 14 V input OVP.
The charger's **OTG boost cannot replace the TPS61023** for the host port, and an earlier proposal that it could was wrong twice over: the boost drives VBUS, the same physical node as the USB-C input, and it only enables while unplugged (→ C.5).
- D22** **Two level-held pins plus the existing SPI link** for the CPU↔EC power handshake: the pins carry attention, SPI carries the reason. A single wire encoding poweroff, reboot, ack and refusal by pulse width would be a bug source on a board where both endpoints are also under test. Costs 2 of Helium's spare I/O, onto a budget that does not close (→ Q8, S.2).
`POWER_CTRL` is also the one register that departs from the "address as opcode" model of M.2 – a small closed enumeration with no arguments is cheaper to decode by value than to spread over eight addresses. The departure is deliberate and confined to this block.
- D23** **No `sys_poweroff()`**. Power management adds nothing to the syscall table; it is `/dev/power` and an `ioctl`, and the privilege check comes free from the device node's permissions rather than from a bespoke gate. A single-purpose syscall in a table otherwise built from general primitives would have been the first crack in the rule (→ S.11, J.4).
- D24** **Battery telemetry as double-buffered registers inside Helium**, staging → live → shadow, each copy a single cycle. Two cheaper designs were rejected: a **seqlock**, which moves retry logic onto the constrained CPU with no hard bound; and letting the EC **write telemetry into SRAM** over the debug agent's bus path, which would make battery reporting depend on the cache controller and the arbiter – the one thing the power subsystem is specified not to do (→ S.16).
- D25** **The power registers sit outside the `DEBUG_ENABLE` gate**. Shutdown and battery reporting are production functions; gating them behind the debug strap would break them on every board built with debug disabled. They are read-only from SPI and cannot initiate a bus cycle, so the exception grants nothing the strap exists to withhold (→ R.18).
- D26** **Neon is specified in full** – three video modes, a blitter, a command processor executing display lists out of its own SDRAM, and a resource budget that closes at 77 % of the device (sheet T). It reconciles REV A against this board, which differs from what REV A assumed in four places: the reference clock, the SDRAM size, the existence of `NEON_BUS_BSY`, and where the interrupt goes.

- D27** **Mode 0's text buffer and font live in Neon's block RAM, initialised from the bitstream** – not in video memory, and not loaded by anything. 8 KB plus 4 KB of the 16 KB available, which is expensive; it buys a machine that displays a correct screen the moment **CDONE** rises, with no SDRAM controller, no arbiter, no FIFO and no CPU in existence. **The console is therefore available as the debugging instrument for every subsystem built after it**, which is the whole argument (→ T.1, T.29).
- D28** **One clock domain inside Neon, at 103.125 MHz**, with the pixel stage on a clock enable every second cycle and **PCLK** out through a DDR register. No clock-domain crossing anywhere, so no synchroniser FIFO between scanout and memory and an entire class of intermittent failure removed. **The second PLL is freed** and has two claimants, audio and the VGA bring-up clock (→ T.8, Q54).
REV A's 102.000 MHz is unreachable from this board's 25 MHz oscillator; 103.125 is the nearest legal PLL setting and the vertical total moves 635 → 640 to land back at ~60 Hz (→ Q43).
- D29** **No hardware tile engine and no sprite engine.** The blitter redraws a full 320×200 tilemap in ~1.3 ms, under 8 % of a frame, and blits 32 masked 16×16 sprites in ~0.12 ms; hardware sprites exist to avoid save/restore of background on machines that cannot afford to redraw, and with double buffering and 64 MB that problem does not arise. **Saving ~1,650 LUT, about 21 % of the device** – which is what takes Neon from 95 % occupancy, not routable once audio grows, to 77 %. **The hardware cursor is retained**, because a GUI pointer must track the mouse independently of the rendering loop (→ T.44).
The trade is real and is recorded so it is not re-litigated. What is given up: sprites composited at scanout cost no framebuffer bandwidth and cannot be erased by drawing. What is gained: one rendering model instead of two incompatible paths, and the logic budget to fit the blitter, the command processor and audio on one device.
- D30** **Blitter commands carry (x, y) coordinates, not 24-bit addresses.** Computing **base + y × stride + x** on the 65816 is a shift-and-add per object per frame on a CPU with no multiplier – precisely the work this architecture exists to remove. Neon does it instead, at one sequential multiply per command, ~16 clocks and ~80 LUT. The consequence is larger than the cause: **a destination becomes one 32-bit word, so moving an object is a single patch to an existing display list**, which is what makes per-frame CPU cost ~180 bytes instead of several milliseconds (→ T.42).
- D31** superseded by D36 **Neon's registers live at \$FE:FF00, inside the aperture, not in bank \$FF.** Neon already decodes bank **\$FE**; giving it a second window in the privileged bank would cost a decode in Helium and a wall on Q22 for nothing.
The half of this that survives is the **palette**, which stays in the aperture at **\$FE:8000** with 256 entries – it is bulk data, written a kilobyte at a time, and nothing about it is privileged (→ T.49). The half that was wrong is "for nothing": a second window buys the protection property of D36, which is not nothing.
- D32** **PEEK is deleted; the graphics aperture is readable.** REV A made it write-only, with a slow address/data register pair for diagnostic reads, on the premise that Neon could not stall the CPU. **NEON_BUS_BSY** already exists (B.6) and stalls PHI2 across the row activation, so a read simply works at a few hundred nanoseconds – about six times faster than the mechanism it replaces, and with no register file to build. Its protocol is D37. **The software model is still written as though the aperture were write-only**, because nothing in the design

needs to read pixels and anything that does will run at aperture speed (→ T.15, T.16).

D33 Backlight EN and PWM belong to the EC, not to Neon. The EC is awake before Neon is configured and while the switched domain comes down, and it already owns every rail enable. Closes Q6 and returns two pins to a budget that needs them (→ T.52).

D34 Neon's interrupt goes to Helium's PIC, not to the CPU's IRQB. Masking and priority come free from a controller that already exists, the CPU keeps one interrupt entry path, and it is one pin on each FPGA rather than a wire into the socket. Vblank, raster compare and the display list's SIGNAL are aggregated inside Neon and demultiplexed by the kernel reading IRQ_STATUS (→ T.47).

D35 Px-doubling becomes Mode 2b. What D06 proposed as a global policy – draw at 512×300, emit every pixel twice – returns as one mode among three, with 2×2 replication filling the panel exactly. Mode 1 gives the GUI native 1024×600 at 1 bpp and Mode 2 gives games replication at 8 bpp, so no subsystem pays for another's compromise. Supporting 2b alongside 2a costs approximately nothing in logic: it is a register field driving the replication counters and the line stride (→ T.33).

D36 Neon's registers live in bank \$FF, at \$FF:8000 – \$FF:80FF, and bank \$FF gains its first off-chip tenant. It reverses D31, which had put them inside the \$FE aperture, and it does so for a reason that outranks the tidiness argument on both sides: **\$FE is user-mappable and \$FF is not.** A process granted a framebuffer mapping could otherwise reach LIST_BASE and CMD_CTRL.LIST_RUN and aim the command processor – which reads and writes the whole 64 MB with no MMU in front of it – anywhere it liked. D31 could only answer that with a rule the mapping ioctl had to remember; **this makes it a property of the architecture**, putting Neon's control surface behind the same wall as the MMU's (→ T.21, J.5).

It costs no pins and no new mechanism. Neon already taps A0–A15 and already captures the bank byte off D0–D7 during PHI2 low, so recognising a second bank is the same comparator with a different constant, and NEON_BUS_SEL simply broadens to mean "Helium has validated this cycle for Neon" across both windows (→ B.6).

The base is \$FF:8000 so Helium's decode is one address bit – "A15 set in bank \$FF → not mine" – rather than a range comparison. Helium keeps \$FF:0000 – \$FF:7FFF whole, so the block allocation Q22 still owes can proceed with no risk of colliding with Neon later. The genuinely new gateware is small and specific: **Helium must release D0–D7 on a read in that window**, which is the same turnaround an \$FE read already performs, and **Neon must gate on the strobe rather than on the address alone** – ABORTB cancels an instruction on the CPU, it cannot un-write a register on the far side of the bus. No stall is needed: the file is fabric registers and answers inside the PHI2 high phase (→ T.23).

D37 The CPU can read VRAM, through a bus-busy stall – and the protocol is "stall first, ask afterwards". `NEON_BUS_SEL` selects Neon's bus; while `NEON_BUS_BSY` is asserted Helium holds PHI2 low; when Neon deasserts it the data is in Neon's output register and Helium raises the clock. The alternative – Neon asserting a wait line fast enough to hold off the rising edge – is a race inside a ~62 ns low phase across an asynchronous inter-FPGA crossing, and it is unnecessary: **Helium decodes the bank itself, so it knows a read of \$FE is in progress without being told.** It stalls unconditionally and the race disappears (→ T.18, B.6).

Stalling low rather than stretching the high phase is what keeps it cheap. Extending the high phase would need a fresh datasheet reading on holding a static W65C816S with the clock high, which nobody has done; stalling low reuses the exact clock gate Helium owns for cache fills and the permission D16 already established. It also falls out right on the bus, since D0–D7 carries the bank byte while PHI2 is low and Neon therefore must not drive then.

Four requirements travel with the decision, and the first would otherwise ship as an intermittent bug. **A read drains the write FIFO before it is issued** – writes are posted, so a read that overtakes a pending write returns the previous contents, failing according to arbiter load, which means "read back what was written" would pass almost every time. **The arbiter bounds the wait** – a row activation is ~60 ns and irrelevant, a read queued behind a whole blitter operation is up to 1.5 ms with the CPU clock stopped (→ Q29). **Helium's fill watchdog covers the path** (→ F.11, Q57). And **the software model stays write-only**, because the restriction was doing real work: it is why save-under is a blitter copy and why assets flow one way (→ T.19).

The same wire serves Helium with no clock involved. When the debug agent reads VRAM through an external cycle, `BE` is low and there is no PHI2 to stall – Helium just waits for busy to clear before sampling D0–D7. `NEON_BUS_BSY` is a ready line that Helium *happens* to use to gate PHI2 when the requester is the CPU. And it is less slow than it sounds: a read loop runs at ~70 % of the speed of the same loop against SRAM, because the bottleneck is the 65816, not the SDRAM (→ T.19).

D38 Neon's SDRAM is partitioned by function across its four banks, and this is a first-order decision rather than an allocator preference. Banks 0 and 1 hold the front and back framebuffers, bank 2 the even-numbered window backing stores, bank 3 the odd ones plus the font atlas, icon sheets and audio buffers. **The invariant is that during composition the blitter's reads, its writes and scanout are always in three different banks**, so every ACTIVATE is issued inside another bank's data phase. Collapse it and delivered bandwidth halves, from ~160 to ~80 MB/s – a 2D blit crosses a stride at every line boundary, and a shared bank turns each crossing into a PRECHARGE plus ACTIVATE that overlaps with nothing (→ U.16). **Closes Q50.**

Two obligations come with it. The **controller must map the bank bits to the high address bits**, or the partition cannot be expressed as an address range at all – the obvious alternative of interleaving banks on low bits to spread sequential traffic would destroy the whole policy. And **the kernel's VRAM allocator is no longer free to place a surface wherever it likes**; the partition is part of the interface, not an optimisation it may ignore.

- D39 Neon's arbiter is fixed-priority – refresh, scanout, audio, CPU, blitter – with preemption only at burst boundaries and a minimum burst of 8 words.** Scanout is not merely favoured but *shown* to be safe: worst case 23 clocks, 223 ns, against a 512-word FIFO that a stall of that length drains by 5.8 words. This is the inverse of the Amiga's **BLTPRI**, which let the blitter starve the CPU; here the blitter can starve nothing. **Placing the CPU above the blitter is also what bounds D37's read stall** – a read waits for one burst, never for a whole blit (→ U.25, U.27). **Closes Q57 and the Neon half of Q29.**
- The margin is the point rather than the number. If the clock falls to 51.5625 MHz under Q44's fallback, if the burst grows, if audio's demand doubles, the stall stays two orders of magnitude inside the FIFO. **A real-time guarantee resting on an unverified estimate is not a guarantee**, and a scanout underrun has no degradation mode – the line is simply corrupt.
- D40 A prototype board is inserted before the target board**, and it is the first schedule decision this log has had to record. Helium and Neon are merged into one ECP5 on a commercial Colorlight i9 module with the CPU, SRAM, EC, VGA and audio on a carrier of our own, so the whole software roadmap runs on real gateware while the KiCad capture proceeds in parallel. **The blocked dependency was serial and is now two tracks**: Q1's missing panel datasheet stops the schematic and stopped everything behind it, and it stops nothing on the carrier. Accepted cost: one board that is thrown away, and a discipline section – P.12–P.18 – that has to be enforced at every merge or the prototype becomes a rewrite (→ sheet P).
- The reason the trade is worth taking is asymmetric, not marginal. A gateware defect found on the carrier costs a bitstream rebuild; the same defect found after E1.3 costs a board spin, and the parts of the design most likely to carry defects – MMU, cache, arbiter, blitter, compositor – are precisely the parts that need no board-specific silicon to exercise.
- D41 One i9 module, not two.** Two modules would give Helium and Neon separate SDRAM, which is the target topology, but each is locked at 8 MB with no expansion path and the pair doubles cost, carrier complexity and the inter-module wiring. One module reaches the same software milestones, and the topology difference is handled where it belongs – as a leak-table row and P.15's memory-port abstraction, which is a discipline problem rather than a hardware one.
- D42 VGA through an R-2R ladder, not HDMI.** An earlier costing attributed an HDMI connector to the i9 extension board; it belongs to a Muse Lab board for the i5, a different product. The correction is an improvement rather than a loss: a TMDS encoder is 400–600 LUT of gateware that the target board deletes, whereas 6-bit-per-channel VGA is 20 pins from a budget with 24 spare and lands on **exactly the 18-bit palette entry the target panel takes** (D05, P.16). One less thing built here that has to be unbuilt there.
- D43 The prototype carries the target's 1 MB ×8 SRAM, not a ×16 part.** An earlier costing for the carrier specified an IS61WV102416 with UB/LB. That re-widens what D15 deliberately narrowed and would have the carrier rehearse a byte-lane topology the target does not have – while claiming, in the same paragraph, to be rehearsing the shared-net trick of F.2 that only exists because the part is ×8. The AS7C38096A-10TIN of F.5 goes on both boards.

- D44 Mode 0 geometry stays 128 × 32 on both boards; the prototype letterboxes instead of reflowing.** At 640 × 480 the console shows an 80 × 30 window into the same buffer, with T.25's `TEXT_START + text_row*128 + column` unchanged; at 1024 × 600 all 128 columns fit exactly and the 32 rows sit inside a vertical border. **Making the geometry a synthesis parameter was the alternative and was rejected:** it adds a configuration axis the target never uses, and it forces the console driver and every scrollbar consumer to ask the hardware how wide the screen is (→ P1).
- D45 The SPI-SD controller gets a register block of its own, with two data paths and a plain register file rather than address-as-opcode.** The card is unreachable from software until it has one, and the shape of it follows from what it has to carry: LBA, a physical destination and status are **state that persists between commands**, which is exactly the case M.2's address-as-opcode convention does not cover – that convention applies to the four triggers, whose argument is a block count, and to nothing else. Both a **FIFO path** and a **DMA path** are specified, because the FIFO reaches E6 without touching the arbiter or the cache and demand paging cannot use it: 512 bytes through the accumulator is ~440 μs of CPU at 8 MHz against 164 μs of SPI, and a FIFO-only design retracts G.1's reason for putting the card behind Helium in the first place. Card initialisation stays in software behind a raw-command register, per B.2 (→ sheet G).
- The DMA path is what forces `CACHE_INVAL_FRAME` into sheet M, and that command is the part of this decision most likely to be skipped by whoever writes the gateway first. Helium acquires a second bus master writing SDRAM behind a write-back cache, and the dirty line left over the destination frame is not a staleness bug but a delayed overwrite of correct data (→ M.12, G.9).
- D46 The expansion slot is a second instance of the inter-device contract, not a second bus.** A card gets `/EXP_SEL_n`, `/EXP_BSY` and `/EXP_IRQ`, T.18's stall-first read protocol and F.11's watchdog – the same shape Neon has, which is what Q56 chose the names for. **One polarity is deliberately reversed:** `/EXP_BSY` idles *not busy*, because busy-by-default in a slot means a machine with nothing plugged in does not boot, and the race that polarity was defending against is killed instead by a mandatory stall window before Helium starts sampling. Cost: **three pins for the first slot, one for each after it**, with the bank byte demultiplexed by a '573 on the board and the slot buffered off the CPU nets by a '245 – both of which cost packages rather than FPGA pins (→ sheet X, X.11).
- D47 A card is reachable in two spaces and neither costs a pin.** Control is a 256-byte window in bank `$FF`, which inherits L.10's keystone – no card in any slot is reachable from user code by construction – with Helium decoding the window and the card decoding inside it, exactly as T.21 splits Neon's. Bulk is a **1 MB aperture per slot carved out of the frame numbers F.10 deliberately leaves unpopulated**, so a PTE can name a card with the TLB and the permission bits still in front of it; the address reaches the connector on A11–A19, which Helium already drives and which are idle while the SRAM's `/CE` is (F.2). Aperture frames are non-cacheable and the gateway enforces it rather than trusting the PTE flag (→ X.8, X.9).

- D48** **No third-party bus mastering, and the reason is coherence rather than arbitration.** M.12's rule – invalidate before a read into memory, flush before a write out of it – is honourable by Helium's own SD engine because Helium owns the cache, and is not honourable by a card somebody else designed. DMA stays a privilege of the northbridge; `/EXP_BR` and `/EXP_BG` are routed to the connector and left unconnected at the FPGA, in the same spirit as F.10's unpopulated second `/CS` (→ X.16).
- D49** **The slow tier is a third I2C bus with Helium as its only master, and its windows are mirrors rather than proxies.** `I2C-EXP` avoids multi-master arbitration on either existing bus – C.13's R3 forbids a bus with two masters, not a third bus – and its two pins have been in Q8's allocation since before the split left them without a device. A window cannot perform the transaction on demand: a byte over I2C is ~90 µs at 100 kHz against a 125 ns bus cycle, three orders of magnitude past what M.6 permits to stall PHI2. So each window is a copy in block RAM refreshed by a sequencer, using **S.16's staging → live → shadow mechanism with a device table in front of it** – the atomicity and staleness properties come along unchanged (→ X.23, X.24).
- D50** **The embedded controller becomes a single RP2354B, and what makes it the right part is what it deletes rather than what it adds.** **48 GPIO** retires the MCP23017 that D.6 needed and with it the awkward question of which power domain an expander driving always-on enables belongs to (Q41). **2 MB of flash inside the package** removes a component from the always-on rail and a net from the board. **A hardware USB host controller** removes the bit-banged PIO stack from the critical path, along with its rule that `D+` and `D-` sit on consecutive GPIOs. The cost is paid in one place and paid fully: **committing the native controller to host mode makes `BOOTSEL` over USB structurally unavailable**, because a USB-A receptacle sources VBUS and reaching it from a PC would need two supplies opposing each other – so the one recovery path that needed no header is replaced by three that need no USB (→ sheet D, D.9, D.13). The console follows the port. There is no device-mode USB, so the USB-CDC half of R.19's mirrored console is gone and the console is the UART on `J-DBG` – the same 5-pin header that carries SWD, serviced by one probe over one cable.
- D51** **NVFS breaks its own format rather than keeping ordering discipline, and the break is accepted on the strength of five changes.** A **journal** replaces write ordering, because ordering is a correctness argument spread across every mutating operation – which is where draft 2's crash harness found its bugs – and a journal is one argument in one place, with idempotent single-pass recovery on a machine that has no on-target `fsck`. A **2048-byte block equal to the MMU page** makes demand-paging an executable exactly one block read and every index a shift. **Extents** remove the indirect-block read at ~2.5 ms each. **Fixed 64-byte directory slots** remove the in-band length field that is the largest single source of directory corruption in this class of filesystem – a cost Y3.10 still pays. And **POSIX-identical mode and type encodings** remove every translation table from both drivers and from the layer above. The price is `mkfs`, `nvfsck` and the read-path driver rewritten, and every existing volume evacuated by copy (→ sheet Y2, Y2.1). **One action has to happen before any draft 3 code is written:** tag and archive the draft 2 tooling, because evacuating a draft 2 volume depends on it and there will be no way to reconstruct it afterwards.

- D52** The media contract: one MBR primary partition of type **0x7F**, starting at an LBA that is a multiple of four, and no wear levelling of our own. **0x7F** is the byte reserved for local and experimental use; **0x7A** and **0x7B**, which an earlier draft proposed, are unsafe – they are the hidden-variant aliases of two FAT16 types under the **+0x10** convention, so some host tools would silently treat an NVFS partition as a hidden FAT. The alignment makes one NVFS block exactly four sectors, and **mkfs** refuses an unaligned partition rather than working around it, because working around it turns every transfer into two. Erase-block management and wear levelling belong to the card's own translation layer, and NVFS issues no discard (→ Y2.25).
- D53** A VFS layer exists, and it is specified before the second filesystem is implemented rather than after. Introducing it now costs one module and a mechanical rework of NVFS's call surface; introducing it after ext2 costs the same rework twice plus a syscall layer that has already grown branches. Its shape is decided by what neither driver can do: **lookup** returns an identifier and never a vnode, because a driver cannot know what is cached or what mount is grafted onto the result; **path resolution**, **symlink following** and **mount crossing** are VFS-owned, because neither driver can see across a mount boundary; and the **vnode cache** is VFS-owned, because two copies of one inode diverge silently on append and the logic is identical in both drivers. The locking discipline is written now and compiles to nothing until the Amiga milestone – retrofitting locking into a vnode cache fails intermittently rather than cleanly (→ sheet Y1).
- D54** ext2 is the interoperability format and is a transfer medium, not the system volume. Chosen over FAT32 for reasons that are all about this CPU: **little-endian on disk**, so every field is a direct load with no byte-swap path; a small documented structure set with no chain to walk; a real directory tree with permissions, links and symlinks; and no long-filename encoding layer. The recommendation is **read-only** – gates E2.0 and E2.1 – because that covers the actual use case at roughly a quarter of the cost and with none of the corruption risk, while the write path adds the allocators, truncation, the counter invariants and the whole ordering discipline ext2 has no journal to replace (→ sheet Y3, Q101).
- D55** One block cache below the VFS, not one per driver, with a sequence-number hook for the journal. Two caches means two pools sized by guesswork, no global LRU, and one mount's buffers idle while the other thrashes. Uniform 2048-byte buffers, of which an ext2 mount uses half – 50 % waste while an ext2 mount is active, accepted because ext2 is a transfer medium and a two-size allocator costs more code than the memory it saves. A **commit_seq** per buffer and a **flushed_seq** per mount stop a global LRU from writing a metadata block back before its journal record has committed, which a naive shared cache would do at an arbitrary moment and break silently. Four bytes per buffer and one comparison in the eviction path (→ Y1.14, Q91).
- D56** The filesystem stack is prototyped on a host PC before it is written for this machine, as a third series that blocks nothing. Three filesystems behind one internal table is the cheapest way to discover what D53's interface actually has to be, and a bug found under a debugger and a sanitizer is a bug not found on a 65816 with a serial console. **FAT32 first, then ext2, then NVFS; every read path before any write path** – so that a bad decision in the shared layers is caught before it has been baked into three write paths (→ P.26).

Open questions

- Q1** blocking ER-TFT101-1 module datasheet: backlight current/voltage, strap exposure on the FPC, GT911 connector, whether a panel reset or standby line is brought out to the FPC at all – **and whether the module needs any supply beyond 3V3 and the backlight string**. Most HX8282 designs generate AVDD, VGH and VGL on the module itself, and sheet C assumes this one does: the panel appears in its tree as a single 200 mA tenant of 3V3_MAIN. If it does not, the power tree gains rails and that row is wrong (C.17). Order the panel in the **capacitive variant**. Blocks the KiCad schematic for sheets 1.5/1.6.
-
- Q2** HDL choice (Verilog vs VHDL) – constrains the reference softcore (P65816 is VHDL).
-
- Q3** Specific SDRAM controller among the three open-source candidates (adaptation and verification in E3).
-
- Q4** EC↔kernel protocol – **closed for power**: shutdown, reboot and battery now have a specified transport, register set and failure behaviour (sheet S). What remains under this heading is the **HID event path** and the RTC, neither of which travels the power path. Two things are owed, not one: the event *format* (V.30 requires it fit 32 bytes and coalesce), and the *transport*, which sheet H still records only as "the EC channel". The proposal on record is the EC as I2C master with Helium as a slave plus a dedicated **IRQ_HID** line – three pins already carried in Helium's budget – but it predates the REV C bus split, and Helium sits in the switched domain while C.8 reserves the always-on bus to the charger, the gauge and the button. So it survives on **I2C-SW** or not at all, and that is a decision rather than an inheritance (→ Q35, Q41).
D49 constrains this proposal without settling it. Helium becoming a master on a bus of its own is precisely so that it never has to become a master on I2C-SW – which leaves the HID proposal's *EC master, Helium slave* arrangement intact, on the bus it was written for. What must not happen is the two being conflated later: **I2C-EXP** is Helium-as-master and I2C-SW is Helium-as-slave, and a single I2C block inside Helium serving both roles on two buses is a gateware decision nobody has taken (→ X.22).
-
- Q5** Exact format of the BIOS → kernel info block.
-
- Q6** **Closed – the RP2354B**. Backlight **EN** and PWM stay with the EC, which is the only party awake both before Neon is configured and while the switched domain is coming down, and which already holds every rail enable through its expander. Neon recovers two pins its own budget needs and **BACKLIGHT_EN** leaves the register map (→ D33, T.52).
-
- Q7** **Closed, and in D04's favour**. There is no data link. Three *control* lines survive between the FPGAs and no more – **NEON_BUS_SEL** inbound, **NEON_BUS_BSY** outbound, and Neon's aggregated interrupt into Helium's PIC – and the 8 or 16 pins the old budget reserved for a link were reserved for something that does not exist. **They belong to the CPU bus tap Neon needs in order to see the \$FE aperture at all**, which the same budget never counted; the substitution is recounted in Q12 and it costs 10 pins net (→ B.6, T.51).

- Q8** **blocking** The 17 free pins on Helium do not cover SPI-SD, the UART console and the debug port. Allocated today: inter-FPGA link 8, I2C 2, RP2354B handshake 3, spare 4. Sheets D and F both depend on interfaces that have no pins – the budget does not close as written.
- Sheet X is the first thing to arrive at this question with an answer rather than a demand.** The expansion slot is priced at three pins and the slow bus at zero – its two are the "I2C 2" already in the allocation above, which the C.13 bus split left without a device – and both are deliberately sized to be refusable: dropping the second slot is one pin, dropping the slot entirely is three, and neither costs a milestone (→ Q81, X.3).
- Sheet R sharpens the deficit rather than easing it: the debug agent needs `DBG_CSN`, `DEBUG_ENABLE`, `BE` out of Helium and optionally `DBG_IRQN`, and it turns the CPU-side address and data pins bidirectional (→ Q27). It only shares `SCK / MOSI / MISO` with the configuration bus, which is the one economy available.
-
- Q9** How many page tables actually stay resident? The 256 KB fixed region must hold the kernel (64 KB), the vectors, bank `$00` and the active tables at 32 KB each. K.2 used to promise 16 residents out of 512 KB; the honest figure now looks closer to four.
-
- Q10** SRAM footprint compatibility: confirm in the datasheets that A20 on the 2 MB `AS7C316096` lands on an NC pin of the 1 MB `AS7C38096A` in TSOP-II 44. Usual for Alliance, unverified.
-
- Q11** SDRAM column count: confirm the 512 Mbit part uses 10 column bits against the 256 Mbit part's 9, with the pinout otherwise identical – the premise the fallback footprint rests on.
-
- Q12** **Recounted, and it closes at 100 of ~107** – but only after two errors in the old estimate cancelled each other, which is the part worth knowing. The old figure reserved **16 pins for an inter-FPGA link D04 had already deleted**, and never counted the **26-pin CPU bus tap** that replaced it; the substitution costs 10 and the budget was wrong in the dangerous direction. Against that, VGA costs 2 dedicated pins rather than 14 (the R-2R ladders hang off the panel's own RGB bus), the backlight's 2 went to the EC (Q6), and the bank byte is captured off D0–D7 during PHI2 low instead of eight dedicated lines. Full table in T.51. **18 bpp remains the recommendation and 24 bpp still does not fit.** What is left under this heading is narrower and is what still blocks sheet 1.5: sign-off against the real TQ144 pinout, I/O bank by I/O bank, with the SDRAM's 39 pins kept in banks that can meet 103 MHz and the panel's 22 kept together – and with the GBIN-capable pins read off Lattice's own TQ144 pinout rather than assumed, since the oscillator has to land on one in all three devices (B.7).
-
- Q13** **Is the softcore even the right lever?** Given that memory and not the core is what caps this machine (→ E.10), effort may be better spent on the cache and memory subsystem than on Argon. Worth deciding explicitly before anything is invested in FPGA-C.
-
- Q14** Argon's role and memory path – two halves of one question. Does the softcore *replace* the physical W65C816S on the shared bus or *coexist* with it? And if populated, does it use the standard bus into Helium, with the same PHI2 and cache constraints as the physical chip, or a dedicated wider link into Helium's cache? Without the latter its advantage is marginal.
- Related to Q7: both ask what private FPGA-to-FPGA paths this board actually has.

- Q15** The real iCE40 Fmax of a candidate core: synthesize P65816 stripped of its SNES bus-timing fidelity on the target part and read the placed-and-routed figure. Every speed estimate in sheet E is guesswork until this number exists.
- Q16** Base core: P65816 cleaned up and given a flat memory interface, for maximum compatibility, versus FT816 for speed first – pending an instruction-coverage audit of FT816. Tied to Q2, since P65816 is VHDL.
- Q17** Freeze the canonical user virtual map *within* \$02–\$FD: where `.text` ends and `.data` begins, stack size, heap start.
The bank boundaries themselves are settled and not part of this question – an early draft placing user `.text` at \$01:0000 was rejected against L.10.
- Q18** Bank \$00 – **settled in principle**: each process's stack pages are pinned for its whole lifetime, and the vectors, trampolines and kernel stack occupy one or two privileged pages mapped identically in every process (→ L.11). What is left is sizing: the exact split of those one or two pages between the three tenants, how large the kernel stack should be, and whether there is one shared kernel stack or one per process.
- Q19** Audit Calypsi's calling convention and its direct-page pseudo-register usage. Two things fall out of it: the exact context-switch save set, and how syscall arguments are passed.
- Q20** Specify the binary format header – field layout, magic value, versioning.
- Q21** Shared read-only text: is libc's `.text` duplicated in every process, or mapped shared read-only across page tables? The PTE R/X bits already allow it and the memory win is free, but it puts page reference counting into the loader.
- Q22** Bank \$FF register map – **Helium's MMU block is specified** (→ sheet M), the **power block now exists too** (→ sheet S), and the **peripheral block has acquired its first tenant and its register set** – the storage block of G.6, specified as offsets precisely because this question is open. All three want a 256-byte wall of their own. Still unassigned: three base addresses, and the timer and interrupt controller entirely. Keep the walls between blocks so the decode and the privilege check stay trivial.
And the count has grown again, in a direction that argues for settling the map rather than extending it piecemeal. Sheet X asks for one wall per expansion slot – proposed at \$FF:7E00 and \$FF:7F00, the top of Helium's half against Neon at the bottom of the other one – plus one for the slow bus's block. That is now **six blocks and a timer** in a bank whose allocation is a five-minute decision nobody has taken, and the two sheet X adds are the only ones that can be moved for free, since no gateway and no header depends on them yet (→ Q82).
The debug agent claims no space here and must not be given any – its register file is SPI-only by decision D18 (→ R.4).
Neon is back on this list, and it is the reason to settle the rest of the map now. Its registers occupy \$FF:8000–\$FF:80FF – the bank's first off-chip tenant, and **the one block Helium does not answer** (D36). The base was chosen so that Helium's test is a single address bit rather than a range comparison: everything at or above \$FF:8000 belongs to Neon, everything below is Helium's to allocate as it likes. So the walls this question asks for now have a natural outer boundary, and the timer, the interrupt controller, the peripheral block and the power block (Q40) can be placed in \$FF:0000–\$FF:7FFF in any order without ever colliding with Neon (→ T.22).

- Q23** What a panel larger than 10.1" would cost. RGB-TTL dies above ~10", so a 14" class panel means a **transmitter bridge** – an ANX6345, taking the parallel RGB Neon already produces and running eDP link training itself in hardware, configured over I2C by the RP2354B before the CPU leaves reset.
- On iCE40 that is effectively the only route: driving LVDS needs a 7:1 output serializer and native eDP needs hard SerDes, and neither exists on this family – so the two alternatives an ECP5 would have had are both closed here. Knock-on costs: ~22 pins on Neon at 18 bpp against a budget already near its limit (→ Q12), a framebuffer and bandwidth budget to re-derive at the new resolution, a larger backlight boost than the PT4110 sizing in C.6, and touch to re-specify – laptop panels are usually non-touch, which puts the GT911 of D07 in question.
- Q24** ASID width, still unfixed. It drives both EBR consumption per TLB entry and how often an ASID must be recycled – and every recycle carries a mandatory flush (→ M.8). Four bits (16 contexts) and six (64) have both been floated; validate against the EBR budget alongside the 32–64 TLB entries already planned.
- Q25** **Settled** – how the RP2354B reaches Helium's control registers with the 65816 held in reset. Neither option as posed: not bus mastering, and not the configuration path either, but a **dedicated chip select onto the shared configuration SPI data lines**, driving a debug agent that performs the access from inside Helium. An internal write to `$FF:00xx` meets the same comparator a CPU store would, so sheet M's whole command model is exercisable with no CPU (→ D18, R.21).
- Q26** **Closed by the part, and not in the direction the question expected.** It asked whether the EC's flash should grow to 16 MB so that backup bitstreams and the BIOS could live in it and the machine could cold-boot with no card. The RP2354B's flash is **in the package at 2 MB and cannot grow**: an external part could be added beside it, but it would reintroduce the always-on component and the net that D.8 deleted, and it would need its own recovery story. **The SD stays the sole boot path** (D08, G.2), and what used to be the argument for a bigger flash – a machine that boots with an empty slot – is now an explicitly declined capability rather than an open option.
- Q27** blocking Debug agent pins and the **BE** handoff, all of it ahead of schematic capture: route **BE** from Helium to the CPU socket; make Helium's CPU-side address and data pins **inout** and check the I/O bank and pin-count impact on the TQ144 sheet; allocate **DBG_CSN** on both ends; place the **DEBUG_ENABLE** strap with its pull and jumper footprint; place the 3-pin UART header; and decide whether **DBG_IRQN** is worth a pin or the console polls (→ R.23, Q8). The turnaround count between **BE** and Helium driving must be derived from the W65C816S datasheet at 3.3 V, not guessed.
- Q28** The debug agent's **SPI-to-core clock domain crossing**: synchroniser depth and the command handshake, specified formally rather than left to whoever writes the module. It also sets the real ceiling on the 16 MHz link. This is where subtle intermittent failures live, and they would be attributed to the memory system rather than to the tool observing it.

- Q29** **Arbiter priority order** now that the debug agent is a client: where it sits against the CPU port, the fill engine and the walker. SDRAM refresh stays highest; the rest is a decision, and `ERR_ARB` exists because the agent has to give up rather than wait forever (→ R.17).
- D45 adds a fourth client and it is the easiest of them to place.** The SPI-SD DMA engine moves 512 bytes at a time and is fed by a link three orders of magnitude slower than the SDRAM, so it can sit at the bottom of the order and lose every contest without ever becoming the bottleneck – but it needs a bounded give-up like the debug agent's, and `SD_STATUS.ERR_ARB` exists for the same reason `ERR_ARB` does (→ G.10).
- Neon's arbiter is now a second instance of this question, and there the answer is a hard bound rather than a preference.** A CPU read of the `$FE` aperture stops PHI2 until Neon answers (D37), so whatever the blitter is doing when the read arrives is added directly to interrupt latency – up to 1.5 ms if a full-screen keyed copy has to finish first. **The CPU read port needs high priority and the blitter must be preemptible at a bounded burst boundary,** and the resulting worst case has to be stated in microseconds, not left implicit (→ Q57).
-
- Q30** **Trace buffer depth** against the EBR left after the TLB and the cache tags. 1024 records at ~48 bits is the proposal, roughly six blocks on the HX8K; it competes directly with the 32–64 TLB entries of L.7 and with whatever Q24 settles for the ASID width.
-
- Q31** blocking **The AON↔switched crossings audit:** a net-by-net list of every connection over the domain boundary, each checked against R1–R3 (C.13). It deserves a review pass of its own rather than being folded into general ERC, because it is the item on the power sheet most likely to produce a board-level bug that is hard to diagnose – an AON pull-up back-feeding a dead rail has no symptom on a meter and a very confusing one on a scope. Blocks schematic sign-off.
-
- Q32** **BQ25896 behaviour when the input steps 5 V → 9 V mid-operation.** The cold-start path depends on it: the EC boots on the default 5 V contract and the CH224K then renegotiates underneath a running system. The part is designed for this; the datasheet confirmation has not been done.
-
- Q33** **Measure 3V3_MAIN and 1V2 at E1 and revise the budget** (C.17). The SDRAM and VCCIO rows dominate the total and both scale with bus activity, so the TPS63020's ~45 % margin is only as good as two estimates. A measurement above the assumption forces a higher-current part, which is a BOM change, not a firmware one.
-
- Q34** **TPS63900 availability and its actual Iq.** The AON rail is chosen on quiescent current alone, and the ~17 mW standby figure and the "the pack self-discharges before it flattens" claim both rest on it. The TPS63001 is the fallback, at a cost in standby drain that would have to be re-derived. Tied to the same BOM pass as CH224K versus its successor CH224A, the K being phased out.

- Q35** Where the PCF8563 RTC lives now that the I2C bus has split in two. It wants to keep time while the machine is off, which argues the always-on bus; but that bus is otherwise reserved to the charger and gauge, and every device added to it costs standby current permanently. The alternative – RTC on the switched bus with its own backup cell – moves the cost to a battery and a footprint (→ H).
X.22 adds a third option that did not exist when this was written, and it is the better version of the second one. On I2C-EXP – Helium's own bus, switched domain, no EC in the middle – the PCF8563 keeps time from its backup cell whether or not anything is powered, because a crystal does not care who owns the bus; and the kernel reads the clock as a mirrored window instead of asking the EC for it. It costs the same cell and footprint as the switched-bus option and deletes a transport. It becomes window 0 of a tier that exists for exactly this kind of device.
- Q36** The three press durations against the BQ25896's own QON thresholds. Short (< 1 s), long (~4 s) and the charger's hardware reset have to be separable by a user with one button. If the EC's 4 s collides with the charger's window, the EC's number moves – the charger's is fixed in silicon (→ S.6).
- Q37** EC policy on a repeated NAK, and the related question of whether a battery-critical request may be declined at all. A kernel that refuses twice is either busy or broken and the EC cannot tell which; escalating too eagerly loses data, escalating too late loses the machine. The battery case has a short argument on one side: the battery does not negotiate. Kernel and EC firmware must agree, and today neither is written (→ S.15).
- Q38** Carving the always-accessible SPI region – EC_PWR_STATE, TELEM_CTRL and the staging bank outside the DEBUG_ENABLE gate – without disturbing the debug agent's existing map (R.18). Gateware work, and the addresses proposed in sheet R are proposals.
- Q39** The grace period and the EC watchdog are a matched pair and have no canonical home. 3 s inside 10 s today, with ~4.4 s of margin. A grace period longer than the watchdog turns every shutdown into a forced one *silently*, since the two are indistinguishable from outside until someone loses data. Both numbers must live in one place, in a form both the kernel and the EC firmware build from (→ S.15).
- Q40** Base address of the power and telemetry block within bank \$FF – the 16-byte telemetry window plus POWER_CTRL and POWER_STATUS, behind their own 256-byte wall (→ Q22). Blocks the gateware decode and the kernel headers equally, and it is a five-minute decision that two independent pieces of work are waiting on.
- Q41** Closed – there is no expander. The question was which domain the MCP23017 lives in, given that it must sit on the always-on side to drive an enable at all, which put it on a bus the power sheet reserves for the charger and gauge and put its outputs across the domain boundary under R1 and R2. The RP2354B's 48 GPIO absorb everything the expander was carrying, so the part, the bus contention and the crossing all disappear together (D.6, D.8). The recount that replaces it is a different exercise: every GPIO assignment validated against the function-mux table (→ Q105).

- Q42 Reboot: warm or cold?** The EC re-runs the power-up sequence after the rails drop, which is a cold restart by construction. Whether the kernel needs to distinguish the two – and whether anything should survive a reboot that does not survive a poweroff – is unanswered, and the answer decides whether `REQ_REBOOT` is one bit or the start of a state machine.
- Q43** blocking **Neon's PLL constants and the final modeline.** `DIVR=0 DIVF=32 DIVQ=3` gives 103.125 MHz core and 51.5625 MHz pixel from the 25 MHz oscillator, and 1344×640 puts the refresh at 59.95 Hz – but **those blanking values are a self-consistent set, not figures from a datasheet**, and the module datasheet is the same one Q1 is still waiting for. The pixel clock and the active geometry will not change; only the porches will, and with them the exact refresh. Blocks schematic sheet 1.5 and the timing generator equally (→ T.4, E0.4).
- Q44** blocking **Timing closure at 103.125 MHz on the HX8K**, with the SDRAM controller and the blitter placed together. Controllers in this range are documented on the family; that is not the same as a placed-and-routed design with everything else beside it. The fallback is single-rate at 51.5625 MHz, halving peak bandwidth to ~103 MB/s – every figure in sheet T halves and the architecture survives, but the margins do not. Synthesise a stripped controller early rather than discovering this at E5 (→ T.10).
- Q45 Neon's EBR relief, and it must not be chosen before synthesis.** The 32 blocks are fully allocated with zero margin. Two reliefs exist: **reduce the font to 128 glyphs** – full ASCII plus box-drawing, freeing 4 blocks, and it should be a synthesis parameter from the first day – or **multiplex the 16 text-buffer blocks into graphics use**, freeing 8 KB at the cost of address muxes and of destroying screen contents across a mode switch. **The font blocks must not be reclaimed under any scheme**: their bitstream initialisation is the entire mechanism behind D27. Decide once the blitter's real EBR demand is known (→ T.12).
- Q46 Closed by D36, and structurally rather than by a rule.** The question was how to keep Neon's register file out of a user framebuffer mapping, since a process reaching `LIST_BASE` and `CMD_CTRL.LIST_RUN` commands an engine that reads and writes all 64 MB with no MMU in front of it. The answer is that **the register file is no longer in the aperture**: it moved to `$FF:8000`, and bank `$FF` is unmappable to user space by construction (L.10). The aperture is now data all the way through and the mapping ioctl has nothing left to remember. What survives is one negative test at E8 – a process that asks for a mapping of every page it can name must still not reach a Neon register – kept because an architectural guarantee deserves a test precisely when nobody expects it to fail (→ T.21, J.5).
- Q47 The console's device name, and the one requirement that comes with it.** The Neon text console and the serial line cannot both be `/dev/tty`; this document now calls the screen `/dev/con` and leaves the UART as `/dev/tty`, which is a naming decision, not an architectural one, and it should be ratified before the driver table is numbered. The requirement is not cosmetic: **the console driver must hold its own scrollbar in system memory and repaint from it**, because a graphics mode may reclaim the text buffer's block RAM (Q45) and a mode switch and back can destroy the screen. Normal for a virtual console, expensive only if discovered late (→ T.55).

- Q48 Ratify the coordinate-based command format and budget what it costs.** Three items travel together: the sequential multiplier in the command processor (~16 clocks, ~80 LUT); the clip unit **rejecting a fully off-screen rectangle before issuing any memory traffic**, on which the off-screen parking idiom for unused list entries entirely depends; and the processor **sign-extending coordinates and clipping negatives**, so an object can hang off the left or top edge. None is optional and all three are cheap; they are open only because nobody has written them down as requirements on the gateway (→ T.42, T.43).
- Q49 Does COPY_EXPAND read its 1-bpp source at a bit-granular x offset, or must glyph atlas entries be byte-aligned?** The answer decides whether a proportional UI font can be packed tightly in the atlas or must be padded to 8-pixel boundaries, which costs VRAM and, more importantly, decides the width-table format the GUI is written against. Cheap to answer now, disruptive to change once fonts exist.
- Q50 Who owns Neon's 64 MB.** Framebuffers, glyph and icon atlases, pre-rendered level buffers, save-under pools and command lists all live there and nothing partitions them today; the addresses in the worked examples are placeholders. Two constraints belong to whatever allocator emerges: source and destination of a copy should sit in **different SDRAM banks** so the controller interleaves them instead of paying a row activation per alternation, and command lists must not be moved while **LIST_RUN** is set (→ T.14).
- Q51 Command-list versioning between gateway and kernel.** The explicit word count in every command exists so an unknown opcode can be skipped rather than desynchronising the stream – a decision made precisely because the two will drift during development. What is missing is the other half: how a kernel discovers which opcodes the gateway it is talking to actually implements. **NEON_VER** is one byte and a table has to hang off it somewhere (→ T.43).
- Q52 How far the service port reaches: SDRAM, or block RAM only.** Full reach costs an arbiter port shared with the CPU write FIFO and perhaps 100 LUT, and buys bulk asset loading straight from the card at ~2.5 MB/s against the CPU's ~1.3, at no cost to the 65816. Block RAM only is simpler and still gives the boot display and the debug channel, which are the two things no milestone can do without. The bulk-load benefit is real but nothing depends on it (→ T.46).
- Q53 Verify Calypsi generates absolute-long addressing for (volatile uint8_t *)0xFE0000** without reloading a bank register on every access. Every line of C that touches Neon depends on it, and so does the honesty of the ~140 µs per-frame figure. It belongs with the calling-convention and direct-page audit already outstanding (→ Q19, T.62).

Q54 The VGA bring-up clock, and whether VGA survives at all. 640×480 wants 25.175 MHz and the video domain gives 25.78 – 2.4 % fast, which most monitors tolerate and none is specified to. Neon's second PLL is free and could synthesise it exactly, but audio wants that PLL too – less badly than it looks, since grounding the DAC's `SCK` puts the master clock inside the PCM5102A (→ T.9). The prior question is whether VGA is still worth its 2 pins and its gateware **now that Mode 0 puts a working console on the panel itself at E1.7**, three stages before VGA was ever going to be useful (→ T.9).

Audio's half of this question is now closed, and it closed harder than T.9 expected. The iCE40 PLL is integer-only, and neither 44.1 nor 48 kHz is reachable from 25 MHz at any setting – 441 and 3,125 do not factor into $(DIVR+1) \leq 16$ times a power of two. **The second PLL cannot produce an exact audio clock**, so audio has no claim on it at all rather than a softer one, and this question reduces to whether VGA is worth two pins and its gateware (W.11, Q77).

Q55 Per-window VRAM buffers with compositing – deferred, not rejected. Each window owning a buffer and the framebuffer being produced by compositing in z-order removes the damage problem entirely: occluded content still exists, so moving a window is a recomposite rather than a repaint from application state. With 64 MB and ~143 MB/s it is affordable – ~15 KB per window on top of the 75 KB framebuffer – which is not true of most machines this size. It is materially more gateware and more software, and **it must not be attempted before the flat framebuffer and its damage model are working** (→ T.57, T.58).

Q56 Closed. The three inter-FPGA nets are named, and the naming carries the philosophy: `NEON_BUS_SEL` (Helium → Neon) selects Neon's bus, `NEON_BUS_BSY` (Neon → Helium) holds the CPU while that bus is busy, and `NEON_IRQ` aggregates Neon's three interrupt sources into Helium's PIC. `VRAM_SEL` and `BWAIT` are retired – the first described less than half of what it does now that Neon has two windows, the second described a mechanism that was going to stretch the wrong signal. Polarities and pulled states are settled with them, including the one deliberate departure from B.8: **`NEON_BUS_BSY` is busy by default**, because for that net the pulled state must be the safe one (→ B.6, D37).

- Q57** The read-stall timeout, and the bound the arbiter has to guarantee – one number and one policy, and neither has an owner. The watchdog window for a VRAM read must be long enough to survive a refresh plus a legitimate arbiter queue and short enough that a missing Neon is reported rather than merely felt; it now serves three clients rather than two – a fill, a Neon that never answers, and a card that asserts `/EXP_BSY` and dies (X.12) – which is an argument for settling one number rather than three. It belongs in the same place as the fill watchdog's, not beside it (F.11). The policy is Q29's: the CPU read port needs priority high enough, and the blitter preemption granularity fine enough, that the worst case is microseconds and not the 1.5 ms of a full-screen keyed copy. **Both are cheap to decide and expensive to discover** – the failure mode is scheduler jitter that appears only under drawing load (→ D37, T.19).
- Q58** blocking Which blitter: operations or channels? Sheet T specifies an operation-based engine taking `(x, y)` commands from a FIFO or a display list; sheet U specifies the Amiga four-channel engine with 256 bitwise logic functions and raw pointer-plus-modulo addressing. They are different machines, not different register maps: the minterm LUT cannot express a colour-key compare and the operation set cannot express XOR. **The decisive figure is setup cost** – ~30 stores, ~19 µs of CPU per blit, which is nothing for a compositor doing a few large blits and fatal for the 24-sprite frame of T.59, where it is ~450 µs against the 140 µs budgeted for the whole frame. **The recommendation is U.4's third option**: this datapath with sheet T's command layer generating the pointers, which is roughly LUT-neutral and keeps both. Decide before RTL; it reaches the kernel, the toolchain and both milestones (→ U.1).
- A third argument arrives from software and it is about capability, not cost. The toolkit needs XOR – selection highlight, pressed states, the text caret, the focus rectangle, any rubber-band outline – and sheet T's operation set has no raster op at all. Without one, every one of those becomes a fill plus a keyed copy from an inverted duplicate: twice the traffic, twice the commands, and a visual language rewritten around it (V.14).
- Q59** Does the GUI run at 1 bpp or 8 bpp, and does Mode 1 survive? T.30 calls the 1-bpp barrel shifter "not optional" for a GUI; U.10 observes that at 8 bpp on a 16-bit bus the shifter degenerates to a byte swap, **deleting ~400 LUT and buying 256 colours instead of two**. The costs are a 614,400 B framebuffer against 76,800 and 36.8 MB/s of scanout against 4.6 – both comfortable against 64 MB and ~150 MB/s. A **1024 × 600 × 8 bpp mode does not exist in sheet T** and every figure in sheet U assumes it. Whether it replaces Mode 1 or joins it decides what the window server is written against (→ U.12).
- It also decides every icon, and that is the expensive half. Sheet V is written for 1 bpp throughout – gray as dither, selection as invert, depth as a lpx frame and a shadow line – and redrawing that visual language for 256 colours is weeks of art rather than an afternoon of code. **Answer this before drawing a single icon**, which is the toolkit's own open item folded in here rather than asked twice (V.13). And 2 bpp, floated as a middle path, is the worst of the three on this device: the shifter argument is not linear, so 2 bpp pays most of 1 bpp's ~400 LUT and buys four colours instead of 256.

Q60 Flat framebuffer with repaint, or backing stores with recomposition? T.55 recommends the flat model and defers backing stores (Q55); U.31 recommends backing stores with damage-limited recomposition and shows the budget carries ~90 full-screen composites a second, so full recomposition is always available as a correct fallback. **The disagreement is downstream of Q59** – T argued against a 1-bpp screen where repaint is cheap, U against 8-bpp stores with 64 MB behind them. The consequences are opposite and neither is retrofittable: under one, occluded pixels are gone and the window manager repaints from application state; under the other they are preserved and a move is a recomposite. **Answer Q59 first.**

Sheet V is the software half of this question and it argues the same way U did, from a different direction: with backing stores the OS gets *smaller* – `layers.library` and its cliprect arithmetic disappear, clients never receive refresh events, a hung client still displays, and the event set drops to ten. It also sharpens the cost. At 1 bpp a full recomposite is ~1.0 ms and the screen can be rebuilt fifteen times a frame; **at 8 bpp it is ~8 ms and a typical desktop at 1.8× overdraw consumes essentially the whole frame**, so damage-limited recomposition stops being an optimisation held in reserve and becomes the routine path from the first day (→ V.11).

Q61 blocking **Block RAM no longer closes.** Sheet T is at 32 of 32 with zero margin (T.12); sheet U's tally adds **2 blocks for audio that T never counted and 2 for the 8-deep snapshot queue against T's 1** for the command FIFO, so the combined design wants 34–35 before the blitter's burst buffers. The reliefs are known – a 128-glyph font frees 4 (Q45), and a 2-bpp cursor rather than 4-bpp gives back half a block (U.29) – but **what was insurance is now spent**, and it has to be decided before RTL rather than after synthesis (→ U.38).

A third claimant now exists and it is not vapour: `DRAW_GLYPHS` (Q64) wants a source buffer and is the single largest CPU saving available to the GUI. **It is blocked on this question, not on its own merit.**

Q62 blocking **Where the window server lives, and it is decided by D36 rather than by preference.** The first pass at sheet V made `wserver` a user task owning the `$FE` aperture and called that the protection model. Half of it works – J.5's `ioctl` maps the aperture, and pixels, atlases, palette and command lists are all reachable that way. The other half does not: `CMD_PORT`, `LIST_BASE`, `CMD_CTRL.LIST_RUN`, `DISPLAY_BASE`, `VRAM_PAGE` and the cursor position are in bank `$FF`, **which is unmapable to user space by construction, so command emission is a privileged act on this machine.** The recommendation is the split the glossary already draws: **the compositor is kernel code** – vblank handler, z-ordered surface list, top-level command list, swap – **and the window server is a user task** doing hit-testing, focus, drag, menus, decoration and client validation, reaching the hardware through a `/dev/neon` node and an `ioctl`, which adds no syscalls and keeps D23 intact. **The prize for getting it this way round is real:** a compromised window server can corrupt what is on screen and nothing else, because it cannot aim a command processor that reads and writes all of VRAM with no MMU in front of it (→ V.4, V.16).

One wrinkle travels with it, and it answers the aperture-addressing question from an angle that first pass did not expect. The aperture shows 32 KB at a time and `VRAM_PAGE` is privileged, so **a user-space server pays a syscall every time it writes cross a 32 KB boundary.** Reprogramming costs one byte store and no stall – the cost is the call, not the hardware – so the mitigations are to keep the list arena, string cache and decoration assets within few pages, and to let the `ioctl` take page *and* offset. If crossings dominate in measurement, list building moves into the kernel with the compositor: one module, not a redesign (→ V.18).

Q63 Ports, signals and messages in the syscall table – the first addition to J.3's dispatch table since it was written, and it deserves the scrutiny D23 applied to power. Five calls: `port_create`, `port_send`, `port_recv`, `task_wait`, `task_signal`, with **fixed 32-byte messages copied by the kernel** and bulk payloads through the `mshare` that already exists. They pass the test `sys_poweroff()` failed – general primitives, not one subsystem's verb in a syscall's clothes. **What must not happen is two blocking mechanisms:** N.3 already has per-device blocked queues, and a process waiting on a device *and* a message would have no single place to wait. Fold the device queues onto signal bits so `task_wait` is the only blocking primitive, or do not add it (→ V.28, V.29).

The cost to design against is **~12 µs per message** – ~100 cycles of kernel copy at 8 MHz. Fine for window notifications and input events, and the reason **motion coalescing is not optional**: never more than one `EV_MOUSEMOVE` per frame per client. It also puts a requirement on Q4's outstanding HID event format – an event must fit 32 bytes and must carry **absolute position and a button bitmap**, because a format expressed as deltas and edges cannot be coalesced at all.

Q64 DRAW_GLYPHS, and it is complementary to the string cache rather than a replacement for it. One command taking (`font_base`, `glyph_w`, `glyph_h`, `string_ptr_in_sdram`, `count`, `dx`, `dy`). Costed against an 80-character line: per glyph is ~1.5 ms of CPU emission, a cached composed string is ~18 µs but ~1.5 ms the first time, and **DRAW_GLYPHS** is ~80 µs **with no first time**. So **the cache wins for stable labels and DRAW_GLYPHS wins for text that changes** – the editor, the terminal, the file listing, which is what a desktop is for. A 128 × 32 console repaint is ~3.2 ms against ~76 ms per glyph. It also deletes 256 KB of strip and all the LRU machinery. Cost it in LUT and EBR against the 128-glyph font relief before committing to either (→ Q61, V.26).

Q65 Command-list fetch is a fifth arbiter stream and D38 never counted it. During a composite pass the command processor fetches from SDRAM while the blitter reads a backing store and writes the back buffer. For an even-numbered window those are three different banks and the invariant holds; **for an odd-numbered window the fetch and the pixel read are both in bank 3** and every alternation costs a PRECHARGE and ACTIVATE. The traffic is small – a 20-window composite list is a few hundred bytes – but the stall is per crossing, not per byte. Three answers are available: give the lists a bank, interleave window parity so a backing store and the list describing it are never in the same bank, or measure and conclude it is noise. **Cheap now, awkward at G2** (→ V.32).

Q66 The blitter reaches 16 MB, not 64, and it should be said in sheet T rather than inferred. `DISPLAY_BASE`, `DRAW_BASE`, `LIST_BASE` and sheet U's four channel pointers are all **24-bit** (T.48, U.19), and U.16's partition is written as four 4 MB ranges spanning `$000000`–`$FFFFFF`. Nothing in the GUI needs more – at 8 bpp a hundred backing stores is 12 MB and the framebuffers 1.2 – but it means **the partition is a 16 MB window into the device, not a division of it**. Decide whether the address registers widen to 26 bits or the upper 48 MB is simply unaddressable by the drawing engine, and write it down either way (→ V.33).

- Q67** blocking **Are the i9 extension board's PMOD headers direct, unbuffered, bidirectional FPGA I/O?** Several Colorlight products route GPIO through unidirectional 74HC245 buffers; the module itself carries raw balls to the SODIMM, so this concerns the extension board alone. If they are buffered the 65816 data bus cannot be wired to them and P.19's Stage A is dead as written. **Moot if Stage A is skipped** in favour of going straight to the carrier PCB, which is the cheaper way to close this question – continuity meter on arrival, or do not buy the extension board at all.
- Q68** blocking **Ball assignment for the prototype:** map P.05's 82 signals onto specific i9 balls, respecting ECP5 bank grouping and keeping the CPU bus inside as few banks as practical. It is E0.5 again on different silicon, with one extra constraint the target board does not have – the SODIMM's own zoning, which decides which signals may share a block at all.
- Q69** **Clock plan for the merged die:** PLL outputs for the Helium domain, the Neon domain and the pixel clock, from the module's 25 MHz on P3, with P.14 requiring the two core domains to be genuinely separate. Confirm the ECP5's PLL count covers it before the first module is written. The target board answers the same question with three devices and one oscillator (B.7); here it is one device and one oscillator, which is harder rather than easier.
- Q70** **Measured current draw of the i9 module,** PHYs running and held in reset. P.20 budgets ~500–700 mA held down and ~700 mA–1.1 A running, and both are unmeasured estimates. It decides whether USB-C at 3 A is comfortable or merely adequate, and it is measured at P0 with a bench supply before the carrier schematic is finalised.
- Q71** **R-2R ladder values and termination,** verified against a 75 Ω monitor input. 6-bit ladders need 1 % resistors and the loaded output has to be checked rather than assumed; a ladder that is right in theory and 20 % low into 75 Ω produces a picture that looks fine until someone compares two monitors.
- Q72** **The prototype's target PHI2 on the carrier.** Stage A is fixed at 1–2 MHz by the wire-wrap (P.19); Stage B should run the target's 8 MHz (B.1), but that is an intention rather than a measurement – the shared nets have three loads on a carrier with a SODIMM connector in the path, which the target board does not have. **Every stall and fill number taken below 8 MHz is an existence proof and nothing more,** and the sheet should say which clock each measurement was taken at.
- Q73** **Which parts of G0–G8 are unreachable without a power path.** P.02 drops battery, charger, gauge and sequencing, so sheet S's \$FF power block has no hardware on the carrier and everything resting on it – the battery indicator, the shutdown dialogue, the stale-telemetry behaviour of E8's test list – cannot be exercised. Decide whether those are stubbed against a simulated power block on the prototype or simply deferred, and write the answer into sheet V's staging rather than discovering it at G6.
- Q74** **A cache flush that never completes has no recovery path, and no existing watchdog covers it.** F.11's watchdog bounds a cache fill and R.17's bounds a debug transaction, but both work by resuming a stalled PHI2 – and the asynchronous flushes of M.6 deliberately never stall it, so neither applies. A hung SDRAM controller leaves the kernel spinning on BUSY indefinitely, with no clock stopped and no fault raised. Wanted: a counter in Helium bounding

`CACHE_FLUSH_FRAME` and `CACHE_FLUSH_ALL`, escalating to NMI or `ABORTB` – which also means spending one of the bits `FAULT_CAUSE` currently reserves at 15:6.

Q75 Where the `COP` signature byte lands on the stack – `PC-1` or `PC-2`, verified against the WDC manual rather than inferred. J.3 takes the service number from the accumulator and no longer reads the byte, so nothing on the syscall path depends on the answer; a tracer, a disassembler or a static binary verifier does, and a wrong constant there produces a silent misdispatch rather than a visible failure.

Q76 Shutdown has no mute step, and the power-down click was never considered. The RC on the DAC's `XSMT` exists because a rail edge through an un-muted `PCM5102A` is a click in the headphones (sheet H); it protects power-up only. S.13 sequences an orderly shutdown all the way to `POWER_CTRL` with nothing muting the output, and the power-down click is the more audible of the two because the listener still has the headphones on. Two parts, both cheap and both needing to be decided together: `XSMT` driven from `AUD_CTRL.MUTE` rather than strapped, costing one Neon pin against the 99 of W.38, and a mute step inserted between stopping the window server and flushing the filesystem – after the session's last sound, before the switched domain moves (→ W.40).

Q77 The sample rate is 44,146 Hz and the question is whether to write that down or spend a part on 44,100. No PLL setting reaches either standard rate from the board's 25 MHz oscillator: the iCE40 PLL is integer-only and 44.1 kHz needs a denominator factor of $441 = 3^2 \times 7^2$, 48 kHz one of $3,125 = 5^5$, neither expressible as $(DIVR+1) \leq 16$ times a power of two (W.11). So the second PLL cannot help audio at all, and audio's half of Q54 is closed – not weakened, as T.9 has it. The integer divisor 73 at 32fs gives +0.104 %, 1.8 cents of pitch and 3.75 s/hour of drift, against two exact alternatives that are both refused here: a second oscillator breaks B.7's one-oscillator star, and a fractional NCO hands the DAC's internal PLL a `BCK` jittered by a full 9.7 ns clock period. The recommendation is to accept it and publish it in `AUD_RATE`, so no piece of software ever assumes a rate the hardware is not producing (→ W.9, W.12).

Q78 What happens when a fifth process wants sound. Four hardware channels are a finite resource and there is no software mixer available to virtualise them – W.1 is arithmetic, not preference: the 65816 has 181 cycles per stereo frame at 8 MHz and mixing four streams does not fit in them. The recommendation is the interface that admits it: channel 3 reserved for the session's system sounds, channels 0–2 allocated to open in order, `ENODEV` to the fourth caller – rather than silently folding a late arrival into somebody else's channel. What is still open is whether the window server may preempt an application's channel for an alert, which is a session-policy question of the same shape as S.14's cancellation dialogue and belongs with sheet V rather than with the driver (→ W.44).

- Q79** When the SD DMA engine is actually built, and what stands in for it until then. D45 specifies both paths but they are not the same size of job: the FIFO is a page of gateware, the DMA engine is an arbiter client with a coherence rule and a `CACHE_INVAL_FRAME` beside it. The staging that falls out of this is that E6 and P5.j can be reached on the FIFO alone, with the block driver already written against the DMA register names so nothing above it changes when the engine lands. What has to be decided is whether that is deliberate staging or an accident waiting to become permanent – **a FIFO-only machine cannot page**, and the swap reserve of G.4 is the feature the whole card topology was chosen for.
- Q80** The SPI-SD clock, and whether the card can be driven fast enough to matter. Every throughput figure in sheet G assumes 25 MHz – the ceiling for SPI mode on a modern card, and one this board has no reason to miss – but the number is inherited rather than measured, and it is derived from Helium's core clock by an integer divisor that also has to produce ≤ 400 kHz for initialisation (G.11). Both divisors, the achievable rate on real cards over the '3257 mux, and the resulting per-block time belong in one place before the block driver's timeouts are written against a guess.
- Q81** `blocking` How many expansion slots the board carries, and it is the only variable in sheet X that fights Q8. One slot is three pins of Helium, two are four, and none is zero – every other cost in that sheet is a connector, a '573 and a '245, none of which touches the FPGA. The question is not really "how many": it is whether a slot survives a pin budget that does not close for the interfaces the machine cannot boot without, and the honest ordering is that SPI-SD, the UART and the debug port are all senior to it. What must not happen is the decision being taken by default at layout, because the connector and its buffers are cheap to route now and impossible to add later (→ X.3, Q27).
- Q82** Base addresses for the slot windows and the slow-bus block within bank \$FF – \$FF:7E00 and \$FF:7F00 for the slots is a proposal chosen for being at the top of Helium's half, and the slow bus has no proposal at all. Both are part of Q22 rather than separate from it, and they are the two allocations in that bank that can still be moved for nothing (→ X.7).
- Q83** The aperture's frame range and, more importantly, its size. X.8 proposes frames \$F000–\$F1FF and \$F200–\$F3FF out of the half F.10 leaves unpopulated; the range is arbitrary and the **1 MB size is not** – it is exactly what A0–A19 reaches, and widening it means new pins on a budget that has none. Two things to settle with it: whether the range should sit at the very top of the frame space so that a future second SDRAM device could still grow contiguously beneath it, and whether the kernel's frame allocator needs to know the range exists at all or simply never receives those numbers (→ Q81).
- Q84** Ratifying the card contract, which is the part of sheet X that outlives the project's own hardware. `SLOT_ID`, `SLOT_CLASS` and `SLOT_STATUS` at fixed offsets are what make enumeration and interrupt demultiplexing cost nothing (X.13, X.14) – but a class numbering that is invented one card at a time becomes unfixable, and the obligations of X.20 are a promise made to people who will never read this document. Wanted: the class list, and the contract restated on a single page that ships with the connector rather than inside a synthesis document.

- Q85** The slow bus is specified as a mechanism and not yet as a register file. D49 settles the shape – mirrors refreshed by a sequencer, S.16's staging → live → shadow, a raw-transaction escape for initialisation – and leaves open how many windows there are, how large each may be, the descriptor encoding, what the sequencer does when several windows come due in the same pass, and whether the raw escape is enough to bring up every device this tier is likely to host. The map in sheet X is a proposal at that resolution and is marked as one. **Nothing depends on it yet**, which is the argument for writing it against a real second tenant rather than against an imagined one (→ X.28).
- Q86** Where the expansion tiers land in the E-series, and the answer should be "late". Neither is required by a milestone: E6 boots without them and E8 is a laptop without them. What is required *now* is schematic space and reserved pins before layout (E1.3), which is the classic shape of a decision that gets taken by omission. The staging that suggests itself is the slow bus first – it is a sequencer and a table, and the RTC is a real tenant – with the parallel slot's gateware after the machine has something to plug into it, and both entered on sheet P rather than left in this sheet (→ X.30).
- Q87** Where the filesystem work goes in the sequential plan, and it is one question rather than three. Sheet Y1's V0–V5, sheet Y3's E2.0–E2.7 and sheet P's F0–F9 are three gate series that were each written without the others in view, and **V2 gates E2.1** – the NVFS driver has to move behind the vnode table before a second driver is written against it – while F4 is the host-side rehearsal of both. None of them is currently slotted against P5 or E6, and the calendar cost of inserting them before the Apple II milestone needs an explicit decision rather than a default.
- Q88** blocking **Nothing on this board holds a wall clock, and the filesystem needs one.** Timestamps are `u32` seconds since 1970; with no source, every file the machine writes carries zero – which the format defines as "unknown" and `nvfsck` must not report as an error (Y2.4), so nothing breaks, but nothing on the card can be dated either. The options are a real-time-clock part, which is a board change; the EC holding time across a battery-backed always-on domain, which is nearly free given C.9 already keeps that domain alive; or the user setting the clock at each boot, which is honest and irritating. **This is a hardware gap rather than a filesystem question and it should be settled before the board is frozen**, because the middle option costs nothing if it is designed in and a part if it is not (I.4, Y1.17).
- Q89** NVFS journal sizing gives 1.6 % of the volume to the journal at every size, and on a large card that is too much. The default ring is `total/64` clamped to 256–8192 blocks, which is 1 MiB on a 64 MiB volume and 128 MiB on an 8 GiB one. A cap of about 4096 blocks is proposed. It affects `mkfs` only, and existing volumes are unaffected (Y2.5).
- Q90** blocking **Does NVFS expose a stable 32-bit file identifier that survives modification?** Y1.7's vnode cache is keyed on it, and the inode number plus `i_generation` satisfies it – but this has to be confirmed against the implementation rather than the specification **before V0**, because if any identity in NVFS turns out to be derived from extent location or anything else that changes on write, either the cache key or the format has to move.

- Q91** The shared block cache carries a hook only one driver uses, and that is a smell worth stating. D55 chose it because the alternative's cost – two caches, two pools, no global LRU – is permanent while this one's is a single field. The counter-argument is that a mechanism with one user is a mechanism nobody maintains. Revisit at V5, which may trail V4 with ext2 on a private cache in the meantime.
- Q92** Is the host track's internal table path-keyed or inode-keyed? libfuse offers both shapes and **only the inode-keyed one transfers**, because path resolution, symlink following and mount crossing are all VFS responsibilities here and a driver never sees a path (Y1.21, P.28). Cheap to fix at one implementation and expensive at three, so it is checked before the first filesystem gets far.
- Q93** NVFS does not store `..`, and a directory's link count is therefore always 2 rather than 2 + subdirectories. That removes the link-count bookkeeping which is a persistent source of `fsck` errors, at the cost of a directory found by inode number having no path back to its parent, and of a host binding having to synthesise the POSIX count if any tool depends on it. **There are 8 reserved bytes in the inode** if a stored parent is ever wanted instead. Confirm the trade or spend them (Y2.13).
- Q94** The vnode cache size and the per-process descriptor table size are both guesses – 24 and 16 – and should be revisited once the GUI's file access patterns are known rather than before. The cache must exceed the maximum number of simultaneously open files by a margin, since directory vnodes along active paths are held too (Y1.15's budget table).
- Q95** Is there a `devfs`, or do character devices stay outside the filesystem namespace? J.4 routes an `open` either to the filesystem or to the device table, which works and is not a namespace. Putting devices inside it is what would require `open` and `close` slots in the per-vnode table that Y1.10 deliberately omits – two slots, added cheaply at that point, but only if the question is asked before V3 rather than after.
- Q96** blocking Can an SD card's write cache be reliably flushed, and if not, what do the durability guarantees actually mean? Y2.16's journal assumes a write is durable once the card leaves the busy state and `CMD13` confirms it; Y3.9's ordering rules assume the same of every flush point. SD has no cache-flush command comparable to eMMC's. **If the assumption is wrong, both filesystems' crash-consistency arguments are weaker than they are written**, and the way to find out is a power-loss rig rather than a datasheet. Verify during SD bring-up and record the result (G.11).
- Q97** `data=ordered` costs a barrier per transaction, and the cost has not been measured. Y2.18 forbids the faster alternative because letting data lag the commit exposes the previous occupant of a reallocated block through a newly written file, which on a single-user machine with no other protection is both a correctness and a privacy failure. **If the barrier proves intolerable the alternative is journaling small writes with the metadata, not relaxing the rule.** Measure before the format is frozen.
- Q98** Is the ext2 driver required to handle arbitrary host-created volumes, or only one documented `mke2fs` invocation? The answer decides how much of Y3.4's feature table has to be implemented rather than merely refused, and whether Y3.6's htree handling is needed at all – formatting with the index off removes that problem at source.

- Q99 Resident or loadable: ~19 KB of ext2 driver competes with the kernel for a bank.** Read-only is ~9 KB, and the VFS is ~5.5 KB on top of whatever NVFS costs (Y1.15, Y3.13). **Address space is the constraint rather than code size**, and there is no loadable-module story in this document. Check it against J.1's bank \$01 before E2.1, not after.
- Q100 ext2's pinned group descriptor table does not scale.** 8 KB for a 2 GiB volume, 128 KB for a 32 GiB card (Y3.14), which is not payable. Either an LRU sub-cache over the descriptors or a stated maximum supported volume size, decided before the write path exists.
- Q101 Read-only ext2 or read-write, which is the largest single scope decision in the filesystem stack.** D54 recommends read-only: it covers the actual use case at roughly a quarter of the cost and carries none of the risk of destroying data rather than merely failing to read it. The write path should be revisited only when a concrete need to write ext2 on this machine appears (Y3.16).
- Q102 blocking Verify a Low Speed device behind the hub, on real hardware, with a real keyboard.** The host must emit a PRE packet before each Low Speed transaction, and that is historically the most fragile path in this family's host stack. It is U6, and it is the gate on which the machine either has a keyboard or does not (D.10).
- Q103 What happens when a user plugs a hub into a downstream port.** The on-board hub consumes the single guaranteed level, and in a portable computer a second one will be attached. The tier limit has to be raised and multi-level handling verified rather than assumed; **the fallback is defined now – detect the downstream hub during enumeration and refuse it with a console message rather than failing silently** (D.10, U7).
- Q104 Confirm the EC's stepping from the package marking on the parts as received,** not from the distributor's part number. It is a two-minute check on delivery and an expensive discovery afterwards (D.8).
- Q105 Validate every EC GPIO assignment against the part's function-mux table.** D.6 closes the pin count with room to spare; that is a different exercise from confirming each function can actually be reached on the pin it was assigned to, and it belongs before schematic capture (Q41 reduced to this).
- Q106 How JP-BYPASS is implemented, and whether it survives to production.** A 3-pin shunt or two 0 Ω pads with one populated; at 12 Mbps either is electrically free. It is worth keeping on the prototype for certain – it is the only thing that separates "the host stack does not work" from "the hub does not work" (D.11).
- Q107 blocking Write and archive the RAM flash-writer stub before it is needed.** Tier 2 recovery writes only SRAM, so without the stub it cannot reach flash at all, and **a recovery tool that has to be rebuilt from source during an emergency is not a recovery tool**. About 200 lines, verified by CRC-32 readback, archived beside every EC firmware release (D.14).
- Q108 How much of the QSPI bus is exposed on J-QSPI, and how it is guarded.** Those nets reach the in-package flash die, so the header trades recovery access against noise coupling into the running system: short stubs, routed away from the SDRAM and both switching nodes, ground-guarded if there is room, series damping on the header side (D.15).

- Q109 Which hub IC.** The documented four-port part with per-port power control and overcurrent sense is recommended on no-black-boxes grounds; a cheaper SSOP part with gang-only control is defensible if hand-assembly ease dominates on the prototype. **Confirm the Hi-Speed-to-Full-Speed upstream fallback on the actual part** before committing (D.16).
- Q110 Downstream power policy: an aggregate hardware limit, or bMaxPower accounting in EC firmware.** Worst case is 1.55 A and 7.75 W from a 1S battery, which is not deliverable. **The firmware policy is preferable** – it is the honest behaviour, it is visible to the user, and it is the kind of policy an EC should own – with the hardware limit as the fallback if the boost converter cannot carry the transients (D.17).
- Q111 Verify the TPS61023's output capability at the chosen aggregate USB limit with the battery at 3.4 V,** before the schematic is frozen. It is the same converter D.5 already enables on demand, now asked for considerably more (C.10).
- Q112 blocking Confirm on the received module that CKE , CS# and the DQM lines of its SDRAM are tied off rather than driven.** The finding is community reverse-engineering rather than a vendor document, it is a continuity check that takes minutes, and it decides whether the carrier must carry its own SDRAM – which P.11a argues it must, and which invalidates most of P.11's allocation if it holds.
- Q113 XSMT should be driven rather than strapped, and the open half is which party drives it.** A Neon pin is natural while the machine runs and wrong at both edges that matter; the EC is awake before Neon is configured and while the rails come down, which is T.52's argument for the backlight applied unchanged. **No functional strap belongs on a passive-only net** (W.7, W.40, Q76).
- Q114 Calypsi's default far pointer is 32 bits, not the 24 this document assumed,** and the correction reaches the ABI rather than the prose: struct layout, the calling convention and every kernel and driver bank-crossing convention depend on which qualifier each object takes. **__far24** should be preferred wherever a far pointer is unavoidable – the fourth byte is dead weight in a 24-bit address space. Settle it per object in E0.8's audit, against compiled output rather than the manual (O.2).
- Q115 ABORTB cannot trap WAI or STP executed by unprivileged code,** so a user process can halt the machine. A watchdog is the cheap answer and the EC already holds one (D.7); opcode detection at fetch inside Helium is the complete one and is what another FPGA-based 65816 machine already does for a different purpose. Decide before untrusted code is run, which is later than it sounds (E.22).
- Q116 blocking The ABORT go/no-go bench test, and it comes before the OS design is committed rather than after.** Assert abort on a plain load and confirm the correct address is stacked, **RTI** re-executes it, and every register is intact; repeat on a store and confirm the write was gated. **If plain accesses are not reliably restartable on the actual silicon, the fallback is a stall-only design with statically mapped banks,** and that is a different operating system (E.24, P3.k).

Q117 What to do if the 103.125 MHz domain will not close on the prototype's speed grade. Ranked: pipeline harder, drop the pixel clock, then – last – cross-check the timing in the vendor's proprietary tool. The third is worth naming precisely because A.6 makes the open toolchain a principle: checking a design the open flow builds is a smaller concession than building with it, and it is still one (P.25).

The document's terminology in one place. Every acronym is also expanded on first use where it appears, so this sheet is deliberately redundant – it exists to be jumped into, not read through. It is grouped by domain, and within each group runs roughly in the order the concepts appear. The figures have a sheet of their own (Z2).

Virtual memory and the MMU → sheet K · sheet L

TERM	MEANING
Virtual memory	A layer of indirection between the addresses the CPU generates and those that reach the RAM. Every process here sees a clean, contiguous 16 MB of it, and the indirection is Helium's: the 65816 has no MMU of its own.
Virtual address	The address the CPU emits – what the program "believes". 24 bits here.
Physical address	The address that reaches the SRAM/SDRAM chips. 27 bits here. 27 bits here – a 128 MB ceiling, with the SDRAM frames at the bottom of that space and the pinned SRAM frames at the top (L.6).
Page	A fixed-size block of the virtual space. 2 KB. 8,192 of them per process, and the size is deliberately decoupled from the 256 B granularity a fill actually moves.
Frame	A block of the same size in physical memory. To translate is to pair a page with a frame. 32,768 of them across the SDRAM, tracked by a bitmap in the kernel.
VPN	Virtual Page Number – high bits of the virtual address; identifies the page. 13 bits.
Offset	Low bits; the byte within the page. Never translated. 11 bits.
MMU	Memory Management Unit – the hardware that translates. Not in the CPU here: Helium implements it. The TLB, the hardware walker, the permission checks and the ABORTB that reports a failure are all Helium's.
Page table	An array indexed by VPN holding the PTEs. One per process, 32 KB. Flat, not multi-level: 8,192 entries × 32 bits, held in the pinned SRAM region so the walker reads it directly.
PTE	Page Table Entry – one entry of that table. 32 bits: frame number plus flags. 16 bits of frame number plus the flags below – which is what puts the physical ceiling at 128 MB.
PTE flags	P present in RAM · W writable · X executable · U reachable from user mode · D dirty (modified since load) · A accessed recently · NC non-cacheable · PIN non-evictable · SW free for the kernel's own use.
TLB	Translation Lookaside Buffer – cache of VPN→frame translations inside the MMU. Avoids re-reading the table on every access.

TERM	MEANING
TLB hit / miss	Success or failure in that cache. A miss costs one table walk; it is not an error.
TLB reach	Entries × page size – the working set the TLB can cover before it starts thrashing. 64 KB here with 32 entries; the main thing a larger page buys.
Page walk / walker	The traversal of the page table that resolves a miss. Hardware here (≈100 ns, four 8-bit SRAM accesses); a software exception in other architectures.
ASID	Address Space IDentifier – tag marking which process each TLB entry belongs to. Without it the whole TLB would be flushed on every context switch.
Page fault	Exception raised on access to a page with <code>P=0</code> . Recoverable. Delivered as an ABORT; the handler grows a heap or stack, resolves a copy-on-write, or kills the process (L.14).
Demand paging	Loading pages only when touched rather than all up front. Deferred to post-v1.
Eager loading	Mapping and filling every segment at <code>exec</code> time. What v1 does, in place of demand paging.
Swapping	Evicting rarely used pages to storage in order to free frames. Order matters: flush the frame's dirty sub-blocks to SDRAM before writing it to the card (L.16).
Pinning	Marking a page as non-evictable. Required for the kernel, the interrupt vectors and the bank <code>\$00</code> stack. It is a physical address range rather than a cache attribute, so an access there skips tag lookup entirely and completes at fixed latency.
Memory protection	Preventing a process from reading or writing outside its own space. Achieved through the PTE flags, not through translation itself.
Cache line	The tagged unit of Helium's cache – 2 KB, one tag per line, but filled and written back in 256 B sub-blocks.
Sub-block	The 256 B unit a fill actually moves, with its own valid and dirty bits. Keeps a miss at ~6 μs instead of ~41 μs and bounds interrupt latency.
Set associative	Cache organization where a given address may sit in any of N positions ("ways") within one set. Four ways here; direct mapping would put 32,768 physical lines in competition for 384 slots.
Tag	The bits stored alongside a cache line saying which physical line it currently holds. Kept in EBR, ~1.5 KB, beside the TLB.
Dirty victim	The line a fill has to evict when it has been written to – it must go back to SDRAM first, which is why a miss can cost double.
Write-back	Cache policy where writes stay in the cache and reach memory only on eviction. Implied by the dirty bits. It is also why a frame is flushed before eviction and invalidated before a DMA engine fills it.
Backing store	The memory a cache is a cache of. Here the SDRAM, which the PTEs always name.

TERM	MEANING
Flush vs invalidate	The two ways to empty a cache line, and they are not interchangeable. Flush writes dirty contents back and then marks the line clean – used before the CPU's data has to be visible to someone else. Invalidate drops the line without writing anything, dirty or not – used when the contents are about to be overwritten by someone else. Getting them backwards loses data in one direction and merely wastes a burst in the other (M.12, G.9).
Copy-on-write	Sharing a page read-only between processes and duplicating it only when one writes. An ABORT case (L.4). The downgrade to read-only is one <code>TLB_INVALID_PAGE</code> , not a full flush.
Static core	A CPU whose state survives an arbitrarily slow or stopped clock. The W65C816S is one, which is what makes the PHI2 stall legal.
Clock gating	Stopping and restarting PHI2 in gateway. Must be glitch-free, and the first pulse on resume must meet the minimum high width. The same gate serves the cache-miss stall, Neon's busy stall and the machine's low-power state.
Trampoline	The short privileged stubs in bank <code>\$00</code> that receive every vector – the CPU forces PBR=0 on entry – and <code>JSL</code> straight into the real kernel in <code>\$01</code> (→ L.11).
ASID recycling	Reassigning a used ASID to a new process. Legal only after flushing that ASID's TLB entries; skipping the flush leaks the old address space into the new one, silently (→ M.8).
Address as opcode	Command convention where each operation has its own address and the value written is its argument, not a command code (→ M.2).
WDM	Opcode <code>\$42</code> , reserved by WDC and executed as a two-byte, two-cycle no-op. Considered as a command channel and rejected; it stays a no-op, and a softcore must implement it as one.
Memory barrier	An instruction forcing memory ordering. Not needed here and not provided: the 65816 has no prefetch queue and no store buffer, so every bus cycle is already in program order.
Watchdog	Timer that resumes PHI2 and raises NMI or ABORT if a fill overruns – without it a hung fill freezes the machine with no clock and no diagnostic path. It covers Neon's read stall too, which is what makes an unfitted Neon a fault report rather than a dead board (F.11).
Refresh	The periodic recharge SDRAM needs to keep its contents. Must be interleaved into the fill state machine, not deferred until it finishes.

CPU, bus and gateway → sheet B · sheet F

TERM	MEANING
Bank	A 64 KB slice of the 65816's 24-bit space, selected by DBR/PBR. The virtual bank map is in L.10. The map is fixed: <code>\$00</code> stack, direct page and vectors · <code>\$01</code> kernel · <code>\$02–\$FD</code> user · <code>\$FE</code> VRAM window · <code>\$FF</code> privileged I/O.
DBR · PBR · DP	Data Bank, Program Bank and Direct Page registers – the 65816 state that decides which bank an access lands in.

TERM	MEANING
Native mode	The 65816's 16-bit, 24-bit-address mode, entered with <code>CLC/XCE</code> . The opposite is 6502 emulation mode. The M and X flags then pick 8- or 16-bit accumulator and index registers at run time.
PHI2	The CPU's master clock, generated by Helium. Target 8 MHz. Target 8 MHz at 3.3V. Helium may hold it in either state indefinitely – the core is fully static – which is what a cache-miss stall does.
BE	Bus Enable – the pin that makes the physical CPU tristate its address, data and RWB drivers and release the bus. Driven by Helium, which uses it far more often than Argon would (→ R.3).
RESB	The 65816's active-low reset. Released by the RP2354B as the last step before the CPU runs. Pulled down when no bitstream is loaded, so a blank board holds the CPU in reset instead of running it into an undriven bus.
ABORTB	W65C816S pin that cancels the instruction in progress without side effects. The system's fault mechanism. Asserted by the MMU on a missing page or a permission violation, and the instruction restarts once the kernel has fixed the mapping.
VPA / VDA	Pins indicating whether the cycle is an instruction fetch or a data access. They make checking the <code>X</code> flag possible.
RDY	Bidirectional, open-drain: Helium pulls it low to freeze the CPU while it takes the shared nets (F.4), and the CPU pulls it low itself during <code>WAI</code> . Cache-miss stalls do not use it – D16 moved those to PHI2 gating, since some W65C816S revisions ignore RDY during write cycles.
Northbridge	Borrowed PC term for the chip sitting between CPU and memory. Here it is Helium: MMU, cache, arbiter, timer and I/O.
Gateware	The logic loaded into an FPGA – the FPGA equivalent of firmware. Written in HDL, not compiled to instructions.
Softcore	A CPU implemented as gateware rather than as a chip. Argon's optional 65816 is one (→ sheet E). The footprint ships unpopulated, and what a core owes the board is a pin list as much as an instruction set (E.13).
Fmax	The highest clock a placed-and-routed design will meet timing at. A property of the design <i>on a given part</i> – quoting one across FPGA families is meaningless.
CPI	Cycles Per Instruction. The 65816 spends 2–7, many wasted on bus protocol. Lowering it buys throughput without touching the critical path, which is why it beats chasing clock.
Microcode	The internal table that sequences each instruction. On a soft core it sits in block RAM, and registering its output is the standard first move against the critical path.
Cycle-exact	A core reproducing the original's timing cycle for cycle. Needed to emulate a specific machine; unnecessary here, where only programmer-visible behaviour must match (→ E.6).

TERM	MEANING
ILP	Instruction-Level Parallelism – independent work a core can overlap. An accumulator architecture offers little, which bounds what any 65816 core can gain from pipelining.
Accumulator architecture	A design where most operations route through one register. Compact to encode, heavy on memory traffic, and inherently serial.
Bus arbiter	The logic deciding who drives the shared bus in each cycle: CPU, cache fill, video, or refresh. The CPU is stalled rather than queued: Helium takes the shared nets with BE and hands the clock back afterwards.
PIC	Programmable Interrupt Controller – collects device IRQs, applies priorities, and raises IRQ/NMI to the CPU. Neon's three sources arrive on one wire and are demultiplexed by the kernel reading IRQ_STATUS .
EBR	Embedded Block RAM – 32 dual-port blocks of 512 B inside each iCE40, 16 KB in total. On Helium its scarcity is why only the TLB and the cache tags fit on-chip while the page table lives in external SRAM; on Neon it is the whole of Mode 0, and it is fully allocated with zero margin (T.12). The prototype board's ECP5 has ~243 KB of it, roughly fifteen times as much, which is the single most misleading number on that platform (P.04).
Bitstream initialisation	Giving a block RAM its contents from the compiled bitstream rather than loading them at run time. What makes Neon's font and text buffer valid before any software exists – the single most useful property of the design (D27).
HDL	Hardware Description Language – Verilog or VHDL. The choice is still open (Q2).
Stub	In layout, a track branching off a bus. Long stubs ruin signal integrity at ~100 MHz, hence the short comb of F.13.
Expansion slot	The parallel connector of sheet X: the machine's own bus, buffered, with a card as a third tenant of the contract Helium already has with Neon. Unpopulated in the first build.
/EXP_SEL · /EXP_BSY · /EXP_IRQ	The slot's three nets – select, stall, interrupt – mirroring NEON_BUS_SEL / BSY / IRQ , with /EXP_BSY idling not busy because an empty slot must not stall the machine (X.11).
Slot aperture	The 1 MB per slot the MMU can map into a process, carved out of the frame numbers F.10 leaves unpopulated and addressed on A11–A19, which are idle while the SRAM's /CE is (X.8). Always non-cacheable.
Register window vs aperture	The two ways a card is reachable, and the split is T.21's: control in bank \$FF , unmappable to user space by construction; bulk through the aperture, mappable with permissions in front of it.
Geographic addressing	Telling a card which slot it is in with connector pins strapped differently in each position, so the same board works in either one with no jumper (X.14).

TERM	MEANING
Bus master	A device that drives the address bus instead of the CPU. Here: Helium, and nothing else – a card cannot honour the cache-coherence discipline of M.12, so D48 refuses it the bus rather than trusting it (X.16).
I2C-EXP	The third I2C bus, Helium's own and the only one where Helium is master. The slow expansion tier: devices mirrored into I/O windows rather than reached transactionally (X.22).
Window (slow bus)	A copy of a device's registers in Helium's block RAM, refreshed by a sequencer and read by the CPU at bus speed. A mirror, not a proxy – a byte over I2C is ~90 µs and nothing may stall PHI2 for that (X.23).

Board, power and configuration → sheet C · sheet D · sheet S

TERM	MEANING
EC	Embedded Controller – the always-on microcontroller handling power, boot and housekeeping. Here, the RP2354B in QFN-80. It stays off the shared bus entirely, and its in-package 2 MB flash is the board's only mandatory non-volatile storage (D.8).
Rail	A supply voltage distributed across the board (3V3_AON, 3V3_MAIN, 1V2, VPP...). "Sequencing" is the order they come up in. Every logic rail here is 3.3V, which is what keeps level shifters off the board.
SYS	The node after the charger, fed by USB or battery indifferently. Everything hangs off it rather than off the battery.
Power-path	Charger topology that powers the system and charges the cell at once, so the machine runs on USB with a flat or absent battery.
1S	One lithium cell in series – a nominal ~3.7V pack. Two 1S cells in <i>parallel</i> are still 1S: more capacity, same voltage, no balancing.
Fuel gauge	Chip that estimates remaining charge from voltage and current history. Here the MAX17048, read by the EC over I2C. Distinct from an ADC, which reports a voltage and models nothing.
Buck-boost	Converter holding its output steady whether the input is above or below it – needed because a cell crosses 3.3V as it drains.
Boost	Converter that only steps voltage up. Used for the backlight and the 5V host VBUS.
LD0	Low-DropOut regulator – a linear regulator, which wastes the voltage it drops as heat. The 1.2V rail used to be one and is now a buck (D20).
Always-on · AON	The domain powered whenever SYS exists: the EC, its flash, the PD sink, the charger/gauge bus and the power button. Nothing else belongs there.
Switched domain	Everything whose rail the EC can turn off – the FPGAs, the CPU, the memories, the panel, the touch bus. Off by default at EC reset.
Iq	Quiescent current – what a converter draws on its own, at no load. Decisive for an always-on rail, irrelevant for a switched one.

TERM	MEANING
PG • Power Good	An open-drain output asserting that a converter has reached regulation. Used here as the sequencing interlock: the EC waits on it before raising the next rail.
PD • Power Delivery	The USB-C protocol by which a sink asks for more than the default 5V, negotiated over the CC conductors. Here 9V, requested by a resistor.
OTG	A charger mode in which the device sources 5V onto its own VBUS. Present on the BQ25896 and deliberately unused (D21).
Ship mode	A charger state disconnecting the battery from SYS entirely, leaving only leakage. The machine's deepest off state; the button's QON pin wakes it.
QON	The BQ25896 pin that resets the battery FET from hardware, with no firmware involved. The floor under every other way of turning the machine off.
Ordered shutdown	The graceful path: the kernel flushes and confirms before any rail drops. Its opposite is <i>forced</i> , where the EC stops waiting for an answer.
Watchdog	Here, the EC's 10-second bound on how long a kernel may take to acknowledge a shutdown request. Without one, a hung kernel hides itself as a machine that will not turn off.
Staging • live • shadow	The three copies of the telemetry bank. The EC dribbles bytes into staging, one commit publishes them to live, one latch snapshots live into shadow – so an 8-bit CPU can read a 16-bit value without it changing underneath (S.16).
Tearing	Reading a multi-byte value while it is being updated, and getting halves from two different samples. The reason for the snapshot mechanism above.
SSPI	Slave SPI – the iCE40 configuration mode in which an external master (the RP2354B) clocks the bitstream in.
Bitstream	The compiled gateway file loaded into an FPGA at every power-up. iCE40s are SRAM-based: they forget on power-off.
CRESET_B / CDONE	The iCE40's configuration handshake: held low to start loading, raised by the FPGA when configuration succeeded.
Bring-up	First powering of a new board, block by block, verifying each before enabling the next. Each stage of sheet P closes on an explicit TEST ▶ , not on a judgement.
Free-run	Diagnostic where the CPU is fed a constant NOP so it just counts through addresses – proves clock, reset and address bus without any memory.
Handoff	Transfer of a shared resource between two owners – here the microSD passing from the RP2354B to Helium through the '3257 mux. The RP2354B releases the card before Helium's controller claims it – nothing arbitrates it in hardware.

TERM	MEANING
TQFP · PLCC · TSOP · BGA	Chip packages. The first three have accessible leads and can be hand-soldered; BGA hides its balls underneath and cannot, which is why it is excluded (D01).

Video, audio and peripherals → sheet H · sheet T · sheet U · sheet W

TERM	MEANING
Chip RAM	Amiga term reused here: Neon's own 64 MB SDRAM – framebuffers, atlases, level buffers, command lists and audio DMA – outside the CPU hierarchy and reached through the <code>\$FE</code> window.
Framebuffer	The region of memory holding the pixels currently on screen. Exposed to processes as <code>/dev/fb</code> . It lives in Neon's SDRAM rather than in system memory: the CPU reaches it through the <code>\$FE</code> aperture and never has to read it back.
VRAM window	The 64 KB opening in bank <code>\$FE</code> through which the CPU reaches video memory, gated by the MMU's <code>NEON_BUS_SEL</code> . In graphics modes its low 32 KB slides over SDRAM by <code>VRAM_PAGE</code> .
NEON_BUS_SEL	Helium → Neon: "this cycle is validated for you." Qualifies both of Neon's windows – <code>\$FE</code> after translation and the permission check, <code>\$FF:8000 – \$FF:80FF</code> after the bank decode and the privilege check (B.6).
NEON_BUS_BSY	Neon → Helium: "I cannot serve this cycle yet." Busy by default , so an absent or unconfigured Neon stalls the machine into a watchdog report rather than onto an undriven bus. Its meaningful edge is the deassertion, which means the data is ready (T.18).
Bus-busy stall	How the CPU reads VRAM: Helium decodes the read, holds PHI2 low unconditionally, and raises it when Neon clears busy. Stall first, ask afterwards – which removes the race a wait line asserted against the rising edge would have had (D37).
Aperture	The same opening, named from Neon's side. Data only – Neon's control registers are not in it but at <code>\$FF:8000</code> , because <code>\$FE</code> is user-mappable and <code>\$FF</code> is not (D36).
Px-doubling	Drawing at a lower resolution and emitting each pixel more than once in both axes to fill 1024×600. Not a policy here but a mode: Mode 2b at 2×2, and Mode 2a at 3×3 (D35).
RGB-TTL	Parallel video interface: one wire per colour bit plus sync and clock. No bridge chip needed, at the cost of many pins. Driven here at 18 bpp – RGB666.
VSYNC	The pulse marking the end of a frame. Raised as an IRQ so the GUI can redraw without tearing. One of the three sources aggregated onto <code>NEON_IRQ</code> .
Cell · glyph	One character position – a glyph code plus an attribute byte · the pixel pattern for one code, held in the font. 8 × 16 px here, the standard VGA text cell.

TERM	MEANING
Text buffer	The 4096 cells of Mode 0, 8 KB in Neon's block RAM, initialised from the bitstream so the screen is correct at power-on with no software involved.
Ring buffer	A buffer addressed modulo its size, so advancing a pointer rotates the visible contents without moving data. <code>TEXT_START</code> scrolls the console this way – 256 bytes written instead of 8,192 moved.
Blitter	Block image transferrer – hardware that fills, copies and combines rectangular memory regions with no CPU involvement. The centre of Neon's graphics half.
Channel	One of the blitter's four memory ports under the model of sheet U – A, B, C read, D writes. Each carries a pointer, a signed modulo and an enable; a disabled read channel supplies a constant and costs no bandwidth.
Minterm · logic function	One row of the truth table combining channels A, B and C. Three inputs give eight rows, so the whole function is one byte – the <code>LF</code> – selecting any of 256 operations . <code>\$0C</code> copies, <code>\$CA</code> cookie-cuts, <code>\$3C</code> is XOR.
Cookie-cut	Compositing a masked sprite over a background in one pass: mask on A, pixels on B, destination read back on C, <code>LF = \$CA</code> . Needs a mask surface in memory, which a colour key does not.
Modulo	The signed value added to a channel's pointer at the end of every line – the difference between a bitmap's stride and the width being moved. It is what makes a blit two-dimensional , and it expresses clipping with no extra hardware.
Surface	Any rectangular pixel buffer in Neon's SDRAM: framebuffer, window backing store, font atlas, icon sheet.
Backing store	A surface holding one window's complete contents, including the parts currently hidden behind other windows – exactly what a flat framebuffer does not keep (Q60).
Compositor	The kernel code that assembles the visible framebuffer from the backing stores, driving the blitter. Distinct from the window manager, which decides what should be where. Which model it runs is still open: repaint every window intersecting the damage, or recompose from backing stores (Q60).
Descending mode	Blitting with the pointers decrementing rather than incrementing. Required whenever source and destination overlap and the move is down-and-right – a window drag is the case.
Damage · dirty rectangle	A region of the screen whose composited content is no longer valid and must be rebuilt. Under U.31 a bug here costs performance, not corruption, because full recomposition stays correct.
Cookie-cut vs colour key	The two ways to make a sprite non-rectangular. A key designates one palette index transparent and needs no extra memory; a mask is a second bitmap and needs generating and keeping in step. The blitter model decides which is available (Q58).

TERM	MEANING
Colour key	A palette index designated transparent; pixels of that value are not written during a copy. What makes a sprite a sprite here, with no sprite engine.
Barrel shifter	Logic shifting a word by any amount in one cycle. Required to place a 1-bpp image at an arbitrary horizontal position, which is the expensive part of Mode 1 and not optional.
Stride	The byte distance from the start of one image row to the next. Independent per source and destination, which is what lets one copy window a viewport out of a much wider buffer.
Command list	A sequence of drawing commands in SDRAM, executed autonomously by Neon. The ANTIC display list and the Amiga Copper, generalised to include the blitter.
Display list patching	Writing into a list while Neon executes it – the intended usage pattern, not an abuse. Moving an object is one 32-bit store to one word (T.41).
Page flip	Displaying one framebuffer while drawing into another, then exchanging them. <code>SWAP_BUFFERS</code> , applied at vblank so a half-drawn frame is never shown.
Damage model	Dirty rectangles, z-order traversal and repaint from application state. Unavoidable with a flat framebuffer, because moving a window does not restore what was under it – the pixels were overwritten and are gone.
Save-under	Copying a region about to be occluded and restoring it afterwards. Works here despite the CPU not needing to read VRAM, because the <i>blitter</i> moves it and the pixels never leave VRAM.
Service port	Neon's four configuration SPI pins, reused after <code>CDONE</code> as a control channel from the EC. The reason boot progress appears on screen before a CPU exists.
Clock enable	A signal gating a register's update without gating its clock. Used to derive Neon's 51.5625 MHz pixel rate inside one 103.125 MHz domain, so there is no second clock domain to cross.
Row activation	The delay when SDRAM must open a new row before its data is reachable, ~60 ns. Why glyph lookup stays in block RAM, and why an aperture read has to stall the CPU.
SDRAM bank	One of four independent arrays inside the device; accesses to different banks interleave with no row-activation penalty. A layout convention for copies, not something hardware enforces (Q50).
FPC	Flexible Printed Circuit – the flat ribbon connecting the panel. "FPC-50" is its 50-contact connector.
eDP	Embedded DisplayPort – serial panel interface. Rejected for v1 as it needs a bridge chip (D05).
R-2R	A resistor-ladder DAC – the cheapest way to get analogue VGA levels out of FPGA pins. Bring-up only on the target board; the whole video output on the prototype , at 6 bits per channel into 1 % resistors (P. 08).

TERM	MEANING
I2S	Serial audio bus between the FPGA and the DAC – BCK the bit clock, LRCK the frame clock, DIN the data. Unrelated to I2C despite the name.
fs · 32fs	The sample rate · a bit clock of 32 BCK per stereo frame, which is exactly two 16-bit slots with no padding. The frame length decides the divider's granularity , which is why this machine uses 32fs and not the more common 64fs (W.6).
SCK grounded	Strapping that moves the PCM5102A's master clock inside the part, onto its own PLL locked to BCK . Costs one pin less and means the sample rate is whatever Neon's divider produces (W.5).
44,146 Hz	This machine's actual sample rate – 103.125 MHz / (32 × 73). Not 44,100, because the iCE40 PLL is integer-only and 44.1 kHz needs a denominator factor of 441 that no setting can produce. +1.8 cents of pitch, published in AUD_RATE so nothing has to assume it (W.11, Q77).
XSMT	The DAC's soft-mute pin, held low through an RC while the rail rises – the anti-pop . It protects power-up only; the power-down click is Q76.
Saturating add	Clamping a sum at full scale instead of letting it wrap. Four channels at full volume exceed full scale, and a wrap turns the loudest moment in the material into full-amplitude noise (W.27).
Underrun	A DMA buffer running dry before software refills it. Here the channel holds its last sample rather than emitting zero – a step to zero is a click – and sets a sticky flag, because an underrun that cannot be observed is a bug report that says "it crackles sometimes" (W.22).
QSPI	Quad SPI – 4-bit-wide serial bus. Historical here: it went with the PSRAM (D13) and no longer appears on the board.
HID	Human Interface Device – the USB class covering keyboards and mice.
USB host controller	The EC's native USB, assigned to host mode in hardware – it handles hub enumeration, the PRE packets Low Speed devices need, and bulk transfers in silicon. The assignment is one-way: there is no device port and therefore no USB recovery path (D.9).
PRE packet	The Full Speed preamble a host must emit before every transaction with a Low Speed device behind a hub. Historically the most fragile path in this family's host stack, which is why Q102 is a bring-up gate rather than an assumption.
Hub cascade	A second hub level, created whenever a user plugs a hub into a downstream port. The on-board hub already consumes the one level guaranteed to work (D.10).
J-DBG · J-QSPI	The EC's two headers. The first carries SWD and the console UART, serviced by one probe over one cable; the second carries the bootrom UART and direct access to the flash die (D.12).
Rescue debug port	An access point in the EC's debug architecture that can reset the chip even when its firmware is hostile to debugging – a tight loop, a routine that reconfigures the debug pins, a crashed power sequencer. The reason SWD is the primary path rather than the convenient one (D.13).

TERM	MEANING
Bootrom UART boot	The EC's second recovery tier: two straps at reset put the bootrom on a 1 Mbaud UART carried on flash-interface pins. It writes SRAM only , which is why the RAM flash-writer stub exists (D.14).
Mux	Multiplexer – switch routing one set of signals to one of several destinations. The '3257 gives the microSD two possible owners.
DMA	Direct Memory Access – a device reading or writing memory itself, without the CPU moving each byte. The SPI-SD engine is the only one on this board, and it is the whole reason <code>CACHE_INVALID_FRAME</code> exists (M.12).

Operating system and toolchain → sheet I · sheet J · sheet N · sheet O

TERM	MEANING
BIOS	Here it means two things kept distinct: the RP2354B is the "board BIOS" (everything pre-CPU), and <code>BIOS.BIN</code> is the "system BIOS" (everything post-reset).
Info block	The structure the BIOS hands the kernel at load time: RAM size, device map, battery state, RTC time. Format still open (Q5).
Monitor	Minimal interactive debugger: examine and alter memory, load over serial. Two of them exist – one in the BIOS, running on the 65816, and the EC console of sheet R, which covers the same ground from outside the CPU and is available a whole stage earlier.
Kernel	The resident, privileged core of the OS. Lives in virtual bank <code>\$01</code> , pinned in SRAM. Entered only through <code>COP</code> or an interrupt, both of which raise privilege on the vector fetch.
Syscall	A service request from a process to the kernel. Invoked here through the <code>COP</code> instruction, with the service number in the accumulator.
PCB	Process Control Block – the per-process record holding saved registers, ASID and page-table pointer. Not the printed circuit board, which this document always spells out.
Context switch	Switching process; entails saving state to the PCB and pointing the MMU at a different page table.
Preemptive	The kernel takes the CPU back on its own (on the timer tick) rather than waiting for the process to yield it.
Round-robin	Scheduling that simply cycles through ready processes in turn, each getting one quantum.
Tick	The periodic timer interrupt driving scheduling. 100 Hz here. 100 Hz, from a free-running counter in Helium – never from cycle counts, which stop being time the moment PHI2 can stall.
Zombie	A finished process whose exit status is still held for its parent to collect with <code>wait</code> .
Driver	Kernel module handling one device behind a fixed 5-function interface, exposed as a <code>/dev/*</code> node.

TERM	MEANING
devfs	The synthetic filesystem where those <code>/dev/*</code> nodes live – no bytes on the SD card.
ioctl	The escape hatch of the unified I/O interface: device-specific operations that are neither read nor write, such as mapping the framebuffer.
FAT32	The filesystem this document once put on the boot card, and now the <i>first</i> of the three the host track implements (P.26) – chosen there for being the cheapest way to get a real filesystem mounted, not for being used here. The boot card is NVFS (G.3).
BSS	The zero-initialised data of a binary. Carries no bytes in the file: the loader just maps it and clears it.
Relocation	Patching a binary's addresses to match where it was actually loaded. The MMU removes the need: every process sees the same addresses.
ABI	Application Binary Interface – the contract user binaries rely on. Drivers may be rewritten as long as it holds.
JSL / RTL	The 65816's long call and return, crossing banks. The basis of the large memory model and of syscall stubs.
Large model	Compiler model where code is addressed across all banks with JSL/RTL. Paired here with a fixed DBR so data access stays cheap.
Toolchain	The full chain from source to loadable artefact. Open end to end here: KiCad · Yosys · nextpnr · IceStorm · pico-sdk · ca65/64tass.

Filesystems → sheet Y1 · sheet Y2 · sheet Y3

TERM	MEANING
VFS	Virtual File System – the layer between the syscall table and the filesystem drivers. It owns path resolution, the mount table, vnode identity and the error space, because none of those can be decided inside a driver that cannot see across a mount boundary (Y1.1).
vnode	The in-memory object for one <i>in-use file</i> , cached and reference-counted, identified by its mount and a stable file identifier. Not the same as an open file: two <code>open()</code> calls share a vnode and hold separate offsets (Y1.5).
Inode	The on-disk record of a file – type, permissions, size, times, and where its data is. 128 bytes in both NVFS and the ext2 configuration used here. A name is not part of it: names live in directories and point at inode numbers, which is what makes hard links possible.
Superblock	The block that describes the whole volume: sizes, region locations, counters, feature flags. NVFS keeps a byte-identical backup at the last block, written only by <code>mkfs</code> and <code>nvfsck</code> , and never silently falls back to it (Y2.6).
Bitmap · summary	One bit per block or inode saying whether it is allocated, plus – in NVFS – a free-count per bitmap block so the allocator can skip a full one without reading it (Y2.7).

TERM	MEANING
Extent	A run of consecutive blocks described by one entry: first logical block, first physical block, length. Replaces the indirect tree for files laid down in one or a few runs, which is most of them (Y2.10).
Indirect block	ext2's alternative: a block full of block pointers, with double and triple levels above it. Cheap to implement, expensive to read – each level is another transfer before any data moves (Y3.5).
Hole	A logical block covered by no extent and no pointer. Reads as zeros and occupies nothing.
Journal	A log of metadata changes written before the changes reach their home locations, so that recovery is a forward scan and a sequence of block copies. NVFS's is physical, redo-only, metadata-only and circular , which is what makes replay idempotent (Y2.15).
Redo-only · idempotent	The journal holds new contents, never old ones, so replay copies blocks to fixed destinations – running it twice is the same as running it once, and a crash <i>during</i> recovery is harmless.
Checkpoint	Writing a committed transaction's blocks to their home locations and advancing the journal's start pointer. May be deferred and batched; advancing it before those writes are durable is the one way to lose data (Y2.16).
Barrier	The point at which every write issued before it is durable before any issued after. On SD that means waiting out the busy state and confirming – an assumption rather than a guarantee (Q96).
data=ordered	The rule that a transaction's file data reaches the medium before its commit block, so metadata can never reference a block whose contents are stale. The faster alternative is refused here (Y2.18).
Orphan	An inode whose link count reached zero while it was still open. Threaded onto a list in the superblock and freed at the next mount, which is what stops a crash from leaking space (Y2.19).
Deferred delete	Unlinking a file that is still open: the name goes, the blocks are released when the last reference does. Makes deleting an open file safe rather than catastrophic (Y1.18).
Feature flags	Three masks in the superblock. An unknown <i>incompatible</i> bit refuses the mount, an unknown <i>read-only</i> bit mounts read-only, an unknown <i>compatible</i> bit is ignored – and the implementation must name the bit it did not recognise (Y2.20, Y3.4).
htree · DIR_INDEX	ext2's hashed directory index, hidden inside fake entries so that a linear reader sees the directory correctly. Reading is safe and writing is not : an insert that does not update the index leaves a file that exists, occupies space, appears in <code>readdir</code> and cannot be opened by name (Y3.6).
readdir cookie	An opaque value a driver hands back to resume iteration. NVFS advances a slot index and ext2 a byte offset; neither is expressible in the other's terms, so the VFS stores it and never interprets it (Y1.3).

TERM	MEANING
fsck · e2fsck · nvfsck	Host-side repair tools. There is no on-target fsck and none is planned for either filesystem: NVFS recovers from its journal and its orphan list, and ext2's recovery is the host's (Y2.22, Y3.2).
Differential testing	Performing every operation twice – once through the implementation under test and once through a reference driver – and comparing. What makes the host track tractable (Y3.15).
Crash injection	Hooking a specific write number, truncating the image there, and checking that what remains is clean or trivially recoverable. The harness that found draft 2's ordering bugs (Y2.2).
FUSE	Filesystem in Userspace – the Linux mechanism the host track mounts through. Its low-level, inode-keyed API is the one that transfers; the path-based one does not, because a driver here never sees a path (P.28).
MBR type 0x7F	The partition type byte reserved for local and experimental use, and the one an NVFS volume carries. 0x7A and 0x7B are unsafe – they alias hidden-FAT types (D52).

The windowing OS → sheet V

TERM	MEANING
wserver	The window server: window objects, z-order, input routing, focus, decoration and client validation. A user task here, because none of that needs a privileged instruction – the compositor beside it is kernel code, because command emission does (Q62).
tk · gcl	The client-side libraries: the widget toolkit and the drawing layer that appends ops to the shared buffer. Both link into the application; neither is a server component.
Shared command buffer	The 4 KB page mshare maps writable in the client and readable in the server. Drawing ops accumulate in it and flush with one syscall – a syscall per primitive would cost more than the primitive (V.20).
Content list · composite list	The two kinds of command list a GUI keeps in SDRAM: one draws into a window's backing store and runs when its content changes , the other blits that store into the back buffer and runs every frame . Text lives in the first, which is why an idle window costs nothing.
String cache	Composed runs of glyphs held as a strip in SDRAM, so drawing a repeated label is one COPY_RECT instead of one command per character. Attacks emission cost, not bandwidth – and proportional fonts fall out of it free, since it stores runs rather than glyphs.
DRAW_GLYPHS	A proposed command taking a string pointer and a count, so a line of text is one command with no cache at all. Complementary to the cache rather than a replacement : the cache wins for stable labels, this wins for text that changes (Q64).

TERM	MEANING
Refresh event	The message an Amiga client got when part of its window became visible again, obliging it to repaint. There is none here: the backing store already holds the pixels, so a client paints when its own state changes and never otherwise.
layers.library	The Amiga layer that answered <i>what is behind this window and who repaints it</i> – cliprects, <code>SIMPLE_REFRESH</code> , <code>SMART_REFRESH</code> , <code>SUPER_BITMAP</code> . Per-window backing stores delete the question, and the library with it.
Intuition · IDCMP · BOOPI	Amiga's window system, its event protocol and its widget object model. Named here as the reference the design departs from: <code>wserver</code> , a ten-event set, and a static vtable toolkit respectively.
Z-order	The front-to-back ordering of windows. The compositor draws back to front – painter's algorithm – so occlusion needs no region arithmetic at all: the front window is simply drawn afterwards.
Decoration	The frame, title bar and gadgets the server draws into the window's own backing store , above the content area and clipped away from the client. Composited as one wide blit rather than four narrow ones, which is what U.18 asks for.
Live drag	Moving a window by recompositing every frame rather than dragging an XOR outline and moving the pixels once. Affordable here, and the default.
Hit test	Walking the z-list front to back to find which window a pointer position belongs to. Decoration hits stay in the server; content hits are delivered to the client in window-relative coordinates.
Port · signal · message	The Exec model, proposed for this kernel: a port bound to one bit of a task's 32-bit signal mask, fixed 32-byte messages copied by the kernel, and <code>task_wait</code> as the single blocking primitive (Q63).
Coalescing	Collapsing a burst of events into the latest one – never more than one motion event per frame per client. Not an optimisation at ~12 µs of kernel copy per message, and it constrains the HID event format to absolute state rather than deltas.
Overdraw	Total composited area ÷ screen area, counting the desktop clear. The number that decides whether full recomposition fits: ~1.8× is comfortable at 1 bpp and consumes the whole frame at 8 (V.10).
Widget vtable	The toolkit's dispatch: a static <code>init · render · event · layout · attr</code> table per class, deliberately mirroring the kernel's five-function driver interface. ~15 cycles per call – right at widget granularity, wrong at pixel granularity.
Upload	The escape hatch for imagery Neon's primitives cannot produce: the client rasterises into system RAM and the server pushes it through the aperture. ~15 ms for a 200 × 100 patch at 8 bpp – correct to provide, correct to discourage.

Debug and instrumentation → sheet R

TERM	MEANING
Debug agent	The block inside Helium that performs memory and bus accesses on command from the RP2354B. A requester in the arbiter, not an external master.
Internal access	A transaction satisfied entirely inside Helium – SRAM, SDRAM or Helium's own registers. Nothing appears on the CPU bus.
External cycle	A transaction where Helium drives the CPU bus pins, so devices outside it see a cycle indistinguishable from the 65816's. The only way to reach the \$FE aperture.
PHI2 stall	Freezing the CPU clock to steal bus time or to halt. Legal because the core is static; the same gating logic serves cache fills, halts, external cycles and Neon's read stall.
Cycle stealing	Taking the bus within a window the CPU is not using, instead of stalling it. The cheaper of the two paths when it is available. Available only where the CPU leaves a gap; where it does not, the stall above is the fallback.
Trace buffer	Ring buffer in EBR recording one entry per bus cycle – address, data, and the control pins. A logic analyser built into the gateway.
Trigger arming	The address-and-mask comparison that starts or stops a capture, and the act of enabling it. Position selects whether the captured window sits before, around or after the match.
Scan chain	The register-readout path debuggable CPUs provide. The W65C816S has none, which is why its registers can only be inferred or dumped by a stub (→ R.15).
Debug stub	A short routine that pushes every CPU register to a known location. Authoritative and intrusive – it perturbs the state it reports, so the console never runs it silently.
W1C	Write-1-to-Clear – a status bit cleared by writing a one to it, so two readers cannot clear each other's flags by accident.
March test	Memory test that walks a pattern up and down the array in a defined order, catching stuck, coupled and address-decode faults that a simple write-read pass misses.
Walking ones address-in-address	The two other standard patterns: one bit moved through each word, and each cell holding its own address. The second is what finds a swapped address line.
Turnaround	The gap a bus needs between one driver releasing and the next asserting. Sized in clocks from the datasheet, never guessed – too short shorts two drivers, too long wastes the cycle.
Clock domain crossing	Passing a signal between two unrelated clocks – here the SPI link and the Helium core. Needs synchronisers and a handshake, and is a classic home of intermittent faults.
GDB stub	Firmware speaking the GDB remote serial protocol, letting a host debugger drive the target. Deferred until the command set stops moving (→ R.24).

TERM	MEANING
Breakpoint · watchpoint	Halt on reaching an address · halt on touching a datum. The first is BRK or a trace-trigger comparator; the second is deferred.

The prototype board

TERM	MEANING
Prototype board	The single-ECP5 carrier of sheet P – Helium and Neon merged on one commercial module, with the CPU, SRAM, EC, VGA and audio on a board of our own. Temporary packaging of the design, never a variant of it (D40).
Colorlight i9 v7.2	The commercial module the prototype is built around: an LFE5U-45F with 8 MB SDRAM, 8 MB SPI flash, two unused Ethernet PHYs and a 25 MHz oscillator, on a DDR2 SODIMM edge connector. Sold for LED-panel drivers and widely used as a cheap FPGA board.
ECP5	Lattice's larger FPGA family. Rejected for the target board by D01 – BGA only in the useful sizes, and the project's rule is hand-solderable – which is exactly why it is acceptable on a module somebody else soldered.
SODIMM	The 200-pin edge connector the i9 module plugs into. It carries raw FPGA balls , not buffered I/O, which is what makes the module usable as a general-purpose carrier at all.
PMOD	The 2 × 6 header convention used by the i9 extension board. Whether its pins are direct or pass through unidirectional buffers decides whether the wire-wrap stage is possible (Q67).
DAPLink	The on-board debug probe of the i9 extension board, giving JTAG plus a serial bridge. Absent on a carrier that hosts the SODIMM socket directly, which is why the RP2354B picks up ECP5 JTAG there (P.10).
prjtrellis · nextpnr-ecp5	The ECP5 half of the same open toolchain – Yosys and nextpnr, different backend. Added to E0.1 by the prototype; no proprietary tools on either board.
Letterboxing	Showing a window into a buffer larger than the display rather than reflowing the buffer. How Mode 0 keeps its 128 × 32 geometry on a 640 × 480 monitor (D44).
Leak	A prototype convenience that survives into the target design and becomes a defect there – shared memory, one clock domain, abundant EBR, a hardcoded size. Sheet P carries the list, and it is reviewed at every merge, not at the transition.

The twelve figures are numbered across the whole document rather than per sheet (A. 13), and each one lives in the sheet it illustrates – so this is the only place they can be seen as a set. The glossary is Z1.

FIG.	SHEET	WHAT IT SHOWS
Fig. 1	B – Global architecture	The four inhabitants of the shared bus, with the RP2354B governing configuration and reset from outside it
Fig. 2	C – Power supply	Power tree REV C: USB-C and CH224K into the BQ25896 power-path, SYS, and the five converters hanging off it
Fig. 3	D – Embedded controller	The nine-step boot sequence – steps 1–6 the EC's, from step 7 the 65816's
Fig. 4	F – Physical memory	Memory structure: the SRAM on the CPU's own nets, Helium driving the translated half and owning the SDRAM alone
Fig. 5	K – Virtual memory concepts	Field-width asymmetry – the virtual split falls out of the CPU's 24 bits, the physical one is the MMU's choice
Fig. 6	L – Virtual memory management	Address translation: TLB with ASID, the hardware walker over the flat table in SRAM, and the ABORTB fault path
Fig. 7	P – Step-by-step build	The two build tracks, prototype above and target below, with both milestones at their exact point
Fig. 8	R – Debug agent	The debug agent as a requester inside Helium: the EC commands over SPI and never drives the bus
Fig. 9	S – Power control	The power control path – two level pins, the SPI link, and telemetry latched into atomic snapshots
Fig. 10	T – Neon	Neon: the text path that depends on nothing but the bitstream, and the graphics path fed by commands rather than pixels
Fig. 11	U – Blitter and compositor	The blitter datapath and the bank partition that is worth a factor of two in delivered bandwidth
Fig. 12	V – The windowing OS	The windowing stack, and the privilege boundary falling on command emission rather than on the framebuffer

noVa64	SYNTHESIS DOCUMENT – INDEX AND CONTENT OUTLINE – COMPLETE, SHEETS A–Z2
Seven parts · ten areas · sheets A–X and Y1–Y3 · Figs. 1–12 · appendix Z1–Z2 Self-contained: every sheet is the specification of what it covers, not a summary of one held elsewhere – power in C and S, the embedded controller, USB and the recovery paths in D, the kernel–hardware interface in M, the debug agent in R, the filesystem stack in Y1, Y2 and Y3, Neon in T, U and W, the windowing OS in V, the expansion bus in X, the build in P	
All 3.3V · hand-solderable · open toolchain	DOC v0.1 · ARCHITECTURE REV A · POWER REV C · EC REV B · CPU REV A · MEMORY REV A · FS REV A · NEON REV B · AUDIO REV A · OS REV A · GUI REV A · PROTOTYPE REV A · AUG 2026